

La Bible DevOps & Cloud

Du premier terminal Linux jusqu'aux pipelines DevSecOps sur Kubernetes : les 30 projets du programme [DevOps-Projects](#) expliqués pas à pas, pour un vrai débutant.

Linux Réseau AWS Azure Git Docker Kubernetes Terraform Ansible
Jenkins GitHub Actions GitLab CI Azure DevOps Helm Argo CD SonarQube
Trivy Prometheus Grafana

```
$ git clone https://github.com/sanctifie/I0.git  
$ cd I0 && echo "C'est parti."
```

Table des matières

Comment utiliser cette bible	4
PARTIE 0 — AVANT DE COMMENCER	7
Le DevOps, c'est quoi ?	8
Le cloud, c'est quoi ?	10
Préparer ton poste de travail	12
Ouvrir et sécuriser ton compte AWS	14
Maîtriser les coûts	16
Les autres comptes à créer	17
PARTIE 1 — LES FONDAMENTAUX	18
Leçon 1 — Linux pour le DevOps	19
Leçon 2 — Le réseau, sans douleur	24
Leçon 3 — Les services AWS de base	26
Projet 03 — Linux pour les ingénieurs Cloud & DevOps	30
Projet 02 — Architecture VPC scalable sur AWS	36
Projet 01 — Application Java sur une architecture 3-tiers AWS	43
PARTIE 2 — CONTENEURS & PREMIERS PIPELINES CI/CD	51
Leçon 4 — Git pour de vrai	52
Leçon 5 — Docker et les conteneurs	53
Leçon 6 — La CI/CD	56
Leçon 7 — Jenkins	57
Leçon 8 — Ansible	59
Projet 04 — Application Django sur AWS avec ECR et ECS	60
Projet 05 — Déployer du code dans un conteneur Docker avec Jenkins	65
Projet 06 — Le pipeline CI/CD avancé avec les grands outils DevOps	70
Projet 07 — Le parcours DevOps avec Azure DevOps	78
Projet 10 — Pipeline CI/CD pour une application .NET sur Azure	84
Projet 14 — Pipeline CI/CD de bout en bout avec GitHub Actions pour une application Android	88
PARTIE 3 — KUBERNETES & INFRASTRUCTURE AS CODE	92
Leçon 9 — Kubernetes	93
Leçon 10 — Terraform	97
Leçon 11 — Helm	100
Leçon 12 — GitOps et Argo CD	101
Projet 08 — Kubernetes de bout en bout sur EKS : le jeu 2048	103
Projet 11 — Infrastructure 2-tiers sur AWS avec Terraform	107
Projet 12 — Super Mario sur Kubernetes, avec Terraform	110
Projet 15 — Application e-commerce 3-tiers sur EKS avec Helm	113
Projet 16 — Projet DevOps temps réel : de Jenkins à Kubernetes avec Argo CD	117
Projet 17 — Déployer une application sur AKS avec Azure DevOps et Cloud Shell	122
Projet 18 — Pipeline Jenkins GitOps pour une application Java : Maven, SonarQube, Docker, Argo CD	125
Projet 19 — De zéro à la production : EKS et CI/CD avec Jenkins et Terraform	129
Projet 20 — Déployer du Terraform sur Azure avec un pipeline Azure DevOps	133
PARTIE 4 — CLOUD NATIF AWS & SERVERLESS	138
Leçon 13 — La CI/CD « maison » d'AWS et le serverless	139
Projet 21 — Pipeline CI/CD avec les outils DevOps d'AWS	141

Projet 22 — Architecture 100 % serverless sur AWS avec CI/CD	144
Projet 26 — Automatiser l'infrastructure AWS avec Terraform et GitLab CI/CD	147
PARTIE 5 — DEVSECOPS & OBSERVABILITÉ	151
Leçon 14 — DevSecOps : la sécurité dans le pipeline	152
Leçon 15 — Observabilité : monitoring, logs, alertes	154
Projet 09 — DevSecOps : clone de Netflix, CI/CD, monitoring et e-mails	156
Projet 13 — Clone de Zomato : déploiement sécurisé avec une CI/CD DevSecOps	161
Projet 23 — DevSecOps : déploiement blue/green du clone de Swiggy sur ECS avec AWS CodePipeline	164
Projet 24 — Pipeline DevSecOps temps réel pour une application web .NET	168
Projet 25 — DevSecOps : l'application Java Petshop avec Jenkins, Docker, Ansible et Kubernetes	171
Projet 27 — DevSecOps : l'application Reddit sur EKS avec Argo CD, et son monitoring	175
Projet 28 — DevSecOps : déployer une interface de chatbot OpenAI sur EKS avec Jenkins et Terraform	179
Projet 29 — CI/CD d'une application de vote en microservices avec Azure DevOps et Argo CD	182
Projet 30 — Le projet final : Blog App sur EKS avec Nexus, SonarQube, Trivy et monitoring	186
ANNEXES — RECETTES, AIDE-MÉMOIRE ET GLOSSAIRE	192
Annexe A — Les recettes d'installation	193
Annexe B — Aide-mémoire	199
Annexe C — Glossaire	201
Annexe D — Planning type et suivi	204

Comment utiliser cette bible

Cette bible accompagne le programme **DevOps-Projects** : 30 projets réels, classés du plus simple au plus avancé. Le dépôt d'origine donne surtout des listes de tâches (« crée un VPC », « installe Jenkins »...). Ici, chaque tâche est **expliquée** : ce qu'on fait, la commande exacte, et surtout *pourquoi* on le fait.

À qui s'adresse ce livre ?

À quelqu'un qui part de zéro, ou presque. Tu n'as pas besoin de savoir programmer, ni de connaître Linux, ni d'avoir déjà ouvert la console AWS. Il te faut :

- un ordinateur (Windows, macOS ou Linux) avec 8 Go de RAM au minimum ;
- une connexion Internet ;
- une carte bancaire (AWS et Azure la demandent, même pour l'offre gratuite) ;
- environ **6 à 9 mois** à raison de 8 à 10 heures par semaine pour tout le parcours ;
- de la curiosité, et la patience de relire un message d'erreur jusqu'au bout.

Comment le livre est organisé

Partie	Contenu	Projets
0 — Avant de commencer	Ce qu'est le DevOps, le cloud, préparer son poste, sécuriser son compte AWS, maîtriser ses coûts	—
1 — Fondamentaux	Leçons : Linux, réseau, AWS de base. Puis les premiers projets	03, 02, 01
2 — Conteneurs & CI/CD	Leçons : Git, Docker, CI/CD, Jenkins, Ansible. Puis les projets	04, 05, 06, 07, 10, 14
3 — Kubernetes & IaC	Leçons : Kubernetes, Terraform, Helm, GitOps. Puis les projets	08, 11, 12, 15–20
4 — Cloud natif & Serverless	Leçons : services managés AWS, serverless. Puis les projets	21, 22, 26
5 — DevSecOps & Observabilité	Leçons : sécurité de la chaîne logicielle, monitoring. Puis les projets	09, 13, 23, 24, 25, 27–30
Annexes	Recettes réutilisables (installer Jenkins, un cluster kubernetes...), aide-mémoire, glossaire	—

Chaque projet suit **le même plan**, pour que tu saches toujours où tu en es :

1. **Fiche projet** : niveau, durée, coût estimé, compétences travaillées.
2. **Ce qu'on construit** : l'architecture, avec un schéma.
3. **Les concepts** à comprendre avant de taper la moindre commande.
4. **Pas à pas** : les étapes numérotées, chaque commande expliquée.
5. **Vérification** : comment prouver que ça marche.
6. **Dépannage** : les erreurs les plus fréquentes et leurs solutions.
7. **Nettoyage** : ce qu'il faut détruire pour ne pas payer.
8. **Pour aller plus loin** et **questions d'entretien**.

Les encadrés

💡 ASTUCE

Un raccourci, une bonne pratique, une façon plus confortable de faire.

🧠 POURQUOI ?

L'explication de fond. **Ne les saute pas** : c'est ce qui fait la différence entre quelqu'un qui recopie des commandes et un ingénieur DevOps.

⚠️ ATTENTION

Un piège classique dans lequel presque tout le monde tombe au moins une fois.

🚫 DANGER — COÛTS / SÉCURITÉ

Un risque d'argent (une facture) ou de sécurité (une fuite de clés). À lire deux fois.

🧹 NETTOYAGE OBLIGATOIRE

La liste de ce qu'il faut supprimer à la fin d'un projet. Un cluster Kubernetes oublié peut coûter plus de 150 € par mois.

Conventions dans les commandes

```
# Une ligne qui commence par # est un commentaire : elle n'est pas exécutée.
sudo apt update      # $ ou rien devant = commande à taper dans ton terminal
<MON_IP>             # entre chevrons = à remplacer par TA valeur (sans les chevrons)
```

BASH

- Les commandes sont écrites pour **Ubuntu 24.04** (serveurs) et **Amazon Linux 2023** quand c'est précisé.
- La région AWS utilisée dans les exemples est **eu-west-3 (Paris)**. Tu peux en choisir une autre, mais reste cohérent dans tout un projet.
- Quand une commande dépend d'un numéro de version (Kubernetes, Prometheus, SonarQube...), le livre utilise une variable comme **VERSION=...** et t'indique où trouver la dernière version. **Les versions changent vite ; la méthode, elle, reste la même.**

CE QUI A CHANGÉ DEPUIS L'ÉCRITURE DU DÉPÔT D'ORIGINE

Le dépôt DevOps-Projects date de 2022-2024. Certaines choses ont changé, et cette bible a été mise à jour en conséquence :

- **AWS CodeCommit** n'accepte plus de nouveaux clients (depuis juillet 2024) → on utilise GitHub avec **AWS CodeConnections**.
- **Azure DevOps Starter** a été retiré par Microsoft → on reconstruit le même pipeline à la main (c'est d'ailleurs plus formateur).
- Les **Launch Configurations** EC2 sont dépréciées → on utilise des **Launch Templates**.
- L'**offre gratuite AWS** a changé en juillet 2025 : les nouveaux comptes reçoivent des **crédits** (jusqu'à 200 \$) valables 6 mois, et les instances éligibles sont les **t3.micro / t4g.micro** plutôt que **t2.micro**.
- Le backend S3 de **Terraform** sait maintenant verrouiller l'état sans DynamoDB (**use_lockfile = true** , Terraform 1.10 et plus).
- **SonarQube Community** s'appelle désormais **SonarQube Community Build**.
- **OWASP Dependency-Check** demande une clé d'API NVD (gratuite) pour ne pas mettre des heures à télécharger sa base.

La méthode de travail (à appliquer à chaque projet)

1. **Lis le projet en entier avant de commencer.** Comprends le schéma.
2. **Crée le dossier** `projects/NN-nom/` dans le dépôt IO, avec un fichier `NOTES.md`.
3. **Avance étape par étape.** Après chaque étape, vérifie que ça marche avant de passer à la suivante. Une erreur à l'étape 3 qu'on découvre à l'étape 9, c'est une heure perdue.
4. **Note tout** dans `NOTES.md` : les commandes, les erreurs rencontrées et comment tu les as résolues. Ce journal deviendra ton portfolio et ta mémoire.
5. **Commit souvent** (`git add` , `git commit` , `git push`).
6. **Nettoie** à la fin (encadré ), puis vérifie la facturation le lendemain.
7. **Coche la case** dans le `README.md` du dépôt.
8. **Explique le projet à voix haute** en 3 minutes, comme en entretien. Si tu bloques, c'est que tu n'as pas encore tout compris : relis les encadrés .

LE MODÈLE DE NOTES.MD

```
# Projet NN – Titre

## Objectif (en une phrase, avec tes mots)

## Architecture (schéma ou description)

## Journal
### Jour 1 – date
- Fait : ...
- Problème : <message d'erreur exact>
- Cause : ...
- Solution : ...

## Ce que j'ai appris (5 points)

## Nettoyage effectué (liste + date)
```

MARKDOWN

Comment déboguer (la compétence n°1)

Un ingénieur DevOps passe une bonne partie de son temps à comprendre pourquoi quelque chose ne marche pas. Voici une méthode qui marche presque toujours :

1. **Lis le message d'erreur en entier**, en particulier la *dernière* ligne et la *première* ligne qui mentionne ton code ou ta configuration.
2. **Reformule le problème** : « le navigateur n'arrive pas à joindre le port 8080 de la machine X ».
3. **Remonte la chaîne, couche par couche**. Le service tourne-t-il (`systemctl status`) ? Écoute-t-il sur le bon port (`ss -tlnp`) ? Le pare-feu de la machine laisse-t-il passer ? Le *Security Group* AWS laisse-t-il passer ? L'IP est-elle la bonne ?
4. **Consulte les logs** : `journalctl -u <service>` , `docker logs` , `kubectl logs` , `kubectl describe` , la console du job Jenkins.
5. **Change une seule chose à la fois**, puis reteste.
6. **Cherche le message exact** sur Internet, entre guillemets. Tu n'es presque jamais le premier.
7. Si tu bloques plus de 45 minutes, **fais une pause** ou explique le problème à quelqu'un (ou à un canard en plastique). La moitié du temps, la solution apparaît pendant l'explication.

✿ LE MODÈLE DES « COUCHES »

Un problème réseau, c'est comme une lettre qui n'arrive pas. Avant d'accuser le facteur, vérifie que l'adresse est juste, que la boîte aux lettres existe, que le nom est bien sur la boîte, et que personne n'a fermé le portail. On vérifie toujours **du plus simple au plus compliqué**, et **de l'intérieur vers l'extérieur** (d'abord sur la machine elle-même avec `curl localhost:8080`, puis depuis l'extérieur).

PARTIE 0

Avant de commencer

Comprendre ce qu'est le DevOps et le cloud, préparer ton poste de travail, ouvrir et sécuriser tes comptes, et apprendre à ne jamais recevoir de mauvaise surprise sur ta facture.

Le DevOps, c'est quoi ?

Le problème que le DevOps résout

Imagine une entreprise des années 2005. D'un côté, les **développeurs** (*Dev*) écrivent le code des applications. Leur objectif : livrer de nouvelles fonctionnalités, vite. De l'autre, les **opérationnels** (*Ops*, les administrateurs système) font tourner les serveurs. Leur objectif : que rien ne tombe en panne. Donc changer le moins possible.

Les deux équipes ont des objectifs opposés. Le résultat :

- les développeurs « jettent le code par-dessus le mur » une fois tous les 6 mois ;
- la mise en production se fait un samedi soir, à la main, avec un document Word de 40 pages ;
- quelque chose casse, chacun accuse l'autre (« ça marche sur ma machine ! ») ;
- on corrige dans l'urgence, et on attend encore 6 mois avant la prochaine livraison.

Le **DevOps** est né vers 2009 pour casser ce mur. Ce n'est **ni un outil, ni un poste, ni une équipe** au départ : c'est une **culture** et un ensemble de **pratiques** pour que développement et exploitation travaillent ensemble, et pour livrer du logiciel **souvent, rapidement et de façon fiable**.

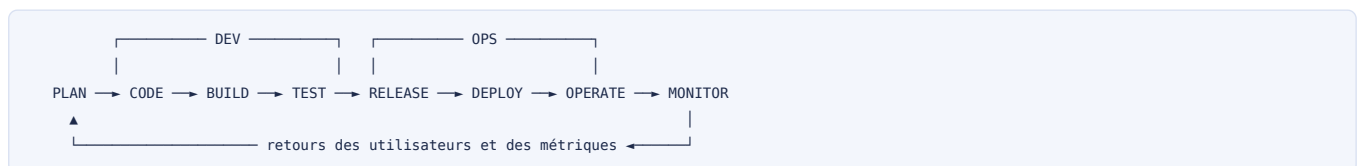
🌱 LA CUISINE D'UN RESTAURANT

Avant le DevOps, c'est un restaurant où les cuisiniers inventent des plats sans jamais parler aux serveurs, et où les serveurs refusent tout nouveau plat de peur de se tromper. Le DevOps, c'est une brigade où tout le monde suit les mêmes fiches recettes (*le code*), où chaque plat est goûté avant de sortir (*les tests automatiques*), où le passe-plat est organisé et rapide (*le pipeline CI/CD*) et où on regarde en permanence si les clients finissent leur assiette (*le monitoring*).

Les piliers : le modèle CALMS

Lettre	Pilier	Ce que ça veut dire concrètement
C	<i>Culture</i>	Responsabilité partagée : « you build it, you run it ». Pas de chasse au coupable après un incident (<i>post-mortem blameless</i>).
A	<i>Automation</i>	Tout ce qui est répétitif est automatisé : tests, build, déploiement, création de serveurs.
L	<i>Lean</i>	Livrer par petits morceaux, souvent. Éliminer les temps d'attente et le travail inutile.
M	<i>Measurement</i>	Mesurer pour décider : temps de déploiement, taux d'erreurs, temps de rétablissement.
S	<i>Sharing</i>	Partager le savoir, les outils, la documentation, les tableaux de bord.

La boucle DevOps



Chaque étape de la boucle a ses outils. Tu vas tous les utiliser dans ce parcours :

Étape	Rôle	Outils du parcours
Plan	Organiser le travail	Azure Boards, GitHub Issues
Code	Écrire et versionner le code	Git , GitHub, GitLab, Azure Repos
Build	Compiler, empaqueter	Maven, Gradle, npm, Docker
Test	Vérifier la qualité et la sécurité	JUnit, SonarQube , Trivy , OWASP Dependency-Check
Release	Stocker les livrables versionnés	JFrog Artifactory, Nexus , Docker Hub, ECR, ACR
Deploy	Mettre en production	Jenkins , GitHub Actions , GitLab CI, Azure Pipelines, AWS CodePipeline, Argo CD , Helm
Operate	Faire tourner l'infrastructure	AWS , Azure , Kubernetes , Terraform , Ansible
Monitor	Observer et alerter	Prometheus , Grafana , CloudWatch, Application Insights

Les grandes notions que tu vas croiser partout

Intégration continue (CI, *Continuous Integration*). À chaque fois qu'un développeur pousse du code, un robot récupère le code, le compile et lance les tests automatiquement. Si quelque chose casse, on le sait en quelques minutes, pas en quelques mois.

Livraison / déploiement continu (CD, *Continuous Delivery / Deployment*). Le code qui a passé la CI est automatiquement empaqueté et prêt à être déployé (*delivery*), voire déployé automatiquement en production (*deployment*).

Infrastructure as Code (IaC). Au lieu de cliquer dans une console pour créer des serveurs, on **décrit l'infrastructure dans des fichiers texte** (Terraform, CloudFormation). Ces fichiers sont versionnés dans Git, relus, testés et rejoués à l'identique. C'est la fin du serveur « que seul Jean-Michel sait reconfigurer ».

Conteneurs. Une application empaquetée avec *tout* ce dont elle a besoin (bibliothèques, configuration) dans une « boîte » standard (Docker) qui tourne pareil partout : sur ton portable, en test, en production.

Orchestration. Quand on a des centaines de conteneurs, il faut un chef d'orchestre qui les place sur des machines, les redémarre s'ils plantent et les multiplie quand le trafic augmente : **Kubernetes**.

GitOps. Git devient la seule source de vérité : l'état voulu du système est dans Git, et un outil (Argo CD) s'assure en permanence que la réalité correspond à Git.

DevSecOps. On intègre la sécurité **dans** le pipeline (analyse du code, des dépendances, des images) au lieu de la vérifier à la fin. C'est ce qu'on appelle le *shift left* : déplacer les contrôles vers la gauche de la boucle, donc plus tôt.

Observabilité. Pouvoir répondre à « que se passe-t-il dans mon système en ce moment, et pourquoi ? » grâce aux **métriques**, aux **logs** et aux **traces**.

Le métier d'ingénieur DevOps / Cloud

Dans la vraie vie, « DevOps » est devenu un intitulé de poste. Un ingénieur DevOps ou Cloud passe ses journées à :

- construire et maintenir des **pipelines CI/CD** ;
- écrire de l'**Infrastructure as Code** (Terraform surtout) ;
- gérer des **clusters Kubernetes** et des services cloud ;
- automatiser (scripts Bash/Python, Ansible) ;
- mettre en place **monitoring et alertes**, et intervenir sur les incidents ;
- sécuriser l'ensemble (droits IAM, secrets, scans) ;
- aider les développeurs à livrer plus vite (*platform engineering*).

LES MESURES DORA

Les équipes DevOps sont souvent évaluées avec 4 indicateurs, issus des recherches DORA (*DevOps Research and Assessment*) :

1. **Fréquence de déploiement** : combien de fois on livre en production.
2. **Délai de mise en production** (*lead time*) : temps entre un commit et sa mise en production.
3. **Taux d'échec des changements** : pourcentage de déploiements qui causent un incident.
4. **Temps de rétablissement** (*MTTR*) : temps pour réparer après un incident.

Les meilleures équipes déploient plusieurs fois par jour, en moins d'une heure, avec peu d'échecs, et réparent en moins d'une heure. Tout le parcours sert à acquérir les compétences qui permettent ça.

QUESTIONS D'ENTRETIEN

- Qu'est-ce que le DevOps, en une phrase ? *Une culture et des pratiques qui rapprochent développement et exploitation pour livrer souvent, vite et de façon fiable, grâce à l'automatisation et à la mesure.*
- Quelle différence entre *continuous delivery* et *continuous deployment* ? *En delivery, le déploiement en production reste déclenché par un humain. En deployment, il est automatique dès que tous les tests passent.*
- Qu'est-ce que l'Infrastructure as Code et pourquoi l'utiliser ? *Décrire l'infrastructure dans des fichiers versionnés. Bénéfices : reproductibilité, relecture, historique, rapidité, moins d'erreurs humaines.*

Le cloud, c'est quoi ?

L'idée

Le **cloud computing**, c'est **louer** des ressources informatiques (serveurs, stockage, bases de données, réseau...) à la demande, via Internet, et **payer à l'usage**, au lieu d'acheter et d'installer ses propres serveurs.

🌱 L'ÉLECTRICITÉ

Au XIX^e siècle, chaque usine avait son propre générateur. Puis on s'est branché sur le réseau électrique : on paie ce qu'on consomme, sans s'occuper de la centrale. Le cloud fait la même chose pour l'informatique. AWS, Azure et Google Cloud sont les « centrales ».

Les avantages :

- **Élasticité** : on passe de 2 à 200 serveurs en quelques minutes, puis on redescend.
- **Paieement à l'usage** : pas d'investissement de départ. *Attention : ça veut aussi dire qu'une ressource oubliée coûte de l'argent en continu.*
- **Services managés** : base de données, file de messages, Kubernetes... le fournisseur gère les mises à jour et la haute disponibilité.
- **Portée mondiale** : des datacenters sur tous les continents.

IaaS, PaaS, SaaS : qui gère quoi ?

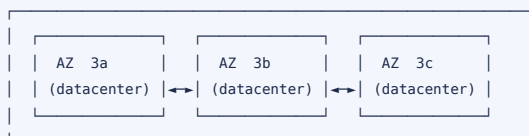
Modèle	On loue...	Tu gères	Exemples
On-premise	rien (tu possèdes tout)	tout, du bâtiment à l'application	ton propre datacenter
IaaS (<i>Infrastructure as a Service</i>)	des machines virtuelles, du réseau, du stockage	l'OS, les mises à jour, le runtime, l'application	AWS EC2, Azure VM
PaaS (<i>Platform as a Service</i>)	une plateforme prête à exécuter ton code	ton code et sa configuration	AWS Elastic Beanstalk, Azure App Service, AWS Lambda
SaaS (<i>Software as a Service</i>)	un logiciel fini	tes données et tes utilisateurs	Gmail, GitHub, Slack

Plus on monte, moins on a de travail d'administration, mais moins on a de contrôle.

Régions et zones de disponibilité

Les fournisseurs découpent le monde en **régions** (par exemple `eu-west-3` = Paris, `eu-west-1` = Irlande, `us-east-1` = Virginie). Chaque région contient plusieurs **zones de disponibilité** (*Availability Zones, AZ*), comme `eu-west-3a`, `eu-west-3b`, `eu-west-3c` : ce sont des datacenters physiquement séparés (alimentation, réseau, refroidissement indépendants) mais reliés par des liaisons très rapides.

Région eu-west-3 (Paris)



🧠 POURQUOI C'EST IMPORTANT

Si tu mets tous tes serveurs dans une seule AZ et que ce datacenter a un problème (incendie, coupure), ton application tombe. **La haute disponibilité commence par répartir les serveurs sur au moins 2 AZ.** C'est exactement ce que demandent les projets 01 et 02.

Le modèle de responsabilité partagée

Le fournisseur sécurise **le cloud lui-même** (bâtiments, matériel, hyperviseur, réseau mondial). **Toi**, tu sécurises **ce que tu mets dans le cloud** : tes comptes et droits d'accès (IAM), tes données, la configuration de tes pare-feux, les mises à jour de tes systèmes, tes secrets.

🔴 LA CAUSE N°1 DES INCIDENTS CLOUD

Ce ne sont pas des pirates de génie, ce sont des **erreurs de configuration** : un bucket S3 laissé public, une clé d'accès AWS publiée par erreur sur GitHub, un port de base de données ouvert à tout Internet, un compte root sans double authentification. Les robots scannent GitHub en permanence : **une clé AWS publiée est exploitée en quelques minutes** (souvent pour miner de la cryptomonnaie à tes frais).

AWS, Azure, Google Cloud

Le parcours utilise surtout **AWS** (le leader du marché) et un peu **Azure**. Les concepts sont les mêmes partout, seuls les noms changent :

Concept	AWS	Azure	Google Cloud
Machine virtuelle	EC2	Virtual Machines	Compute Engine
Réseau privé	VPC	VNet	VPC
Stockage d'objets	S3	Blob Storage	Cloud Storage
Base SQL managée	RDS / Aurora	Azure SQL / Database for MySQL	Cloud SQL
Kubernetes managé	EKS	AKS	GKE
Registre d'images	ECR	ACR	Artifact Registry
Fonctions serverless	Lambda	Functions	Cloud Functions / Cloud Run
Droits d'accès	IAM	Entra ID + RBAC	IAM
Monitoring	CloudWatch	Azure Monitor / App Insights	Cloud Monitoring
DNS	Route 53	Azure DNS	Cloud DNS
CI/CD intégré	CodePipeline / CodeBuild	Azure DevOps Pipelines	Cloud Build

Préparer ton poste de travail

Tout au long du parcours, tu vas travailler dans un **terminal** et un **éditeur de code**. Installons-les proprement une fois pour toutes.

Étape 1 — Un terminal Linux

La quasi-totalité des serveurs du cloud tournent sous Linux. Autant travailler dans le même environnement.

Sous Windows : installe WSL2 (*Windows Subsystem for Linux*). C'est un vrai Ubuntu qui tourne dans Windows.

```
# Dans PowerShell lancé "en tant qu'administrateur"
wsl --install -d Ubuntu-24.04
# Redémarre l'ordinateur, puis lance "Ubuntu" depuis le menu Démarrer.
# Il te demande de créer un nom d'utilisateur et un mot de passe Linux.
```

POWERSHELL

Installe aussi **Windows Terminal** (Microsoft Store) : plus confortable que l'ancienne console.

Sous macOS : le Terminal est déjà là. Installe le gestionnaire de paquets **Homebrew** (instructions sur brew.sh), qui permet d'installer tous les outils avec `brew install ...`.

Sous Linux : tu as déjà tout. Les commandes de ce livre sont pour Ubuntu/Debian.

✓ VÉRIFICATION

Ouvre ton terminal et tape `uname -a`. Sous WSL ou Linux, tu dois voir `Linux` dans la réponse. Sous macOS, `Darwin`.

Étape 2 — Les outils de base

Dans ton terminal Ubuntu (WSL ou Linux) :

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl wget unzip jq tree vim nano ca-certificates gnupg make python3-pip
```

BASH

- `sudo` : exécuter la commande en tant qu'administrateur (*superuser do*). Il demande ton mot de passe Linux.
- `apt` : le gestionnaire de paquets d'Ubuntu, l'équivalent d'un « App Store » en ligne de commande.
- `jq` : un outil pour lire et filtrer du JSON (les réponses de l'AWS CLI sont en JSON).

Sous macOS : `brew install git curl wget jq tree`.

Étape 3 — Visual Studio Code

Télécharge **VS Code** sur code.visualstudio.com. Extensions recommandées :

Extension	Pourquoi
WSL (Windows uniquement)	Ouvrir les dossiers de WSL directement : tape <code>code .</code> dans le terminal Ubuntu
HashiCorp Terraform	Coloration et autocomplétion des fichiers <code>.tf</code>
YAML (Red Hat)	Valide les fichiers YAML (Kubernetes, pipelines)
Docker / Kubernetes	Explorer conteneurs et clusters
GitLens	Voir l'historique Git ligne par ligne
AWS Toolkit	Parcourir tes ressources AWS

Étape 4 — Configurer Git et GitHub

```
git config --global user.name "Ton Nom"
git config --global user.email "ton.email@exemple.com"
git config --global init.defaultBranch main
git config --global pull.rebase false
```

BASH

Crée ensuite une **clé SSH** pour t'authentifier auprès de GitHub sans taper de mot de passe :

```
ssh-keygen -t ed25519 -C "ton.email@exemple.com"
# Appuie sur Entrée pour accepter l'emplacement par défaut (~/.ssh/id_ed25519)
```

BASH

```
# Choisis une "passphrase" (un mot de passe qui protège la clé)
cat ~/.ssh/id_ed25519.pub
```

Copie la ligne affichée (elle commence par `ssh-ed25519`) et colle-la sur GitHub : **Settings → SSH and GPG keys → New SSH key**.

🔑 COMMENT MARCHE UNE CLÉ SSH ?

Une paire de clés, c'est un **cadenas** et sa **clé**. La clé **publique** (`.pub`) est le cadenas : tu peux la distribuer à qui tu veux (GitHub, des serveurs). La clé **privée** (sans extension) est la seule capable d'ouvrir ce cadenas : **elle ne quitte jamais ta machine et ne se partage jamais**. Quand tu te connectes, le serveur te lance un défi que seule la clé privée peut résoudre. Tu retrouveras exactement ce mécanisme avec les *key pairs* EC2, les agents Jenkins et Ansible.

```
ssh -T git@github.com
# Réponse attendue : "Hi <ton-pseudo>! You've successfully authenticated..."
git clone git@github.com:sanctifie/I0.git
cd I0
```

BASH

Active aussi la **double authentification (2FA)** sur GitHub (Settings → Password and authentication).

Étape 5 — Les outils DevOps (à installer au fur et à mesure)

Tu n'as pas besoin de tout installer aujourd'hui : chaque projet indique ce qu'il lui faut. Voici les commandes de référence pour Ubuntu/WSL :

```
# --- AWS CLI v2 ---
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
unzip -q awscliv2.zip && sudo ./aws/install && rm -rf aws awscliv2.zip
aws --version

# --- Terraform (dépôt officiel HashiCorp) ---
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp.gpg] https://apt.releases.hashicorp.com $(lsb_release -cs) main" \
| sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install -y terraform
terraform -version

# --- kubectl (binaire officiel, dernière version stable) ---
curl -LO "https://dl.k8s.io/release/$(curl -Ls https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -m 0755 kubectl /usr/local/bin/kubectl && rm kubectl
kubectl version --client

# --- eksctl ---
curl -sL "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_Linux_amd64.tar.gz" | tar xz -C /tmp
sudo mv /tmp/eksctl /usr/local/bin/ && eksctl version

# --- Helm ---
curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
helm version
```

BASH

Docker : sous Windows/macOS, installe **Docker Desktop** (il s'intègre à WSL2 : coche *Use the WSL 2 based engine*). Sous Linux, voir l'annexe A.

💡 POURQUOI `CURL ...` | BASH EST À MANIER AVEC PRÉCAUTION

`curl URL | bash` télécharge un script et l'exécute immédiatement, sans que tu le lises. C'est pratique, mais tu fais une confiance totale à l'URL. Ne le fais qu'avec des sources officielles (ici, le dépôt de Helm). En entreprise, on préfère télécharger le script, le lire, puis l'exécuter.

Ouvrir et sécuriser ton compte AWS

C'est l'étape la plus importante de toute la partie 0. Un compte AWS mal sécurisé peut te coûter des milliers d'euros. Prends 45 minutes pour la faire sérieusement.

Étape 1 — Créer le compte

1. Va sur aws.amazon.com → **Create an AWS Account**.
2. Utilise une adresse e-mail dédiée (par exemple prenom+aws@gmail.com) et un mot de passe long et unique (gestionnaire de mots de passe recommandé).
3. Choisis le type de compte **Personal**.
4. Renseigne ta carte bancaire. AWS fait une pré-autorisation de 1 \$ environ.
5. Choisis le plan **Free** (gratuit) si on te le propose : il te donne des crédits et **bloque les services les plus chers** tant que tu ne passes pas au plan payant. Pour les projets avancés (EKS, NAT Gateway...), tu devras passer au plan payant : ce sera le moment de relire la section sur les coûts.

L'OFFRE GRATUITE AWS DEPUIS JUILLET 2025

Les nouveaux comptes reçoivent **100 \$ de crédits** à l'ouverture, plus jusqu'à **100 \$** supplémentaires en réalisant des petites missions d'initiation dans la console (lancer une instance EC2, créer un budget...). Le plan gratuit dure **6 mois** ou jusqu'à épuisement des crédits. Les conditions évoluent : vérifie la page *AWS Free Tier* au moment où tu ouvres ton compte. Dans tous les cas, **les crédits ne remplacent pas les alertes de budget** : configure-les quand même.

Étape 2 — Protéger le compte root

Le compte avec lequel tu viens de t'inscrire est le **root** : il a tous les droits, y compris fermer le compte et changer la carte bancaire. On ne l'utilise **presque jamais**.

1. Connecte-toi en root → en haut à droite, ton nom → **Security credentials**.
2. Section **Multi-factor authentication (MFA)** → **Assign MFA device** → *Authenticator app* (Google Authenticator, Microsoft Authenticator, Authy, 1Password...), ou mieux, une clé physique (YubiKey).
3. Vérifie qu'il n'existe **aucune access key** pour le root. S'il y en a une, supprime-la.

RÈGLE D'OR

Jamais d'access key sur le compte root. Jamais. Et MFA obligatoire. Le root ne sert qu'à quelques actions rares (facturation, fermeture du compte).

Étape 3 — Créer ton utilisateur d'administration

Deux options. La première est la méthode recommandée par AWS aujourd'hui ; la seconde est plus simple et suffit pour apprendre.

Option A (recommandée) : IAM Identity Center.

1. Console → service **IAM Identity Center** → *Enable*.
2. **Users** → **Add user** : ton prénom, ton e-mail.
3. **Permission sets** → **Create** → *Predefined* → [AdministratorAccess](#).
4. **AWS accounts** → coche ton compte → *Assign users* → ton utilisateur + le permission set.
5. Tu reçois un e-mail d'invitation. Tu obtiens une **URL de portail d'accès** (de la forme <https://d-xxxxxxxxx.awsapps.com/start>) : mets-la en favori, c'est désormais par là que tu te connectes. Active la MFA à la première connexion.

Côté terminal :

```
aws configure sso
# SSO session name : io
# SSO start URL : https://d-xxxxxxxxx.awsapps.com/start
# SSO region : eu-west-3 (ou la région où tu as activé Identity Center)
# → une page s'ouvre dans le navigateur pour valider
# Default client Region : eu-west-3
# Default output format : json
# Profile name : io-admin

aws sts get-caller-identity --profile io-admin
export AWS_PROFILE=io-admin # pour ne pas répéter --profile
aws sso login # à refaire quand la session expire
```

Option B (plus simple) : un utilisateur IAM classique.

1. Console → **IAM** → **Users** → **Create user** → nom [admin-<prenom>](#), coche *Provide user access to the AWS Management Console*.
2. *Attach policies directly* → [AdministratorAccess](#).
3. Connecte-toi avec cet utilisateur (URL de connexion du compte affichée sur le tableau de bord IAM) et **active sa MFA**.

4. Pour le terminal : *Security credentials* → **Create access key** → *Command Line Interface*.

```
aws configure
# AWS Access Key ID      : AKIA...
# AWS Secret Access Key  : *****
# Default region name    : eu-west-3
# Default output format  : json
aws sts get-caller-identity
```

BASH

Les clés sont stockées dans `~/.aws/credentials`.

● PROTÉGER TES ACCESS KEYS

- Ne les mets **jamais** dans un fichier versionné par Git, ni dans un Dockerfile, ni dans un script partagé, ni dans une capture d'écran.
- Ajoute ce fichier `.gitignore` à la racine du dépôt IO dès maintenant :

```
# Secrets et état local
*.pem
*.key
.env
*.tfstate
*.tfstate.*
.terraform/
terraform.tfvars
credentials
kubeconfig
```

- Si une clé fuit : **IAM** → **l'utilisateur** → **Security credentials** → **Deactivate puis Delete** immédiatement, puis vérifie la facturation et CloudTrail.
- Préfère les **rôles IAM** dès que c'est possible (une instance EC2, un pipeline) : ils donnent des identifiants temporaires renouvelés automatiquement, sans clé stockée nulle part.

Étape 4 — Alertes de budget (obligatoire)

1. Console → **Billing and Cost Management** → **Budgets** → **Create budget**.
2. *Use a template* → **Zero spend budget** : tu reçois un e-mail dès que tu dépenses 0,01 \$ au-delà de l'offre gratuite.
3. Crée un second budget **Monthly cost budget** de **20 \$** (ou le montant que tu acceptes), avec alertes à 50 %, 80 % et 100 %.
4. **Billing** → **Billing preferences** → active *Receive Free Tier usage alerts* et les alertes de facturation CloudWatch.
5. Active **Cost Explorer** (gratuit) pour voir d'où viennent tes dépenses.

💡 LE RITUEL DE FIN DE SESSION

À la fin de chaque séance de travail, ouvre **Billing** → **Bills** (ou Cost Explorer, regroupé par service) et la page **EC2** → **Instances** de *chaque région que tu as utilisée*. Ça prend 1 minute et ça évite 99 % des mauvaises surprises. AWS affiche aussi une vue globale : **EC2 Global View**.

Étape 5 — Quelques réglages utiles

- **Région par défaut** : en haut à droite de la console, choisis *Europe (Paris)* `eu-west-3` et garde-la. Une ressource créée dans une autre région est **invisible** tant que tu ne changes pas de région dans la console : c'est la cause n°1 du « mais où est passée mon instance ? ».
- **CloudTrail** : activé par défaut pour 90 jours d'historique des actions (*Event history*). Utile pour savoir qui a fait quoi.
- **Tags** : prends l'habitude de taguer chaque ressource avec `Project=io-NN` et `Owner=<toi>`. Ça permet de filtrer les coûts par projet dans Cost Explorer.

Maîtriser les coûts

Le parcours peut coûter **presque rien** ou **plusieurs centaines d'euros** : tout dépend de ta rigueur pour tout détruire à la fin de chaque séance.

Ce qui coûte cher (même quand ça ne fait rien)

Ressource	Coût approximatif	Remarque
NAT Gateway	~0,05 \$/h ≈ 35 \$/mois + données	Facturée à l'heure dès sa création. Projets 01, 02, 11, 22.
Cluster EKS (plan de contrôle)	0,10 \$/h ≈ 73 \$/mois	En plus des nœuds EC2. Projets 06, 08, 12, 15, 16, 19, 27, 28, 30.
Load Balancer (ALB/NLB)	~0,025 \$/h ≈ 18 \$/mois	Chaque Service Kubernetes de type LoadBalancer en crée un !
Transit Gateway	~0,05 \$/h par attachement	Projets 01 et 02. Deux VPC = 2 attachements ≈ 73 \$/mois.
Instance t3.large / t2.large	~0,09 \$/h ≈ 65 \$/mois	Jenkins + SonarQube en ont besoin (projets 09, 24, 25...).
RDS Multi-AZ	double du prix d'une instance simple	Projet 01 : fais du Single-AZ pour apprendre.
Elastic IP non attachée	~0,005 \$/h	Toute IPv4 publique est payante depuis 2024.
Volumes EBS / snapshots	~0,09 \$/Go/mois	Survivent à l'instance si « Delete on termination » est décoché.
AKS / VM Azure	variable	Azure facture aussi à l'heure.

Les bons réflexes

1. **Arrête** (*Stop*) les instances EC2 quand tu fais une pause d'une journée ; **détruis** (*Terminate*) tout à la fin d'un projet. Une instance arrêtée ne coûte plus en calcul, mais ses disques EBS et son IP élastique continuent de coûter.
2. Pour les projets en **Terraform**, [terraform destroy](#) est ton meilleur ami.
3. Pour **EKS**, supprime d'abord les [Service](#) de type [LoadBalancer](#) et les [Ingress](#) (sinon les Load Balancers restent orphelins et bloquent la suppression du VPC), puis [eksctl delete cluster](#).
4. Travaille par **sessions concentrées** : monte l'infra, fais le projet, détruis. Si un projet prend plusieurs jours, écris l'infra en Terraform pour la recréer en 5 minutes le lendemain.
5. Utilise **AWS Nuke** ou la console *Resource Groups & Tag Editor* (recherche par tag [Project](#)) pour retrouver ce qui traîne.

● LE SCÉNARIO CATASTROPHE CLASSIQUE

Tu crées un cluster EKS un vendredi pour le projet 08, ça marche, tu es content, tu fermes l'ordinateur. Tu reviens trois semaines plus tard : cluster (50 \$) + 2 nœuds (40 \$) + NAT Gateway (25 \$) + 2 Load Balancers (27 \$) ≈ **140 \$**. Personne ne te prévient, à part ton alerte de budget. D'où l'étape 4.

Les autres comptes à créer

Service	Pour quoi	Quand
GitHub	Héberger le code, GitHub Actions	Maintenant
Docker Hub	Publier des images Docker	Projet 05
Azure (azure.microsoft.com/free)	200 \$ de crédit sur 30 jours + services gratuits 12 mois	Projet 07 (ouvre-le juste avant, pour profiter des 30 jours)
Azure DevOps (dev.azure.com)	Pipelines, Repos, Boards	Projet 07
SonarCloud (sonarcloud.io)	Analyse de qualité de code en ligne, gratuite pour les dépôts publics	Projet 01
JFrog (offre gratuite / essai)	Dépôt d'artefacts Maven et Docker	Projet 01 / 06
GitLab	GitLab CI	Projet 26
TMDB (themoviedb.org)	Clé d'API pour les clones Netflix	Projet 09
NVD (nvd.nist.gov/developers/request-an-api-key)	Clé d'API pour OWASP Dependency-Check	Projet 09

⚠ UNE ADRESSE E-MAIL, UN MOT DE PASSE PAR SERVICE

Utilise un gestionnaire de mots de passe (Bitwarden, 1Password, KeePassXC) et active la double authentification partout où c'est possible. En DevOps, tes comptes donnent accès à de l'infrastructure qui coûte de l'argent.

PARTIE 1

Les fondamentaux

Linux, réseau et premiers services AWS. Trois leçons, puis trois projets : un TP Linux complet, une architecture réseau VPC haute disponibilité, et une application Java déployée sur une architecture 3-tiers.

Leçon 1 — Linux pour le DevOps

Linux est le système d'exploitation de l'immense majorité des serveurs, des conteneurs Docker et des nœuds Kubernetes. Tu n'as pas besoin d'en être expert pour commencer, mais tu dois être à l'aise avec ce qui suit.

Le terminal et le shell

Le **terminal** est la fenêtre où tu tapes du texte. Le **shell** est le programme qui interprète ce que tu tapes ; sur la plupart des serveurs, c'est **Bash**. L'invite de commande ressemble à ceci :

```
ubuntu@ip-172-31-5-10:~$  
|      |      |  $ = utilisateur normal (# = root)  
|      |      |  ~ = tu es dans ton dossier personnel  
|      |      |  nom de la machine  
|      |      |  ton nom d'utilisateur
```

Une commande a toujours la même forme : `commande -options arguments`. Par exemple `ls -l /etc` : la commande `ls` (lister), l'option `-l` (format long), l'argument `/etc` (le dossier à lister).

💡 LES 5 RÉFLEXES QUI FONT GAGNER DES HEURES

- `Tab` complète automatiquement les noms de commandes et de fichiers. Appuie deux fois pour voir les possibilités.
- `↑` rappelle les commandes précédentes. `Ctrl + R` cherche dans l'historique.
- `Ctrl + C` interrompt la commande en cours. `Ctrl + L` efface l'écran.
- `man ls` affiche le manuel de `ls` (touche `q` pour quitter). `ls --help` donne un résumé.
- `history` liste les dernières commandes tapées.

L'arborescence des fichiers

Sous Linux, tout part de la **racine** `/`. Il n'y a pas de `C:\`.

Dossier	Contenu
<code>/</code>	La racine de tout
<code>/home/<user></code>	Les dossiers personnels des utilisateurs (<code>~</code> est un raccourci vers le tien)
<code>/root</code>	Le dossier personnel de l'utilisateur root
<code>/etc</code>	Les fichiers de configuration (<code>/etc/ssh/sshd_config</code> , <code>/etc/passwd</code> ...)
<code>/var</code>	Les données variables : logs (<code>/var/log</code>), données de services (<code>/var/lib/jenkins</code>)
<code>/opt</code>	Les logiciels installés « à la main » (<code>/opt/maven</code> , <code>/opt/sonarqube</code>)
<code>/usr/bin</code> , <code>/usr/local/bin</code>	Les programmes exécutables
<code>/tmp</code>	Fichiers temporaires, effacés au redémarrage
<code>/dev</code>	Les périphériques (disques : <code>/dev/xvda</code> , <code>/dev/nvme1n1</code>)
<code>/proc</code>	Infos du noyau sur les processus et le système (fichiers virtuels)
<code>/mnt</code> , <code>/data</code>	Points de montage de disques supplémentaires

Chemin absolu : commence par `/` (`/etc/nginx/nginx.conf`), valable de n'importe où. **Chemin relatif** : part du dossier courant (`nginx/nginx.conf`). `.` désigne le dossier courant, `..` le dossier parent.

Naviguer et manipuler les fichiers

```
pwd                # où suis-je ? (print working directory)  
ls -la             # lister, y compris fichiers cachés (commençant par .)  
cd /var/log        # aller dans /var/log  
cd ..              # remonter d'un niveau  
cd ~               # revenir à son dossier personnel  
cd -               # revenir au dossier précédent  
  
mkdir projet       # créer un dossier  
mkdir -p a/b/c     # créer toute l'arborescence d'un coup  
touch notes.txt    # créer un fichier vide (ou mettre à jour sa date)  
cp notes.txt copie.txt # copier  
cp -r dossier/ sauv/  # copier un dossier (récursif)
```

BASH

```
mv copie.txt ancien.txt # déplacer OU renommer
rm ancien.txt           # supprimer un fichier (pas de corbeille !)
rm -r dossier/          # supprimer un dossier et son contenu
rmdir dossier_vide      # supprimer un dossier vide
```

● 'RM -RF' NE PARDONNE PAS

Il n'y a **pas de corbeille** sous Linux. `rm -rf /` (en root) efface tout le système. Relis toujours une commande `rm -r` avant d'appuyer sur Entrée, surtout si elle contient une variable (`rm -rf $DOSSIER/` avec une variable vide devient `rm -rf /`).

Lire et écrire dans les fichiers

```
cat fichier           # afficher tout le fichier
less fichier          # afficher page par page (q pour quitter, / pour chercher)
head -n 20 fichier    # 20 premières lignes
tail -n 20 fichier    # 20 dernières lignes
tail -f /var/log/syslog # suivre un fichier en direct (logs !) - Ctrl+C pour arrêter
wc -l fichier         # compter les lignes

echo "bonjour" > f.txt # écrire dans f.txt (ÉCRASE le contenu)
echo "suite" >> f.txt  # ajouter à la fin de f.txt
```

Les **éditeurs** : `nano fichier` est le plus simple (les raccourcis sont affichés en bas, `Ctrl+O` pour enregistrer, `Ctrl+X` pour quitter). `vim` est partout et très puissant, mais déroutant au début :

Dans vim	Action
i	passer en mode insertion (pour taper du texte)
Échap	revenir en mode normal
:w / :q / :wq / :q!	enregistrer / quitter / les deux / quitter sans enregistrer
yy puis p	copier la ligne, la coller en dessous
10p	coller 10 fois
dd	supprimer la ligne
/mot puis n	chercher « mot », occurrence suivante
:s/ancien/nouveau/g	remplacer partout dans le fichier
u	annuler

Chercher et filtrer

```
grep "error" app.log      # lignes contenant "error"
grep -i "error" app.log   # sans tenir compte des majuscules
grep -r "password" /etc/  # chercher récursivement dans un dossier
grep -v "DEBUG" app.log   # lignes NE contenant PAS DEBUG

find / -name "f3" 2>/dev/null # trouver tous les fichiers nommés f3
find /var/log -name "*.log" -mtime -1 # fichiers .log modifiés il y a moins d'un jour

sed -i 's/DevOps/devops/g' f3 # remplacer dans un fichier sans l'ouvrir
```

Le **pipe** `|` envoie la sortie d'une commande à l'entrée de la suivante. C'est la philosophie Unix : de petits outils qu'on assemble.

```
cat /var/log/auth.log | grep "Failed password" | wc -l # combien de tentatives SSH échouées ?
ps aux | grep java                                     # les processus java
ls / | wc -l                                           # combien d'éléments dans / ?
```

`2>/dev/null` redirige les messages d'erreur (le flux n°2) vers le « trou noir » `/dev/null`. Pratique avec `find`, qui se plaint de chaque dossier qu'il n'a pas le droit de lire.

Utilisateurs et groupes

Linux est **multi-utilisateur**. Chaque fichier appartient à un **utilisateur** et à un **groupe**. Chaque utilisateur a un **groupe principal** et peut appartenir à des **groupes secondaires**.

```
whoami           # qui suis-je ?
id               # mon UID, mon groupe principal, mes groupes
sudo useradd -m -s /bin/bash user1 # créer user1 avec un dossier personnel et bash
```

```
sudo passwd user1      # lui définir un mot de passe
sudo groupadd devops    # créer un groupe
sudo usermod -g devops user2 # changer le groupe PRINCIPAL de user2
sudo usermod -aG aws user1 # AJOUTER user1 au groupe secondaire aws (-a est vital !)
sudo userdel -r user5    # supprimer un utilisateur et son dossier
su - user1              # devenir user1 (demande SON mot de passe)
sudo -i                 # devenir root
exit                    # revenir à l'utilisateur précédent
```

Les comptes sont listés dans `/etc/passwd`, les groupes dans `/etc/group`, les mots de passe (chiffrés) dans `/etc/shadow`.

⚠ L'OUBLI DU '-A'

`usermod -G docker bob` remplace tous les groupes secondaires de bob par `docker`. `usermod -aG docker bob` les **complète**. Oublier le `-a` peut retirer un utilisateur du groupe `sudo` ... et te verrouiller hors de ta propre machine.

sudo : un utilisateur peut exécuter des commandes en tant que root s'il est dans le groupe `sudo` (Ubuntu) ou `wheel` (Amazon Linux/RHEL), ou s'il est autorisé dans `/etc/sudoers`. Modifie ce fichier uniquement avec `sudo visudo`, qui vérifie la syntaxe avant d'enregistrer.

Les permissions

```
$ ls -l script.sh
-rwxr-x--- 1 alice devops 1024 oct 4 10:00 script.sh
|_|_|_|      |_|  |_|
|_|_|_|      |    | groupe propriétaire
|_|_|_|      |    | utilisateur propriétaire
|_|_|_|      |    |
|_|_|_|      |    | droits des AUTRES      : --- (rien)
|_|_|_|      |    | droits du GROUPE       : r-x (lire, exécuter)
|_|_|_|      |    | droits du PROPRIÉTAIRE : rwx (lire, écrire, exécuter)
|_|_|_|      |    | type : - fichier, d dossier, l lien
```

Droit	Sur un fichier	Sur un dossier	Valeur
r	lire le contenu	lister le contenu	4
w	modifier	créer/supprimer des fichiers dedans	2
x	exécuter	entrer dedans (cd)	1

On additionne les valeurs : `rwX` = 7, `r-X` = 5, `r--` = 4. Donc `chmod 750` = `rwXr-X---`.

```
chmod 755 script.sh      # rwxr-xr-x
chmod +x script.sh       # ajouter le droit d'exécution
chmod 400 ma-clé.pem     # lecture seule pour moi : OBLIGATOIRE pour une clé SSH
sudo chown user1 /dir1   # changer le propriétaire
sudo chown user1:devops /f2 # changer propriétaire et groupe
sudo chgrp devops /dir1  # changer seulement le groupe
sudo chown -R jenkins:jenkins /var/lib/jenkins # récursif
```

● `CHMOD 777`

Tu verras souvent `chmod 777` dans des tutoriels (y compris certains projets du dépôt d'origine, par exemple sur `/var/run/docker.sock`). Ça donne **tous les droits à tout le monde** : c'est une faille de sécurité. La bonne solution est presque toujours d'ajouter l'utilisateur au bon groupe (`usermod -aG docker jenkins`) puis de redémarrer le service ou la session.

Tu verras souvent `chmod 777` dans des tutoriels (y compris certains projets du dépôt d'origine, par exemple sur `/var/run/docker.sock`). Ça donne **tous les droits à tout le monde** : c'est une faille de sécurité. La bonne solution est presque toujours d'ajouter l'utilisateur au bon groupe (`usermod -aG docker jenkins`) puis de redémarrer le service ou la session.

Les paquets

Distribution	Gestionnaire	Installer	Mettre à jour
Ubuntu / Debian	<code>apt</code>	<code>sudo apt install nginx</code>	<code>sudo apt update && sudo apt upgrade</code>
Amazon Linux 2023 / RHEL / Fedora	<code>dnf</code>	<code>sudo dnf install nginx</code>	<code>sudo dnf upgrade</code>
Amazon Linux 2 / CentOS 7 (anciens)	<code>yum</code>	<code>sudo yum install nginx</code>	<code>sudo yum update</code>

`apt update` rafraîchit la **liste** des paquets disponibles ; `apt upgrade` installe les **mise à jour**. Pour installer un logiciel absent des dépôts officiels (Jenkins, Docker, Terraform...), on ajoute le dépôt de l'éditeur avec sa clé GPG (qui garantit que les paquets viennent bien de lui), comme tu l'as fait pour Terraform.

Processus et services

```
ps aux                # tous les processus
top (ou htop)         # moniteur en temps réel (q pour quitter)
kill 1234              # demander au processus 1234 de s'arrêter
```

```
kill -9 1234 # le tuer de force (dernier recours)

sudo systemctl status nginx # état du service
sudo systemctl start nginx # démarrer
sudo systemctl stop nginx # arrêter
sudo systemctl restart nginx # redémarrer
sudo systemctl enable nginx # démarrer automatiquement au boot
sudo systemctl enable --now nginx # les deux à la fois
sudo journalctl -u nginx -f # suivre les logs du service
sudo systemctl daemon-reload # après avoir créé/modifié un fichier .service
```

systemd est le gestionnaire de services de Linux. Un service est décrit par un fichier *unit* dans `/etc/systemd/system/`. Tu en écriras plusieurs (Tomcat, Prometheus, SonarQube). Voici un modèle commenté :

```
# /etc/systemd/system/monapp.service
[Unit]
Description=Mon application
After=network.target # démarrer après le réseau

[Service]
User=monapp # ne JAMAIS faire tourner une app en root si on peut l'éviter
ExecStart=/opt/monapp/bin/start.sh
Restart=on-failure # redémarrer automatiquement en cas de crash
Environment=JAVA_HOME=/usr/lib/jvm/java-17

[Install]
WantedBy=multi-user.target # activé au démarrage normal du système
```

Disques et systèmes de fichiers

```
df -h # espace utilisé par système de fichiers (h = lisible)
du -sh /var/log # taille d'un dossier
lsblk # les disques et partitions
free -h # mémoire vive
```

Pour utiliser un nouveau disque, il faut : **1**) le formater (créer un système de fichiers), **2**) le monter dans un dossier, **3**) rendre le montage permanent dans `/etc/fstab`. Tu le feras concrètement dans le projet 03.

Réseau (côté machine)

```
ip a # mes adresses IP
ss -tlnp # quels ports sont en écoute et par quel programme
curl -I http://localhost:8080 # tester un service web (en-têtes seulement)
ping 8.8.8.8 # la machine joint-elle Internet ?
nslookup google.com (ou dig) # résolution DNS
ssh -i cle.pem ubuntu@1.2.3.4 # se connecter à une machine distante
scp -i cle.pem fichier ubuntu@1.2.3.4:/tmp/ # copier un fichier vers elle
```

Variables d'environnement et scripts

```
echo $HOME # afficher une variable
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64 # définir pour la session
echo 'export PATH=$PATH:/opt/maven/bin' >> ~/.bashrc # rendre permanent
source ~/.bashrc # recharger sans se reconnecter
```

`PATH` est la liste des dossiers où le shell cherche les commandes. Si tu installes Maven dans `/opt/maven` et que `mvn` est « command not found », c'est que `/opt/maven/bin` n'est pas dans le `PATH`.

Un **script Bash** est un fichier texte contenant des commandes :

```
#!/bin/bash
# sauvegarde.sh - archive les logs du jour vers S3
set -euo pipefail # s'arrêter à la première erreur (bonne pratique)

DATE=$(date +%F) # ex : 2026-10-04
ARCHIVE="/tmp/logs-$DATE.tar.gz"

tar -czf "$ARCHIVE" /var/log/monapp/
```

```
aws s3 cp "$ARCHIVE" "s3://mon-bucket-logs/$DATE/"
echo "Sauvegarde $DATE terminée"
```

```
chmod +x sauvegarde.sh && ./sauvegarde.sh
```

BASH

cron exécute des tâches planifiées. `crontab -e` ouvre ta table de tâches :

```
#  minute (0-59)  heure (0-23)  jour du mois  mois  jour de semaine (0=dimanche)
#  |              |              |              |              |
#  0              2              *              *              *      /opt/scripts/sauvegarde.sh
# => tous les jours à 2h00
```



AVANT DE PASSER AU PROJET 03

Sur ta machine (WSL), sans regarder les réponses ci-dessus :

1. Crée l'arborescence `~/labo/a/b/c` en une commande.
2. Écris trois lignes dans `~/labo/notes.txt`, puis affiche seulement la dernière.
3. Rends un script exécutable uniquement par toi (`700`).
4. Trouve combien de lignes de `/etc/passwd` contiennent `nologin`.
5. Affiche les ports en écoute sur ta machine.

Leçon 2 — Le réseau, sans douleur

Le réseau est la compétence qui sépare le plus nettement les débutants des ingénieurs confirmés. 80 % des « ça ne marche pas » en cloud sont des problèmes réseau. Voici l'essentiel.

Adresses IP

Une **adresse IPv4** identifie une machine sur un réseau : quatre nombres de 0 à 255, par exemple `192.168.1.10`. Chaque nombre fait 8 bits, donc une adresse fait 32 bits.

Certaines plages sont **privées** (utilisables uniquement à l'intérieur d'un réseau, jamais routées sur Internet) :

Plage privée	Notation CIDR
10.0.0.0 – 10.255.255.255	<code>10.0.0.0/8</code>
172.16.0.0 – 172.31.255.255	<code>172.16.0.0/12</code>
192.168.0.0 – 192.168.255.255	<code>192.168.0.0/16</code>

Toutes les autres adresses sont **publiques**, joignables depuis Internet.

La notation CIDR (le `/16`, `/24` ...)

`172.32.0.0/16` signifie : « les **16 premiers bits** sont fixes, le reste est libre ». Il reste $32 - 16 = 16$ bits libres, soit $2^{16} = 65\,536$ adresses.

CIDR	Bits libres	Nombre d'adresses	Exemple
<code>/16</code>	16	65 536	un VPC entier : <code>172.32.0.0/16</code>
<code>/20</code>	12	4 096	un gros sous-réseau
<code>/24</code>	8	256	un sous-réseau : <code>172.32.1.0/24</code> = de 172.32.1.0 à 172.32.1.255
<code>/28</code>	4	16	un tout petit sous-réseau
<code>/32</code>	0	1	une seule adresse : <code>82.64.10.5/32</code> (« seulement mon IP »)
<code>/0</code>	32	toutes	<code>0.0.0.0/0</code> = tout Internet

💡 LA RÈGLE SIMPLE

Plus le nombre après le `/` est **petit**, plus le réseau est **grand**. Pour les `/16`, `/24`, `/32`, il suffit de compter : `/8` fixe le premier nombre, `/16` les deux premiers, `/24` les trois premiers, `/32` les quatre.

AWS réserve **5 adresses** dans chaque sous-réseau (la première, les trois suivantes et la dernière). Un `/24` donne donc 251 adresses utilisables.

Ports et protocoles

Une IP désigne une machine ; un **port** désigne un service sur cette machine. C'est comme l'adresse d'un immeuble (IP) et le numéro d'appartement (port).

Port	Service
22	SSH
80 / 443	HTTP / HTTPS
3306	MySQL
5432	PostgreSQL
8080	Jenkins, Tomcat (convention)
9000	SonarQube
8081	Nexus
9090 / 3000	Prometheus / Grafana
6443	API Kubernetes
30000–32767	Kubernetes NodePort

TCP garantit que les données arrivent complètes et dans l'ordre (web, SSH, bases de données). **UDP** est plus léger mais sans garantie (DNS, streaming).

DNS

Le DNS traduit un nom (`www.exemple.com`) en adresse IP. Types d'enregistrements utiles :

Type	Rôle	Exemple
A	nom → adresse IPv4	<code>app.exemple.com</code> → <code>13.37.1.2</code>
CNAME	nom → autre nom	<code>www.exemple.com</code> → <code>mon-lb-123.elb.amazonaws.com</code>
Alias (AWS)	comme CNAME, mais utilisable à la racine du domaine et gratuit	<code>exemple.com</code> → ALB
TXT	texte libre (vérifications)	validation de certificat

HTTP en 30 secondes

Le navigateur envoie une **requête** (`GET /login HTTP/1.1`), le serveur répond avec un **code de statut** :

Code	Signification	Ce que ça t'apprend
200	OK	tout va bien
301/302	redirection	
403	interdit	droits / authentification
404	introuvable	mauvaise URL ou appli non déployée au bon endroit
500	erreur serveur	l'application a planté : regarde ses logs
502/503/504	mauvaise passerelle / indisponible / délai dépassé	le proxy ou le load balancer n'arrive pas à joindre l'application derrière lui

Les briques d'un réseau

- **Routeur** : fait passer les paquets d'un réseau à un autre, selon une **table de routage** (« pour aller vers X, passe par Y »).
- **Passerelle par défaut** : la route utilisée quand aucune autre ne correspond (`0.0.0.0/0`).
- **NAT** (*Network Address Translation*) : permet à des machines en IP privée de **sortir** sur Internet en partageant une IP publique, sans être **joignables** depuis Internet. C'est le rôle de ta box Internet chez toi.
- **Pare-feu** : autorise ou bloque le trafic selon des règles (IP source, port, protocole).
- **Load balancer** (répartiteur de charge) : reçoit le trafic et le répartit entre plusieurs serveurs ; retire automatiquement les serveurs en panne.
- **Reverse proxy** : un serveur (Nginx...) qui reçoit les requêtes et les transmet à une application derrière lui.
- **Bastion** (*jump host*) : une machine exposée à Internet uniquement en SSH, qui sert de point d'entrée unique pour administrer les machines privées.

🌿 LA RÉSIDENCE FERMÉE

Un **VPC**, c'est une résidence privée. Les **sous-réseaux publics** sont les bâtiments en bordure de rue, avec une porte sur la rue (l'**Internet Gateway**). Les **sous-réseaux privés** sont les bâtiments au fond du parc : on ne peut pas y entrer depuis la rue, mais leurs habitants peuvent sortir faire des courses en passant par la loge du gardien (la **NAT Gateway**), qui note l'aller pour laisser passer le retour. Le **bastion** est le gardien, seul autorisé à faire entrer les visiteurs. Les **Security Groups** sont les serrures de chaque appartement.

🔧 TESTE-TOI

1. Combien d'adresses dans `10.0.0.0/20` ? (4 096)
2. `172.32.1.0/24` et `172.32.2.0/24` se chevauchent-ils ? (Non)
3. Que signifie une règle de pare-feu « port 22, source `0.0.0.0/0` » ? (SSH ouvert à tout Internet)
4. Un navigateur renvoie 502 Bad Gateway derrière un load balancer. Où chercher en premier ? (L'application derrière : tourne-t-elle, sur le bon port, le Security Group autorise-t-il le load balancer ?)

Leçon 3 — Les services AWS de base

Les trois premiers projets utilisent une douzaine de services AWS. Voici ce que chacun fait, avec les notions qu'il faut avoir en tête.

IAM — qui a le droit de faire quoi

IAM (*Identity and Access Management*) gère les identités et les permissions.

Notion	Ce que c'est
User	une personne (ou un programme) avec des identifiants permanents
Group	un ensemble d'utilisateurs qui partagent des permissions
Role	une identité sans mot de passe qu'on « endosse » temporairement : une instance EC2, une fonction Lambda, un pipeline, un pod Kubernetes
Policy	un document JSON qui liste ce qui est autorisé ou interdit

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject"],
    "Resource": "arn:aws:s3:::mon-bucket-config/*"
  }]
}
```

JSON

Lecture : « autoriser (**Allow**) l'action *lire un objet S3* (`s3:GetObject`) sur tous les objets du bucket `mon-bucket-config` ». Chaque ressource AWS a un identifiant unique, son **ARN** (*Amazon Resource Name*), de la forme `arn:aws:service:région:compte:ressource`.

🧠 LE PRINCIPE DU MOINDRE PRIVILÈGE

On donne **exactement** les droits nécessaires, rien de plus. Le projet 02 te demande explicitement de ne **pas** donner `AmazonS3FullAccess` à tes instances, mais seulement la lecture d'un bucket. Si une instance est piratée, l'attaquant n'aura accès qu'à ce bucket, pas à tout ton compte.

Un **instance profile** est l'enveloppe qui permet d'attacher un rôle IAM à une instance EC2. Les programmes sur l'instance (AWS CLI, SDK) obtiennent alors automatiquement des identifiants temporaires : **aucune clé à stocker**.

EC2 — les machines virtuelles

Notion	Explication
Instance	une machine virtuelle
AMI (<i>Amazon Machine Image</i>)	le modèle de disque à partir duquel on crée l'instance : Ubuntu 24.04, Amazon Linux 2023... ou ta propre image
Type d'instance	la taille : <code>t3.micro</code> (2 vCPU, 1 Go), <code>t3.medium</code> (2 vCPU, 4 Go), <code>t3.large</code> (2 vCPU, 8 Go)...
Key pair	la paire de clés SSH : AWS garde la publique, tu télécharges la privée (<code>.pem</code>) une seule fois
EBS	les disques réseau attachés à l'instance
Security Group	le pare-feu de l'instance
User data	un script exécuté une fois , au premier démarrage de l'instance (installation automatique)
Elastic IP	une IP publique fixe, qui ne change pas quand on arrête/redémarre l'instance

📄 LES NOMS D'UTILISATEUR PAR DÉFAUT SELON L'AMI

`ubuntu` pour Ubuntu, `ec2-user` pour Amazon Linux et RHEL, `admin` pour Debian. Se tromper donne l'erreur `Permission denied (publickey)`.

Lancer une instance (console), le geste de base que tu referas cent fois :

1. **EC2** → **Instances** → **Launch instances**.
2. **Name** : `labo-linux`. Ajoute un tag `Project=io-03`.
3. **AMI** : **Ubuntu Server 24.04 LTS** (ou Amazon Linux 2023).
4. **Instance type** : `t3.micro` (éligible à l'offre gratuite).
5. **Key pair** : **Create new key pair** → nom `io-key`, type ED25519 ou RSA, format `.pem` → le fichier se télécharge.
6. **Network settings** : VPC par défaut, **Auto-assign public IP** : **Enable**. **Create security group** : autorise **SSH depuis My IP** (et pas *Anywhere*).
7. **Storage** : 8 à 20 Go gp3.
8. **Launch instance**.

```
# Sur ton poste (WSL/macOS) : range la clé et protège-la
mkdir -p ~/.ssh && mv ~/Downloads/io-key.pem ~/.ssh/ # sous WSL : /mnt/c/Users/<toi>/Downloads/io-key.pem
chmod 400 ~/.ssh/io-key.pem
ssh -i ~/.ssh/io-key.pem ubuntu@<IP_PUBLIQUE>
```

⚠ « UNPROTECTED PRIVATE KEY FILE! »

SSH refuse une clé lisible par d'autres utilisateurs. C'est pour ça qu'on fait `chmod 400`. Sous WSL, si la clé est restée sur le disque Windows (`/mnt/c/...`), `chmod` ne fonctionne pas : copie-la d'abord dans `~/.ssh/`.

Security Groups et NACL — les pare-feux

Un **Security Group** (SG) est un pare-feu attaché à une ressource (instance, base de données, load balancer).

- On écrit uniquement des règles **Allow** (tout ce qui n'est pas autorisé est bloqué).
- Il est **stateful** : si une requête entrante est autorisée, la réponse sortante l'est automatiquement.
- La **source** d'une règle peut être une plage d'IP (`82.64.10.5/32`) ou un autre **Security Group**. C'est extrêmement puissant : « autoriser le port 3306 depuis le SG des serveurs d'application » reste vrai même quand les serveurs changent d'IP avec l'auto-scaling.

```
Internet —80→ [ SG-alb ] Load balancer
                | 8080 (source = SG-alb)
                ▼
            [ SG-app ] Serveurs d'application
                | 3306 (source = SG-app)
                ▼
            [ SG-db ] Base de données RDS
```

Les **NACL** (*Network ACL*) sont des pare-feux au niveau du **sous-réseau**, **stateless** (il faut autoriser l'aller ET le retour) et acceptent des règles Deny. On les laisse généralement à leur valeur par défaut (tout autorisé) et on travaille avec les Security Groups.

VPC — ton réseau privé dans le cloud

Composant	Rôle
VPC	ton réseau isolé, défini par une plage CIDR (<code>172.32.0.0/16</code>) dans une région
Subnet	une sous-plage du VPC (<code>172.32.1.0/24</code>), située dans une seule AZ
Route table	les règles de routage d'un ou plusieurs sous-réseaux
Internet Gateway (IGW)	la porte entre le VPC et Internet (une par VPC, gratuite)
Subnet public	un sous-réseau dont la table de routage envoie <code>0.0.0.0/0</code> vers l' IGW
Subnet privé	un sous-réseau sans route vers l'IGW
NAT Gateway	placée dans un subnet public, elle permet aux subnets privés de sortir sur Internet (mises à jour, téléchargements) sans être joignables. Payante à l'heure .
Transit Gateway	un routeur central qui relie plusieurs VPC (et des réseaux d'entreprise) entre eux
VPC Peering	une liaison directe entre deux VPC (alternative plus simple et gratuite à l'heure au Transit Gateway)
VPC Flow Logs	l'enregistrement des connexions réseau (acceptées/rejetées), envoyé vers CloudWatch Logs ou S3
VPC Endpoint	accès privé à un service AWS (S3, Secrets Manager...) sans passer par Internet

```
VPC 172.32.0.0/16
├─ Subnet public 172.32.1.0/24 (AZ a) — route 0.0.0.0/0 → IGW   ← NAT GW, bastion, load balancer
├─ Subnet public 172.32.2.0/24 (AZ b) — route 0.0.0.0/0 → IGW
├─ Subnet privé 172.32.3.0/24 (AZ a) — route 0.0.0.0/0 → NAT GW ← serveurs d'application
└─ Subnet privé 172.32.4.0/24 (AZ b) — route 0.0.0.0/0 → NAT GW ← base de données
```

🧠 POURQUOI METTRE LES SERVEURS DANS DES SUBNETS PRIVÉS ?

Un serveur sans IP publique et sans route entrante depuis Internet **ne peut pas être attaqué directement** depuis Internet. Seul le load balancer (public) le contacte, sur un seul port. Les administrateurs passent par le bastion ou par **SSM Session Manager**. C'est la base de toute architecture sérieuse.

Les load balancers (ELB)

Type	Couche	Usage
ALB (<i>Application Load Balancer</i>)	7 (HTTP/HTTPS)	Routage par chemin (<code>/api</code> → service A) ou par nom de domaine, terminaison HTTPS, WAF. Le choix par défaut pour le web.

Type	Couche	Usage
NLB (Network Load Balancer)	4 (TCP/UDP)	Très hautes performances, IP fixes, tout protocole TCP. Utilisé dans les projets 01 et 02.

Un load balancer envoie le trafic vers un **Target Group** (un groupe de cibles : instances, IP ou fonctions Lambda) et effectue des **health checks** : il interroge régulièrement chaque cible (par exemple `GET /` sur le port 80) et cesse d'envoyer du trafic aux cibles en mauvaise santé.

Auto Scaling — s'adapter à la charge

Un **Auto Scaling Group** (ASG) maintient un nombre d'instances entre un **minimum**, un **désiré** et un **maximum**. Si une instance tombe en panne, il la remplace. Si la charge augmente (par exemple CPU > 70 %), il en ajoute.

Pour savoir quoi lancer, il utilise un **Launch Template** : AMI, type d'instance, key pair, Security Group, rôle IAM, user data. *(Les anciennes « Launch Configurations » mentionnées dans le dépôt d'origine sont dépréciées ; on utilise les Launch Templates.)*

🧠 LA CONSÉQUENCE FONDAMENTALE DE L'AUTO-SCALING

Les instances deviennent **jetable**s : elles peuvent être détruites et recrées à tout moment. **Tu ne dois jamais configurer une instance à la main**. Tout doit être dans l'AMI ou dans le user data. C'est le passage de l'informatique « animal de compagnie » (un serveur chouchouté avec un nom) à l'informatique « bétail » (des instances identiques et interchangeables).

AMI personnalisées : Global AMI et Golden AMI

Les projets 01 et 02 parlent de *Global AMI* et de *Golden AMI*. C'est une stratégie en couches :

```
AMI officielle (Amazon Linux 2023)
  | + AWS CLI, agent CloudWatch, agent SSM, durcissement sécurité
  ▼
« Global AMI » — base commune de l'entreprise
  |
  | + Nginx      → « Golden AMI Nginx »
  | + Tomcat+JDK → « Golden AMI Tomcat »
  | + Maven+Git  → « Golden AMI Maven »
```

Créer une AMI : configure une instance, puis **Actions → Image and templates → Create image**. L'instance redémarre, et quelques minutes plus tard l'AMI apparaît dans **EC2 → AMIs**. On peut ensuite lancer autant d'instances identiques qu'on veut. *(En entreprise, on automatise cette fabrication avec **Packer** ou **EC2 Image Builder**.)*

🧠 POURQUOI PRÉINSTALLER DANS L'AMI PLUTÔT QUE TOUT FAIRE EN USER DATA ?

Une instance lancée depuis une Golden AMI est prête en 1 minute ; avec un user data qui installe tout, il faut 5 à 10 minutes, et le script peut échouer si un dépôt Internet est indisponible. Pour l'auto-scaling, la vitesse de démarrage compte. Le user data ne sert alors qu'à la configuration finale (déployer la dernière version de l'appli, renseigner une adresse).

S3 — le stockage d'objets

S3 stocke des fichiers (« objets ») dans des **buckets**. Le nom d'un bucket est **unique au monde** (`io-config-<ton-prenom>-2026`). Usages : fichiers de configuration, artefacts de build, sauvegardes, logs, sites statiques, état Terraform.

```
aws s3 mb s3://io-config-alex-2026      # créer un bucket (make bucket)
aws s3 cp app.conf s3://io-config-alex-2026/ # envoyer un fichier
aws s3 ls s3://io-config-alex-2026/      # lister
aws s3 sync ./logs s3://io-logs-alex-2026/ # synchroniser un dossier
aws s3 rb s3://io-config-alex-2026 --force # supprimer le bucket et son contenu
```

BASH

Par défaut, un bucket est **privé** et l'option *Block Public Access* est activée. Laisse-la.

RDS — les bases de données managées

RDS fournit MySQL, PostgreSQL, MariaDB, etc. sans gérer le serveur : sauvegardes automatiques, mises à jour, réplication. En mode **Multi-AZ**, une copie de secours synchrone tourne dans une autre AZ et prend le relais automatiquement en cas de panne. On la place dans un **DB subnet group** (des subnets privés sur au moins 2 AZ) et on ne l'expose **jamais** sur Internet.

CloudWatch — métriques, logs et alarmes

- **Metrics** : CPU, réseau, disque des instances (par défaut toutes les 5 minutes). **La mémoire et l'espace disque ne sont pas mesurés par défaut** : il faut installer l'**agent CloudWatch** sur l'instance. C'est le « push custom memory metrics » des projets 01 et 02.
- **Logs** : centralisation des logs (applications, VPC Flow Logs, Lambda) dans des *log groups* composés de *log streams*.
- **Alarms** : « si la métrique X dépasse le seuil Y pendant Z minutes, alors notifie (via **SNS**, par e-mail) ou déclenche une action (auto-scaling) ».

SSM Session Manager — SSH sans SSH

Systems Manager Session Manager ouvre un terminal sur une instance **depuis la console AWS ou la CLI, sans port 22 ouvert, sans clé, sans bastion**.

Conditions : l'agent SSM est installé (préinstallé sur Amazon Linux et Ubuntu officiels), l'instance a un rôle IAM avec la politique

[AmazonSSMManagedInstanceCore](#), et elle peut joindre les points d'accès SSM (via NAT ou via des VPC endpoints).

```
aws ssm start-session --target i-0123456789abcdef0
# nécessite le "Session Manager plugin" installé sur ton poste
```

BASH

Route 53 — le DNS d'AWS

Route 53 héberge des **zones DNS** (*hosted zones*) et peut aussi vendre des noms de domaine (environ 15 \$/an pour un [.com](#)). Une zone hébergée coûte 0,50 \$/mois. Pour les projets, un domaine est **optionnel** : tu peux utiliser directement le nom DNS fourni par le load balancer ([xxx.elb.eu-west-3.amazonaws.com](#)).

🔑 QUESTIONS D'ENTRETIEN

- Différence entre Security Group et NACL ? *SG : au niveau de l'instance, stateful, règles Allow uniquement. NACL : au niveau du subnet, stateless, Allow et Deny.*
- Comment une instance dans un subnet privé télécharge-t-elle des mises à jour ? *Via une NAT Gateway placée dans un subnet public (ou via des VPC endpoints pour les services AWS).*
- Pourquoi utiliser un rôle IAM plutôt que des access keys sur une instance ? *Identifiants temporaires renouvelés automatiquement, rien à stocker ni à faire tourner, aucun risque de fuite dans le code.*
- ALB ou NLB ? *ALB pour HTTP/HTTPS avec routage applicatif ; NLB pour du TCP/UDP brut, des performances extrêmes ou des IP fixes.*

Projet 03 — Linux pour les ingénieurs Cloud & DevOps

FICHE PROJET

Niveau	★☆☆☆ Débutant
Durée	3 à 5 heures
Coût	~0 € (instance <code>t3.micro</code> + volume de 5 Go pendant quelques heures)
Compétences	utilisateurs, groupes, permissions, arborescence, chemins relatifs/absolus, <code>sed</code> , <code>vim</code> , <code>find</code> , disques EBS, systèmes de fichiers, montage
Prérequis	Partie 0 terminée, leçon 1 (Linux) lue
Dépôt d'origine	<code>DevOps-Project-03/README.md</code>

Ce qu'on construit

Une instance EC2 Linux sur laquelle tu vas jouer le rôle d'administrateur système : créer des utilisateurs et des groupes, construire une arborescence de dossiers, gérer finement qui possède quoi, manipuler des fichiers sous différentes identités, puis ajouter un disque supplémentaire et le monter.

C'est un projet sans architecture compliquée, mais **toutes** les commandes que tu vas taper reviendront dans les 29 projets suivants.

L'arborescence à construire

Le dépôt d'origine montre l'arborescence sous forme d'image. La voici, reconstituée à partir des consignes :

```
/
├─ dir1/
│   └─ f1
├─ dir2/
│   └─ dir1/
│       └─ dir2/
│           └─ dir10/
├─ dir4/
├─ dir6/
├─ dir7/
│   └─ dir10/
├─ dir8/
├─ f2
└─ opt/
    └─ dir14/
        └─ dir10/
```

⚠ ON TRAVAILLE DIRECTEMENT SOUS `/

Créer des dossiers à la racine n'est **pas** une bonne pratique sur un vrai serveur ; ici, c'est un exercice sur une machine jetable. Tu la détruiras à la fin.

Étape 0 — Lancer l'instance

Suis la procédure « Lancer une instance » de la leçon 3 : Ubuntu 24.04, `t3.micro`, key pair `io-key`, SSH autorisé depuis **My IP**, tag `Project=io-03`. **Note** l'**AZ** de l'instance (onglet *Networking*, par exemple `eu-west-3a`) : tu en auras besoin à l'étape 7.

```
ssh -i ~/.ssh/io-key.pem ubuntu@<IP_PUBLIQUE>
sudo apt update && sudo apt upgrade -y
sudo hostnamectl set-hostname labo-linux      # un nom lisible dans l'invite
exec bash                                     # recharger le shell pour voir le nouveau nom
```

BASH

Étape 1 — En tant que super-utilisateur

Passes root pour toute cette étape :

```
sudo -i
```

BASH

1.1 — Créer user1, user2, user3 avec un mot de passe.

```
for u in user1 user2 user3; do
    useradd -m -s /bin/bash "$u"      # -m : crée /home/$u ; -s : shell bash
    passwd "$u"                      # tape un mot de passe deux fois
done
```

BASH

POURQUOI `-M -S /BIN/BASH` ?

Sans `-m`, l'utilisateur n'a pas de dossier personnel. Sans `-s /bin/bash`, il hérite de `/bin/sh`, un shell minimal sans complétion ni historique. Sur Ubuntu, la commande `adduser` (avec un `d` en plus) fait tout ça de façon interactive ; `useradd` est la commande bas niveau, la même sur toutes les distributions, donc celle qu'on utilise dans les scripts.

1.2 — Créer les groupes devops et aws.

```
groupadd devops
groupadd aws
```

BASH

1.3 — Faire de devops le groupe principal de user2 et user3.

```
usermod -g devops user2
usermod -g devops user3
```

BASH

1.4 — Ajouter aws comme groupe secondaire de user1.

```
usermod -aG aws user1
id user1 user2 user3      # vérification
```

BASH

```
uid=1001(user1) gid=1001(user1) groups=1001(user1),1005(aws)
uid=1002(user2) gid=1004(devops) groups=1004(devops)
uid=1003(user3) gid=1004(devops) groups=1004(devops)
```

`gid` est le groupe principal ; `groups` liste tous les groupes. Le groupe principal est celui qui est attribué aux fichiers que l'utilisateur crée.

1.5 — Créer l'arborescence.

```
mkdir -p /dir1 /dir2/dir1/dir2/dir10 /dir4 /dir6 /dir7/dir10 /dir8 /opt/dir14/dir10
touch /dir1/f1 /f2
tree -L 4 /dir1 /dir2 /dir7 /opt/dir14      # (sudo apt install tree si absent)
```

BASH

`-p` crée les dossiers parents manquants et ne râle pas si le dossier existe déjà.

1.6 et 1.7 — Donner /dir1, /dir7/dir10 et /f2 au groupe devops et à l'utilisateur user1.

```
chown user1:devops /dir1 /dir7/dir10 /f2
ls -ld /dir1 /dir7/dir10 /f2
```

BASH

```
drwxr-xr-x 2 user1 devops 4096 ... /dir1
drwxr-xr-x 2 user1 devops 4096 ... /dir7/dir10
-rw-r--r-- 1 user1 devops   0 ... /f2
```

`ls -ld` affiche les informations **du dossier lui-même** (sans `-d`, `ls` afficherait son contenu).

Étape 2 — En tant que user1

Le problème : créer des utilisateurs et des groupes demande les droits root. Un utilisateur normal ne peut pas le faire. Pour que l'exercice soit faisable, on donne à user1 le droit d'utiliser `sudo` :

```
usermod -aG sudo user1      # (toujours en root) ; sur Amazon Linux le groupe s'appelle wheel
exit                        # quitter root, on redevient ubuntu
sudo su - user1             # devenir user1 (le - charge son environnement)
```

BASH

👁️ OBSERVE D'ABORD L'ÉCHEC

Avant d'ajouter user1 au groupe `sudo`, essaie `useradd user4` en tant que user1. Tu obtiens `useradd: Permission denied`. Lire et comprendre ce message est le vrai but de l'exercice : c'est le système de droits qui fonctionne.

```
whoami      # user1
sudo useradd -m -s /bin/bash user4 && sudo passwd user4
sudo useradd -m -s /bin/bash user5 && sudo passwd user5
sudo groupadd app
```

BASH

```
sudo groupadd database
exit
```

Étape 3 — En tant que user4

user4 doit écrire dans `/dir6`, `/`, `/dir1` et `/dir2/...`, qui appartiennent à root (sauf `/dir1`, à user1). Il n'a aucun droit d'écriture dedans. Deux possibilités :

- **Option pédagogique** : essaie sans droits, constate les `Permission denied`, puis donne `sudo` à user4 (`sudo usermod -aG sudo user4` depuis ubuntu) et préfixe les commandes par `sudo`.
- **Option « propre »** : donner l'accès en écriture via un groupe. Par exemple, rendre `/dir6` accessible en écriture au groupe devops et y ajouter user4. C'est ainsi qu'on fait en entreprise : **jamais de sudo pour tout le monde**.

On prend l'option pédagogique pour avancer :

```
sudo usermod -aG sudo user4    # en tant que ubuntu
sudo su - user4
sudo mkdir /dir6/dir4          # 3.1
sudo touch /f3                 # 3.2
sudo mv /dir1/f1 /dir2/dir1/dir2/ # 3.3 : déplacer f1 dans /dir2/dir1/dir2
sudo mv /f2 /f4                # 3.4 : renommer = déplacer vers un nouveau nom
ls /dir2/dir1/dir2/ /          # vérification
exit
```

BASH

'MV' SERT À LA FOIS À DÉPLACER ET À RENOMMER

Pour Linux, renommer un fichier, c'est le « déplacer » vers un autre nom dans le même dossier. Il n'existe pas de commande `rename` dédiée dans les outils de base.

Étape 4 — En tant que user1

```
sudo su - user1

# 4.1 - Créer /home/user2/dir1 (le dossier d'un AUTRE utilisateur : il faut sudo)
sudo mkdir /home/user2/dir1

# 4.2 - Aller dans /dir2/dir1/dir2/dir10 et créer /opt/dir14/dir10/f1 en chemin RELATIF
cd /dir2/dir1/dir2/dir10
sudo touch ../../../../opt/dir14/dir10/f1
ls -l /opt/dir14/dir10/
```

BASH

COMPTER LES '..'

Depuis `/dir2/dir1/dir2/dir10`, chaque `..` remonte d'un niveau : `dir10` → `dir2` → `dir1` → `dir2` → `/`. Il faut donc **4** `..` pour arriver à la racine, puis on redescend dans `opt/dir14/dir10`. Astuce pour vérifier : `realpath ../../../../opt/dir14/dir10` affiche le chemin absolu correspondant.

```
# 4.3 - Déplacer ce fichier dans le dossier personnel de user1
sudo mv /opt/dir14/dir10/f1 ~/
sudo chown user1: ~/f1    # il appartenait à root (créé avec sudo) : on le récupère

# 4.4 - Supprimer /dir4 récursivement
sudo rm -r /dir4

# 4.5 - Vider /opt/dir14 (supprimer tout ce qu'il contient, mais garder le dossier) en une commande
sudo rm -rf /opt/dir14/*
ls -la /opt/dir14

# 4.6 - Écrire le texte dans /f3
echo "Linux assessment for an DevOps Engineer!! Learn with Fun!!" | sudo tee /f3
cat /f3
exit
```

BASH

POURQUOI 'SUDO ECHO "...'" > /F3' NE MARCHE PAS

Dans `sudo echo texte > /f3`, c'est **ton shell** (non root) qui ouvre `/f3` pour la redirection `>`, avant même que `sudo` s'exécute. Résultat : `Permission denied`. La solution standard est `echo texte | sudo tee /f3` : c'est `tee`, lancé en root, qui écrit le fichier (`tee -a` pour ajouter à la fin). Tu reverras ce piège souvent.

'RM -RF /OPT/DIR14/*' ET LES FICHIERS CACHÉS

L'étoile `*` ne correspond **pas** aux fichiers cachés (ceux qui commencent par un point). Pour tout supprimer, y compris les fichiers cachés :
`sudo find /opt/dir14 -mindepth 1 -delete`.

Étape 5 — En tant que user2

```
sudo usermod -aG sudo user2      # en tant que ubuntu
sudo su - user2

sudo touch /dir1/f2              # 5.1
sudo rm -r /dir6                 # 5.2 (contient dir4 : il faut -r)
sudo rm -r /dir8                 # 5.3

# 5.4 - Remplacer "DevOps" par "devops" dans /f3 sans éditeur
sudo sed -i 's/DevOps/devops/g' /f3
cat /f3
```

DÉCORTIQUER `SED -I 'S/DEVOPS/DEVOPS/G' /F3`

- `sed` (*stream editor*) transforme du texte ligne par ligne.
- `-i` modifie le fichier **sur place** (*in-place*). Sans `-i`, le résultat est seulement affiché.
- `s/ancien/nouveau/` : substitution.
- `g` : toutes les occurrences de la ligne, pas seulement la première.

C'est l'outil roi pour modifier un fichier de configuration dans un script (tu l'utiliseras dans des pipelines pour changer un tag d'image dans un manifest Kubernetes, projet 18).

5.5 — Avec vi, copier la ligne 1 et la coller 10 fois.

```
sudo vim /f3
```

Dans vim : place-toi sur la ligne 1 (`gg`), tape `yy` (copier la ligne), puis `10p` (coller 10 fois), puis `:wq` et Entrée.

5.6 — Remplacer « Engineer » par « engineer » en une seule commande. Deux méthodes :

```
sudo sed -i 's/Engineer/engineer/g' /f3
# ou dans vim : :%s/Engineer/engineer/g
grep -c engineer /f3      # doit afficher 11 (le nombre de lignes qui contiennent "engineer")
```

5.7 — Supprimer /f3.

```
sudo rm /f3
exit
```

Étape 6 — En tant que root

```
sudo -i

# 6.1 - Trouver tous les fichiers nommés f3 et afficher leurs chemins absolus
find / -name "f3" 2>/dev/null
# (Il ne devrait plus rien y avoir, puisqu'on a supprimé /f3. Crée-en un pour tester : touch /tmp/f3)

# 6.2 - Compter le nombre de fichiers dans /
ls -A / | wc -l      # tous les éléments (dossiers compris)
find / -maxdepth 1 -type f | wc -l    # seulement les fichiers ordinaires

# 6.3 - Afficher la dernière ligne de /etc/passwd
tail -n 1 /etc/passwd
```

```
user5:x:1007:1007::/home/user5:/bin/bash
```

Chaque ligne de `/etc/passwd` contient, séparés par `:` : nom, `x` (le mot de passe est dans `/etc/shadow`), UID, GID principal, commentaire, dossier personnel, shell.

Étape 7 — Créer et attacher un volume EBS de 5 Go

Dans la console AWS :

1. **EC2** → **Elastic Block Store** → **Volumes** → **Create volume**.
2. *Volume type* : `gp3` ; *Size* : `5` GiB.
3. *Availability Zone* : **la même que ton instance** (celle notée à l'étape 0).
4. Tag `Name=io-03-data`, `Project=io-03` → **Create volume**.
5. Sélectionne le volume → **Actions** → **Attach volume** → choisis ton instance → *Device name* : `/dev/sdf` → **Attach**.

🔴 POURQUOI LA MÊME AZ ?

Un volume EBS est un disque **réseau** hébergé dans une AZ précise. Il ne peut être attaché qu'à une instance de la **même** AZ. Pour le déplacer, on fait un *snapshot* (copie stockée dans S3, valable pour toute la région) puis on recrée un volume dans l'autre AZ.

En ligne de commande, l'équivalent est :

```
aws ec2 create-volume --size 5 --volume-type gp3 --availability-zone eu-west-3a \
--tag-specifications 'ResourceType=volume,Tags=[{Key=Project,Value=io-03}]'
aws ec2 attach-volume --volume-id vol-xxxxxxx --instance-id i-xxxxxxx --device /dev/sdf
```

BASH

Étape 8 — Créer le système de fichiers et monter le disque

```
sudo -i
lsblk
```

BASH

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINTS
nvme0n1	259:0	0	8G	0	disk	
└─nvme0n1p1	259:1	0	7G	0	part	/
nvme1n1	259:2	0	5G	0	disk	← notre nouveau disque, vide

📁 `/DEV/SDF` EST DEVENU `/DEV/NVME1N1`

Sur les instances récentes (familles t3, m5 et suivantes, dites « Nitro »), les volumes EBS apparaissent comme des disques NVMe. Le nom choisi dans la console (`/dev/sdf`) n'est qu'une étiquette. Repère le bon disque grâce à sa **taille** (5G) et à l'absence de point de montage.

```
# 8.1 - Créer un système de fichiers ext4 sur le disque
file -s /dev/nvme1n1          # "data" = vide, rien à perdre. S'il affiche un système de fichiers, STOP.
mkfs.ext4 /dev/nvme1n1

# 8.2 - Monter le disque sur /data
mkdir -p /data
mount /dev/nvme1n1 /data

# 8.3 - Vérifier
df -h /data
```

BASH

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/nvme1n1	4.9G	24K	4.6G	1%	/data

🔴 `MKFS` EFFACE TOUT

`mkfs` détruit toutes les données du disque ciblé. Vérifie trois fois le nom du périphérique. Formater `/dev/nvme0n1` (le disque système) rendrait l'instance inutilisable.

Bonus indispensable : rendre le montage permanent. Après un redémarrage, `/data` ne sera plus monté. Il faut l'ajouter à `/etc/fstab` :

```
blkid /dev/nvme1n1          # récupère l'UUID du système de fichiers
echo "UUID=<ton-uuid> /data ext4 defaults,nofail 0 2" >> /etc/fstab
umount /data && mount -a    # teste fstab : aucune erreur = c'est bon
df -h /data
```

BASH

On utilise l'**UUID** plutôt que `/dev/nvme1n1` parce que les noms NVMe peuvent changer d'un démarrage à l'autre. L'option `nofail` évite que l'instance refuse de démarrer si le disque est absent.

✅ VÉRIFICATION FINALE

- `id user1` montre le groupe `aws` ; `id user2` montre `devops` comme groupe principal.
- `ls -ld /dir1 /dir7/dir10` montre `user1 devops`.
- `/dir4`, `/dir6`, `/dir8`, `/f3` n'existent plus ; `/f4` et `/dir1/f2` existent.
- `ls ~user1/` contient `f1`.
- `df -h` affiche `/data` ; après `sudo reboot` et reconnexion, `/data` est toujours monté.

Dépannage

🔑 DÉPANNAGE

- **Permission denied (publickey)** au SSH : mauvais utilisateur (`ubuntu` pour Ubuntu, `ec2-user` pour Amazon Linux), mauvaise clé, ou clé pas en `chmod 400` .
- **SSH qui tourne dans le vide puis `Connection timed out`** : le Security Group n'autorise pas ton IP (elle a peut-être changé : ta box en change parfois). Mets à jour la règle avec *My IP*.
- **`user1 is not in the sudoers file`** : `user1` n'est pas dans le groupe `sudo` , ou il ne s'est pas reconnecté depuis l'ajout (les groupes sont lus à la connexion : `exit` puis `sudo su - user1`).
- **Le volume n'apparaît pas dans la liste d'attachement** : il n'est pas dans la même AZ que l'instance.
- **L'instance ne redémarre plus après une modification de `fstab`** : erreur dans la ligne ajoutée. Utilise la *EC2 Serial Console* ou détache le disque racine et corrige-le depuis une autre instance. D'où l'importance de tester avec `mount -a` et d'utiliser `nofail` .

Nettoyage

🔧 NETTOYAGE OBLIGATOIRE

1. **Terminate** l'instance (EC2 → Instances → Instance state → Terminate).
2. **Supprime le volume de 5 Go** : il n'est **pas** supprimé avec l'instance, car il a été créé séparément (EC2 → Volumes → état *available* → Delete).
3. Vérifie qu'aucun volume ni Elastic IP ne traîne.
4. Garde la key pair `io-key` (gratuite) pour les projets suivants.

Pour aller plus loin

- Écris un script Bash `setup.sh` qui reproduit **tout** l'exercice (étapes 1 à 6) automatiquement, et range-le dans `projects/03-linux/` du dépôt IO. C'est exactement ce qu'on appelle l'automatisation.
- Lance une nouvelle instance en passant ce script dans le **user data** : la machine se configure seule au démarrage.
- Remplace l'accès par `sudo` de `user4` par une solution à base de groupe et de permissions (`chgrp devops /dir6 && chmod 775 /dir6`), plus conforme au moindre privilège.
- Découvre les **ACL** (`setfacl` / `getfacl`) pour donner des droits à plusieurs groupes sur un même dossier.

🔑 QUESTIONS D'ENTRETIEN

- Quelle différence entre groupe principal et groupe secondaire ? *Le principal est attribué aux fichiers créés par l'utilisateur ; les secondaires donnent des droits supplémentaires.*
- Que signifie `chmod 640` ? *Propriétaire : lecture/écriture ; groupe : lecture ; autres : rien.*
- Comment ajouter un disque à une instance Linux et le rendre persistant ? *Créer et attacher le volume (même AZ), `mkfs` , `mount` , puis une entrée dans `/etc/fstab` avec l'UUID et `nofail` .*
- Pourquoi `sudo echo x > /fichier` échoue-t-il ? *La redirection est faite par le shell non privilégié ; utiliser `tee` .*

Projet 02 — Architecture VPC scalable sur AWS

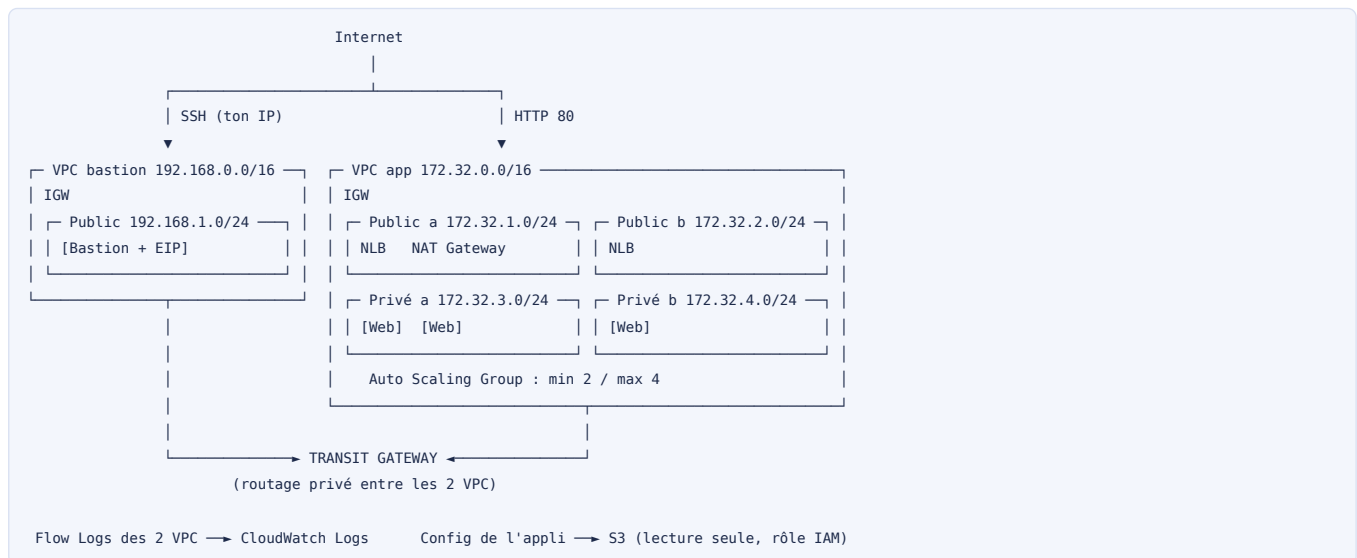
FICHE PROJET

Niveau	★☆☆☆☆ Débutant+
Durée	6 à 10 heures (en 2 ou 3 sessions)
Coût	~0,20 \$/heure tant que tout tourne (NAT Gateway, Transit Gateway, NLB). Compte 1 à 3 \$ si tu détruis à la fin de chaque session.
Compétences	VPC, subnets, tables de routage, IGW, NAT, Transit Gateway, Flow Logs, bastion, Golden AMI, agent CloudWatch, IAM role, S3, Launch Template, Auto Scaling, NLB, Route 53, SSM
Prérequis	Projet 03, leçons 2 et 3
Dépôt d'origine	DevOps-Project-02/Project-02.md + application html-web-app

Ce qu'on construit

Un site web hautement disponible et capable de grandir tout seul :

- un VPC « **bastion** » ([192.168.0.0/16](#)) qui contient uniquement la machine d'administration ;
- un VPC « **application** » ([172.32.0.0/16](#)) qui contient des serveurs web **privés**, répartis sur 2 zones, gérés par un **Auto Scaling Group** (2 à 4 instances) ;
- un **Network Load Balancer** public qui distribue le trafic ;
- un **Transit Gateway** qui relie les deux VPC pour que le bastion puisse administrer les serveurs ;
- les **Flow Logs** des deux VPC dans CloudWatch, pour l'audit réseau ;
- une **Golden AMI** qui contient Apache, Git, l'agent CloudWatch et l'agent SSM ;
- un **bucket S3** qui contient la configuration de l'application.



Les concepts du projet

POURQUOI DEUX VPC SÉPARÉS ?

C'est une pratique courante en entreprise : un **VPC de gestion** (ou « shared services ») contient les outils d'administration (bastion, outils de monitoring, VPN), et des **VPC applicatifs** contiennent les charges de production. On isole ainsi les accès : même si une application est compromise, elle n'a pas de chemin direct vers les outils d'admin, et inversement on contrôle précisément les flux autorisés entre les deux.

POURQUOI UN TRANSIT GATEWAY PLUTÔT QU'UN VPC PEERING ?

Pour deux VPC, un *peering* suffirait et serait moins cher. Mais un peering ne relie que deux VPC, et il n'est pas transitif (si A↔B et B↔C, A ne peut pas joindre C). Quand une entreprise a 10, 50 ou 200 VPC, elle utilise un **Transit Gateway** : un routeur central auquel tous les VPC se raccordent (topologie en étoile). Le projet te fait manipuler le vrai outil.

POURQUOI UN NLB ET PAS UN ALB ?

Le projet demande un NLB. Un ALB conviendrait aussi parfaitement pour un site web (et c'est ce que tu utiliseras dans les projets suivants). Le NLB a la particularité de **conserver l'adresse IP du client** jusqu'à l'instance et de travailler en TCP pur.

Plan d'adressage

Écris-le **avant** de cliquer. C'est le travail de l'architecte réseau.

VPC	Subnet	CIDR	AZ	Type	Contient
bastion 192.168.0.0/16	bastion-public-a	192.168.1.0/24	a	public	bastion
app 172.32.0.0/16	app-public-a	172.32.1.0/24	a	public	NAT GW, nœud NLB
app	app-public-b	172.32.2.0/24	b	public	nœud NLB
app	app-private-a	172.32.3.0/24	a	privé	serveurs web, attachement TGW
app	app-private-b	172.32.4.0/24	b	privé	serveurs web, attachement TGW

💡 RÈGLE D'OR DE L'ADRESSAGE

Les plages des VPC qu'on veut relier **ne doivent pas se chevaucher**. 192.168.0.0/16 et 172.32.0.0/16 sont disjointes : c'est bon. Si les deux VPC utilisaient 10.0.0.0/16, le routage entre eux serait impossible.

❗ 172.32.0.0/16 N'EST PAS UNE PLAGE PRIVÉE !

Les plages privées sont 172.16.0.0 à 172.31.255.255. 172.32.x.x est en réalité une plage **publique** appartenant à quelqu'un d'autre. Dans un VPC isolé, ça fonctionne, mais tes serveurs ne pourraient pas joindre les vraies machines qui utilisent ces adresses sur Internet. Le dépôt d'origine utilise cette plage, on la garde pour coller au projet, mais **en entreprise, utilise une plage privée** (par exemple 10.32.0.0/16).

Étape 1 — Créer la Golden AMI

1.1 — Lancer une instance de construction. Dans le VPC par défaut : Amazon Linux 2023, `t3.micro`, key pair `io-key`, SSH depuis My IP, nom `ami-builder`.

1.2 — Créer un rôle IAM pour l'instance. IAM → Roles → *Create role* → *AWS service* → *EC2* → attache :

- `AmazonSSMManagedInstanceCore` (Session Manager),
- `CloudWatchAgentServerPolicy` (envoi des métriques et logs).

Nom : `io-web-role`. Puis EC2 → ton instance → *Actions* → *Security* → *Modify IAM role* → `io-web-role`.

1.3 — Installer les logiciels.

```
ssh -i ~/.ssh/io-key.pem ec2-user@<IP>
sudo dnf update -y
sudo dnf install -y httpd git amazon-cloudwatch-agent
aws --version                # l'AWS CLI v2 est préinstallée sur Amazon Linux 2023
systemctl status amazon-ssm-agent # l'agent SSM aussi
sudo systemctl enable httpd    # Apache démarrera automatiquement au boot
```

1.4 — Configurer l'agent CloudWatch pour envoyer la mémoire et le disque.

```
sudo tee /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json > /dev/null <<'EOF'
{
  "agent": { "metrics_collection_interval": 60 },
  "metrics": {
    "namespace": "IO/WebServers",
    "append_dimensions": { "InstanceId": "${aws:InstanceId}", "AutoScalingGroupName": "${aws:AutoScalingGroupName}" },
    "metrics_collected": {
      "mem": { "measurement": ["mem_used_percent"] },
      "disk": { "measurement": ["used_percent"], "resources": ["/"] }
    }
  }
}
EOF
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
  -a fetch-config -m ec2 -s -c file:/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json
sudo systemctl enable amazon-cloudwatch-agent
```

🐛 LE `<<'EOF' ... EOF`

C'est un **heredoc** : tout le texte entre les deux `EOF` est envoyé à la commande (ici `tee`, qui l'écrit dans le fichier). Les guillemets autour de `'EOF'` empêchent Bash d'interpréter les `${...}` : ils arrivent tels quels dans le fichier, et c'est l'agent CloudWatch qui les remplacera par l'ID de l'instance.

Après 2 à 3 minutes : CloudWatch → *Metrics* → *All metrics* → namespace `IO/WebServers` → tu dois voir `mem_used_percent`.

1.5 — Créer l'AMI. EC2 → ton instance → *Actions* → *Image and templates* → *Create image* → nom `io-golden-web-v1` → *Create image*. Attends l'état **Available** (EC2 → AMIs), puis **termine** l'instance `ami-builder`.

Étape 2 — Préparer l'application et sa configuration

2.1 — Mettre le code dans ton dépôt IO. Copie le dossier `html-web-app` du dépôt d'origine dans `projects/02-vpc/app/` de ton dépôt, puis pousse-le.

```
cd ~/IO
git clone --depth 1 https://github.com/ZaheerrAhmed/DevOps-Projects /tmp/dp
mkdir -p projects/02-vpc && cp -r /tmp/dp/DevOps-Project-02/html-web-app projects/02-vpc/app
git add projects/02-vpc && git commit -m "Projet 02 : application web" && git push
```

BASH

(Le dépôt d'origine parle de Bitbucket ; GitHub fait exactement la même chose.)

2.2 — Créer le bucket de configuration.

```
BUCKET=io-02-config-<ton-prenom>
aws s3 mb s3://$BUCKET --region eu-west-3
echo 'APP_ENV=production
APP_MESSAGE=Bienvenue sur le site du projet 02' > app.env
aws s3 cp app.env s3://$BUCKET/app.env
```

BASH

2.3 — Donner au rôle le droit de lire ce bucket, et rien d'autre. IAM → Roles → `io-web-role` → *Add permissions* → *Create inline policy* → onglet JSON :

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow", "Action": "s3:GetObject", "Resource": "arn:aws:s3:::io-02-config-<ton-prenom>/*" },
    { "Effect": "Allow", "Action": "s3:ListBucket", "Resource": "arn:aws:s3:::io-02-config-<ton-prenom>" }
  ]
}
```

JSON

Nom : `read-io-02-config`. C'est le « *Do not grant S3 Full Access* » de l'énoncé.

Étape 3 — Construire les deux VPC

Tu peux utiliser l'assistant **VPC → Create VPC → « VPC and more »**, qui crée VPC, subnets, tables de routage, IGW et NAT en une fois. Mais **fais-le une première fois à la main** : c'est en créant chaque brique que tu comprends comment elles s'assemblent.

3.1 — VPC bastion.

1. VPC → *Your VPCs* → *Create VPC* → *VPC only* → nom `io-bastion-vpc`, CIDR `192.168.0.0/16`.
2. Subnets → *Create subnet* → VPC `io-bastion-vpc` → nom `bastion-public-a`, AZ `eu-west-3a`, CIDR `192.168.1.0/24`.
3. Internet gateways → *Create* → `io-bastion-igw` → *Actions* → *Attach to VPC* → `io-bastion-vpc`.
4. Route tables → *Create* → `bastion-public-rt` dans `io-bastion-vpc` → *Routes* → *Edit routes* → *Add route* : `0.0.0.0/0` → Internet Gateway `io-bastion-igw`.
Puis *Subnet associations* → associe `bastion-public-a`.
5. Sur le subnet : *Actions* → *Edit subnet settings* → coche **Enable auto-assign public IPv4 address**.

3.2 — VPC application.

1. VPC `io-app-vpc`, CIDR `172.32.0.0/16`.
2. Quatre subnets selon le plan d'adressage (`app-public-a/b`, `app-private-a/b`).
3. IGW `io-app-igw` attachée à `io-app-vpc`.
4. Table `app-public-rt` : `0.0.0.0/0` → `io-app-igw`, associée aux **deux subnets publics**.
5. **NAT Gateway** : *NAT gateways* → *Create* → nom `io-app-nat`, subnet `app-public-a`, *Connectivity type* : Public → **Allocate Elastic IP** → *Create*.
6. Table `app-private-rt` : `0.0.0.0/0` → NAT Gateway `io-app-nat`, associée aux **deux subnets privés**.

⚠ LA NAT GATEWAY VA DANS LE SUBNET PUBLIC

C'est l'erreur la plus courante du projet. La NAT Gateway a besoin d'une route vers l'IGW pour sortir : elle doit être dans un subnet **public**. Ce sont les tables de routage des subnets **privés** qui pointent vers elle.

i HAUTE DISPONIBILITÉ DE LA NAT

Ici, une seule NAT Gateway (dans l'AZ a) sert les deux subnets privés. Si l'AZ a tombe, les serveurs de l'AZ b perdent leur accès Internet sortant. En production, on met **une NAT Gateway par AZ**, chacune avec sa propre table de routage privée. On ne le fait pas ici pour diviser le coût par deux.

Étape 4 — Relier les VPC avec un Transit Gateway

1. VPC → *Transit gateways* → *Create* → nom `io-tgw`, laisse les options par défaut (*Default route table association* et *propagation* activées) → *Create*. Attends l'état **Available** (quelques minutes).
2. *Transit gateway attachments* → *Create* :
 - `io-tgw-att-bastion` : TGW `io-tgw`, type VPC, VPC `io-bastion-vpc`, subnet `bastion-public-a`.
 - `io-tgw-att-app` : VPC `io-app-vpc`, subnets `app-private-a` et `app-private-b` (un par AZ).
3. **Ajouter les routes dans les VPC** (le TGW sait router entre ses attachements, mais les VPC doivent savoir qu'il faut passer par lui) :

- `bastion-public-rt` : `172.32.0.0/16` → *Transit Gateway* `io-tgw`.
- `app-private-rt` : `192.168.0.0/16` → `io-tgw`.

🧠 LE ROUTAGE, C'EST TOUJOURS DANS LES DEUX SENS

Le bastion envoie un paquet vers `172.32.3.15` : sa table de routage dit « `172.32.0.0/16` → TGW ». Le TGW le livre dans le VPC app. Le serveur répond vers `192.168.1.20` : sa table de routage doit dire « `192.168.0.0/16` → TGW ». Sans cette route retour, la réponse part vers la NAT (route par défaut) et se perd. Quand une connexion entre deux réseaux échoue, vérifie **toujours l'aller ET le retour**.

Étape 5 — Les Flow Logs

1. CloudWatch → *Log groups* → *Create log group* → `/io/vpc-flow-logs`, rétention **3 jours** (pour limiter le coût).
2. Dans ce groupe, crée deux *log streams* : `bastion-vpc` et `app-vpc`. (En pratique, AWS crée automatiquement un stream par interface réseau ; créer ces deux streams sert surtout à respecter l'énoncé.)
3. Rôle IAM pour les Flow Logs : IAM → *Create role* → *Custom trust policy* :

```
{
  "Version": "2012-10-17",
  "Statement": [{ "Effect": "Allow", "Principal": { "Service": "vpc-flow-logs.amazonaws.com" }, "Action": "sts:AssumeRole" }]
}
```

Ajoute une politique inline autorisant `Logs:CreateLogGroup`, `Logs:CreateLogStream`, `Logs:PutLogEvents`, `Logs:DescribeLogGroups`, `Logs:DescribeLogStreams` sur `*`. Nom du rôle : `io-flowlogs-role`.

4. VPC → `io-bastion-vpc` → onglet *Flow logs* → *Create flow log* → *Filter* : **All** → destination *CloudWatch Logs* → groupe `/io/vpc-flow-logs` → rôle `io-flowlogs-role`. Recommence pour `io-app-vpc`.

Une ligne de Flow Log ressemble à ceci :

```
2 123456789012 eni-0a1b2c3d 82.64.10.5 192.168.1.20 51234 22 6 12 3400 1696400000 1696400060 ACCEPT OK
compte interface IP source IP dest port src/dst proto paquets octets début fin décision
```

Le champ `REJECT` est précieux pour diagnostiquer un Security Group trop restrictif.

Étape 6 — Le bastion

1. Security Group `io-bastion-sg` dans `io-bastion-vpc` : entrée **SSH 22 depuis My IP**. (L'énoncé dit « depuis Internet » ; restreindre à ton IP est bien plus sûr.)
2. Lance une instance : Amazon Linux 2023, `t3.micro`, VPC `io-bastion-vpc`, subnet `bastion-public-a`, SG `io-bastion-sg`, key pair `io-key`, nom `io-bastion`.
3. **Elastic IP** : EC2 → *Elastic IPs* → *Allocate* → *Actions* → *Associate* → instance `io-bastion`.

Étape 7 — Le Launch Template

7.1 — Security Group des serveurs web (`io-web-sg`, dans `io-app-vpc`) :

Type	Port	Source	Pourquoi
SSH	22	<code>io-bastion-sg</code> ...	voir l'encadré ci-dessous
HTTP	80	<code>io-nlb-sg</code> (créé à l'étape 9)	seul le load balancer peut joindre les serveurs

⚠️ RÉFÉRENCER UN SECURITY GROUP D'UN AUTRE VPC

Une règle de SG ne peut référencer un SG d'un **autre** VPC qu'à travers un *peering* (et, plus récemment, via certains Transit Gateways selon la configuration). Le plus simple et le plus fiable ici : source `192.168.1.0/24` (le subnet du bastion) pour le port 22.

7.2 — Le user data : le script exécuté au premier démarrage de chaque serveur.

```
#!/bin/bash
set -euxo pipefail
exec > /var/log/user-data.log 2>&1 # tout tracer pour pouvoir déboguer

BUCKET=io-02-config<ton-prenom>
REPO=https://github.com/<ton-pseudo-github>/IO.git

# 1. Récupérer le code et le placer à la racine du serveur web
git clone --depth 1 "$REPO" /tmp/io
cp -r /tmp/io/projects/02-vpc/app/* /var/www/html/

# 2. Récupérer la configuration depuis S3 (grâce au rôle IAM, sans aucune clé)
aws s3 cp "s3://$BUCKET/app.env" /etc/app.env
source /etc/app.env
```

```
# 3. Afficher quelle instance a répondu (utile pour voir le load balancing)
TOKEN=$(curl -sX PUT http://169.254.169.254/latest/api/token -H "X-aws-ec2-metadata-token-ttl-seconds: 300")
IID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/instance-id)
AZ=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/placement/availability-zone)
echo "<p>$APP_MESSAGE - servi par $IID ($AZ)</p>" > /var/www/html/whoami.html

systemctl restart httpd
```

🔗 `169.254.169.254` ?

C'est le **service de métadonnées** d'EC2 (IMDS) : une adresse spéciale, joignable uniquement depuis l'instance, qui lui donne des informations sur elle-même (ID, AZ, rôle IAM...). Le **TOKEN** correspond à **IMDSv2**, la version sécurisée, obligatoire par défaut sur les nouvelles AMI.

7.3 — Créer le Launch Template. EC2 → Launch Templates → Create :

- Nom `io-web-lt` ; coche *Provide guidance... Auto Scaling*.
- AMI : **My AMIs** → `io-golden-web-v1`.
- Type : `t3.micro`. Key pair : `io-key`.
- *Network settings* : **ne choisis pas de subnet** (c'est l'ASG qui décidera), SG `io-web-sg`.
- *Advanced details* : IAM instance profile `io-web-role` ; Metadata version : V2 only ; User data : colle le script.

Étape 8 — Target Group et Auto Scaling Group

1. EC2 → *Target Groups* → *Create* → type **Instances**, nom `io-web-tg`, protocole **TCP 80**, VPC `io-app-vpc`. *Health checks* : protocole HTTP, chemin `/`. Ne sélectionne aucune instance → *Create*.
2. EC2 → *Auto Scaling Groups* → *Create* :
 - Nom `io-web-asg`, Launch Template `io-web-lt`.
 - VPC `io-app-vpc`, subnets `app-private-a` et `app-private-b`.
 - *Attach to an existing load balancer* → *Choose from your load balancer target groups* → `io-web-tg`.
 - Coche *Turn on Elastic Load Balancing health checks* (ainsi une instance dont le site ne répond plus sera remplacée, pas seulement une instance éteinte).
 - Capacité : **désirée 2, min 2, max 4**.
 - *Scaling policy* : *Target tracking*, *Average CPU utilization* = **50 %**.

En 2 à 3 minutes, deux instances apparaissent, une dans chaque AZ.

Étape 9 — Le Network Load Balancer

1. Security Group `io-nlb-sg` dans `io-app-vpc` : entrée **TCP 80** depuis `0.0.0.0/0`.
2. EC2 → *Load Balancers* → *Create* → **Network Load Balancer** :
 - Nom `io-web-nlb`, *Internet-facing*, IPv4.
 - VPC `io-app-vpc`, mappings : `app-public-a` et `app-public-b`.
 - Security group : `io-nlb-sg`. (*Un SG ne peut être ajouté à un NLB qu'à sa création.*)
 - *Listener* TCP 80 → `io-web-tg`.
3. Reviens sur `io-web-sg` et ajoute la règle HTTP 80 depuis `io-nlb-sg`.
4. Target Groups → `io-web-tg` → onglet *Targets* : attends que les 2 instances soient **healthy**.

Étape 10 — Route 53 (optionnel, si tu as un domaine)

Route 53 → *Hosted zones* → ta zone → *Create record* : nom `projet02`, type **CNAME**, valeur = le *DNS name* du NLB. (*Encore mieux : type A avec **Alias** vers le NLB.*) Sans domaine, utilise directement le nom DNS du NLB.

Validation

✓ VÉRIFICATION

1. **Depuis Internet** : ouvre <http://<DNS-du-NLB>/> → la page de l'application s'affiche. Ouvre <http://<DNS-du-NLB>/whoami.html> plusieurs fois (navigation privée, ou `curl` en boucle) : l'ID d'instance change d'une requête à l'autre.

```
for i in $(seq 1 10); do curl -s http://<DNS-du-NLB>/whoami.html; done
```

BASH

2. **Par le bastion, en SSH** : on utilise le **transfert d'agent SSH** pour rebondir sans copier la clé privée sur le bastion.

```
ssh-add ~/.ssh/io-key.pem # charger la clé dans l'agent local
ssh -A ec2-user@<EIP-du-bastion> # -A : transférer l'agent
ssh ec2-user@172.32.3.x # depuis le bastion, vers un serveur privé (IP visible dans la console)
# ou en une commande, depuis ton poste :
ssh -J ec2-user@<EIP-du-bastion> ec2-user@172.32.3.x # -J = ProxyJump
```

BASH

3. **Par Session Manager** : EC2 → une instance web → *Connect* → onglet **Session Manager** → *Connect*. Un terminal s'ouvre sans SSH ni bastion.

4. **Auto-réparation** : *terminate* une des instances web. Dans les minutes qui suivent, l'ASG en relance une nouvelle, qui rejoint le Target Group.

5. **Montée en charge** (bonus) : connecte-toi à une instance et lance `sudo dnf install -y stress-ng && stress-ng --cpu 2 --timeout 600`. Au bout de quelques minutes, l'ASG ajoute des instances (onglet *Activity*).

6. **Flow Logs** : CloudWatch → [/io/vpc-flow-logs](#) → des lignes **ACCEPT** et **REJECT** apparaissent.

● NE COPIE JAMAIS TA CLÉ PRIVÉE SUR LE BASTION

Copier `io-key.pem` sur le bastion (`scp`) est tentant, mais si le bastion est compromis, l'attaquant récupère la clé de **toutes** tes machines. Le transfert d'agent (`-A`) ou `ProxyJump` (`-J`) gardent la clé sur ton poste.

Dépannage

🔧 DÉPANNAGE

- **Targets unhealthy** : connecte-toi à une instance (Session Manager) et regarde `/var/log/user-data.log`. Causes typiques : le `git clone` échoue (dépôt privé, ou pas de route vers la NAT), le `aws s3 cp` échoue (`AccessDenied` : politique IAM ou nom de bucket incorrect), Apache n'est pas démarré, ou le SG `io-web-sg` n'autorise pas le port 80 depuis le NLB.
- **Le user data ne s'est pas exécuté** : il doit commencer exactement par `#!/bin/bash`. Les logs système sont dans `/var/log/cloud-init-output.log`.
- **Le bastion n'arrive pas à joindre les serveurs** : vérifie les deux routes TGW (aller et retour), l'état *Available* des attachements, et la règle SSH de `io-web-sg` (source `192.168.1.0/24`).
- **Session Manager indique « instance not connected »** : rôle sans `AmazonSSMManagedInstanceCore`, ou pas d'accès sortant (NAT absente ou mal routée). Patienter aussi 2 à 3 minutes après le démarrage.
- **Les instances n'ont pas Internet** : la NAT est-elle dans un subnet public ? La table privée pointe-t-elle vers la NAT ? Les subnets privés sont-ils bien associés à cette table ?

Nettoyage

🧹 NETTOYAGE OBLIGATOIRE

Dans **cet ordre** (les dépendances empêchent sinon certaines suppressions) :

1. Auto Scaling Group `io-web-asg` → *Delete* (termine les instances).
2. Load balancer `io-web-nlb`, puis Target Group `io-web-tg`.
3. Instance `io-bastion` → *Terminate* ; puis **libère l'Elastic IP** (*Release*).
4. **NAT Gateway** `io-app-nat` → *Delete*, puis libère son Elastic IP.
5. Transit Gateway : supprime les **deux attachements**, puis le TGW.
6. Flow logs, puis le log group `/io/vpc-flow-logs`.
7. Les deux VPC (*Delete VPC* supprime aussi subnets, tables de routage et IGW).
8. Bucket S3 (`aws s3 rb s3://io-02-config-<prenom> --force`).
9. Garde l'AMI `io-golden-web-v1` si tu veux la réutiliser (le stockage du snapshot coûte quelques centimes par mois), sinon *Deregister* l'AMI et supprime son snapshot (EC2 → Snapshots).

Pour aller plus loin

- Remplace le NLB par un **ALB** et compare.
- Ajoute **HTTPS** avec un certificat gratuit **ACM** (nécessite un domaine).
- Supprime complètement le bastion et le port 22 : administre uniquement par Session Manager (c'est la tendance actuelle).
- Remplace la NAT Gateway par des **VPC endpoints** S3 et SSM : les serveurs n'ont plus du tout besoin d'Internet (mais il faut alors fournir le code autrement, par exemple via S3).
- Refais tout le projet en **Terraform** une fois la partie 3 terminée. Tu verras la différence : 1 heure de clics contre `terraform apply`.

QUESTIONS D'ENTRETIEN

- Comment une instance sans IP publique accède-t-elle à Internet ? Et comment l'administrer ?
- Quelle différence entre Transit Gateway et VPC peering ?
- Que se passe-t-il si une AZ entière tombe dans ton architecture ? *(Le NLB et l'ASG basculent sur l'AZ restante ; mais la NAT unique est un point de défaillance.)*
- Pourquoi utiliser une Golden AMI ? Quels sont ses inconvénients ? *(Démarrage rapide et fiable ; mais il faut reconstruire l'AMI à chaque mise à jour de sécurité : on automatise avec Packer ou EC2 Image Builder.)*

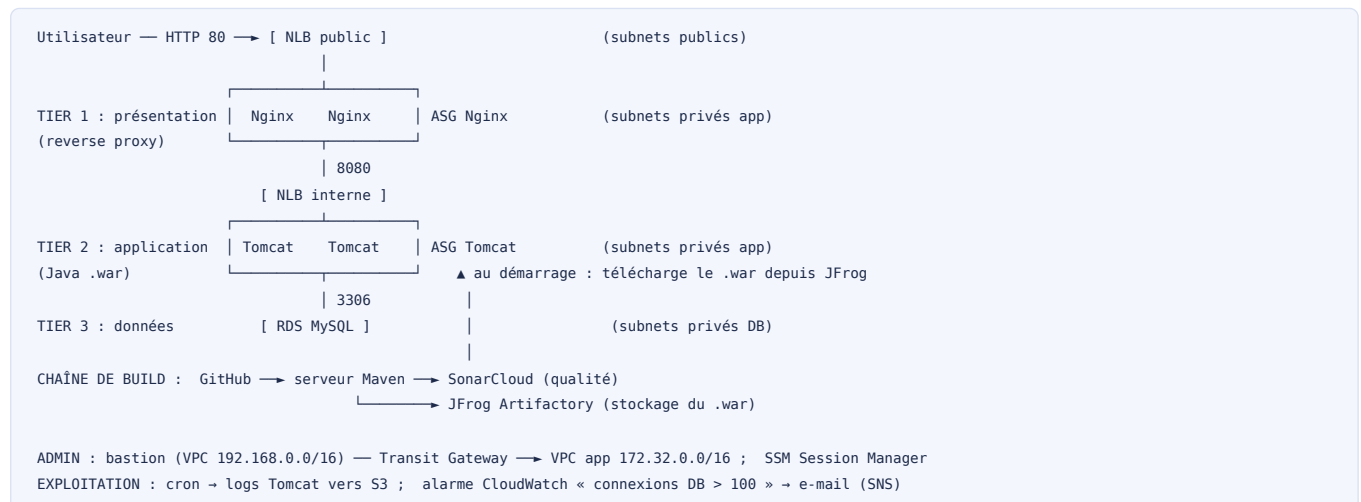
Projet 01 — Application Java sur une architecture 3-tiers AWS

FICHE PROJET

Niveau	★★★☆☆ Intermédiaire
Durée	12 à 20 heures (4 à 6 sessions)
Coût	~0,30 \$/h quand tout tourne (NAT, TGW, 2 NLB, RDS, 4 à 6 petites instances). Détruis entre les sessions.
Compétences	architecture 3-tiers, Golden AMI multiples, Maven, <code>settings.xml</code> , dépôt d'artefacts JFrog, analyse SonarCloud, RDS MySQL, NLB interne et externe, Nginx reverse proxy, Tomcat, systemd, SSM Parameter Store, cron, CloudWatch Alarms, SNS
Prérequis	Projet 02 terminé (on réutilise tout son réseau)
Dépôt d'origine	DevOps-Project-01/Project-01.md + application Java-Login-App

Ce qu'on construit

Une application web Java de connexion/inscription d'employés, déployée selon l'architecture la plus classique du web : **3 tiers** (3 couches), chacune isolée, redondante et capable de grandir.



Les concepts du projet

POURQUOI 3 TIERS ?

Chaque couche a un rôle et des besoins différents :

- **Présentation** (Nginx) : reçoit le trafic Internet, sert les fichiers statiques, termine le HTTPS, protège l'application.
- **Application** (Tomcat + Java) : exécute la logique métier. C'est souvent la couche la plus gourmande, qu'on veut faire grandir indépendamment.
- **Données** (MySQL) : stocke l'information. C'est la couche la plus précieuse et la plus protégée.

Séparer les couches permet de les **faire grandir indépendamment**, de les **sécuriser séparément** (la base n'accepte que les connexions des serveurs Tomcat) et de les **mettre à jour séparément**.

POURQUOI UN DÉPÔT D'ARTEFACTS (JFROG) ?

Le serveur qui **compile** le code (Maven) n'est pas celui qui l'**exécute** (Tomcat). Entre les deux, il faut un endroit fiable où stocker les livrables versionnés (`dptweb-1.0.war`, `dptweb-1.1.war` ...) : c'est le **dépôt d'artefacts** (*artifact repository*). Avantages : on déploie exactement le binaire qui a été testé, on peut revenir à une version précédente, et les nouveaux serveurs de l'ASG téléchargent tout seuls la bonne version au démarrage. Les alternatives sont **Nexus** (projet 30), **AWS CodeArtifact**, ou un simple bucket S3.

POURQUOI UNE ANALYSE SONARCLOUD ?

SonarCloud (version en ligne de SonarQube) lit le code source et détecte les **bugs**, les **failles de sécurité**, le **code dupliqué** et les mauvaises pratiques. Une **Quality Gate** (porte de qualité) définit des seuils : si le nouveau code ne les respecte pas, la porte est « rouge » et on peut bloquer la livraison. Tu vas voir que cette application contient une vraie faille, une **injection SQL**, et que SonarCloud la trouve.

Étape 0 — Préparer le réseau

Reconstruis le réseau du projet 02 (VPC bastion, VPC app, IGW, NAT, Transit Gateway, bastion, routes). Ajoute **deux subnets privés pour la base de données** dans `io-app-vpc` :

Subnet	CIDR	AZ	Table de routage
db-private-a	172.32.5.0/24	a	app-private-rt
db-private-b	172.32.6.0/24	b	app-private-rt

(Une base RDS n'a pas besoin d'accès Internet ; on aurait pu leur créer une table de routage sans route par défaut. C'est encore plus strict.)

Ajoute à ton rôle `io-web-role` (ou crée un rôle `io-app-role`) les politiques `AmazonSSMManagedInstanceCore` et `CloudWatchAgentServerPolicy`, plus les droits de lire tes paramètres SSM (étape 5).

Étape 1 — Les AMI : Global, puis trois Golden

1.1 — La Global AMI : Amazon Linux 2023 avec AWS CLI (préinstallée), agent SSM (préinstallé) et agent CloudWatch configuré pour la mémoire. C'est exactement l'étape 1 du projet 02, **sans Apache**. Nomme l'AMI `io-global-v1`.

1.2 — Golden AMI Nginx. Lance une instance depuis `io-global-v1`, puis :

```
sudo dnf install -y nginx
sudo systemctl enable nginx
```

Crée l'AMI `io-golden-nginx-v1`, termine l'instance.

1.3 — Golden AMI Tomcat. Lance une instance depuis `io-global-v1`, puis :

```
# Java 11 (Amazon Corretto, la distribution OpenJDK d'Amazon)
sudo dnf install -y java-11-amazon-corretto-headless
java -version

# Tomcat 9 - dernière version 9.0.x sur https://dlcdn.apache.org/tomcat/tomcat-9/
TOMCAT_VERSION=9.0.<xx>
cd /tmp
curl -fLO "https://dlcdn.apache.org/tomcat/tomcat-9/v${TOMCAT_VERSION}/bin/apache-tomcat-${TOMCAT_VERSION}.tar.gz"
sudo mkdir -p /opt/tomcat
sudo tar -xzf apache-tomcat-${TOMCAT_VERSION}.tar.gz -C /opt/tomcat --strip-components=1

# Un utilisateur système dédié : Tomcat ne doit pas tourner en root
sudo useradd -r -M -d /opt/tomcat -s /sbin/nologin tomcat
sudo chown -R tomcat:tomcat /opt/tomcat
sudo rm -rf /opt/tomcat/webapps/* # retirer les applications d'exemple (surface d'attaque)
```

⚠ TOMCAT 9, PAS TOMCAT 10 OU 11

Tomcat 10 et suivants utilisent les nouveaux noms de paquets Java `jakarta.*` au lieu de `javax.*`. Cette application (Spring Boot 2.2) utilise `javax.*` : sur Tomcat 10, le `.war` se déploie mais renvoie des erreurs 404/500 incompréhensibles. **La version de la plateforme doit correspondre à celle de l'application** : leçon valable bien au-delà de Tomcat.

Crée le service systemd :

```
sudo tee /etc/systemd/system/tomcat.service > /dev/null <<'EOF'
[Unit]
Description=Apache Tomcat 9
After=network.target

[Service]
Type=forking
User=tomcat
Group=tomcat
Environment=JAVA_HOME=/usr/lib/jvm/java-11-amazon-corretto
Environment=CATALINA_HOME=/opt/tomcat
Environment=CATALINA_PID=/opt/tomcat/temp/tomcat.pid
Environment="CATALINA_OPTS=-Xms256m -Xmx512m"
# Fichier optionnel (le "-" devant) qui contiendra les variables de l'application
EnvironmentFile=-/etc/default/dptweb
ExecStart=/opt/tomcat/bin/startup.sh
ExecStop=/opt/tomcat/bin/shutdown.sh
Restart=on-failure

[Install]
WantedBy=multi-user.target
EOF
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now tomcat
curl -I http://localhost:8080 # réponse HTTP (404 normal : webapps est vide)
```

Ajoute la collecte des logs Tomcat dans la configuration de l'agent CloudWatch si tu le souhaites, puis crée l'AMI `io-golden-tomcat-v1`.

1.4 — Golden AMI Maven (le serveur de build).

```
sudo dnf install -y git java-17-amazon-corretto-devel
MAVEN_VERSION=3.9.<ex> # dernière version sur https://maven.apache.org/download.cgi
cd /tmp
curl -fLO "https://d1cdn.apache.org/maven/maven-3/${MAVEN_VERSION}/binaries/apache-maven-${MAVEN_VERSION}-bin.tar.gz"
sudo tar -xzf apache-maven-${MAVEN_VERSION}-bin.tar.gz -C /opt
sudo ln -s /opt/apache-maven-${MAVEN_VERSION} /opt/maven

# Mettre Maven dans le PATH de tous les utilisateurs
sudo tee /etc/profile.d/maven.sh > /dev/null <<'EOF'
export JAVA_HOME=/usr/lib/jvm/java-17-amazon-corretto
export M2_HOME=/opt/maven
export PATH=$M2_HOME/bin:$PATH
EOF
source /etc/profile.d/maven.sh
mvn -version
```

🐛 JAVA 17 POUR BUILDER, JAVA 11 POUR EXÉCUTER ?

Le scanner SonarCloud exige **Java 17 ou plus** pour s'exécuter. Le code, lui, est compilé pour Java 8 (`<java.version>1.8</java.version>` dans le `pom.xml`), donc le `.war` produit tourne sans problème sur le Java 11 de Tomcat. Un JDK récent sait produire du code pour une version plus ancienne : c'est le rôle de l'option `release` du compilateur.

Crée l'AMI `io-golden-maven-v1`.

💡 LE LIEN SYMBOLIQUE `/OPT/MAVEN`

`/opt/maven` pointe vers `/opt/apache-maven-3.9.x`. Pour mettre Maven à jour, on installe la nouvelle version à côté et on déplace le lien : rien d'autre à modifier. C'est une astuce d'administration très courante.

Étape 2 — Préparer JFrog et SonarCloud

2.1 — JFrog. Crée un compte sur jfrog.com (essai gratuit / plateforme cloud). Dans ton instance (`https://<nom>.jfrog.io`) :

1. *Administration* → *Repositories* → *Create repository* → *Local* → *Maven* : clé `libs-release-local`.
2. *Create* → *Remote* → *Maven* : clé `maven-remote` (URL par défaut : Maven Central). Il servira de cache pour les dépendances.
3. *Create* → *Virtual* → *Maven* : clé `libs-release`, qui regroupe `libs-release-local` et `maven-remote`.
4. Ton profil → *Edit profile* → **Generate an Identity Token** (ou *Access token*). Note ton nom d'utilisateur et ce jeton.

🐛 LOCAL, REMOTE, VIRTUAL

- **Local** : tes propres artefacts (ton `.war`).
- **Remote** : un cache d'un dépôt externe (Maven Central). Si Maven Central est indisponible, ou si une dépendance disparaît d'Internet, ton build continue de fonctionner.
- **Virtual** : une seule URL qui agrège les deux. Maven n'a qu'une adresse à connaître.

2.2 — SonarCloud. Connecte-toi sur sonarcloud.io avec ton compte GitHub → + → *Create new organization* (gratuite pour les projets publics) → *Analyze new project* → choisis le dépôt de l'application → méthode d'analyse « **With other CI tools** » / **Manually**, build **Maven**. Note la **clé d'organisation** et la **clé de projet**. *My Account* → *Security* → génère un **token**.

Étape 3 — Préparer le code de l'application

3.1 — Crée un dépôt GitHub `java-login-app` (public, pour SonarCloud gratuit) et copie-y le contenu de `DevOps-Project-01/Java-Login-App` du dépôt d'origine. Ouvre-le dans VS Code.

3.2 — Modifier le `pom.xml`. Remplace le bloc `<distributionManagement>` et ajoute les propriétés Sonar :

```
<properties>
  <java.version>1.8</java.version>
  <!-- versions de plugins récentes, compatibles avec un JDK 17 -->
  <maven-war-plugin.version>3.4.0</maven-war-plugin.version>
  <!-- SonarCloud -->
  <sonar.organization>TON-ORGANISATION</sonar.organization>
  <sonar.projectKey>TON-ORGANISATION_java-login-app</sonar.projectKey>
  <sonar.host.url>https://sonarcloud.io</sonar.host.url>
</properties>
```

```
<distributionManagement>
  <repository>
    <id>central</id>
    <name>libs-release-local</name>
    <url>https://TON-NOM.jfrog.io/artifactory/libs-release-local</url>
  </repository>
</distributionManagement>
```

⚠ POURQUOI SURCHARGER 'MAVEN-WAR-PLUGIN' ?

Spring Boot 2.2 impose par défaut la version 3.2.3 du plugin WAR, qui plante sous Java 16+ avec une erreur du type `Unable to make field ... TreeMap.comparator accessible`. Surcharger la propriété `maven-war-plugin.version` règle le problème sans toucher au reste. Quand une vieille application refuse de se construire sur un JDK récent, c'est presque toujours un plugin trop ancien.

3.3 — Le `settings.xml`, sans mot de passe en clair. Le fichier du dépôt d'origine contient un e-mail et un mot de passe **en clair**. Ne reproduis jamais ça. Utilise des variables d'environnement :

```
<?xml version="1.0" encoding="UTF-8"?>
<settings xmlns="http://maven.apache.org/SETTINGS/1.2.0">
  <servers>
    <server>
      <id>central</id>
      <username>${env.JFROG_USER}</username>
      <password>${env.JFROG_TOKEN}</password>
    </server>
  </servers>
  <mirrors>
    <mirror>
      <id>central</id>
      <mirrorOf>*</mirrorOf>
      <url>https://TON-NOM.jfrog.io/artifactory/libs-release</url>
    </mirror>
  </mirrors>
</settings>
```

- `<servers>` : les identifiants associés à l' `id` `central` (le même `id` que dans `distributionManagement` et dans le miroir).
- `<mirrors>` : « pour télécharger **toutes** les dépendances (`mirrorOf *`), passe par mon dépôt virtuel JFrog ».

3.4 — La configuration de la base de données n'est **pas** écrite dans `application.properties` (le dépôt d'origine y mettait le mot de passe). Spring Boot lit automatiquement les **variables d'environnement** : `SPRING_DATASOURCE_URL` remplace `spring.datasource.url`, etc. On les fournira à Tomcat au démarrage (étape 6). Laisse donc `application.properties` sans identifiants.

🧠 LES « 12 FACTEURS »

La méthodologie *Twelve-Factor App* (une référence du développement cloud) dit : **la configuration doit être séparée du code et fournie par l'environnement**. Le même `.war` doit pouvoir tourner en test et en production ; seules les variables changent. Et un mot de passe ne doit jamais se retrouver dans Git.

3.5 — Le flux Git demandé par l'énoncé.

```
git checkout -b feature/jfrog-sonar
git add pom.xml settings.xml
git commit -m "Configure JFrog et SonarCloud"
git push -u origin feature/jfrog-sonar
```

Sur GitHub : **Compare & pull request** → relis les changements → **Merge pull request** → supprime la branche. Tu viens d'appliquer le flux de travail standard de toutes les équipes : **branche de fonctionnalité** → **pull request** → **relecture** → **fusion dans `main`**.

Étape 4 — Construire et analyser

Lance une instance depuis `io-golden-maven-v1` (dans un subnet privé de `io-app-vpc`, rôle avec SSM), connecte-toi par Session Manager ou par le bastion :

```
git clone https://github.com/<toi>/java-login-app.git && cd java-login-app

export JFROG_USER='ton-user-jfrog'
read -s JFROG_TOKEN && export JFROG_TOKEN      # saisie masquée : rien dans l'historique
read -s SONAR_TOKEN && export SONAR_TOKEN

# Compiler, tester, analyser, puis publier le .war dans JFrog
mvn -s settings.xml clean verify sonar:sonar -Dsonar.token=$SONAR_TOKEN
mvn -s settings.xml deploy -DskipTests
```

🔴 LE CYCLE DE VIE MAVEN

Maven enchaîne des **phases** dans un ordre fixe : `validate` → `compile` → `test` → `package` → `verify` → `install` → `deploy`. Demander une phase exécute toutes les précédentes. `package` produit le `.war` dans `target/` ; `deploy` l'envoie vers le dépôt déclaré dans `distributionManagement`. `clean` vide d'abord le dossier `target/`. `-s settings.xml` dit à Maven d'utiliser ce fichier de configuration.

✅ VÉRIFICATION

- `target/dptweb-1.0.war` existe.
- Dans JFrog : `Artifactory` → `Artifacts` → `libs-release-local` → `com/devopsrealtime/dptweb/1.0/dptweb-1.0.war`.
- Dans SonarCloud : le tableau de bord du projet affiche bugs, vulnérabilités, *security hotspots* et l'état de la Quality Gate. Ouvre les problèmes de sécurité : SonarCloud signale la **requête SQL construite par concaténation** dans `login.java` et `register.java`.

🔴 L'INJECTION SQL DE CETTE APPLICATION

```
String query = "select * from Employee where username='" + userName + "' and password='" + password + "'";
```

JAVA

Si quelqu'un tape `' OR '1'='1` comme mot de passe, la requête devient `... and password=' OR '1'='1'`, toujours vraie : **connexion sans mot de passe**. De plus, les mots de passe sont stockés en clair. C'est une application d'exercice ; ne la laisse pas en ligne plus longtemps que nécessaire. La correction (exercice bonus) : utiliser un `PreparedStatement` avec des paramètres `?`, et hacher les mots de passe (BCrypt).

Bonus : renomme l'artefact pour que l'application soit servie à la racine du site. Ajoute dans le `pom.xml`, section `<build>` : `<finalName>ROOT</finalName>`. Tomcat sert `ROOT.war` sur `/` au lieu de `/dptweb-1.0/`. (Le nom dans JFrog reste versionné ; c'est le nom local qui change. Dans la suite, on renomme simplement le fichier au téléchargement.)

Étape 5 — La base de données RDS

5.1 — Security Groups (dans `io-app-vpc`) :

SG	Entrée	Source
<code>io-db-sg</code>	MySQL 3306	<code>io-tomcat-sg</code> et <code>192.168.1.0/24</code> (bastion)
<code>io-tomcat-sg</code>	8080	<code>172.32.0.0/16</code> (le NLB interne, dont les IP sont dans le VPC) ; 22 depuis <code>192.168.1.0/24</code>
<code>io-nginx-sg</code>	80	<code>io-public-nlb-sg</code> ; 22 depuis <code>192.168.1.0/24</code>
<code>io-public-nlb-sg</code>	80	<code>0.0.0.0/0</code>

5.2 — DB subnet group : RDS → *Subnet groups* → *Create* → `io-db-subnets`, VPC `io-app-vpc`, subnets `db-private-a` et `db-private-b`.

5.3 — Créer la base : RDS → *Create database* :

- *Standard create*, moteur **MySQL**, version **8.4** (la 8.0 est en fin de support standard).
- Modèle : **Free tier** si proposé, sinon *Dev/Test*.
- Identifiant `io-db`, utilisateur `admin`, mot de passe : **Manage master credentials in AWS Secrets Manager** (*recommandé*) ou un mot de passe fort que tu notes.
- Classe `db.t4g.micro`, stockage 20 Go gp3, **pas d'autoscaling du stockage**.
- **Multi-AZ** : l'énoncé le demande. Coche *Create a standby instance* pour voir le fonctionnement, **mais sache que ça double le coût** ; pour l'apprentissage, Single-AZ suffit.
- Connectivité : VPC `io-app-vpc`, subnet group `io-db-subnets`, *Public access* **No**, SG `io-db-sg`.
- *Additional configuration* : sauvegardes 1 jour, désactive la protection contre la suppression.

Note le **endpoint** (ex. `io-db.xxxxxx.eu-west-3.rds.amazonaws.com`).

5.4 — Stocker la configuration dans SSM Parameter Store :

```
aws ssm put-parameter --name /io/01/db/url --type String \
  --value "jdbc:mysql://io-db.xxxxxx.eu-west-3.rds.amazonaws.com:3306/UserDB?sslMode=REQUIRED"
aws ssm put-parameter --name /io/01/db/user --type String --value admin
aws ssm put-parameter --name /io/01/db/password --type SecureString --value '<mot-de-passe>'
aws ssm put-parameter --name /io/01/jfrog/token --type SecureString --value '<token-jfrog>'
```

BASH

Ajoute au rôle des instances Tomcat une politique autorisant `ssm:GetParameter` et `ssm:GetParameters` sur `arn:aws:ssm:eu-west-3:<compte>:parameter/io/01/*` (et `kms:Decrypt` si tu utilises une clé KMS personnalisée ; la clé par défaut `aws/ssm` est autorisée automatiquement).

🔴 'SSLMODE=REQUIRED'

Sur RDS MySQL 8.4, le paramètre `require_secure_transport` est activé par défaut : la base **refuse** les connexions non chiffrées. `sslMode=REQUIRED` demande au pilote JDBC de chiffrer la connexion en TLS. Bonus : c'est aussi une bonne pratique de sécurité.

5.5 — Créer le schéma depuis le bastion (ou une instance Tomcat) :

```
sudo dnf install -y mariadb105 # fournit le client "mysql"
```

BASH

```
mysql -h io-db.xxxxxx.eu-west-3.rds.amazonaws.com -u admin -p
```

```
CREATE DATABASE UserDB;
USE UserDB;
CREATE TABLE Employee (
  id int unsigned auto_increment not null,
  first_name varchar(250),
  last_name varchar(250),
  email varchar(250),
  username varchar(250),
  password varchar(250),
  regdate timestamp,
  primary key (id)
);
SHOW TABLES;
DESCRIBE Employee;
```

SQL

Étape 6 — Le tier application (Tomcat)

6.1 — NLB interne. Target Group `io-tomcat-tg` : instances, TCP **8080**, VPC `io-app-vpc`, health check HTTP sur `/` (ou sur `/login` si `/` renvoie une redirection). Load balancer **Network**, schéma **Internal**, subnets `app-private-a/b`, listener TCP 8080 → `io-tomcat-tg`. Note son nom DNS (`internal-...io-tomcat-nlb-...elb.eu-west-3.amazonaws.com`).

6.2 — Launch Template `io-tomcat-lt` : AMI `io-golden-tomcat-v1`, `t3.small` (Java aime la mémoire), SG `io-tomcat-sg`, rôle IAM avec SSM et lecture des paramètres, IMDSv2, et ce user data :

```
#!/bin/bash
set -euxo pipefail
exec > /var/log/user-data.log 2>&1
REGION=eu-west-3
JFROG_URL=https://TON-NOM.jfrog.io/artifactory/libs-release-local
JFROG_USER=ton-user-jfrog
APP_VERSION=1.0

p() { aws ssm get-parameter --region $REGION --name "$1" --with-decryption --query Parameter.Value --output text; }

# 1. Variables de l'application, lues dans Parameter Store
cat > /etc/default/dptweb <<EOF
SPRING_DATASOURCE_URL=$(p /io/01/db/url)
SPRING_DATASOURCE_USERNAME=$(p /io/01/db/user)
SPRING_DATASOURCE_PASSWORD=$(p /io/01/db/password)
EOF
chmod 600 /etc/default/dptweb

# 2. Télécharger le .war depuis JFrog et le déployer à la racine
curl -fsSL -u "$JFROG_USER:$(p /io/01/jfrog/token)" \
  -o /opt/tomcat/webapps/ROOT.war \
  "$JFROG_URL/com/devopsrealtime/dptweb/$APP_VERSION/dptweb-$APP_VERSION.war"
chown tomcat:tomcat /opt/tomcat/webapps/ROOT.war

systemctl restart tomcat
```

BASH

6.3 — ASG `io-tomcat-asg` : Launch Template `io-tomcat-lt`, subnets `app-private-a/b`, Target Group `io-tomcat-tg`, health checks ELB activés, capacité 2/2/4.

✓ VÉRIFICATION

Depuis le bastion : `curl -I http://<DNS-NLB-interne>:8080/` répond (200 ou 302). Si ce n'est pas le cas, regarde `/var/log/user-data.log` et `/opt/tomcat/logs/catalina.out` sur une instance.

Étape 7 — Le tier présentation (Nginx)

7.1 — NLB public : Target Group `io-nginx-tg` (TCP 80), NLB *Internet-facing* dans `app-public-a/b`, SG `io-public-nlb-sg`, listener TCP 80 → `io-nginx-tg`.

7.2 — Launch Template `io-nginx-lt` : AMI `io-golden-nginx-v1`, `t3.micro`, SG `io-nginx-sg`, rôle SSM, user data :

```
#!/bin/bash
set -euxo pipefail
BACKEND=<DNS-du-NLB-interne>
cat > /etc/nginx/conf.d/dptweb.conf <<EOF
server {
  listen 80 default_server;
  server_name _;
```

BASH

```
location / {
    proxy_pass http://$BACKEND:8080;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
}
EOF
# Désactiver le "server" par défaut du nginx.conf d'Amazon Linux pour éviter le conflit
sed -i 's/listen      80;/listen 8081;/; s/listen      \[::\]:80;/listen \[::\]:8081;/' /etc/nginx/nginx.conf
nginx -t && systemctl reload nginx
```

💡 **`\$HOST` AVEC UN ANTISLASH ?**

Dans un heredoc **sans** guillemets (<<EOF), Bash remplace les variables : `$BACKEND` devient le nom DNS, c'est voulu. Mais `$host` et `$remote_addr` sont des variables de **Nginx**, qui doivent arriver telles quelles dans le fichier : on les protège avec `\`. Mélanger variables du shell et variables de l'application dans un même fichier est une source d'erreurs classique.

💡 **CE QUE FONT LES `PROXY\_SET\_HEADER`**

Sans eux, Tomcat croit que toutes les requêtes viennent de Nginx. `X-Real-IP` et `X-Forwarded-For` transmettent l'IP du vrai visiteur ; `Host` et `X-Forwarded-Proto` permettent à l'application de générer des liens corrects.

7.3 — ASG `io-nginx-asg` : subnets `app-private-a/b`, Target Group `io-nginx-tg`, 2/2/4.

Étape 8 — Validation fonctionnelle

Ouvre <http://<DNS-NLB-public>/> → page de connexion. **Inscris-toi** (*register*), puis **connecte-toi**. Vérifie en base :

```
SELECT id, username, email, regdate FROM UserDB.Employee;
```

SQL

✅ **VÉRIFICATION**

- En tant qu'administrateur, tu te connectes aux instances **par Session Manager** et **par le bastion**.
- En tant qu'utilisateur, tu accèdes à l'application depuis ton navigateur.
- Termine une instance Tomcat : l'application reste accessible, et l'ASG la remplace.

Étape 9 — Post-déploiement : logs vers S3 et alarme

9.1 — Envoyer les logs Tomcat vers S3 avec cron. Crée un bucket `io-01-logs-<prenom>`, autorise le rôle Tomcat à y écrire (`s3:PutObject` sur `arn:aws:s3:::io-01-logs-<prenom>/*`). Ajoute dans le user data Tomcat (ou dans l'AMI) :

```
cat > /usr/local/bin/ship-tomcat-logs.sh <<'EOF'
#!/bin/bash
# Envoie vers S3 les logs Tomcat des jours précédents, puis les supprime localement
set -eou pipefail
BUCKET=io-01-logs-<prenom>
IID=$(cat /var/lib/cloud/data/instance-id)
cd /opt/tomcat/logs
# fichiers datés (catalina.2026-10-03.log, localhost_access_log.2026-10-03.txt...) de plus d'un jour
find . -maxdepth 1 -type f -name '*.20*' -mtime +0 | while read -r f; do
    aws s3 cp "$f" "s3://$BUCKET/$IID/${f#./}" && rm -f "$f"
done
EOF
chmod +x /usr/local/bin/ship-tomcat-logs.sh
dnf install -y cronie && systemctl enable --now crond
echo "15 0 * * * root /usr/local/bin/ship-tomcat-logs.sh >> /var/log/ship-logs.log 2>&1" > /etc/cron.d/ship-tomcat-logs
```

BASH

💡 **POURQUOI C'EST INDISPENSABLE AVEC L'AUTO-SCALING**

Une instance peut être détruite à tout moment par l'ASG : ses logs disparaissent avec elle. Il faut les **sortir** de la machine. Ce script le fait une fois par jour ; en production, on préfère une collecte en continu (agent CloudWatch Logs, Fluent Bit...), qu'on verra plus tard. Ajoute aussi une **règle de cycle de vie** sur le bucket (S3 → *Management* → *Lifecycle rule*) pour passer les logs en stockage froid après 30 jours et les supprimer après 365 jours.

9.2 — Alarme sur les connexions à la base.

1. SNS → *Topics* → *Create topic* → *Standard*, nom `io-alerts` → *Create subscription* → protocole **Email**, ton adresse → confirme le lien reçu par e-mail.
2. CloudWatch → *Alarms* → *Create alarm* → *Select metric* → **RDS** → **Per-Database Metrics** → `io-db` / `DatabaseConnections`.
3. Statistique *Maximum*, période 1 minute ; condition **Greater than 100**.
4. Action : notification vers `io-alerts`. Nom : `io-db-connections-high`.

🔦 TESTER UNE ALARME SANS GÉNÉRER 100 CONNEXIONS

```
aws cloudwatch set-alarm-state --alarm-name io-db-connections-high \
--state-value ALARM --state-reason "Test manuel"
```

BASH

Tu dois recevoir l'e-mail dans la minute.

Dépannage

🔧 DÉPANNAGE

- **502 Bad Gateway sur le NLB public** : c'est Nginx qui répond, mais il n'arrive pas à joindre Tomcat. Vérifie le nom du NLB interne dans `/etc/nginx/conf.d/dptweb.conf`, l'état des cibles de `io-tomcat-tg`, et que `io-tomcat-sg` autorise 8080 depuis le VPC.
- **404 Tomcat** : le `.war` n'a pas été téléchargé (regarde `/var/log/user-data.log` : un 401 signifie un jeton JFrog incorrect, un 404 un mauvais chemin), ou tu es sur Tomcat 10.
- **Page d'erreur après connexion / Communications link failure** : Tomcat n'arrive pas à joindre RDS. Vérifie `io-db-sg` (source `io-tomcat-sg`), l'URL JDBC, et `cat /etc/default/dptweb`. Un `Access denied` vient de l'utilisateur ou du mot de passe ; `Unknown database` signifie que `CREATE DATABASE` n'a pas été fait.
- **Maven : 401 Unauthorized au deploy** : l'`id` du `<server>` ne correspond pas à l'`id` du `<repository>`, ou les variables `JFROG_USER` / `JFROG_TOKEN` ne sont pas exportées dans ce terminal.
- **Maven : Could not resolve dependencies** : le miroir JFrog est mal configuré (URL du dépôt *virtual*) ou le dépôt remote n'a pas accès à Maven Central.
- **SonarCloud : Project not found** : vérifie `sonar.organization` et `sonar.projectKey` (sensibles à la casse) et le token.

Nettoyage

🔧 NETTOYAGE OBLIGATOIRE

1. ASG Nginx et ASG Tomcat (supprime les instances), puis les deux NLB et leurs Target Groups.
2. Instance Maven, bastion, Elastic IPs.
3. **RDS** : `Delete` → décoche *Create final snapshot* (sauf si tu veux le garder) → confirme. Supprime ensuite le secret Secrets Manager s'il a été créé.
4. NAT Gateway, Transit Gateway (attachements d'abord), VPC.
5. Buckets S3, paramètres SSM (`aws ssm delete-parameters --names ...`), topic SNS, alarme.
6. Désenregistre les AMI que tu ne garderas pas et supprime leurs snapshots.
7. Garde le compte SonarCloud et JFrog pour les projets suivants (désactive JFrog s'il s'agit d'un essai payant à la fin).

Pour aller plus loin

- Corrige l'injection SQL avec des `PreparedStatement`, pousse sur une branche, et observe la Quality Gate passer au vert.
- Ajoute un **ALB** avec HTTPS (ACM) à la place du NLB public.
- Remplace le serveur Maven « manuel » par un **pipeline** (c'est exactement ce que font les projets 05 et 06).
- Remplace le téléchargement du `.war` au démarrage par une **AMI contenant déjà l'application** (fabriquée avec Packer à chaque version) : démarrage encore plus rapide et immuable.

🔧 QUESTIONS D'ENTRETIEN

- Décris une architecture 3-tiers sur AWS et la façon dont tu la sécurises.
- Comment fournis-tu un mot de passe de base de données à une application sans l'écrire dans le code ? (*Variables d'environnement alimentées par SSM Parameter Store ou Secrets Manager, rôle IAM pour y accéder.*)
- À quoi sert un dépôt d'artefacts ? Quelle différence entre dépôt local, remote et virtual ?
- Que se passe-t-il pour les logs quand l'ASG détruit une instance ? Comment l'éviter ?
- Qu'est-ce qu'une injection SQL et comment l'empêcher ?

PARTIE 2

Conteneurs & premiers pipelines CI/CD

Git en profondeur, Docker, les principes de la CI/CD, Jenkins et Ansible. Puis six projets : Django sur ECS, Jenkins + Docker, le grand pipeline « tous outils », Azure DevOps, .NET sur Azure et GitHub Actions.

Leçon 4 — Git pour de vrai

Tu as déjà fait `clone`, `add`, `commit`, `push`. En DevOps, Git est le **point de départ de toute automatisation** : un `push` déclenche un pipeline, une *pull request* déclenche des tests, un commit sur un dépôt de configuration déclenche un déploiement (GitOps). Il faut le maîtriser.

Le modèle mental

Répertoire de travail (tes fichiers) —git add→ Zone de staging (index) (ce qui partira au commit) —git commit→ Dépôt local (.git/) —git push→ Dépôt distant (GitHub) ←git pull—

Un **commit** est une photo complète du projet à un instant donné, identifiée par un hash (`a3f9c1e...`), avec un auteur, une date, un message et un lien vers le commit parent. Une **branche** est simplement une étiquette mobile qui pointe vers un commit.

Les commandes du quotidien

```
git status                # que s'est-il passé ? (à taper tout le temps)
git diff                  # ce qui a changé et n'est pas encore "add"
git diff --staged         # ce qui partira au prochain commit
git add fichier / git add -p # ajouter un fichier / choisir morceau par morceau
git commit -m "feat: ajoute le health check"
git log --oneline --graph --all # l'historique en arbre
git switch -c feature/login    # créer une branche et s'y placer
git switch main               # revenir sur main
git pull                     # récupérer et fusionner les changements distants
git push -u origin feature/login # publier la branche (et la suivre)
git merge feature/login      # fusionner une branche dans la branche courante
git restore fichier         # annuler les modifications non committées d'un fichier
git revert <hash>           # créer un commit qui annule un commit précédent (sûr)
git stash / git stash pop    # mettre de côté des modifications en cours, puis les reprendre
git tag v1.0.0 && git push --tags # marquer une version
```

BASH

Les flux de travail (branching strategies)

GitHub Flow (le plus répandu, et celui qu'on utilise) :

1. `main` est toujours déployable.
2. Pour chaque changement, une branche courte : `feature/...`, `fix/...`.
3. On pousse la branche et on ouvre une **Pull Request** (PR, ou *Merge Request* sur GitLab).
4. La CI s'exécute automatiquement sur la PR ; un collègue relit (*code review*).
5. On fusionne dans `main`, ce qui déclenche le déploiement.

Git Flow (plus ancien, plus lourd) ajoute des branches `develop`, `release/*`, `hotfix/*`. Le projet 14 utilise des branches `develop`, `qa` et `main` qui correspondent à des environnements : c'est une variante.

Trunk-based development : tout le monde commite sur `main` (ou des branches de quelques heures), avec des *feature flags* pour masquer le travail en cours. C'est ce que font les équipes les plus rapides.

Les bonnes pratiques

- **Petits commits, messages clairs.** Convention répandue (*Conventional Commits*) : `feat:`, `fix:`, `docs:`, `ci:`, `refactor:`, `chore:`.
- Un `.gitignore` dès le premier commit : jamais de `node_modules/`, `target/`, `.terraform/`, `*.tfstate`, `.env`, clés.
- **Protège `main`** : sur GitHub, *Settings* → *Branches* → *Add rule* : PR obligatoire, CI verte obligatoire.
- **Jamais de secret dans Git.** Si ça arrive : considère le secret comme compromis, **révoque-le immédiatement**. Le supprimer dans un nouveau commit ne suffit pas : il reste dans l'historique.

💡 LES JETONS D'ACCÈS (PERSONAL ACCESS TOKENS)

Pour qu'un outil (Jenkins, Argo CD, un script) accède à GitHub, on n'utilise pas ton mot de passe mais un **jeton** : GitHub → *Settings* → *Developer settings* → *Personal access tokens* → *Fine-grained tokens*. Donne-lui **uniquement** les dépôts et droits nécessaires (par exemple *Contents: Read and write* sur un seul dépôt) et une date d'expiration.

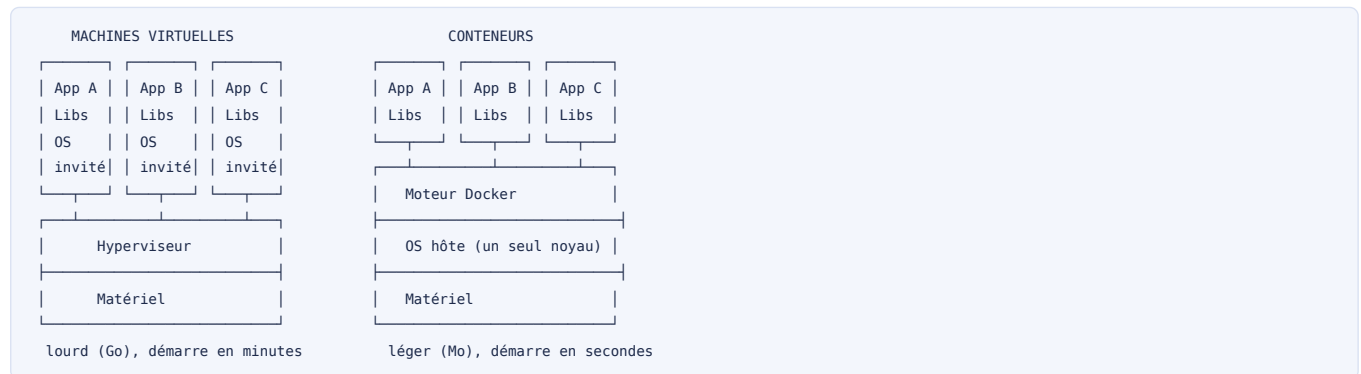
Leçon 5 — Docker et les conteneurs

Le problème

« Ça marche sur ma machine ! » Une application dépend de tout un environnement : une version précise de Python ou de Java, des bibliothèques système, des variables, des fichiers de configuration. Reproduire cet environnement sur chaque serveur est long et fragile.

La solution : le conteneur

Un **conteneur** empaquette l'application **et tout ce dont elle a besoin** dans une unité standard qui tourne de la même façon partout où Docker est installé.



Un conteneur n'embarque **pas** de système d'exploitation complet : il partage le noyau Linux de la machine hôte et s'isole grâce à des fonctions du noyau (*namespaces* pour l'isolement, *cgroups* pour limiter CPU et mémoire). D'où sa légèreté.

🌱 LE CONTENEUR MARITIME

Avant les conteneurs maritimes standardisés, charger un navire prenait des jours : chaque marchandise avait sa forme. Avec la boîte standard, peu importe ce qu'il y a dedans, toutes les grues, tous les bateaux et tous les camions savent la manipuler. Docker fait la même chose pour les logiciels.

Image, conteneur, registre

Notion	Définition	Analogie
Image	un modèle en lecture seule, construit en couches (OS minimal + runtime + appli)	la recette, ou la classe en programmation
Conteneur	une instance en cours d'exécution d'une image	le plat cuisiné, ou l'objet
Dockerfile	le fichier texte qui décrit comment construire l'image	la fiche recette
Registre	un serveur qui stocke et distribue les images	la bibliothèque de recettes
Tag	la version d'une image : <code>nginx:1.27</code> , <code>monapp:v3</code> , <code>monapp:latest</code>	l'édition du livre

Registres courants : **Docker Hub** (public), **Amazon ECR**, **Azure ACR**, **GitHub Container Registry**, **JFrog**, **Nexus**. Le nom complet d'une image indique où elle est stockée : `123456789012.dkr.ecr.eu-west-3.amazonaws.com/monapp:v3`.

Le Dockerfile, ligne par ligne

DOCKERFILE

```
# 1. L'image de base : un Python officiel, version "slim" (allégée)
FROM python:3.12-slim

# 2. Le dossier de travail dans l'image (créé s'il n'existe pas)
WORKDIR /app

# 3. Copier D'ABORD la liste des dépendances, puis les installer
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 4. Copier ENSUITE le code
COPY . .

# 5. Documenter le port écouté par l'application
EXPOSE 8000
```

```
# 6. Ne pas tourner en root
RUN useradd -m appuser
USER appuser

# 7. La commande lancée au démarrage du conteneur
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

🧠 POURQUOI COPIER `REQUIREMENTS.TXT` AVANT LE CODE ?

Chaque instruction crée une **couche**, mise en **cache**. Si une couche n'a pas changé, Docker réutilise le cache et passe à la suivante. Le code change à chaque commit, les dépendances rarement. En copiant `requirements.txt` d'abord, l'étape `pip install` (lente) reste en cache tant que les dépendances ne bougent pas : le build passe de 2 minutes à 5 secondes.

Les instructions à connaître : `FROM`, `WORKDIR`, `COPY` (préfère-le à `ADD`), `RUN`, `ENV` (variable d'environnement), `ARG` (variable disponible seulement pendant le build), `EXPOSE`, `USER`, `CMD` (commande par défaut, remplaçable au lancement), `ENTRYPOINT` (commande fixe), `HEALTHCHECK`.

Le build multi-étapes (*multi-stage*), utilisé dans plusieurs projets (Netflix, Swiggy) : une première étape avec tous les outils de compilation, une seconde ne contenant que le résultat. L'image finale est petite et ne contient pas les outils de build (moins de failles).

```
# Étape 1 : construire l'application React
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Étape 2 : la servir avec Nginx (l'image finale ne contient ni Node ni les sources)
FROM nginx:stable-alpine
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
```

DOCKERFILE

Ajoute toujours un fichier `.dockerignore` (même syntaxe que `.gitignore`) pour ne pas envoyer `node_modules`, `.git` ou des secrets dans le contexte de build.

Les commandes Docker essentielles

```
docker build -t monapp:v1 . # construire l'image à partir du Dockerfile du dossier courant
docker images # lister les images locales
docker run -d --name web -p 8080:8000 monapp:v1
# | | | port HÔTE:port CONTENEUR
# | | nom du conteneur
# | détaché (en arrière-plan)
docker ps / docker ps -a # conteneurs en cours / tous
docker logs -f web # suivre les logs
docker exec -it web bash # ouvrir un shell DANS le conteneur (ou sh)
docker stop web && docker rm web # arrêter puis supprimer
docker rmi monapp:v1 # supprimer l'image
docker tag monapp:v1 moncompte/monapp:v1 # donner un autre nom
docker login && docker push moncompte/monapp:v1
docker pull nginx:stable # télécharger une image
docker system prune -a # grand ménage (images, conteneurs arrêtés, cache)
```

BASH

⚠️ LES DONNÉES D'UN CONTENEUR SONT ÉPHÉMÈRES

Quand on supprime un conteneur, tout ce qu'il a écrit dans son système de fichiers disparaît. Pour garder des données (une base de données, SonarQube, Jenkins), on utilise un **volume** : `docker run -v sonar_data:/opt/sonarqube/data ...`. Tu le verras plusieurs fois.

Docker Compose

Pour lancer plusieurs conteneurs qui travaillent ensemble (une appli, sa base, un cache), on les décrit dans un fichier `compose.yaml` :

```
services:
  web:
    build: .
    ports: ["8080:8000"]
    environment:
      DATABASE_URL: postgres://app:secret@db:5432/app
    depends_on: [db]
  db:
    image: postgres:16
    environment:
```

YAML

```
POSTGRES_USER: app
POSTGRES_PASSWORD: secret
POSTGRES_DB: app
volumes: ["dbdata:/var/lib/postgresql/data"]
volumes:
  dbdata:
```

```
docker compose up -d      # tout démarrer
docker compose logs -f    # suivre les logs de tous les services
docker compose down       # tout arrêter (ajouter -v pour supprimer aussi les volumes)
```

BASH

Les services se joignent **par leur nom** ([db:5432](#)) : Compose crée un réseau privé avec un DNS interne. C'est la même idée que les *Services* Kubernetes.

Installer Docker sur un serveur Ubuntu

Voir l'**annexe A.2** (installation depuis le dépôt officiel de Docker). Version rapide pour un serveur d'exercice :

```
sudo apt update && sudo apt install -y docker.io docker-compose-v2
sudo systemctl enable --now docker
sudo usermod -aG docker $USER      # puis se déconnecter/reconnecter
docker run hello-world
```

BASH

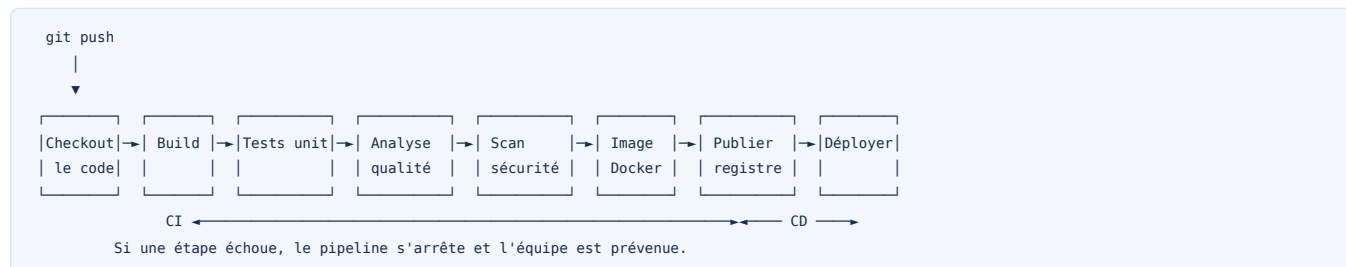
● LE GROUPE 'DOCKER' VAUT ROOT

Un utilisateur du groupe `docker` peut lancer `docker run -v /:/host ...` et lire ou modifier n'importe quel fichier de la machine. Ajouter `jenkins` au groupe `docker` est courant sur un serveur de build, mais c'est un choix conscient : ce serveur doit être bien protégé. Et **jamais** `chmod 777 /var/run/docker.sock` .

Leçon 6 — La CI/CD

Le pipeline

Un **pipeline** est une suite d'étapes automatiques exécutées à chaque changement de code. Il est lui-même décrit **dans un fichier versionné avec le code** (*pipeline as code*) : `Jenkinsfile`, `.github/workflows/*.yaml`, `.gitlab-ci.yml`, `azure-pipelines.yml`, `buildspec.yml`.



Les principes

1. **Le pipeline est la seule route vers la production.** Pas de déploiement manuel « juste cette fois ».
2. **Échouer vite** : les étapes rapides (compilation, tests unitaires) d'abord.
3. **Construire une seule fois, déployer partout** : le même artefact (même image, même tag) passe de test à production.
4. **Versionner les artefacts** de façon traçable : tag d'image = numéro de build ou hash du commit, jamais seulement `latest`.
5. **Les secrets ne sont jamais dans le pipeline en clair** : on utilise le coffre-fort de l'outil (*Jenkins Credentials*, *GitHub Secrets*, *Azure Key Vault*, *AWS Parameter Store*).
6. **Tout est reproductible** : les agents de build sont jetables (conteneurs, instances éphémères).

Les déclencheurs

Déclencheur	Quand	Exemple
Webhook (push)	GitHub prévient l'outil de CI à chaque push	le plus courant et le plus réactif
Pull request	à l'ouverture ou la mise à jour d'une PR	lancer les tests avant la fusion
Poll SCM	l'outil interroge le dépôt régulièrement	si le webhook est impossible (serveur non joignable)
Planifié (cron)	à heure fixe	scan de sécurité nocturne
Manuel	bouton	déploiement en production avec approbation

Les outils de CI/CD du parcours

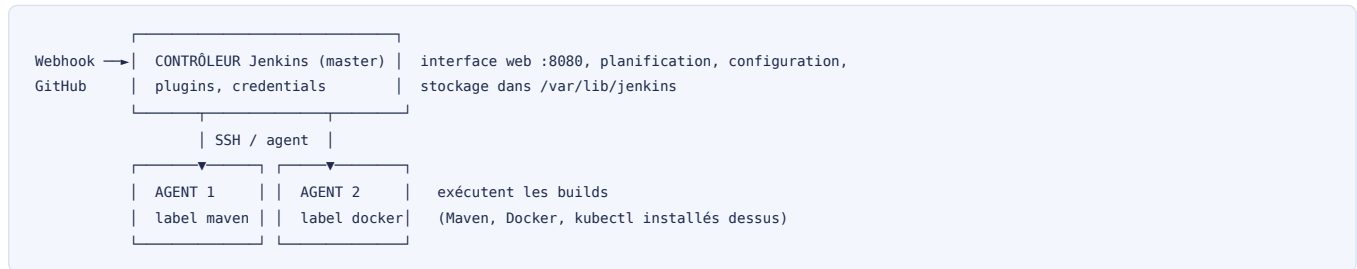
Outil	Hébergement	Fichier	Projets
Jenkins	toi (serveur à installer)	<code>Jenkinsfile</code> (Groovy)	05, 06, 09, 16, 18, 19, 24, 25, 27, 28, 30
GitHub Actions	GitHub (ou tes propres runners)	<code>.github/workflows/*.yaml</code>	14, 22
GitLab CI	GitLab (ou tes runners)	<code>.gitlab-ci.yml</code>	26
Azure Pipelines	Azure DevOps (ou agents auto-hébergés)	<code>azure-pipelines.yml</code>	07, 10, 17, 20, 29
AWS CodePipeline + CodeBuild	AWS	<code>buildspec.yml</code>	21, 23
Argo CD (CD uniquement, GitOps)	dans ton cluster Kubernetes	manifests dans Git	16, 18, 27, 29

Ils font tous la même chose avec une syntaxe différente. Quand tu en maîtrises un, les autres s'apprennent en quelques jours.

Leçon 7 — Jenkins

Jenkins est le serveur d'automatisation open source historique. Il est très répandu en entreprise et omniprésent dans ce parcours.

L'architecture



Le **contrôleur** orchestre ; les **agents** (anciennement *slaves*) exécutent. En production, on met **0 exécuteur** sur le contrôleur : tous les builds tournent sur des agents (sécurité et performances). Les agents sont choisis grâce à des **labels**.

Les notions

- **Job / Item** : une tâche configurée. Types : *Freestyle* (configuration par clics, ancien), **Pipeline** (un Jenkinsfile), **Multibranch Pipeline** (un pipeline par branche, découvert automatiquement).
- **Plugins** : presque tout passe par eux (Git, Docker, SonarQube, Kubernetes...). *Manage Jenkins → Plugins*.
- **Credentials** : le coffre-fort. On y range jetons et mots de passe, référencés par un **ID** dans le Jenkinsfile. *Manage Jenkins → Credentials*.
- **Tools** : les installations d'outils (JDK, Maven, scanner Sonar, Node.js...) déclarées par un nom. *Manage Jenkins → Tools*.
- **Workspace** : le dossier où le job récupère le code et travaille.

Le Jenkinsfile déclaratif

```
pipeline {  
    agent { label 'maven' }           // sur quel agent exécuter  
  
    tools {                           // outils déclarés dans "Manage Jenkins > Tools"  
        jdk 'jdk17'  
        maven 'maven3'  
    }  
  
    environment {                     // variables d'environnement  
        IMAGE = "moncompte/monapp"  
        DOCKERHUB = credentials('dockerhub') // crée DOCKERHUB_USR et DOCKERHUB_PSW  
    }  
  
    options {  
        timeout(time: 30, unit: 'MINUTES')  
        buildDiscarder(logRotator(numToKeepStr: '10'))  
    }  
  
    stages {  
        stage('Checkout') {  
            steps { git branch: 'main', url: 'https://github.com/moi/monapp.git' }  
        }  
        stage('Build & tests') {  
            steps { sh 'mvn -B clean verify' }  
        }  
        stage('Image Docker') {  
            steps {  
                sh 'docker build -t $IMAGE:$BUILD_NUMBER .'  
                sh 'echo $DOCKERHUB_PSW | docker login -u $DOCKERHUB_USR --password-stdin'  
                sh 'docker push $IMAGE:$BUILD_NUMBER'  
            }  
        }  
        stage('Déploiement') {  
            when { branch 'main' } // seulement sur main  
            steps { sh './deploy.sh $BUILD_NUMBER' }  
        }  
    }  
}
```

```

}

post {
    // après les étapes, quel que soit le résultat
    success { echo 'Bravo' }
    failure { echo 'Échec : prévenir l\'équipe' }
    always { cleanWs() } // nettoyer le workspace
}
}

```

Variables fournies par Jenkins : `BUILD_NUMBER` , `BUILD_URL` , `JOB_NAME` , `GIT_COMMIT` , `BRANCH_NAME` (en multibranch), `WORKSPACE` .

💡 LE GÉNÉRATEUR DE SYNTAXE

Dans un job Pipeline, le lien **Pipeline Syntax** ouvre un générateur : tu remplis un formulaire (par exemple « withCredentials »), il te donne la ligne Groovy à coller. Indispensable au début.

L'installation complète de Jenkins est décrite en **annexe A.3** ; tu la feras dans le projet 05.

Leçon 8 — Ansible

Ansible automatise la **configuration** des serveurs : installer des paquets, déposer des fichiers, démarrer des services. Il fonctionne **sans agent** : il se connecte en SSH depuis un **nœud de contrôle** et exécute des tâches décrites en YAML.

```
Nœud de contrôle (Ansible installé)
| inventaire : liste des serveurs
| playbook  : liste des tâches
├─SSH→ serveur web 1
├─SSH→ serveur web 2
└─SSH→ serveur jenkins
```

L'inventaire

```
# inventory.ini
[jenkins_master]
master ansible_host=10.0.1.10

[jenkins_agents]
agent1 ansible_host=10.0.1.20

[all:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/id_ed25519
```

Un playbook

```
# install-java.yml
- name: Préparer les agents Jenkins
  hosts: jenkins_agents
  become: true           # exécuter en sudo
  tasks:
    - name: Installer Java 17 et Maven
      ansible.builtin.apt:
        name: [openjdk-17-jdk, maven, git]
        state: present    # "présent" : idempotent
        update_cache: true

    - name: Créer le dossier de travail
      ansible.builtin.file:
        path: /home/ubuntu/jenkins
        state: directory
        owner: ubuntu
```

```
ansible all -i inventory.ini -m ping           # tester la connexion à tous les serveurs
ansible-playbook -i inventory.ini install-java.yml # exécuter le playbook
```

L'IDEMPOTENCE

On décrit l'**état voulu** (« Java doit être installé »), pas une suite d'actions (« installe Java »). Si Java est déjà là, Ansible ne fait rien et affiche **ok** au lieu de **changed**. On peut donc relancer un playbook 100 fois sans risque. C'est la même philosophie que Terraform (état voulu) et Kubernetes (état désiré).

Terraform vs Ansible : Terraform **crée** l'infrastructure (VPC, instances, bases) ; Ansible **configure** l'intérieur des machines. On les utilise souvent ensemble (projet 06).

QUESTIONS D'ENTRETIEN

- Différence entre une image et un conteneur ? Entre **CMD** et **ENTRYPOINT** ?
- Comment réduire la taille d'une image Docker ? (Image de base *slim/alpine*, *multi-stage*, *.dockerignore*, regrouper les **RUN**, nettoyer les caches.)
- Qu'est-ce qu'un pipeline CI/CD ? Quelles étapes y mets-tu ?
- Contrôleur et agent Jenkins : pourquoi séparer ?
- Qu'est-ce que l'idempotence en Ansible ?

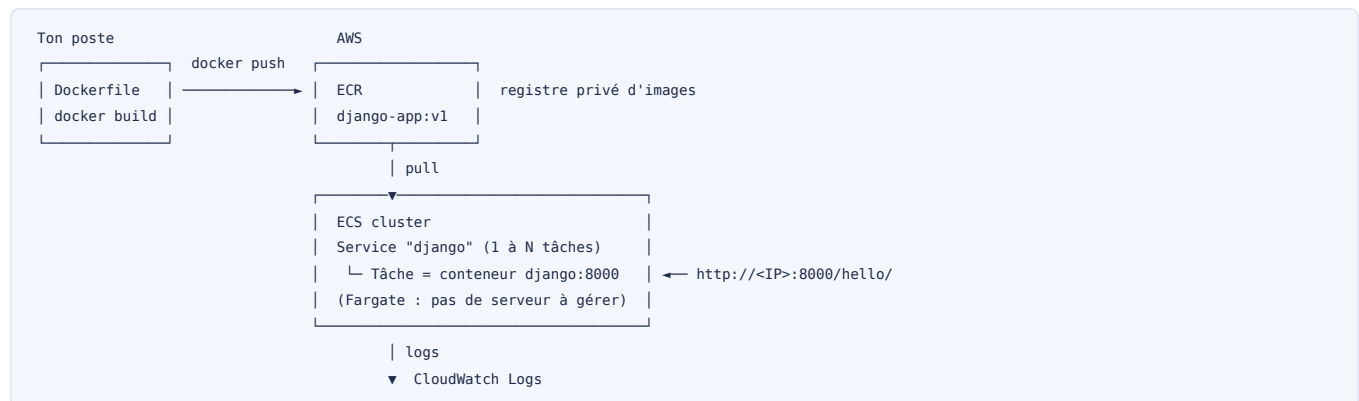
Projet 04 — Application Django sur AWS avec ECR et ECS

FICHE PROJET

Niveau	★☆☆☆ Débutant+
Durée	3 à 5 heures
Coût	quelques centimes (Fargate 0,25 vCPU à l'heure) ; ~0,03 \$/h de plus si tu ajoutes un ALB
Compétences	Dockerfile, build et exécution locale, Amazon ECR, authentification Docker auprès d'AWS, Amazon ECS (cluster, task definition, service), Fargate, CloudWatch Logs
Prérequis	Leçon 5 (Docker), Docker installé sur ton poste
Dépôt d'origine	DevOps-Project-04 (application <code>hello_world_django_app</code> + <code>Dockerfile</code>)

Ce qu'on construit

Ton premier déploiement de **conteneur** dans le cloud : une petite application web Python (Django) est empaquetée en image Docker, stockée dans **ECR** (le registre d'images d'AWS), puis exécutée par **ECS** (le service d'orchestration de conteneurs d'AWS).



Les concepts

Notion ECS	Ce que c'est	Équivalent Kubernetes (pour plus tard)
Cluster	un regroupement logique où tournent les tâches	cluster
Task definition	la « fiche » qui décrit un ou plusieurs conteneurs : image, CPU, mémoire, ports, variables, logs	spécification de Pod
Task (tâche)	une instance en cours d'exécution d'une task definition	Pod
Service	maintient N tâches en vie, les remplace si elles plantent, les relie à un load balancer	Deployment + Service
Launch type	où tournent les tâches : Fargate (serverless, AWS gère les machines) ou EC2 (tes propres instances dans le cluster)	nœuds

FARGATE OU EC2 ?

Le dépôt d'origine utilise des instances EC2 dans le cluster. Avec **Fargate**, tu n'as **aucun serveur** à créer, mettre à jour ou dimensionner : tu dis « 0,25 vCPU et 512 Mo » et AWS s'occupe du reste. C'est le choix le plus simple pour commencer et la tendance actuelle. Le mode EC2 reste utile pour maîtriser finement les coûts à grande échelle ou utiliser du matériel spécifique (GPU). On fait Fargate, et on te montre la variante EC2 en bonus.

Étape 1 — Récupérer et comprendre l'application

```
cd ~/I0 && mkdir -p projects/04-django-ecs && cd projects/04-django-ecs
cp -r ~/tmp/dp/DevOps-Project-04/* . # (cloné au projet 02 ; sinon : git clone --depth 1 https://github.com/ZaheerrAhmed/DevOps-Projects /tmp/dp)
ls
# Dockerfile manage.py requirements.txt hello_world_django_app/ db.sqlite3
```

L'application n'a qu'une page utile : `/hello/`, qui affiche « Hello World ». La racine `/` renvoie une 404 (normal, aucune route n'y est définie).

Étape 2 — Moderniser le Dockerfile

Le Dockerfile d'origine utilise `python:3.9` (fin de vie) et une liste de dépendances sans versions. Voici une version propre :

```
# requirements.txt — figer les versions rend le build reproductible
Django==5.2.*
gunicorn==23.*
```

```
FROM python:3.12-slim

# Pas de fichiers .pyc, logs affichés immédiatement (utile pour CloudWatch)
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1

WORKDIR /usr/src/app

COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

RUN useradd -m django && chown -R django /usr/src/app
USER django

EXPOSE 8000

# gunicorn : un vrai serveur d'applications, contrairement à "runserver" (réservé au développement)
CMD ["gunicorn", "hello_world_django_app.wsgi:application", "--bind", "0.0.0.0:8000", "--workers", "2"]
```

Et un `.dockerignore` :

```
__pycache__/
*.pyc
.git
db.sqlite3
```

🐞 POURQUOI '0.0.0.0' ET PAS '127.0.0.1' ?

`127.0.0.1` (*localhost*) n'accepte que les connexions venant de l'intérieur du conteneur. `0.0.0.0` signifie « écouter sur toutes les interfaces réseau », ce qui permet aux connexions extérieures (redirigées par Docker ou par AWS) d'arriver. Une application qui écoute sur `127.0.0.1` dans un conteneur est injoignable : erreur classique.

⚠️ LE 'SECRET_KEY' ET 'ALLOWED_HOSTS = ["*"]'

`settings.py` contient une clé secrète en clair et accepte n'importe quel nom d'hôte. Acceptable pour un « Hello World », **pas** en production : la clé doit venir d'une variable d'environnement (`os.environ["DJANGO_SECRET_KEY"]`), alimentée par Secrets Manager, et `ALLOWED_HOSTS` doit lister les vrais domaines.

Étape 3 — Construire et tester en local

```
docker build -t hello-world-django-app:v1 .
docker images | grep hello-world-django-app
docker run -d --name django -p 8000:8000 hello-world-django-app:v1
curl http://localhost:8000/hello/
# <html><body>Hello World</body></html>
docker logs django
docker rm -f django
```

💡 TOUJOURS TESTER EN LOCAL D'ABORD

Si ça ne marche pas sur ton poste, ça ne marchera pas dans le cloud, et les erreurs y sont plus longues à diagnostiquer. Cette règle vaut pour tout le parcours.

⚠️ MAC AVEC PUCE APPLE (M1/M2/M3...)

Ton Mac construit des images **ARM**. Fargate tourne par défaut en **x86_64**. Une image ARM lancée sur x86 plante avec `exec format error`. Construis pour la bonne architecture : `docker build --platform linux/amd64 -t ...`. (Autre option : choisir l'architecture ARM64 dans la task definition, moins chère.)

Étape 4 — Créer le dépôt ECR et y pousser l'image

```
export AWS_REGION=eu-west-3
export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
export REPO=hello-world-django-app
```

```
export REGISTRY=$ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com

# 1. Créer le dépôt (avec scan de vulnérabilités à chaque push)
aws ecr create-repository --repository-name $REPO \
  --image-scanning-configuration scanOnPush=true --region $AWS_REGION

# 2. Authentifier Docker auprès d'ECR (jeton valable 12 heures)
aws ecr get-login-password --region $AWS_REGION | docker login --username AWS --password-stdin $REGISTRY

# 3. Donner à l'image son nom complet ECR, puis la pousser
docker tag hello-world-django-app:v1 $REGISTRY/$REPO:v1
docker push $REGISTRY/$REPO:v1
```

🧠 CE QUI SE PASSE À L'ÉTAPE 2

ECR n'a pas de mot de passe fixe. `aws ecr get-login-password` utilise **tes identifiants AWS** (profil configuré en partie 0) pour obtenir un jeton temporaire, qui est passé à `docker login` via l'entrée standard (`--password-stdin` , pour qu'il n'apparaisse ni à l'écran ni dans l'historique). Le dépôt d'origine te faisait exporter tes clés d'accès en variables : avec un profil SSO ou `aws configure` , c'est inutile.

Dans la console : **ECR** → **Repositories** → **hello-world-django-app** → l'image `v1` apparaît, avec le résultat du scan de vulnérabilités.

Étape 5 — Créer le cluster ECS

Console → **Elastic Container Service** → **Clusters** → **Create cluster** :

- Nom : `io-django-cluster`.
- *Infrastructure* : **AWS Fargate (serverless)** uniquement.
- *Monitoring* : tu peux activer *Container Insights* (payant au-delà d'un certain volume ; laisse désactivé pour l'exercice).

Étape 6 — Créer la task definition

ECS → **Task definitions** → **Create new task definition** :

Champ	Valeur
Family	<code>django-task</code>
Launch type	AWS Fargate
OS / Architecture	Linux / X86_64
CPU / Memory	0.25 vCPU / 0.5 GB
Task execution role	<i>Create new role</i> (ou <code>ecsTaskExecutionRole</code>)
Container – name	<code>django</code>
Container – image URI	<code><ACCOUNT_ID>.dkr.ecr.eu-west-3.amazonaws.com/hello-world-django-app:v1</code>
Port mappings	8000 / TCP
Log collection	<i>Use log collection</i> → CloudWatch (coché par défaut)

🧠 LE « TASK EXECUTION ROLE »

C'est le rôle IAM qu'**ECS lui-même** utilise pour **démarrer** ta tâche : télécharger l'image depuis ECR et écrire les logs dans CloudWatch. À ne pas confondre avec le **task role**, qui donne des droits à **ton application** (par exemple lire un bucket S3). Deux rôles, deux usages : encore le moindre privilège.

Étape 7 — Créer le service

Dans le cluster → onglet *Services* → **Create** :

- *Compute options* : Launch type **FARGATE**.
- *Application type* : **Service** ; *Task definition* : `django-task` (dernière révision) ; nom `django-service` ; *Desired tasks* : **1**.
- *Networking* : VPC par défaut, ses subnets publics, **Public IP** : **activé**.
- *Security group* : crée `django-sg` avec une règle entrante **TCP 8000 depuis My IP** (ou `0.0.0.0/0` pour un test rapide).
- *Load balancing* : aucun pour l'instant.

Après une à deux minutes, la tâche passe à l'état **RUNNING**. Clique sur la tâche → onglet *Networking* → **Public IP**.

✅ VÉRIFICATION

```
curl http://<IP_PUBLIQUE_DE_LA_TACHE>:8000/hello/
```

BASH

Tu dois voir « Hello World ». Dans l'onglet *Logs* de la tâche (ou CloudWatch → log group `/ecs/django-task`), tu vois les lignes de unicorn.

Étape 8 — Mettre à jour l'application (le vrai cycle de vie)

Modifie `views.py` (par exemple « Hello World v2 »), puis :

```
docker build -t $REGISTRY/$REPO:v2 .
docker push $REGISTRY/$REPO:v2
```

BASH

Crée une **nouvelle révision** de la task definition avec l'image `:v2` (*Create new revision*), puis mets à jour le service (*Update service* → révision la plus récente → *Force new deployment*). ECS démarre la nouvelle tâche, attend qu'elle soit saine, puis arrête l'ancienne : c'est un **rolling update**.

💡 LE FAIRE EN CLI

```
aws ecs update-service --cluster io-django-cluster --service django-service \
--task-definition django-task --force-new-deployment
```

BASH

C'est exactement la commande qu'un pipeline CI/CD exécutera (projet 23).

Étape 9 (recommandée) — Un load balancer devant le service

L'IP publique d'une tâche **change à chaque redéploiement**. En vrai, on place un **ALB** devant :

1. Crée un SG `django-alb-sg` (HTTP 80 depuis `0.0.0.0/0`) et modifie `django-sg` : 8000 depuis `django-alb-sg` uniquement.
2. Recrée (ou mets à jour) le service avec *Load balancing* → **Application Load Balancer** → *Create a new load balancer* `django-alb`, listener HTTP 80, target group `django-tg` (type **IP**, port 8000), *health check path* `/hello/`.
3. Passe le nombre de tâches à 2 : l'ALB répartit la charge.

⚠️ LE HEALTH CHECK SUR '/'

Si tu laisses le chemin de health check par défaut (`/`), l'ALB reçoit une 404, juge les tâches malades, et ECS les redémarre en boucle. Le health check doit viser une URL qui répond 200 : ici `/hello/`. En entreprise, chaque application expose une route dédiée (`/health`, `/healthz`, `/actuator/health`).

Variante — le mode EC2 (comme dans le dépôt d'origine)

Lors de la création du cluster, choisis *Amazon EC2 instances* : AWS crée un Auto Scaling Group d'instances avec l'AMI optimisée ECS (type `t3.micro`, 1 instance). Dans la task definition, choisis *Launch type* **EC2**, mode réseau *bridge* et un *host port* (par exemple 8000 → 8000). La tâche est placée sur l'instance, joignable via l'IP publique **de l'instance**. Tu vois ainsi ce que Fargate te cache : des serveurs à gérer, mettre à jour et dimensionner.

Dépannage

🔧 DÉPANNAGE

- **La tâche passe de PENDING à STOPPED** : clique sur la tâche arrêtée → *Stopped reason*. Causes classiques :
 - `CannotPullContainerError` : URI d'image erronée, tag inexistant, ou pas d'accès Internet/ECR (tâche dans un subnet privé sans NAT, ou *Public IP* désactivée en subnet public).
 - `exec format error` dans les logs : image ARM sur x86 (voir l'encadré Mac).
 - `Essential container in task exited` : l'application plante au démarrage → lis les logs CloudWatch.
- **curl ne répond pas** : le SG n'autorise pas le port 8000 depuis ton IP, ou tu as oublié `/hello/`.
- **no basic auth credentials au docker push** : le `docker login` a expiré (12 h) ou visait une autre région.
- **Bad Request (400)** de Django : `ALLOWED_HOSTS` ne contient pas le nom utilisé.

Nettoyage

🧹 NETTOYAGE OBLIGATOIRE

1. Service : passe *Desired tasks* à 0, puis **Delete service**.
2. Si créés : ALB `django-alb` et target group `django-tg`.
3. **Delete cluster** `io-django-cluster` (en mode EC2, cela supprime aussi l'ASG et l'instance ; vérifie dans CloudFormation qu'aucune pile ne reste).
4. ECR : `aws ecr delete-repository --repository-name hello-world-django-app --force`.
5. Task definitions : *Deregister* puis *Delete* les révisions (gratuit, mais c'est plus propre).
6. Log group `/ecs/django-task` dans CloudWatch, et les Security Groups créés.

Pour aller plus loin

- Décris toute l'infrastructure (ECR, cluster, task definition, service, ALB) en **Terraform** (partie 3).
- Ajoute un **auto-scaling du service** (*Service auto scaling* : cible CPU à 60 %, 1 à 4 tâches).
- Passe le `SECRET_KEY` par **Secrets Manager** : dans la task definition, section *Environment variables*, type *ValueFrom* avec l'ARN du secret (et donne la permission au task execution role).

- Automatise build → push → déploiement avec **GitHub Actions** (projet 14 pour la syntaxe).

QUESTIONS D'ENTRETIEN

- Quelle différence entre ECS et EKS ? (*ECS : orchestrateur propre à AWS, simple. EKS : Kubernetes managé, standard et portable, plus complexe.*)
- Fargate ou EC2 pour ECS : comment choisis-tu ?
- Différence entre *task role* et *task execution role* ?
- Comment déploies-tu une nouvelle version sans interruption sur ECS ?

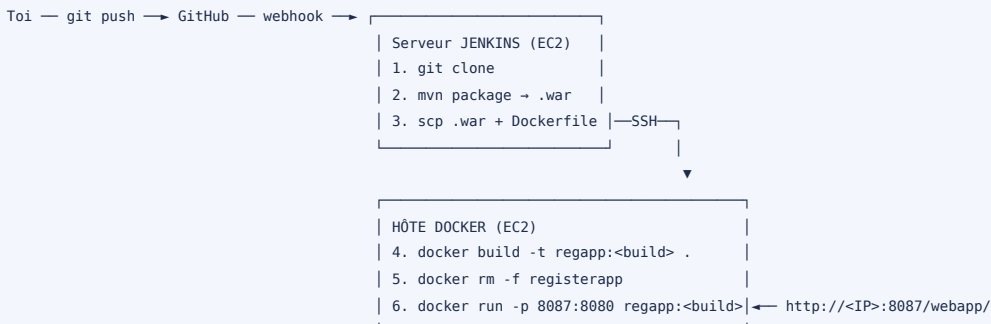
Projet 05 — Déployer du code dans un conteneur Docker avec Jenkins

FICHE PROJET

Niveau	★☆☆☆☆ Débutant+
Durée	5 à 8 heures
Coût	~0,03 \$/h (une <code>t3.small</code> pour Jenkins, une <code>t3.micro</code> pour Docker)
Compétences	installer et configurer Jenkins, plugins, credentials, intégration GitHub (webhook), Maven dans Jenkins, hôte Docker, Dockerfile Tomcat, déploiement par SSH, premier pipeline de bout en bout
Prérequis	Leçons 5 (Docker), 6 (CI/CD), 7 (Jenkins)
Dépôt d'origine	DevOps-Project-05 (application <code>hello-world</code> : Maven multi-modules, <code>.war</code>)

Ce qu'on construit

Une chaîne complètement automatisée : tu modifies le code, tu fais `git push`, et deux minutes plus tard la nouvelle version tourne dans un conteneur Docker, sans aucune action manuelle.



CE QUI CHANGE PAR RAPPORT AU DÉPÔT D'ORIGINE

Le tutoriel d'origine (2023) utilise Amazon Linux 2, Java 11, un job *Freestyle* et le plugin **Publish Over SSH** avec **authentification par mot de passe** (en modifiant `sshd_config`). Nous faisons la même chose de manière plus moderne et plus sûre : Ubuntu 24.04, Java 21, un **Jenkinsfile** (pipeline versionné avec le code) et une **clé SSH** gérée par le plugin **SSH Agent**. Le résultat fonctionnel est identique.

Étape 1 — Le serveur Jenkins

1.1 — Lancer l'instance. Ubuntu 24.04, `t3.small` (2 Go de RAM ; Jenkins + Maven sont à l'étroit en dessous), 20 Go de disque, key pair `io-key`, nom `jenkins`. Security Group `jenkins-sg` : **22** depuis My IP, **8080** depuis My IP.

⚠ LE WEBHOOK GITHUB A BESOIN D'ATTEINDRE JENKINS

Pour que GitHub prévienne Jenkins à chaque push, le port 8080 doit être joignable **depuis les serveurs de GitHub**. Deux options : ouvrir 8080 aux plages d'IP de GitHub (liste publiée sur <https://api.github.com/meta>, clé `hooks`), ou, plus simple pour l'exercice, ouvrir 8080 à `0.0.0.0/0` avec un **mot de passe admin solide**. Ne laisse pas ce serveur en ligne plus longtemps que nécessaire.

1.2 — Installer Java et Jenkins. Applique la recette de l'**annexe A.3**. En résumé :

```
ssh -i ~/.ssh/io-key.pem ubuntu@<IP_JENKINS>
sudo hostnamectl set-hostname jenkins
sudo apt update && sudo apt install -y fontconfig openjdk-21-jre git
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/" \
| sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt update && sudo apt install -y jenkins
sudo systemctl enable --now jenkins
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

BASH

💡 SI 'APT UPDATE' SE PLAINT D'UNE CLÉ (NO_PUBKEY)

Le projet Jenkins renouvelle régulièrement sa clé de signature. Va sur <https://pkg.jenkins.io/debian-stable/> : la page indique le nom exact du fichier de clé en vigueur. Remplace `jenkins.io-2023.key` en conséquence.

1.3 — Premier démarrage. Ouvre `http://<IP_JENKINS>:8080`, colle le mot de passe initial, choisis **Install suggested plugins**, crée ton utilisateur admin (mot de passe fort), garde l'URL proposée.

1.4 — Plugins supplémentaires. *Manage Jenkins* → *Plugins* → *Available plugins* : installe **GitHub Integration** (ou *GitHub*), **Pipeline: Stage View**, **SSH Agent**. (Maven est utilisé via `sh`, pas besoin du plugin *Maven Integration* pour un pipeline.)

1.5 — Déclarer Maven. *Manage Jenkins* → *Tools* → *Maven installations* → *Add Maven* : nom `maven3`, coche *Install automatically* (dernière version 3.9.x). Jenkins téléchargera Maven tout seul au premier build. (Le tutorial d'origine installe Maven à la main dans `/opt` et configure `JAVA_HOME`, `M2`, `M2_HOME` dans `.bash_profile` : c'est possible, mais l'installation automatique est plus simple et reproductible.)

De même, *JDK installations* → nom `jdk21`, `JAVA_HOME` = `/usr/lib/jvm/java-21-openjdk-amd64`.

Étape 2 — L'hôte Docker

2.1 — Lancer l'instance. Ubuntu 24.04, `t3.micro`, nom `dockerhost`, SG `dockerhost-sg` : 22 depuis My IP et depuis le SG `jenkins-sg`, 8081-9000 depuis My IP.

2.2 — Installer Docker et créer l'utilisateur de déploiement.

```
ssh -i ~/.ssh/io-key.pem ubuntu@<IP_DOCKERHOST>
sudo hostnamectl set-hostname dockerhost
sudo apt update && sudo apt install -y docker.io
sudo systemctl enable --now docker

sudo useradd -m -s /bin/bash dockeradmin
sudo usermod -aG docker dockeradmin      # peut utiliser docker sans sudo
sudo mkdir -p /opt/docker && sudo chown dockeradmin:dockeradmin /opt/docker
```

2.3 — Comprendre Tomcat en conteneur (l'étape « manuelle » du tutorial, très formatrice) :

```
sudo docker run -d --name tomcat-test -p 8081:8080 tomcat:9.0-jdk17
curl -I http://localhost:8081      # 404 : webapps est vide dans l'image officielle !
sudo docker exec -it tomcat-test bash
ls /usr/local/tomcat/webapps      # vide
ls /usr/local/tomcat/webapps.dist # les applications par défaut sont ici
cp -R webapps.dist/* webapps/ && exit
curl -I http://localhost:8081      # 200 : la page d'accueil de Tomcat s'affiche
sudo docker rm -f tomcat-test
```

🐛 POURQUOI LA CORRECTION DISPARÂÎT SI ON RECRÉE LE CONTENEUR

Le `cp` a été fait **dans le conteneur**, pas dans l'**image**. Un nouveau conteneur repart de l'image d'origine, sans ta modification. La seule façon durable de modifier un conteneur, c'est de **construire une nouvelle image** avec un Dockerfile. C'est le principe de l'**immuabilité** : on ne modifie jamais un conteneur qui tourne, on en déploie un nouveau.

2.4 — Le Dockerfile de l'application. Il sera envoyé par Jenkins avec le `.war`, donc il vit **dans le dépôt Git** :

```
FROM tomcat:9.0-jdk17
# webapps est vide dans l'image officielle : on y remet les applications par défaut (optionnel)
RUN cp -R /usr/local/tomcat/webapps.dist/* /usr/local/tomcat/webapps
# Déployer notre application
COPY ./*.war /usr/local/tomcat/webapps/
```

Étape 3 — Préparer le dépôt GitHub

```
cd ~ && git clone git@github.com:<toi>/regapp.git      # crée d'abord le dépôt vide "regapp" sur GitHub
cp -r /tmp/dp/DevOps-Project-05/hello-world/* regapp/ && cd regapp
```

Mettre à jour le `pom.xml` racine. Il compile pour Java 1.7 avec un plugin de 2010 : refusé par les JDK récents. Dans le bloc `maven-compiler-plugin` du `<build><plugins>` :

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.13.0</version>
  <configuration>
    <release>8</release>
```

```
</configuration>
</plugin>
```

(Supprime aussi la version [2.3.2](#) du même plugin plus bas dans le fichier, dans la section [reporting](#), si elle crée un conflit.) Vérifie en local si tu as Maven : `mvn -B clean package` doit produire `webapp/target/webapp.war`.

Ajoute le [Dockerfile](#) de l'étape 2.4 à la racine, puis le [Jenkinsfile](#) :

```
pipeline {
    agent any
    tools {
        jdk 'jdk21'
        maven 'maven3'
    }
    environment {
        DOCKER_HOST_IP = '<IP_PRIVÉE_DU_DOCKERHOST>'
        DEPLOY_DIR     = '/opt/docker'
    }
    triggers {
        githubPush()           // déclenché par le webhook GitHub
    }
    stages {
        stage('Build') {
            steps {
                sh 'mvn -B clean package'
            }
        }
        stage('Copier vers l\'hôte Docker') {
            steps {
                sshagent(credentials: ['dockerhost-ssh']) {
                    sh '''
                        scp -o StrictHostKeyChecking=no webapp/target/*.war Dockerfile \
                            dockeradmin@$DOCKER_HOST_IP:$DEPLOY_DIR/
                    '''
                }
            }
        }
        stage('Construire et lancer le conteneur') {
            steps {
                sshagent(credentials: ['dockerhost-ssh']) {
                    sh '''
                        ssh -o StrictHostKeyChecking=no dockeradmin@$DOCKER_HOST_IP "
                            cd $DEPLOY_DIR &&
                            docker build -t regapp:$BUILD_NUMBER . &&
                            docker rm -f registerapp || true &&
                            docker run -d --name registerapp -p 8087:8080 regapp:$BUILD_NUMBER
                        "
                    '''
                }
            }
        }
    }
    post {
        success { echo "Application : http://<IP_PUBLIQUE_DOCKERHOST>:8087/webapp/" }
    }
}
```

```
git add . && git commit -m "Pipeline Jenkins : build Maven + déploiement Docker" && git push
```

🐳 `DOCKER RM -F REGISTERAPP || TRUE`

Le tutoriel d'origine se contente de `docker run -d --name registerapp ...`. Ça marche **une fois** : au deuxième build, Docker refuse (`Conflict. The container name "/registerapp" is already in use`) et le pipeline échoue. Il faut d'abord supprimer l'ancien conteneur. `|| true` évite l'échec au tout premier déploiement, quand le conteneur n'existe pas encore. **Un déploiement automatisé doit fonctionner à la 1^{re} comme à la 100^e exécution.**

🐳 TAGUER L'IMAGE AVEC `\$BUILD\_NUMBER`

Chaque build produit `regapp:1`, `regapp:2`, `regapp:3` ... On sait exactement quelle version tourne, et revenir en arrière consiste à relancer l'image précédente. Avec `regapp:v1` toujours identique, on écrase l'historique.

Étape 4 — La clé SSH Jenkins → hôte Docker

```
# Sur le serveur Jenkins : générer une paire de clés dédiée
ssh-keygen -t ed25519 -f ~/dockerhost_deploy -N "" -C "jenkins-deploy"
cat ~/dockerhost_deploy.pub      # copie cette ligne
```

BASH

```
# Sur l'hôte Docker : l'autoriser pour dockeradmin
sudo -u dockeradmin mkdir -p /home/dockeradmin/.ssh
echo "<la ligne copiée>" | sudo -u dockeradmin tee -a /home/dockeradmin/.ssh/authorized_keys
sudo chmod 700 /home/dockeradmin/.ssh && sudo chmod 600 /home/dockeradmin/.ssh/authorized_keys
```

BASH

Dans Jenkins : *Manage Jenkins* → *Credentials* → *System* → *Global credentials* → *Add credentials* :

- **Kind** : **SSH Username with private key** ; **ID** : `dockerhost-ssh` ; **Username** : `dockeradmin` ;
- **Private Key** : *Enter directly* → colle le contenu de `~/dockerhost_deploy` (la clé **privée**).

Puis **supprime** les fichiers de clé du disque du serveur Jenkins : `rm ~/dockerhost_deploy*`. Elle est désormais chiffrée dans Jenkins.

Étape 5 — Le job et le webhook

5.1 — Créer le job. *New Item* → nom `regapp-pipeline` → **Pipeline** → OK. Section *Pipeline* : *Definition* **Pipeline script from SCM** → **SCM Git** → URL `https://github.com/<toi>/regapp.git` (dépôt public : pas de credentials ; sinon un jeton GitHub) → branche `*/main` → *Script Path* `Jenkinsfile` → **Save**. Lance une première fois **Build Now** (nécessaire pour que Jenkins lise le Jenkinsfile et enregistre le trigger).

5.2 — Le webhook. Sur GitHub : dépôt → *Settings* → *Webhooks* → *Add webhook* :

- **Payload URL** : `http://<IP_PUBLIQUE_JENKINS>:8080/github-webhook/` (le `/` final est obligatoire) ;
- **Content type** : `application/json` ; événement : *Just the push event*.

GitHub envoie un ping : une coche verte doit apparaître à côté du webhook.

💡 SI LE WEBHOOK EST IMPOSSIBLE

Remplace `githubPush()` par `pollSCM('H/2 * * * *')` dans le Jenkinsfile : Jenkins interroge GitHub toutes les 2 minutes. C'est le « Poll SCM » du tutoriel d'origine. Moins réactif, mais ça marche même si Jenkins n'est pas joignable depuis Internet.

Étape 6 — Le test de bout en bout

Modifie `webapp/src/main/webapp/index.jsp` (change le titre), puis `git commit -am "Nouveau titre" && git push`.

✅ VÉRIFICATION

- Dans Jenkins, un build démarre seul en quelques secondes ; la vue *Stage View* montre les 3 étapes au vert.
- Sur l'hôte Docker : `docker ps` montre `registerapp` basé sur `regapp:<numéro du build>` ; `docker images` liste les versions successives.
- `http://<IP_PUBLIQUE_DOCKERHOST>:8087/webapp/` affiche ta page modifiée.

Dépannage

🔧 DÉPANNAGE

- **Permission denied (publickey) à l'étape scp** : la clé publique n'est pas dans `authorized_keys` de `dockeradmin`, mauvais ID de credential, ou permissions trop larges sur `.ssh`.
- **Connection timed out à l'étape scp** : le SG `dockerhost-sg` n'autorise pas le port 22 depuis `jenkins-sg`, ou tu as mis l'IP publique au lieu de l'IP privée (les deux instances sont dans le même VPC : utilise l'IP **privée**).
- **permission denied while trying to connect to the Docker daemon socket** : `dockeradmin` n'est pas dans le groupe `docker`, ou la session a été ouverte avant l'ajout.
- **Source option 7 is no longer supported** : le `pom.xml` n'a pas été mis à jour (étape 3).
- **Le webhook a une croix rouge** : Jenkins n'est pas joignable depuis Internet (SG), ou l'URL ne se termine pas par `/github-webhook/`. GitHub affiche la réponse reçue dans *Recent Deliveries*.
- **Jenkins très lent, ou le build s'arrête sans raison** : manque de mémoire. Ajoute 2 Go de swap (`sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile && sudo mkswap /swapfile && sudo swapon /swapfile`) ou passe en `t3.medium`.

Nettoyage

🔧 NETTOYAGE OBLIGATOIRE

1. Supprime le webhook GitHub (sinon GitHub enverra des requêtes en échec).
2. *Terminate* les instances `jenkins` et `dockerhost`.
3. Supprime les Security Groups `jenkins-sg` et `dockerhost-sg`.
4. Garde le dépôt `regapp` : il resservira.

Pour aller plus loin

- Pousse l'image dans **Docker Hub** ou **ECR** au lieu de la construire sur l'hôte de déploiement : « construire une fois, déployer partout » (c'est la démarche des projets suivants).
- Ajoute une étape `mvn test` séparée et publie les rapports JUnit (`junit '**/target/surefire-reports/*.xml'`).
- Remplace l'hôte Docker par un déploiement Kubernetes : les fichiers `regapp-deploy.yml` et `regapp-service.yml` du dépôt d'origine sont prêts (projet 16).
- Mets Jenkins derrière un reverse proxy Nginx avec HTTPS.

QUESTIONS D'ENTRETIEN

- Comment Jenkins est-il déclenché par un push ? (*Webhook GitHub vers `/github-webhook/`, ou polling.*)
- Où stockes-tu la clé SSH utilisée par Jenkins ? (*Dans les `Credentials Jenkins`, jamais dans le dépôt.*)
- Que se passe-t-il si on relance `docker run --name X` alors que X existe ? Comment rends-tu un déploiement rejouable ?
- Pourquoi ne pas modifier un conteneur en cours d'exécution ?

Projet 06 — Le pipeline CI/CD avancé avec les grands outils DevOps

 FICHE PROJET

Niveau	★★★★☆ Avancé
Durée	15 à 25 heures
Coût	partie A : ~0,10 \$/h (3 instances). Partie B : + ~0,25 \$/h (EKS + 2 nœuds + load balancers)
Compétences	Terraform (VPC, SG, EC2), Ansible (playbooks, inventaire), Jenkins contrôleur/agent, Multibranch Pipeline, webhooks, SonarCloud + Quality Gate, JFrog (artefacts Maven + registre Docker), EKS, Helm, imagePullSecrets , Prometheus & Grafana
Prérequis	Projets 01 et 05. Partie B : leçons 9 à 11 (Kubernetes, Terraform, Helm) de la partie 3
Dépôt d'origine	DevOps-Project-06 (Steps/step1.md à step12.md , Jenkinsfile , Dockerfile , chart Helm sample-app)

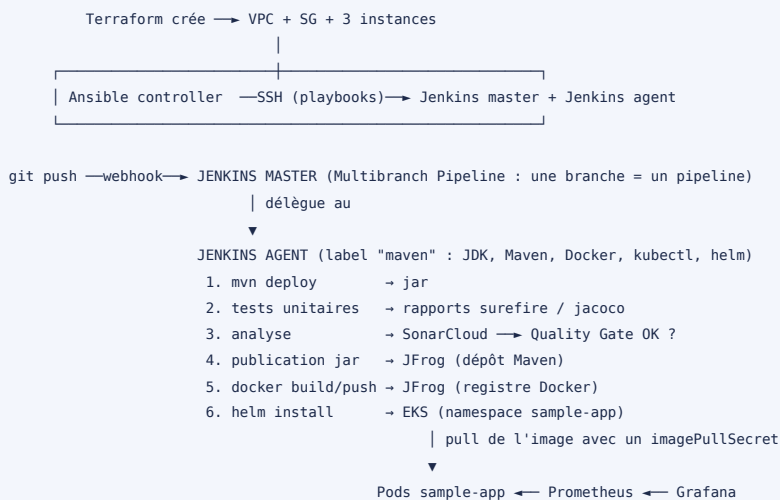
💡 COMMENT ABORDER CE PROJET

C'est le projet le plus complet de la partie 2 : il touche à **tout**. Fais-le en deux temps :

- **Partie A (étapes 1 à 9)** maintenant : infrastructure Terraform, configuration Ansible, Jenkins, SonarCloud, JFrog, image Docker.
- **Partie B (étapes 10 à 12)** après les leçons Kubernetes/Terraform/Helm de la partie 3 : EKS, déploiement Helm, monitoring.

Le code Terraform fourni ici est volontairement simple ; tu le comprendras complètement après la leçon 10. Pour l'instant, considère-le comme une recette.

Ce qu'on construit



L'application est une petite API Spring Boot ([demo-workshop](#)) qui expose des informations sur des dépôts GitHub.

Partie A — Infrastructure, configuration et CI

Étape 1 — Provisionner l'infrastructure avec Terraform

Crée `projects/06-advanced-cicd/terraform/` dans ton dépôt IO, avec ce `main.tf` :

```
terraform {
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
}

provider "aws" {
  region = "eu-west-3"
  default_tags { tags = { Project = "io-06" } }
}

variable "my_ip" { description = "Ton IP publique en /32, ex: 82.64.10.5/32" }
```

```

variable "key_name" { default = "io-key" }

# --- Réseau : un VPC, un subnet public, une sortie Internet ---
resource "aws_vpc" "main" {
  cidr_block      = "10.6.0.0/16"
  enable_dns_hostnames = true
  tags = { Name = "io-06-vpc" }
}

resource "aws_internet_gateway" "igw" { vpc_id = aws_vpc.main.id }

resource "aws_subnet" "public" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.6.1.0/24"
  map_public_ip_on_launch = true
  availability_zone = "eu-west-3a"
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}

# --- Pare-feu : SSH et Jenkins depuis ton IP, tout le trafic interne au VPC ---
resource "aws_security_group" "devops" {
  name = "io-06-sg"
  vpc_id = aws_vpc.main.id

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = [var.my_ip]
  }

  ingress {
    from_port = 8080
    to_port   = 8080
    protocol  = "tcp"
    cidr_blocks = [var.my_ip]          # ajoute les IP des webhooks GitHub si besoin (voir projet 05)
  }

  ingress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = [aws_vpc.main.cidr_block]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# --- Image Ubuntu 24.04 la plus récente ---
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical
  filter {
    name = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }
}

# --- Les 3 machines ---
locals {
  servers = {
    ansible = "t3.micro"
  }
}

```

```

jenkins-master = "t3.small"
jenkins-agent = "t3.medium" # Maven + Docker ont besoin de mémoire
}
}

resource "aws_instance" "server" {
  for_each      = local.servers
  ami           = data.aws_ami.ubuntu.id
  instance_type = each.value
  key_name      = var.key_name
  subnet_id     = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.devops.id]
  root_block_device { volume_size = 20 }
  tags = { Name = "io-06-${each.key}" }
}

output "public_ips" { value = { for k, s in aws_instance.server : k => s.public_ip } }
output "private_ips" { value = { for k, s in aws_instance.server : k => s.private_ip } }

```

```

cd projects/06-advanced-cicd/terraform
terraform init # télécharge le provider AWS
terraform plan -var="my_ip=$(curl -s ifconfig.me)/32" # montre ce qui va être créé
terraform apply -var="my_ip=$(curl -s ifconfig.me)/32" # crée (tape "yes")
terraform output

```

BASH

🧠 CE QUE TU VIENS DE FAIRE

En 100 lignes de texte, tu as décrit un réseau, un pare-feu et trois serveurs. Le fichier est versionné, fiable, et `terraform destroy` supprime **tout** proprement. Compare avec le projet 02, où tu as tout créé à la main. La leçon 10 t'expliquera chaque bloc. Retiens déjà : `resource` = quelque chose à créer, `data` = quelque chose à lire, `variable` = un paramètre, `output` = une valeur à afficher, `for_each` = créer plusieurs ressources à partir d'une liste.

Étape 2 — SSH sans mot de passe entre Ansible et les nœuds

Le nœud Ansible doit pouvoir se connecter aux deux serveurs Jenkins. On copie **temporairement** ta clé sur le contrôleur Ansible (c'est le seul cas acceptable dans ce projet d'apprentissage ; en entreprise on utilise une clé dédiée ou SSM) :

```

scp -i ~/.ssh/io-key.pem ~/.ssh/io-key.pem ubuntu@<IP_PUB_ANSIBLE>:~/.ssh/io-key.pem
ssh -i ~/.ssh/io-key.pem ubuntu@<IP_PUB_ANSIBLE>
chmod 400 ~/.ssh/io-key.pem
sudo apt update && sudo apt install -y ansible
ansible --version

```

BASH

(Variante de l'énoncé : générer une clé avec `ssh-keygen` sur le contrôleur et la copier sur chaque nœud avec `ssh-copy-id`. Sur EC2, l'authentification par mot de passe est désactivée, donc `ssh-copy-id` ne marche pas directement : il faudrait ajouter la clé publique à la main dans `~/.ssh/authorized_keys` de chaque nœud.)

Crée l'inventaire `~/ansible/hosts.ini` avec les IP privées :

```

[jenkins_master]
master ansible_host=10.6.1.X

[jenkins_agent]
agent ansible_host=10.6.1.Y

[all:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/io-key.pem
ansible_ssh_common_args='-o StrictHostKeyChecking=accept-new'

```

INI

```

cd ~/ansible
ansible all -i hosts.ini -m ping
# master | SUCCESS => {"ping": "pong"}
# agent  | SUCCESS => {"ping": "pong"}

```

BASH

Étape 3 — Configurer Jenkins master et agent avec Ansible

`jenkins-master.yml` :

```

- name: Installer le contrôleur Jenkins
  hosts: jenkins_master
  become: true
  tasks:
    - name: Java 21
      ansible.builtin.apt:

```

YAML

```

    name: [fontconfig, openjdk-21-jre]
    update_cache: true

- name: Clé du dépôt Jenkins
  ansible.builtin.get_url:
    url: https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
    dest: /etc/apt/keyrings/jenkins-keyring.asc

- name: Dépôt Jenkins
  ansible.builtin.apt_repository:
    repo: "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/"
    filename: jenkins

- name: Installer Jenkins
  ansible.builtin.apt:
    name: jenkins
    update_cache: true

- name: Démarrer Jenkins
  ansible.builtin.systemd:
    name: jenkins
    state: started
    enabled: true

```

`jenkins-agent.yml` — l'agent est le serveur de **build** : il lui faut Java, Maven, Git et Docker.

```

- name: Préparer l'agent Jenkins (serveur de build Maven + Docker)
  hosts: jenkins_agent
  become: true
  tasks:
    - name: Paquets de build
      ansible.builtin.apt:
        name: [openjdk-21-jdk, maven, git, docker.io, unzip]
        update_cache: true

    - name: Docker démarré
      ansible.builtin.systemd:
        name: docker
        state: started
        enabled: true

    - name: ubuntu peut utiliser docker (les builds Jenkins tourneront sous ubuntu)
      ansible.builtin.user:
        name: ubuntu
        groups: docker
        append: true

    - name: Dossier de travail de l'agent
      ansible.builtin.file:
        path: /home/ubuntu/jenkins
        state: directory
        owner: ubuntu
        group: ubuntu

```

```

ansible-playbook -i hosts.ini jenkins-master.yml
ansible-playbook -i hosts.ini jenkins-agent.yml
ansible-playbook -i hosts.ini jenkins-agent.yml # relance : tout est "ok", rien "changed" → idempotent

```

i MAVEN D'UBUNTU OU MAVEN RÉCENT ?

Le paquet `maven` d'Ubuntu 24.04 fournit Maven 3.8.x, suffisant ici. Le Jenkinsfile d'origine utilise `/opt/apache-maven-3.9.4` : si tu veux exactement cette version, ajoute au playbook une tâche `get_url` + `unarchive` qui télécharge et décompresse l'archive dans `/opt`.

Étape 4 — Relier l'agent au contrôleur

- Ouvre `http://<IP_PUB_MASTER>:8080`, débloque Jenkins (`sudo cat /var/lib/jenkins/secrets/initialAdminPassword`), installe les plugins suggérés, crée ton admin.
- Manage Jenkins → Nodes → Built-In Node → Configure : **Number of executors = 0** (aucun build sur le contrôleur).
- Manage Jenkins → Credentials → Add : SSH Username with private key, ID `agent-ssh`, user `ubuntu`, clé privée = contenu de `io-key.pem`.
- Nodes → New Node : nom `maven-agent`, Permanent Agent →
 - Number of executors : 2 ; Remote root directory : `/home/ubuntu/jenkins` ;
 - Labels : `maven` (c'est ce que demande `agent { node { label 'maven' } }` dans le Jenkinsfile) ;

- *Launch method* : **Launch agents via SSH** ; *Host* : IP **privée** de l'agent ; *Credentials* : `agent-ssh` ; *Host Key Verification Strategy* : *Non verifying* (acceptable pour l'exercice).

5. *Save* → l'agent passe en ligne en quelques secondes (voir son log).

🔴 LE CONTRÔLEUR COPIE LUI-MÊME L'AGENT

Avec « Launch agents via SSH », le contrôleur se connecte à la machine, y dépose `agent.jar` et le lance. Il n'y a donc **rien** à installer côté agent à part Java : c'est pour cela que le playbook de l'énoncé, qui crée un service `jenkins-agent` à la main, n'est pas nécessaire avec cette méthode.

Étape 5 — Credentials GitHub et Multibranch Pipeline

1. Fork (ou copie) l'application `DevOps-Project-06` dans un dépôt GitHub à toi : `demo-workshop`.
2. Crée un **fine-grained token** GitHub (lecture du contenu de ce dépôt, lecture des webhooks).
3. Jenkins → *Credentials* → *Add* → *Username with password* : user = ton pseudo GitHub, password = le token, ID `github-cred`.
4. *New Item* → nom `demo-workshop` → **Multibranch Pipeline** → *Branch Sources* → *Add source* → *GitHub* → credentials `github-cred` → URL du dépôt → *Save*.

Jenkins **scanne** le dépôt : il crée un sous-job pour **chaque branche** qui contient un `Jenkinsfile`, et en crée automatiquement de nouveaux quand tu crées une branche.

Étape 6 — Déclencher le scan par webhook

Installe le plugin **Multibranch Scan Webhook Trigger**. Dans la configuration du job multibranch → *Scan Multibranch Pipeline Triggers* → coche **Scan by webhook** → *Trigger token* : `demo-workshop-token`.

GitHub → dépôt → *Settings* → *Webhooks* → *Add* : URL `http://<IP_PUB_MASTER>:8080/multibranch-webhook-trigger/invoke?token=demo-workshop-token`, type JSON, événements *push*.

Étape 7 — SonarCloud et tests

1. SonarCloud : crée l'organisation et le projet (comme au projet 01), génère un token.
2. Jenkins → *Credentials* → *Secret text*, ID `sonar-token`.
3. Plugin **SonarQube Scanner** → *Manage Jenkins* → *System* → *SonarQube servers* : nom `sonar-server`, URL `https://sonarcloud.io`, token `sonar-token`.
4. *Manage Jenkins* → *Tools* → *SonarQube Scanner installations* : nom `sonar-scanner`, *Install automatically*.
5. À la racine du dépôt, `sonar-project.properties` :

```
sonar.organization=<ton-organisation>
sonar.projectKey=<ton-organisation>_demo-workshop
sonar.projectName=demo-workshop
sonar.sources=src/main
sonar.tests=src/test
sonar.java.binaries=target/classes
sonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/jacoco.xml
sonar.sourceEncoding=UTF-8
```

PROPERTIES

6. Dans SonarCloud : *Administration* → *Webhooks* → URL `http://<IP_PUB_MASTER>:8080/sonarqube-webhook/` (permet à Jenkins de connaître le résultat de la Quality Gate sans attendre indéfiniment).

Étape 8 — Publier le jar dans JFrog

1. JFrog : crée un dépôt local Maven `maven-libs-release-local` et un dépôt local **Docker** `docker-local`. Génère un *access token*.
2. Jenkins → *Credentials* → *Username with password*, ID `jfrog-cred` (ton user JFrog + token).
3. Installe le plugin **Artifactory** (JFrog). (Il est utilisé par l'étape « Jar Publish » du *Jenkinsfile* d'origine via `Artifactory.newServer()`.)

i POURQUOI LE JAR EST DANS 'JARSTAGING/'

Le `pom.xml` contient `<distributionManagement>` vers `file://jarstaging` : `mvn deploy` ne publie pas sur Internet, il « déploie » le jar dans un dossier local `jarstaging/` du workspace. C'est ensuite l'étape *Jar Publish* qui envoie ce dossier vers JFrog avec le plugin Artifactory, en y ajoutant des métadonnées (numéro de build, commit). Deux étapes, deux responsabilités.

Étape 9 — Image Docker et Jenkinsfile complet

Le `Dockerfile` d'origine utilise `openjdk:8`, une image **dépréciée et retirée**. Remplace-le :

```
FROM eclipse-temurin:8-jre
WORKDIR /app
COPY jarstaging/com/satish/demo-workshop/2.1.2/demo-workshop-2.1.2.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

DOCKERFILE

Installe le plugin **Docker Pipeline**. Voici le *Jenkinsfile* complet, adapté et commenté :

```
def registry = 'https://<ton-nom>.jfrog.io'
def imageName = '<ton-nom>.jfrog.io/docker-local/sample_app'
def version = '2.1.2'

pipeline {
```

GROOVY

```

agent { node { label 'maven' } }

stages {
  stage('Build') {
    steps {
      sh 'mvn -B clean deploy -Dmaven.test.skip=true' // compile + "déploie" dans jarstaging/
    }
  }
  stage('Tests unitaires') {
    steps {
      sh 'mvn -B surefire-report:report' // lance les tests et produit le rapport
    }
  }
  stage('Analyse SonarCloud') {
    environment { scannerHome = tool 'sonar-scanner' }
    steps {
      withSonarQubeEnv('sonar-server') {
        sh "${scannerHome}/bin/sonar-scanner"
      }
    }
  }
  stage('Quality Gate') {
    steps {
      timeout(time: 10, unit: 'MINUTES') {
        waitForQualityGate abortPipeline: true // stoppe le pipeline si la porte est rouge
      }
    }
  }
  stage('Publication du jar') {
    steps {
      script {
        def server = Artifactory.newServer url: registry + '/artifactory', credentialsId: 'jfrog-cred'
        def uploadSpec = """{
          "files": [{
            "pattern": "jarstaging/(*)",
            "target": "maven-lib-release-local/{1}",
            "flat": "false",
            "props": "builddid=${env.BUILD_ID};commitid=${env.GIT_COMMIT}",
            "exclusions": ["*.sha1", "*.md5"]
          }]
        }"""
        def buildInfo = server.upload(uploadSpec)
        buildInfo.env.collect()
        server.publishBuildInfo(buildInfo)
      }
    }
  }
  stage('Build image Docker') {
    steps {
      script { app = docker.build("${imageName}:${version}") }
    }
  }
  stage('Push image Docker') {
    steps {
      script {
        docker.withRegistry(registry, 'jfrog-cred') { app.push() }
      }
    }
  }
}
// Partie B : décommenter après l'étape 11
// stage('Déploiement Helm') {
//   steps { sh 'helm upgrade --install sample-app ./sample-app -n sample-app --create-namespace' }
// }
}
}

```

✓ FIN DE LA PARTIE A

- Un `git push` sur `main` déclenche le scan puis le pipeline de la branche `main`.
- SonarCloud affiche l'analyse, avec la couverture de tests (Jacoco).
- JFrog contient le jar (avec les propriétés `builddid` / `commitid`) et l'image `sample_app:2.1.2`.
- Crée une branche `feature/test`, pousse-la : un nouveau sous-job apparaît automatiquement.

Partie B — Kubernetes et monitoring

Étape 10 — Le cluster EKS

Le plus simple et le plus fiable est **eksctl** (voir annexe A.8). Depuis ton poste ou l'agent :

```
eksctl create cluster --name io-06 --region eu-west-3 \
--nodegroup-name workers --node-type t3.medium --nodes 2 --managed
```

BASH

(L'énoncé crée l'EKS avec Terraform : c'est exactement ce que fait le projet 19, en détail. Tu peux revenir ici ensuite et remplacer eksctl par Terraform.)

Sur l'agent Jenkins, installe **kubectl**, **AWS CLI v2** et **Helm** (commandes de la partie 0, étape 5), puis donne-lui accès au cluster. Le plus propre : un **rôle IAM** attaché à l'instance agent, autorisé dans le cluster par une *access entry* :

```
# Depuis ton poste (admin du cluster)
aws eks create-access-entry --cluster-name io-06 --principal-arn arn:aws:iam::<compte>:role/<role-agent>
aws eks associate-access-policy --cluster-name io-06 --principal-arn arn:aws:iam::<compte>:role/<role-agent> \
--policy-arn arn:aws:eks::aws:cluster-access-policy/AmazonEKSClusterAdminPolicy --access-scope type=cluster

# Sur l'agent (en tant qu'ubuntu)
aws eks update-kubeconfig --region eu-west-3 --name io-06
kubectl get nodes
```

BASH

Étape 11 — Déployer l'image depuis JFrog

Kubernetes doit s'authentifier auprès du registre privé JFrog pour télécharger l'image. On crée un **secret de type docker-registry** et on le référence dans le Deployment (`imagePullSecrets`).

Extrais le chart fourni et adapte-le :

```
tar xzf sample-app-1.0.1.tgz # crée le dossier sample-app/ (chart Helm)
```

BASH

Dans `sample-app/templates/deployment.yaml`, mets ton image (`<ton-nom>.jfrog.io/docker-local/sample_app:2.1.2`) et vérifie `imagePullSecrets: [{ name: dockercred }]`.

● DES SECRETS EN CLAIR DANS LE CHART D'ORIGINE

Le `deployment.yaml` d'origine contient des clés d'API Twitter **en clair** (`CONSUMER_SECRET`, `ACCESS_TOKEN_SECRET` ...), et `secret.yaml` contient des identifiants de registre encodés en base64 (ce n'est **pas** du chiffrement : `echo <valeur> | base64 -d` les révèle). **Supprime-les** du chart. Crée le secret du registre en ligne de commande, hors Git :

```
kubectl create namespace sample-app
kubectl create secret docker-registry dockercred -n sample-app \
--docker-server=<ton-nom>.jfrog.io \
--docker-username=<user-jfrog> \
--docker-password=<token-jfrog>
```

BASH

et retire `templates/secret.yaml` du chart. En production, on utilise un outil comme **External Secrets Operator** (qui lit AWS Secrets Manager) ou **Sealed Secrets**.

Décommente l'étape *Déploiement Helm* du Jenkinsfile, pousse. Vérifie :

```
kubectl get pods,svc -n sample-app
kubectl port-forward -n sample-app svc/sample-app-service 8000:8000
curl http://localhost:8000/ # depuis la machine où tourne port-forward
```

BASH

Le service est de type `NodePort` (port 30082 sur chaque nœud). Pour l'exposer à Internet, change-le en `LoadBalancer` (AWS crée un load balancer, payant) ou utilise un Ingress (projet 15).

Étape 12 — Prometheus et Grafana

On installe la pile **kube-prometheus-stack** (Prometheus, Alertmanager, Grafana, exporters, tableaux de bord prêts à l'emploi) avec Helm. Détails en annexe A.10.

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm install monitoring prometheus-community/kube-prometheus-stack -n monitoring --create-namespace
kubectl get pods -n monitoring

# Accès à Grafana sans load balancer (gratuit) :
kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80
# mot de passe admin :
kubectl get secret -n monitoring monitoring-grafana -o jsonpath="{.data.admin-password}" | base64 -d; echo
```

BASH

Ouvre `http://localhost:3000` (utilisateur `admin`) → *Dashboards* → *Kubernetes / Compute Resources / Cluster* : CPU et mémoire du cluster, par namespace, par pod.

(L'énoncé passe les services Prometheus et Grafana en `LoadBalancer` pour y accéder depuis le navigateur. Ça marche, mais chaque service crée un load balancer payant et expose l'outil sur Internet. Le `port-forward` est gratuit et privé.)

Dépannage

🔧 DÉPANNAGE

- **L'agent reste hors ligne** : regarde son log dans Jenkins. `Connection refused/timed out` → IP privée erronée ou SG. `java: not found` → Java absent sur l'agent (playbook non exécuté).
- **waitForQualityGate attend jusqu'au timeout** : le webhook SonarCloud → Jenkins n'est pas configuré, ou Jenkins n'est pas joignable depuis Internet.
- **docker: permission denied dans le pipeline** : l'utilisateur ubuntu a été ajouté au groupe docker **après** le démarrage de l'agent. Déconnecte puis reconnecte l'agent dans Jenkins (*Disconnect / Launch agent*).
- **Pod en ImagePullBackOff** : `kubectl describe pod ...` → `401 Unauthorized` (secret `dockercred` faux ou dans un autre namespace) ou `not found` (nom d'image ou tag faux).
- **helm install : « cannot re-use a name that is still in use »** : utilise `helm upgrade --install`, rejouable à l'infini.
- **error: You must be logged in to the server (Unauthorized)** avec kubectl : l'identité IAM utilisée n'a pas d'access entry dans le cluster.

Nettoyage

🔧 NETTOYAGE OBLIGATOIRE

1. `helm uninstall sample-app -n sample-app` et `helm uninstall monitoring -n monitoring`. Supprime tout service `LoadBalancer` restant (`kubectl get svc -A | grep LoadBalancer`).
2. `eksctl delete cluster --name io-06 --region eu-west-3` (10 à 15 minutes).
3. `terraform destroy -var="my_ip=$(curl -s ifconfig.me)/32"` dans le dossier Terraform.
4. Supprime les webhooks GitHub et SonarCloud.
5. Vérifie dans la console EC2 (instances, volumes, load balancers) et CloudFormation (piles `eksctl-io-06-*`) que tout a disparu.

🔑 QUESTIONS D'ENTRETIEN

- Décris ton pipeline CI/CD idéal, étape par étape, et justifie l'ordre.
- Quelle différence entre Terraform et Ansible ? Pourquoi les utiliser ensemble ?
- Qu'est-ce qu'une Quality Gate et que se passe-t-il si elle échoue ?
- Comment Kubernetes télécharge-t-il une image depuis un registre privé ?
- Pourquoi un Multibranch Pipeline plutôt qu'un simple Pipeline ?

Projet 07 — Le parcours DevOps avec Azure DevOps

FICHE PROJET

Niveau	★★★★☆ Intermédiaire
Durée	12 à 18 heures (7 labs)
Coût	couvert par les 200 \$ de crédit Azure si tu détruis l'AKS entre les sessions (~0,15 \$/h avec 1 nœud). Évite l'Application Gateway (~0,30 \$/h).
Compétences	Azure (groupes de ressources, Entra ID, Storage), Azure DevOps (organisation, projet, Repos, Pipelines, service connections, variable groups), Terraform sur Azure avec état distant, ACR, AKS, Key Vault, Application Insights, tests d'infrastructure (InSpec), monitoring (Container Insights)
Prérequis	Leçons 4 à 6. Un compte Azure (crédit de 200 \$ sur 30 jours)
Dépôt d'origine	DevOps-Project-07/labs/1-... à 7-... (basé sur <i>DevOps-Journey-Using-Azure-DevOps</i> de Thomas Thornton)

Ce qu'on construit

Une chaîne complète **entièrement dans l'écosystème Microsoft** : un pipeline Azure DevOps qui crée l'infrastructure avec Terraform (réseau, registre ACR, cluster AKS, Log Analytics), construit l'image d'une application ASP.NET Core, la pousse dans ACR et la déploie dans AKS, puis teste l'infrastructure et la surveille.

```
Azure Repos —commit→ Azure Pipelines (azure-pipelines.yml)
| stage validate → plan → apply (Terraform, état dans Azure Storage)
|                                     ↳ crée : VNet, ACR, AKS, Log Analytics, App Insights, Key Vault
| stage build → docker build/push → ACR (aspnet:<BuildId>)
| stage deploy → kubectl apply → AKS —pull (rôle AcrPull)→ ACR
| stage test → InSpec vérifie que l'infra est conforme
|
| Application Insights + Container Insights (monitoring)
```

Azure en 2 minutes (pour qui connaît AWS)

Azure	AWS	Remarque
Abonnement (<i>subscription</i>)	compte AWS	l'unité de facturation
Groupe de ressources (<i>resource group</i>)	— (tags, CloudFormation stack)	toute ressource Azure appartient à un groupe ; supprimer le groupe supprime tout ce qu'il contient. Très pratique pour le nettoyage !
Entra ID (ex-Azure AD)	IAM Identity Center	annuaire des utilisateurs et groupes
Service principal / managed identity	utilisateur technique / rôle IAM	identité d'une application ou d'un pipeline
RBAC (rôles <i>Owner</i> , <i>Contributor</i> , <i>Reader</i> , <i>AcrPull</i> ...)	politiques IAM	attribués sur un scope : abonnement, groupe ou ressource
VNet, Storage Account, ACR, AKS, Key Vault	VPC, S3, ECR, EKS, Secrets Manager	

```
# Azure CLI (sur ton poste)
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
az login # ouvre le navigateur
az account show # abonnement courant
az account set --subscription "<nom ou id>"
```

BASH

⚠ LES AGENTS HÉBERGÉS D'AZURE DEVOPS NE SONT PAS ACTIVÉS D'OFFICE

Une nouvelle organisation Azure DevOps n'a **pas** de « parallel job » gratuit sur les agents hébergés par Microsoft : le premier pipeline échoue avec *No hosted parallelism has been purchased or granted*. Deux solutions :

- Demander la gratuité via le formulaire officiel (<https://aka.ms/azpipelines-parallelism-request>) : réponse en 2 à 3 jours ouvrés. **Fais-le dès le début du projet.**
- En attendant, utiliser un **agent auto-hébergé** (*self-hosted*) sur ton poste ou une petite VM (le projet 29 le détaille) : *Project settings* → *Agent pools* → *Default* → *New agent*, puis suivre les 3 commandes affichées.

Lab 1 — Mise en place initiale

1.1 — Organisation et projet Azure DevOps. Va sur <https://dev.azure.com> → *New organization* (choisis la région Europe) → *New project* : `devops-journey`, visibilité privée.

1.2 — Service connection vers Azure. *Project settings* → *Service connections* → *New* → *Azure Resource Manager* → **Workload Identity federation (automatic)** → abonnement → laisse le groupe de ressources vide (accès à l'abonnement) → nom `azure-devops-journey-service-connection` → coche *Grant access permission to all pipelines*.

🔑 WORKLOAD IDENTITY FEDERATION

Le dépôt d'origine crée un *service principal* avec un **secret** (un mot de passe qui expire et qu'il faut renouveler). La fédération d'identité (OIDC) permet au pipeline d'obtenir un jeton temporaire **sans aucun secret stocké** : c'est aujourd'hui la méthode recommandée, et c'est l'équivalent des rôles IAM pour GitHub Actions sur AWS.

L'identité créée a le rôle *Contributor* sur l'abonnement. Pour que Terraform puisse aussi **attribuer des rôles** (étape *AcrPull*), elle a besoin du rôle *User Access Administrator* (ou *Role Based Access Control Administrator*) : Portail Azure → *Subscriptions* → *Access control (IAM)* → *Add role assignment*.

1.3 — Stockage de l'état Terraform. Terraform garde la mémoire de ce qu'il a créé dans un fichier d'**état** (*state*). En équipe ou dans un pipeline, cet état doit être **distant et partagé** : ici, un conteneur Azure Blob Storage.

```
RG=devops-journey-rg
SA=devopsjourney$RANDOM          # nom unique, minuscules et chiffres uniquement
az group create -l francecentral -n $RG
az storage account create -n $SA -g $RG -l francecentral --sku Standard_LRS --min-tls-version TLS1_2
az storage container create --name tfstate --account-name $SA --auth-mode login
echo $SA                          # note-le
```

BASH

1.4 — Groupe Entra ID des administrateurs AKS. Les membres de ce groupe pourront administrer le cluster avec `kubectl`.

```
az ad group create --display-name devopsthehardway-aks-group --mail-nickname devopsthehardway-aks-group
az ad group member add --group devopsthehardway-aks-group --member-id $(az ad signed-in-user show --query id -o tsv)
az ad group show --group devopsthehardway-aks-group --query id -o tsv      # note cet ID
```

BASH

Lab 2 — Déployer l'infrastructure avec Terraform via Azure DevOps

2.1 — Récupérer le code. Le code des labs du dépôt d'origine date de 2021 (provider `azurerm` v2, syntaxe `addon_profile` supprimée depuis). **Utilise la version maintenue** du même parcours, sur le GitHub de l'auteur : <https://github.com/thomast1986/DevOps-Journey-Using-Azure-DevOps> (dossier `labs/2-AzureDevOps-Terraform-Pipeline/`). Importe-le dans Azure Repos : *Repos* → *Import repository* → URL du dépôt.

La structure Terraform est **modulaire** :

```
terraform/
├─ providers.tf      # provider azurerm + backend "azurerm" (l'état distant)
├─ main.tf           # appelle les modules
├─ variables.tf
├─ modules/
│  ├─ vnet/          # réseau + sous-réseaux
│  ├─ acr/           # Azure Container Registry
│  ├─ log-analytics/ # espace de travail pour les logs et métriques
│  └─ aks/           # le cluster Kubernetes
vars/production.tfvars # les valeurs : noms, région, taille des nœuds, ID du groupe AKS admins...
```

Adapte `vars/production.tfvars` : noms uniques (l'ACR doit avoir un nom unique au monde), région (`francecentral`), **1 nœud** `Standard_B2s` ou `Standard_D2s_v5` pour limiter le coût, ID du groupe Entra (lab 1.4), et **désactive l'Application Gateway** (AGIC) si l'option est présente.

2.2 — Installer l'extension Terraform dans l'organisation : *Organization settings* → *Extensions* → *Browse marketplace* → **Terraform** (Microsoft DevLabs).

2.3 — Le pipeline. Voici la structure (variables à adapter) :

```
trigger: none          # lancement manuel pour l'instant (le lab 5 activera le déclenchement auto)
pr: none

pool:
  vmImage: ubuntu-latest # agent hébergé ; ou "name: Default" pour un agent auto-hébergé

variables:
  backendServiceArm: 'azure-devops-journey-service-connection'
  backendAzureRmResourceGroupName: 'devops-journey-rg'
  backendAzureRmStorageAccountName: '<ton-storage-account>'
  backendAzureRmContainerName: 'tfstate'
  backendAzureRmKey: 'terraform.tfstate'
  environment: 'production'

stages:
```

YAML

```

- stage: validate
jobs:
- job: validate
  steps:
- task: TerraformInstaller@1
  inputs: { terraformVersion: 'latest' }
- task: TerraformTaskV4@4
  displayName: init
  inputs:
    provider: azurerem
    command: init
    workingDirectory: '$(System.DefaultWorkingDirectory)/terraform'
    backendServiceArm: $(backendServiceArm)
    backendAzureRmResourceGroupName: $(backendAzureRmResourceGroupName)
    backendAzureRmStorageAccountName: $(backendAzureRmStorageAccountName)
    backendAzureRmContainerName: $(backendAzureRmContainerName)
    backendAzureRmKey: $(backendAzureRmKey)
- task: TerraformTaskV4@4
  displayName: validate
  inputs: { provider: azurerem, command: validate, workingDirectory: '$(System.DefaultWorkingDirectory)/terraform' }

- stage: plan
dependsOn: validate
jobs:
- job: plan
  steps:
- task: TerraformInstaller@1
  inputs: { terraformVersion: 'latest' }
- task: TerraformTaskV4@4
  displayName: init
  inputs: { provider: azurerem, command: init, workingDirectory: '$(System.DefaultWorkingDirectory)/terraform',
    backendServiceArm: $(backendServiceArm), backendAzureRmResourceGroupName: $(backendAzureRmResourceGroupName),
    backendAzureRmStorageAccountName: $(backendAzureRmStorageAccountName),
    backendAzureRmContainerName: $(backendAzureRmContainerName), backendAzureRmKey: $(backendAzureRmKey) }
- task: TerraformTaskV4@4
  displayName: plan
  inputs:
    provider: azurerem
    command: plan
    commandOptions: '-input=false -var-file=../vars/$(environment).tfvars'
    environmentServiceNameAzureRM: $(backendServiceArm)
    workingDirectory: '$(System.DefaultWorkingDirectory)/terraform'

- stage: apply
dependsOn: plan
jobs:
- job: apply
  steps:
# (mêmes étapes installer + init, puis :)
- task: TerraformTaskV4@4
  displayName: apply
  inputs:
    provider: azurerem
    command: apply
    commandOptions: '-input=false -auto-approve -var-file=../vars/$(environment).tfvars'
    environmentServiceNameAzureRM: $(backendServiceArm)
    workingDirectory: '$(System.DefaultWorkingDirectory)/terraform'

```

Pipelines → New pipeline → Azure Repos Git → ton dépôt → Existing Azure Pipelines YAML file → choisis le fichier → Run.

TROIS STAGES : VALIDATE, PLAN, APPLY

validate vérifie la syntaxe (rapide, échoue tôt). **plan** calcule et affiche ce qui va changer, sans rien toucher. **apply** applique. En production, on insère une **approbation manuelle** entre plan et apply (via un *Environment* Azure DevOps avec un *check* d'approbation) : un humain relit le plan avant que l'infrastructure soit modifiée.

VÉRIFICATION

Dans le portail Azure, tu trouves les groupes de ressources créés, avec le VNet, l'ACR, le cluster AKS et l'espace Log Analytics. Dans le conteneur `tfstate`, le fichier `terraform.tfstate` est apparu.

Lab 3 — Construire l'application et la pousser dans ACR

1. *Project settings* → *Service connections* → *New* → *Docker Registry* → *Azure Container Registry* → choisis ton ACR → nom : le nom DNS de l'ACR (ex. `devopsjourneyacr.azurecr.io`).

2. Copie le dossier de l'application `aspnet-core-dotnet-core` (lab 3) dans ton dépôt.
3. Teste-la en local si tu as Docker : `docker build -t devopsjourneyapp . && docker run -p 8080:80 devopsjourneyapp` → `http://localhost:8080`.
4. Ajoute le stage de build au pipeline :

```
variables:
  # ... (variables précédentes)
  repository: 'aspnet'
  dockerfile: '$(Build.SourcesDirectory)/aspnet-core-dotnet-core/Dockerfile'
  containerRegistry: 'devopsjourneyacr.azurecr.io' # nom de la service connection Docker

- stage: build
  dependsOn: apply
  jobs:
  - job: build
    steps:
    - task: Docker@2
      displayName: Build et push vers ACR
      inputs:
        command: buildAndPush
        repository: $(repository)
        dockerfile: $(dockerfile)
        containerRegistry: $(containerRegistry)
        tags: $(Build.BuildId)
```

Après exécution : portail → ACR → *Repositories* → `aspnet` → un tag par build.

Lab 4 — Déployer dans AKS

4.1 — Autoriser AKS à télécharger depuis ACR. Ajoute au `main.tf` (le nom exact des sorties dépend du module) :

```
resource "azurerm_role_assignment" "aks_acr_pull" {
  scope           = module.acr.acr_id           # le registre
  role_definition_name = "AcrPull"
  principal_id    = module.aks.kubelet_object_id # l'identité des nœuds
}
```

POURQUOI L'IDENTITÉ « KUBELET »

Ce sont les **nœuds** du cluster (via le *kubelet*, l'agent Kubernetes de chaque nœud) qui téléchargent les images. On donne donc le rôle *AcrPull* (lecture seule) à **leur** identité managée, sur le registre : moindre privilège, et aucun mot de passe de registre à gérer (contrairement au `imagePullSecret` JFrog du projet 06).

4.2 — Application Insights et Key Vault. Ajoute les modules `appinsights` et `keyvault` (dossier `labs/4-Deploy-App-AKS/terraform/modules` du dépôt maintenu), relance le pipeline, puis range la clé d'Application Insights dans le Key Vault :

```
az extension add --name application-insights
AIKEY=$(az monitor app-insights component show --app devopsjourney -g devopsjourney-rg --query instrumentationKey -o tsv)
az keyvault secret set --vault-name "<ton-keyvault>" --name "AIKEY" --value "$AIKEY"
```

Puis dans Azure DevOps : *Pipelines* → *Library* → + *Variable group* → `devopsjourney` → **Link secrets from an Azure key vault** → choisis le coffre → ajoute `AIKEY`. Référence le groupe dans le pipeline (`variables: - group: devopsjourney`).

LE VARIABLE GROUP LIÉ À KEY VAULT

Le secret ne vit **que** dans Key Vault. Le pipeline le lit au moment de l'exécution ; il est masqué dans les logs (`***`). Si on change la valeur dans Key Vault, tous les pipelines utilisent la nouvelle automatiquement. C'est le même principe que les Credentials Jenkins ou les Secrets GitHub, mais centralisé.

4.3 — Le manifest de l'application (`pipelines/scripts/aspnet.yaml`), corrigé pour les versions actuelles de Kubernetes :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: aspnetcore
spec:
  replicas: 1
  selector:
    matchLabels: { app: aspnetcore }
  template:
    metadata:
      labels: { app: aspnetcore }
    spec:
      containers:
      - name: aspnetcore
        image: devopsjourneyacr.azurecr.io/aspnet:IMAGE_TAG # remplacé par le pipeline
```

```

    ports: [{ containerPort: 80 }]
    resources:
      requests: { cpu: 100m, memory: 128Mi }
      limits:   { cpu: 250m, memory: 256Mi }
    env:
      - name: APPINSIGHTS_INSTRUMENTATIONKEY
        valueFrom: { secretKeyRef: { name: aikey, key: aisecret } }
---
apiVersion: v1
kind: Service
metadata:
  name: aspnetcore
spec:
  type: LoadBalancer      # Azure crée une IP publique
  ports: [{ port: 80 }]
  selector: { app: aspnetcore }

```

⚠ L'INGRESS DU DÉPÔT D'ORIGINE NE FONCTIONNE PLUS

Le fichier d'origine déclare un `Ingress` en `apiVersion: extensions/v1beta1`, **supprimée depuis Kubernetes 1.22**, et dépend de l'Application Gateway (coûteuse). On s'en passe : le `Service` de type `LoadBalancer` suffit pour exposer l'application. (Si tu veux un `Ingress` : `apiVersion: networking.k8s.io/v1` et l'add-on **Application Routing** d'AKS, basé sur NGINX.)

4.4 — Le stage de déploiement :

```

- stage: deploy
  dependsOn: build
  jobs:
    - job: deploy
      steps:
        - task: AzureCLI@2
          displayName: Déployer sur AKS
          inputs:
            azureSubscription: $(backendServiceArm)
            scriptType: bash
            scriptLocation: inlineScript
            inlineScript: |
              set -e
              az aks get-credentials -g devopsjourneyaks-rg -n devopsjourneyaks --admin
              kubectl create secret generic aikey --from-literal=aisecret="$(AIKEY)" \
                --dry-run=client -o yaml | kubectl apply -f -
              sed -i "s/IMAGE_TAG/$(Build.BuildId)/" pipelines/scripts/aspnet.yaml
              kubectl apply -f pipelines/scripts/aspnet.yaml
              kubectl rollout status deployment/aspnetcore --timeout=180s

```

💡 '--DRY-RUN=CLIENT -O YAML | KUBECTL APPLY -F -'

`kubectl create secret` échoue si le secret existe déjà (deuxième exécution du pipeline). Cette astuce génère le YAML du secret puis l'**applique**, ce qui crée **ou** met à jour : la commande devient rejouable. Même philosophie que `docker rm -f ... || true` au projet 05.

✅ VÉRIFICATION

`kubectl get svc aspnetcore` affiche une `EXTERNAL-IP` après une à deux minutes : ouvre-la dans le navigateur.
`kubectl describe pod -l app=aspnetcore | grep Image` montre le tag du dernier build.

Lab 5 — Le vrai CI/CD : déclenchement automatique

Remplace `trigger: none` par :

```

trigger:
  batch: true      # si plusieurs commits arrivent pendant un run, un seul run suivant les regroupe
  branches:
    include: [ main ]

```

Modifie la page d'accueil de l'application (`Views/Home/Index.cshtml` ou équivalent), commit sur `main` : le pipeline démarre seul, construit une nouvelle image et la déploie. **C'est le cœur du DevOps : un commit = une mise en production contrôlée.**

Lab 6 — Tester l'infrastructure avec InSpec

InSpec (Chef/Progress) permet d'écrire des **tests** sur l'infrastructure réelle : « ce Key Vault existe », « ce cluster AKS a bien 1 nœud », « ce groupe de ressources est en France Central ».

```
# azure-inspec-tests/controls/azure_aks.rb
control 'aks-cluster' do
  describe azure_aks_cluster(resource_group: 'devopsjourneyaks-rg', name: 'devopsjourneyaks') do
    it { should exist }
    its('properties.provisioningState') { should cmp 'Succeeded' }
  end
end
```

RUBY

Le dossier [Labs/6-Testing-Infrastructure](#) fournit le profil et le stage de pipeline (qui lit les identifiants nécessaires dans Key Vault). Exécute-le, puis consulte l'onglet *Tests* du run Azure DevOps.

TESTER L'INFRASTRUCTURE

On teste le code applicatif ; pourquoi pas l'infrastructure ? Ces tests détectent les écarts (*drift*) entre ce qui est voulu et ce qui existe, et servent de documentation exécutable. Alternatives modernes : **Terratest** (Go), **terraform test** (intégré à Terraform depuis la 1.6), **Checkov** (analyse statique de sécurité).

Lab 7 — Monitoring et alertes

Dans le portail Azure :

- **Application Insights** (ressource [devopsjourney](#)) : *Live Metrics* (requêtes en temps réel), *Failures*, *Application map*, *Transaction search*.
- **Availability test** : *Availability* → *Add Standard test* → l'URL publique de l'application, testée toutes les 5 minutes depuis plusieurs régions, avec alerte si elle ne répond plus.
- **Container Insights** : cluster AKS → *Insights* → CPU/mémoire par nœud, par pod, logs des conteneurs (requêtes KQL dans Log Analytics).

```
// Log Analytics : les 50 dernières lignes de log de l'application
ContainerLogV2
| where PodName startswith "aspnetcore"
| project TimeGenerated, LogMessage
| order by TimeGenerated desc
| take 50
```

KUSTO

Dépannage

DÉPANNAGE

- **No hosted parallelism has been purchased or granted** : voir l'encadré en début de projet.
- **AuthorizationFailed sur Microsoft.Authorization/roleAssignments/write** : l'identité de la service connection n'a pas le droit d'attribuer des rôles (étape 1.2).
- **Terraform : Error acquiring the state lock** : un run précédent a été interrompu. Dans le portail, sur le blob [terraform.tfstate](#), *Break lease*. Ne le fais que si aucun autre run n'est en cours.
- **Nom d'ACR ou de storage déjà pris** : ils doivent être uniques au monde ; ajoute un suffixe.
- **Pod en ImagePullBackOff** : le rôle [AcrPull](#) n'est pas attribué (ou pas encore propagé : attends quelques minutes).
- **Quota de vCPU dépassé** : les abonnements d'essai ont des quotas faibles ; choisis une taille de VM plus petite ou une autre région.

Nettoyage

NETTOYAGE OBLIGATOIRE

1. Le plus simple : supprimer les **groupes de ressources** créés par Terraform (AKS, ACR, VNet, Log Analytics...). Le groupe [...-node-rg](#) (ou [MC_...](#)) des nœuds AKS est supprimé avec le cluster.

```
az group list -o table
az group delete -n <groupe> --yes --no-wait
```

BASH

(Ou mieux : ajoute un pipeline [terraform destroy](#).)

2. Supprime en dernier [devops-journey-rg](#) (le stockage de l'état).
3. Supprime le groupe Entra [devopsthehardway-aks-group](#) si tu ne le réutilises pas.
4. Vérifie *Cost Management* le lendemain.

QUESTIONS D'ENTRETIEN

- Comment stockes-tu l'état Terraform dans une équipe ? Pourquoi un verrou (*lock*) ?
- Comment un cluster AKS télécharge-t-il des images depuis un ACR privé sans mot de passe ?
- Comment injectes-tu un secret dans un pipeline Azure DevOps ?
- Qu'est-ce que Workload Identity Federation et pourquoi est-ce mieux qu'un secret de service principal ?

Projet 10 — Pipeline CI/CD pour une application .NET sur Azure

FICHE PROJET

Niveau	★☆☆☆ Débutant+
Durée	3 à 5 heures
Coût	gratuit avec l'App Service plan F1 (ou quelques centimes en B1)
Compétences	Azure App Service, pipeline YAML multi-stages (build, test, publish, deploy), artefacts de pipeline, environnements et approbations, Application Insights
Prérequis	Projet 07 (organisation Azure DevOps et service connection)
Dépôt d'origine	DevOps-Project-10/README.md (labs « DevOps Starter »)

DEVOPS STARTER N'EXISTE PLUS

Le lab d'origine utilise **Azure DevOps Starter**, un assistant du portail Azure qui générerait automatiquement un dépôt, un pipeline de build, un pipeline de release et une Web App. **Microsoft l'a retiré**. Nous allons construire exactement le même résultat à la main. C'est beaucoup plus formateur : tu sauras ce que faisait l'assistant, et tu sauras le refaire n'importe où.

Ce qu'on construit

```
Azure Repos (code ASP.NET Core) —commit→ Azure Pipelines
├─ Stage BUILD : restore → build → test → publish → artefact "drop"
└─ Stage DEPLOY (environnement "production", approbation optionnelle)
    └─ Azure App Service (Web App Linux) → https://<app>.azurewebsites.net
        └─ Application Insights (monitoring)
```

Ce que faisait DevOps Starter	Ce que nous faisons
Crée un dépôt avec une appli exemple	<code>dotnet new webapp</code> + push dans Azure Repos
Crée un pipeline de build (<i>classic</i>)	stage <code>Build</code> dans <code>azure-pipelines.yml</code>
Crée un pipeline de release (<i>classic</i>) avec 5 tâches	stage <code>Deploy</code> dans le même YAML
Crée la Web App (et une base Azure SQL)	<code>az webapp create</code> (la base SQL est optionnelle ici)
Branche Application Insights	<code>az monitor app-insights component create</code> + paramètre d'application

YAML PLUTÔT QUE LES PIPELINES « CLASSIC »

Les pipelines *classic* (build et release configurés par clics dans l'interface) ne sont **pas versionnés avec le code** : impossible de relire un changement dans une pull request ou de revenir en arrière. Microsoft recommande désormais les pipelines **YAML multi-stages**, qui font build **et** release dans un seul fichier versionné.

Étape 1 — Créer l'application .NET

```
# SDK .NET sur Ubuntu/WSL (version LTS courante, par exemple 8.0 ou 10.0)
sudo apt update && sudo apt install -y dotnet-sdk-8.0
dotnet --version

mkdir io-dotnet && cd io-dotnet
dotnet new sln -n IoDotnet
dotnet new webapp -n IoDotnet.Web          # application ASP.NET Core Razor Pages
dotnet new xunit -n IoDotnet.Tests        # projet de tests unitaires
dotnet sln add IoDotnet.Web IoDotnet.Tests
dotnet add IoDotnet.Tests reference IoDotnet.Web
dotnet new gitignore

dotnet build && dotnet test
dotnet run --project IoDotnet.Web          # http://localhost:5xxx
```

BASH

Dans Azure DevOps : *Repos* → *Files* → ton projet propose une URL de clonage. Ajoute-la comme remote et pousse :

```
git init && git add . && git commit -m "Application ASP.NET Core initiale"
git remote add origin https://dev.azure.com/<org>/<projet>/_git/io-dotnet
```

BASH

```
git push -u origin main # authentication : bouton "Generate Git Credentials" dans Azure Repos
```

Étape 2 — Créer la Web App et Application Insights

```
BASH
RG=io-dotnet-rg
APP=io-dotnet-$RANDOM # nom unique : il devient <APP>.azurewebsites.net
az group create -n $RG -l francecentral
az appservice plan create -n io-dotnet-plan -g $RG --is-linux --sku F1 # F1 = gratuit
az webapp create -n $APP -g $RG -p io-dotnet-plan --runtime "DOTNETCORE:8.0"

az extension add --name application-insights
az monitor app-insights component create --app $APP-ai -g $RG -l francecentral --kind web
CONN=$(az monitor app-insights component show --app $APP-ai -g $RG --query connectionString -o tsv)
az webapp config appsettings set -n $APP -g $RG --settings APPLICATIONINSIGHTS_CONNECTION_STRING="$CONN"
echo "https://$APP.azurewebsites.net"
```

🧠 UN APP SERVICE, C'EST DU PAAS

Tu ne gères ni VM, ni système d'exploitation, ni serveur web : tu fournis ton application compilée, Azure s'occupe du reste (mises à jour, certificats HTTPS *.azurewebsites.net, mise à l'échelle). C'est l'opposé du projet 01, où tu gérerais tout. Contrepartie : moins de contrôle.

📄 LA BASE AZURE SQL DU LAB D'ORIGINE

DevOps Starter créait aussi une base Azure SQL. Notre application n'en a pas besoin ; si tu veux t'entraîner : `az sql server create + az sql db create --service-objective Basic`, puis une *connection string* dans les paramètres de la Web App. Pense à la supprimer ensuite (c'est elle qui coûte).

Étape 3 — Le pipeline multi-stages

À la racine du dépôt, `azure-pipelines.yml` :

```
YAML
trigger:
  branches:
    include: [ main ]

pool:
  vmImage: ubuntu-latest

variables:
  buildConfiguration: Release
  azureSubscription: 'azure-devops-journey-service-connection' # service connection du projet 07
  webAppName: '<APP>'

stages:
- stage: Build
  jobs:
  - job: Build
    steps:
    - task: UseDotNet@2
      inputs: { packageType: sdk, version: '8.0.x' }

    - script: dotnet restore
      displayName: Restaurer les dépendances

    - script: dotnet build --configuration $(buildConfiguration) --no-restore
      displayName: Compiler

    - task: DotNetCoreCLI@2
      displayName: Tests unitaires
      inputs:
        command: test
        projects: '**/*Tests.csproj'
        arguments: '--configuration $(buildConfiguration) --collect "XPlat Code Coverage"'
        publishTestResults: true # résultats visibles dans l'onglet "Tests"

    - task: DotNetCoreCLI@2
      displayName: Publier l'application
      inputs:
        command: publish
        publishWebProjects: true
        arguments: '--configuration $(buildConfiguration) --output $(Build.ArtifactStagingDirectory)'
        zipAfterPublish: true
```

```

- publish: $(Build.ArtifactStagingDirectory)
  artifact: drop # L'artefact que le stage Deploy récupérera

- stage: Deploy
  dependsOn: Build
  condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'))
  jobs:
    - deployment: DeployWeb
      environment: production # crée l'environnement "production" (historique, approbations)
      strategy:
        runOnce:
          deploy:
            steps:
              - task: AzureWebApp@1
                inputs:
                  azureSubscription: $(azureSubscription)
                  appType: webAppLinux
                  appName: $(webAppName)
                  package: $(Pipeline.Workspace)/drop/**/*.*.zip

```

Pipelines → New pipeline → Azure Repos Git → io-dotnet → Existing YAML file → Run.

🧠 'JOB' VS 'DEPLOYMENT'

Un `deployment` est un job spécial lié à un **environnement**. Azure DevOps garde l'historique de ce qui a été déployé où (quel commit est en production ?), et on peut y attacher des **contrôles** : approbation humaine, plage horaire autorisée, vérification qu'une alerte n'est pas en cours... C'est l'équivalent du « release pipeline » classique, en mieux.

Étape 4 — Examiner ce que fait le pipeline

Reprends les questions du lab d'origine, mais sur ton pipeline :

1. **Le déclencheur** : `trigger` sur `main` = chaque commit lance le pipeline (intégration continue).
2. **Les tâches de build** : restaurer, compiler, tester, publier, puis produire l'artefact `drop` (onglet *Artifacts* d'un run → tu peux télécharger le zip).
3. **L'historique** : *Pipelines* → *ton pipeline* → *Runs*, et dans *Environments* → *production* la liste des déploiements avec leur commit.
4. **Les logs** : clique sur chaque étape d'un run.

Ajouter une approbation : *Pipelines* → *Environments* → *production* → `:` → *Approvals and checks* → *Approvals* → ajoute-toi. Le prochain run s'arrêtera avant le déploiement et attendra ton clic.

Étape 5 — Commit et livraison continue

Dans Azure Repos, édite `IoDotnet.Web/Pages/Index.cshtml` (bouton *Edit*) : change le titre `<h1>` en « Get started right away with Azure DevOps ». *Commit* sur `main`.

✅ VÉRIFICATION

- Le pipeline démarre seul ; les stages *Build* puis *Deploy* passent au vert (après ton approbation si tu l'as activée).
- `https://<APP>.azurewebsites.net` affiche le nouveau titre.
- Onglet *Tests* du run : tous les tests passent.

Bonus — un test qui casse le pipeline. Dans le projet de tests, ajoute un test qui échoue (`Assert.True(false)`) et pousse : le stage *Build* échoue, et **rien n'est déployé**. C'est exactement le rôle de l'intégration continue.

Étape 6 — Application Insights

Portail → Application Insights `<APP>-ai` :

- **Live metrics** : recharge ton site plusieurs fois, observe les requêtes en direct.
- **Performance** : temps de réponse par page.
- **Failures** : ajoute une page qui lève une exception, déploie, appelle-la : l'erreur apparaît avec sa pile d'appels.
- **Availability** : *Add Standard test* sur l'URL du site.

(Pour une télémétrie riche côté code, ajoute le paquet `Microsoft.ApplicationInsights.AspNetCore` ou `OpenTelemetry`. Même sans, l'App Service remonte les métriques de base via la connection string.)

Dépannage

🔧 DÉPANNAGE

- **Deploy :** `Resource doesn't exist` : `webAppName` ne correspond pas, ou la service connection n'a pas accès au groupe de ressources.
- **Le site affiche « Your web app is running and waiting for your content »** : le zip déployé ne contient pas le projet web (vérifie `publishWebProjects` et le contenu de l'artefact `drop`).
- **Erreur 500.30 / « HTTP Error 500.31 »** : version du runtime de la Web App différente de celle de l'application (`--runtime "DOTNETCORE:8.0"` vs `net8.0` dans le `.csproj`).
- **Le plan F1 refuse** : une seule Web App F1 Linux par région et par abonnement ; passe en B1 ou change de région.

Nettoyage

🧹 NETTOYAGE OBLIGATOIRE

```
az group delete -n io-dotnet-rg --yes --no-wait
```

BASH

Le pipeline et le dépôt Azure DevOps sont gratuits : tu peux les garder.

🔑 QUESTIONS D'ENTRETIEN

- Quelle différence entre un pipeline *classic* et un pipeline YAML ?
- Comment empêches-tu un déploiement en production sans validation humaine ? (*Environnement + check d'approbation.*)
- Qu'est-ce qu'un artefact de pipeline et pourquoi le stage Deploy ne recompile-t-il pas ?
- PaaS (App Service) ou conteneurs (AKS) : comment choisis-tu ?

Projet 14 — Pipeline CI/CD de bout en bout avec GitHub Actions pour une application Android

FICHE PROJET

Niveau	★☆☆☆☆ Débutant+
Durée	4 à 6 heures
Coût	gratuit (2 000 minutes/mois de GitHub Actions sur les dépôts privés, illimité sur les dépôts publics)
Compétences	GitHub Actions (workflows, jobs, steps, triggers, secrets, cache, artefacts, outputs, conditions par branche), Gradle, lint, tests et couverture, SonarQube/SonarCloud, branches par environnement, OIDC vers AWS
Prérequis	Leçons 4 et 6
Dépôt d'origine	DevOps-Project-14/android-demo-app (avec <code>.github/workflows/android.yml</code>)

Ce qu'on construit

Une usine logicielle pour une application mobile Android, intégralement dans GitHub :



GitHub Actions : le vocabulaire

Notion	Définition
Workflow	un fichier YAML dans <code>.github/workflows/</code> , déclenché par des événements
Event (on:)	ce qui déclenche : <code>push</code> , <code>pull_request</code> , <code>schedule</code> , <code>workflow_dispatch</code> (bouton manuel)...
Job	un ensemble d'étapes exécutées sur une machine (runner). Les jobs tournent en parallèle , sauf si <code>needs</code> impose un ordre
Step	une commande (<code>run:</code>) ou une action réutilisable (<code>uses:</code>)
Action	un composant prêt à l'emploi, publié sur le Marketplace : <code>actions/checkout</code> , <code>actions/setup-java</code> ...
Runner	la machine qui exécute le job : hébergée par GitHub (<code>ubuntu-latest</code>) ou auto-hébergée
Secret	une valeur chiffrée (<code>Settings</code> → <code>Secrets and variables</code> → <code>Actions</code>), lue avec <code>\${{ secrets.NOM }}</code> , masquée dans les logs
Artifact	un fichier produit par un job, conservé et téléchargeable, transmis à un autre job
Expression	<code>\${{ ... }}</code> : accès aux contextes (<code>github.ref</code> , <code>secrets</code> , <code>steps.<id>.outputs</code> , <code>needs.<job>.outputs</code>)

🔥 CHAQUE JOB DÉMARRE SUR UNE MACHINE VIERGE

Le job `deploy` ne voit **pas** les fichiers du job `build` : ce sont deux machines différentes, effacées après usage. Pour transmettre les APK, on les **téléverse** comme artefact dans `build` et on les **télécharge** dans `deploy`. Pour transmettre une simple valeur (la date), on utilise les **outputs** du job.

Étape 1 — Préparer le dépôt

1. Crée un dépôt GitHub `android-demo-app` et copies-y le contenu de `DevOps-Project-14/android-demo-app`.
2. Crée les branches d'environnement : `git switch -c develop` & `git push -u origin develop`, puis `qa`. Sur GitHub, mets `main` comme branche par défaut.

⚠ UN PROJET ANDROID DE 2021

Le projet utilise l'Android Gradle Plugin 7.0 (bêta) et Gradle 7.0.2, qui exigent **Java 11** : on garde donc JDK 11 dans le workflow. Si le build échoue à cause d'un dépôt de dépendances disparu (`jcenter()` est fermé), remplace `jcenter()` par `mavenCentral()` dans `build.gradle`. Alternative : crée un projet neuf avec **Android Studio** (modèle *Empty Activity*), qui utilisera des versions actuelles (JDK 17), et applique-lui le même workflow.

Étape 2 — Le workflow, modernisé et commenté

Le workflow d'origine utilise des versions d'actions obsolètes (`checkout@v2/v3`, `cache@v1`, `upload-artifact@v3` qui ne fonctionnent plus) et la commande `::set-output`, supprimée par GitHub. Voici la version à jour, `.github/workflows/android.yml` :

```
name: Android CI/CD

on:
  push:
    branches: [ main, qa, develop ]
  pull_request:
    branches: [ main, qa ]
  workflow_dispatch:           # bouton "Run workflow" dans l'onglet Actions

permissions:
  contents: read
  id-token: write              # nécessaire pour l'authentification OIDC vers AWS (étape 5)

jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      build_date: ${ steps.date.outputs.build_date }

    steps:
      - name: Récupérer le code
        uses: actions/checkout@v4
        with:
          fetch-depth: 0        # historique complet : utile à SonarCloud

      - name: Installer JDK 11 (avec cache Gradle intégré)
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '11'
          cache: gradle

      - name: Rendre gradlew exécutable
        run: chmod +x gradlew

      - name: Lint
        run: ./gradlew lint

      - name: Build (APK debug et release)
        run: ./gradlew build

      - name: Tests et couverture Jacoco
        run: ./gradlew jacocoTest      # tâche définie dans le projet (scripts/*.gradle)

      - name: Analyse SonarCloud
        if: github.event_name != 'pull_request' || github.event.pull_request.head.repo.full_name == github.repository
        env:
          SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
          SONAR_HOST_URL: https://sonarcloud.io
        run: ./gradlew sonarqube -Dsonar.projectKey=${ vars.SONAR_PROJECT_KEY } -Dsonar.organization=${ vars.SONAR_ORG }

      - name: Date du build
        id: date
        run: echo "build_date=$(date +%Y%m%d-%H%M%S)" >> "$GITHUB_OUTPUT"

      - name: Renommer les APK avec la date
        run: |
          mkdir -p apk-files/debug apk-files/release
          cp app/build/outputs/apk/debug/app-debug.apk \
            apk-files/debug/app-debug-${ steps.date.outputs.build_date }.apk
          cp app/build/outputs/apk/release/app-release-unsigned.apk \
            apk-files/release/app-release-unsigned-${ steps.date.outputs.build_date }.apk
```

```

- name: Publier les APK comme artefact
  uses: actions/upload-artifact@v4
  with:
    name: apk-files
    path: apk-files/
    retention-days: 7

deploy:
  needs: build
  if: github.ref == 'refs/heads/qa' || github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  environment: ${github.ref_name} # environnements GitHub "qa" et "main" (règles de protection possibles)

steps:
- name: Récupérer les APK
  uses: actions/download-artifact@v4
  with:
    name: apk-files
    path: apk-files/

- name: Afficher les fichiers
  run: ls -R apk-files

- name: S'authentifier auprès d'AWS (OIDC, sans clé)
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: ${secrets.AWS_ROLE_ARN}
    aws-region: eu-west-3

- name: Publier dans S3, dans le dossier de l'environnement
  run: |
    aws s3 cp apk-files/ \
      s3://${vars.APK_BUCKET}/${github.ref_name}/${needs.build.outputs.build_date}/ --recursive

```

🔑 LES POINTS CLÉS À COMPRENDRE

- **>> "\$GITHUB_OUTPUT"** : la façon actuelle de définir une sortie d'étape (remplace `::set-output`). Elle est lue ensuite avec `steps.date.outputs.build_date`, et exposée au job suivant via `outputs: + needs.build.outputs.build_date`.
- **if: sur le job deploy** : on ne publie que pour `qa` et `main`. Une branche `develop` ou une PR s'arrête après le build. C'est la stratégie « une branche = un environnement ».
- **cache: gradle** : Gradle télécharge des centaines de Mo de dépendances ; le cache fait passer le build de 8 à 3 minutes.
- **vars. vs secrets.** : les *variables* (non sensibles, visibles) et les *secrets* (chiffrés, masqués) se configurent au même endroit dans les paramètres du dépôt.
- **La condition sur SonarCloud** : les PR venant d'un *fork* n'ont pas accès aux secrets (sécurité) ; on saute l'analyse dans ce cas plutôt que de faire échouer le workflow.

Étape 3 — SonarCloud

1. sonarcloud.io → importe le dépôt GitHub → méthode d'analyse *GitHub Actions* (désactive l'*Automatic Analysis*, sinon conflit).
2. Dépôt GitHub → *Settings* → *Secrets and variables* → *Actions* :
 - secret `SONAR_TOKEN` ;
 - variables `SONAR_ORG` et `SONAR_PROJECT_KEY`.

Étape 4 — Où publier les APK ?

Le dépôt d'origine publie dans **JFrog Artifactory** hébergé sur une EC2, et ouvre **temporairement** le Security Group à l'IP du runner GitHub avant l'envoi. C'est une technique intéressante (les runners GitHub changent d'IP à chaque exécution), mais elle impose des clés AWS à longue durée de vie dans les secrets. Deux options plus simples :

- **S3** (ci-dessus), avec authentification **OIDC** : aucun secret AWS permanent.
- **JFrog Cloud** : remplace les deux dernières étapes par l'action `jfrog/setup-jfrog-cli@v4` (secrets `JF_URL`, `JF_ACCESS_TOKEN`) puis `jfrog rt upload "apk-files/*" android-artifact/${github.ref_name}`.

Étape 5 — L'authentification OIDC entre GitHub et AWS

🔑 POURQUOI OIDC

Stocker `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` dans GitHub, c'est risquer une fuite de clés permanentes. Avec **OIDC**, GitHub fournit au workflow un jeton signé qui dit « je suis le workflow du dépôt X, branche Y ». AWS le vérifie et donne en échange des identifiants **temporaires** (1 heure) pour un rôle précis. Rien à stocker, rien à faire tourner.

1. IAM → *Identity providers* → *Add provider* → **OpenID Connect** → URL `https://token.actions.githubusercontent.com`, audience `sts.amazonaws.com`.

2. IAM → Roles → Create role → Web identity → ce provider → audience `sts.amazonaws.com` → GitHub organization : ton pseudo, repository : `android-demo-app`. Attache une politique autorisant `s3:PutObject` sur `arn:aws:s3:::<bucket>/*`. Nom : `github-android-deploy`.
3. Crée le bucket (`aws s3 mb s3://io-14-apk-<prenom>`), puis dans GitHub : secret `AWS_ROLE_ARN` (ARN du rôle) et variable `APK_BUCKET`.

La politique de confiance générée ressemble à ceci (vérifie la condition `sub`, elle limite l'accès à ton dépôt) :

```
"Condition": {
  "StringEquals": { "token.actions.githubusercontent.com:aud": "sts.amazonaws.com" },
  "StringLike":    { "token.actions.githubusercontent.com:sub": "repo:<toi>/android-demo-app:*" }
}
```

JSON

Étape 6 — Tester la stratégie de branches

```
git switch develop && echo "// test" >> app/build.gradle && git commit -am "test develop" && git push
# → onglet Actions : seul le job build tourne

git switch qa && git merge develop && git push
# → build puis deploy ; les APK arrivent dans s3://.../qa/<date>/
```

BASH

Ouvre aussi une **pull request** de `develop` vers `qa` : le workflow tourne sur la PR, et son résultat s'affiche dans la PR (coche verte ou croix rouge). Protège `main` et `qa` (*Settings* → *Branches*) en exigeant que le check `build` passe.

✓ VÉRIFICATION

- Onglet *Actions* : runs verts, le job `deploy` apparaît uniquement pour `qa` / `main`.
- Dans un run : section *Artifacts* → `apk-files` téléchargeable.
- SonarCloud : analyse du projet visible.
- S3 : `aws s3 ls s3://io-14-apk-<prenom>/ --recursive`.

Étape 7 (optionnelle) — Notifications

Le workflow d'origine envoie une carte dans **Microsoft Teams**. Plus universel : un *Incoming Webhook* Slack ou Teams, appelé avec `curl` dans une étape `if: always()` :

```
- name: Notification
  if: always()
  run: |
    curl -s -X POST -H 'Content-Type: application/json' \
      -d "{\"text\":\"${{ github.workflow }} sur ${{ github.ref_name }} : ${{ job.status }}\"}" \
      "${{ secrets.CHAT_WEBHOOK_URL }}"
```

YAML

Dépannage

🔧 DÉPANNAGE

- `Unsupported class file major version` ou erreur de plugin Gradle : mauvaise version de JDK (ici, 11).
- `Could not resolve ... jcenter` : remplace `jcenter()` par `mavenCentral()`.
- `Not authorized to perform sts:AssumeRoleWithWebIdentity` : condition `sub` de la politique de confiance qui ne correspond pas au dépôt/branche, ou permissions : `id-token: write` absent.
- **SonarCloud** : `You are running CI analysis while Automatic Analysis is enabled` : désactive l'analyse automatique dans SonarCloud.
- **Le job deploy ne se lance jamais** : la condition `if` compare `github.ref` à `refs/heads/qa` ; vérifie le nom exact de ta branche.

Nettoyage

🧹 NETTOYAGE OBLIGATOIRE

Supprime le bucket S3 (`aws s3 rb s3://io-14-apk-<prenom> --force`), le rôle IAM `github-android-deploy` et le provider OIDC si tu ne le réutilises pas (il resservira au projet 22). Les artefacts GitHub expirent seuls (7 jours ici).

🔧 QUESTIONS D'ENTRETIEN

- Comment deux jobs GitHub Actions s'échangent-ils des fichiers ? Et une valeur ?
- Comment limites-tu un déploiement à certaines branches ?
- Pourquoi préférer OIDC à des clés d'accès dans les secrets ?
- À quoi sert le cache dans un pipeline, et quel est son risque ? (*Gain de temps ; risque de cache corrompu ou périmé : la clé de cache doit dépendre des fichiers de dépendances.*)

PARTIE 3

Kubernetes & Infrastructure as Code

Kubernetes de A à Z, Terraform, Helm et GitOps avec Argo CD. Puis neuf projets : EKS, Terraform 2-tiers, Super Mario, e-commerce avec Helm, Jenkins + Argo CD, AKS, GitOps Java, EKS en Terraform et Azure DevOps + Terraform.

Leçon 9 — Kubernetes

Pourquoi Kubernetes ?

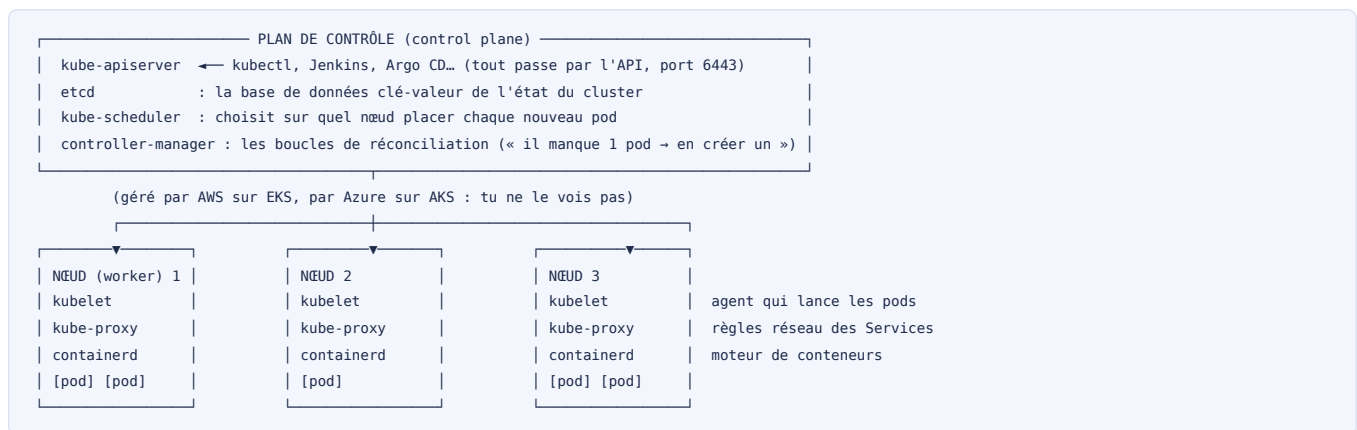
Docker lance des conteneurs sur **une** machine. En production, on a des dizaines de machines et des centaines de conteneurs. Qui décide sur quelle machine lancer chaque conteneur ? Qui le relance s'il plante ? Qui en ajoute quand le trafic augmente ? Qui met à jour sans coupure ? Qui répartit le trafic entre les copies ?

Kubernetes (abrégié **K8s**) est l'orchestrateur qui fait tout cela. Créé par Google, open source depuis 2014, c'est aujourd'hui le standard de fait. Tous les clouds le proposent en version managée : **EKS** (AWS), **AKS** (Azure), **GKE** (Google).

LE CHEF D'ORCHESTRE ET LA PARTITION

Tu ne dis pas à Kubernetes « lance un conteneur sur la machine 3 ». Tu lui donnes une **partition** (un fichier YAML) : « je veux 3 copies de mon application, chacune avec 256 Mo de mémoire, accessibles sur le port 80 ». Kubernetes s'occupe de faire jouer l'orchestre, et si un musicien s'arrête, il en fait entrer un autre. Tu décris l'**état désiré** ; Kubernetes ramène en permanence l'**état réel** vers l'état désiré. C'est la **boucle de réconciliation**.

L'architecture d'un cluster



Les objets à connaître

Pod — la plus petite unité : un ou plusieurs conteneurs qui partagent la même IP et peuvent partager des volumes. En pratique, presque toujours **un conteneur par pod**. Un pod est **éphémère** : s'il meurt, il n'est pas ressuscité, il est **remplacé** par un nouveau (avec une nouvelle IP).

Deployment — gère un ensemble de pods identiques (*replicas*) : il en maintient le nombre voulu et orchestre les **misés à jour progressives** (*rolling updates*) et les retours arrière. Il s'appuie sur un **ReplicaSet** qu'il crée pour toi. C'est l'objet que tu utiliseras 90 % du temps.

Service — une adresse **stable** (nom DNS + IP virtuelle) devant un groupe de pods choisis par **labels**. Les pods vont et viennent ; le Service reste.

Type de Service	Accessible depuis	Usage
ClusterIP (défaut)	l'intérieur du cluster uniquement	communication entre microservices (http://backend:8080)
NodePort	l'extérieur, via <code><IP d'un nœud>:<30000-32767></code>	tests, clusters sans load balancer
LoadBalancer	Internet, via un load balancer du cloud créé automatiquement	exposer une application (attention : un LB payant par Service)

Ingress — un point d'entrée HTTP unique qui route vers plusieurs Services selon le **nom de domaine** ou le **chemin** (`/api` → service api, `/` → service frontend). Nécessite un **Ingress Controller** (NGINX Ingress, AWS Load Balancer Controller qui crée un ALB...). Un seul load balancer pour toutes les applications : bien plus économique.

Namespace — un découpage logique du cluster (`dev`, `prod`, `monitoring`, `argocd` ...). Les noms doivent être uniques dans un namespace.

ConfigMap / Secret — la configuration (non sensible / sensible) injectée dans les pods comme variables d'environnement ou fichiers. Attention : un Secret est seulement **encodé en base64**, pas chiffré (à protéger avec le chiffrement etcd, des droits RBAC stricts, ou un gestionnaire externe).

Volumes, PersistentVolumeClaim (PVC), StorageClass — le stockage persistant. Un pod demande « 10 Go » (PVC) ; la StorageClass crée le disque (un volume EBS sur AWS, grâce au **pilote CSI EBS**).

StatefulSet — comme un Deployment, mais pour les applications avec état (bases de données) : noms stables (`mysql-0`, `mysql-1`) et un disque dédié par pod. **DaemonSet** : un pod sur **chaque** nœud (agents de logs, de monitoring). **Job / CronJob** : tâches ponctuelles ou planifiées.

HorizontalPodAutoscaler (HPA) — ajuste le nombre de replicas selon le CPU ou d'autres métriques.

RBAC, ServiceAccount — qui a le droit de faire quoi dans le cluster ; identité des pods.

Un manifest complet, commenté

YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  namespace: demo
  labels: { app: web }
spec:
  replicas: 3 # 3 copies
  selector:
    matchLabels: { app: web } # ce Deployment gère les pods portant app=web
  strategy:
    type: RollingUpdate
    rollingUpdate: { maxSurge: 1, maxUnavailable: 0 } # jamais moins de 3 pods prêts pendant une mise à jour
  template:
    metadata:
      labels: { app: web }
    spec:
      containers:
        - name: web
          image: nginx:1.27
          ports: [{ containerPort: 80 }]
          resources:
            requests: { cpu: 100m, memory: 128Mi } # réservé pour le placement (100m = 0,1 CPU)
            limits: { cpu: 500m, memory: 256Mi } # plafond (au-delà de la mémoire : le pod est tué, "OOMKilled")
          readinessProbe: # « suis-je prêt à recevoir du trafic ? »
            httpGet: { path: /, port: 80 }
            initialDelaySeconds: 5
          livenessProbe: # « suis-je encore vivant ? » sinon redémarrage
            httpGet: { path: /, port: 80 }
            periodSeconds: 10
          env:
            - name: APP_ENV
              valueFrom: { configMapKeyRef: { name: web-config, key: env } }
---
apiVersion: v1
kind: Service
metadata:
  name: web
  namespace: demo
spec:
  type: ClusterIP
  selector: { app: web } # envoie le trafic aux pods app=web
  ports:
    - port: 80 # port du Service
      targetPort: 80 # port du conteneur
---
apiVersion: v1
kind: ConfigMap
metadata: { name: web-config, namespace: demo }
data:
  env: production
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web
  namespace: demo
spec:
  ingressClassName: nginx # ou "alb" avec l'AWS Load Balancer Controller
  rules:
    - host: web.exemple.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend: { service: { name: web, port: { number: 80 } } }
```

LES LABELS ET SÉLECTEURS : LA COLLE DE KUBERNETES

Rien n'est relié par un nom ou une IP. Un Deployment trouve ses pods, un Service trouve ses cibles, **grâce aux labels** (`app: web`). Si le `selector` du Service ne correspond pas exactement aux labels des pods, le Service n'a aucune cible et ton application est injoignable, sans message d'erreur. C'est le bug n°1 des débutants : vérifie avec `kubectl get endpoints web`.

kubectl, l'outil de tous les jours

```
kubectl get nodes                # les nœuds
kubectl get pods -n demo -o wide # les pods (et sur quel nœud, avec quelle IP)
kubectl get all -n demo          # pods, services, deployments, replicasets
kubectl get pods -A              # dans tous les namespaces
kubectl apply -f fichier.yaml    # créer / mettre à jour (déclaratif, rejouable)
kubectl delete -f fichier.yaml   # supprimer ce que décrit le fichier
kubectl describe pod <pod> -n demo # DÉTAILS + ÉVÉNEMENTS (le premier réflexe en cas de problème)
kubectl logs <pod> -n demo [-f] [--previous] # logs (--previous : ceux du conteneur précédent qui a planté)
kubectl exec -it <pod> -n demo -- sh # shell dans le conteneur
kubectl port-forward svc/web 8080:80 -n demo # accéder à un Service depuis ton poste : http://localhost:8080
kubectl scale deploy/web --replicas=5 -n demo
kubectl set image deploy/web web/nginx:1.28 -n demo # mise à jour de l'image
kubectl rollout status deploy/web -n demo
kubectl rollout undo deploy/web -n demo # revenir à la version précédente
kubectl get events -n demo --sort-by=.lastTimestamp
kubectl config get-contexts / use-context # changer de cluster
kubectl explain deployment.spec.strategy # documentation intégrée des champs
```

BASH

💡 UN ALIAS QUI FAIT GAGNER DES HEURES

```
echo "alias k=kubectl" >> ~/.bashrc
echo 'source <(kubectl completion bash)' >> ~/.bashrc
echo 'complete -o default -F __start_kubectl k' >> ~/.bashrc
```

BASH

Et installe **k9s** (<https://k9scli.io>) : une interface en mode texte pour naviguer dans le cluster.

Lire l'état d'un pod

Statut	Signification	Quoi faire
Pending	pas encore placé sur un nœud	describe : pas assez de CPU/mémoire ? PVC non satisfait ?
ContainerCreating	téléchargement de l'image, montage des volumes	patienter ; sinon describe
Running	au moins un conteneur tourne	vérifier READY 1/1
ImagePullBackOff / ErrImagePull	impossible de télécharger l'image	nom/tag faux, registre privé sans imagePullSecret , droits
CrashLoopBackOff	le conteneur démarre puis plante en boucle	kubectl logs --previous : c'est l'application qui échoue
OOMKilled	dépassement de la limite mémoire	augmenter limits.memory ou corriger la fuite
READY 0/1 en Running	la readinessProbe échoue	vérifier le chemin/port de la probe

Où faire tourner Kubernetes pour apprendre ?

Option	Coût	Pour quoi
kind ou minikube sur ton poste	gratuit	apprendre les objets, tester des manifests (fortement recommandé avant d'aller sur EKS)
kubeadm sur des EC2 (annexe A.6)	quelques centimes/heure	comprendre l'installation d'un cluster (projets 09, 24, 25)
EKS (annexe A.8)	~0,10 \$/h + nœuds	le vrai service managé AWS (projets 06, 08, 12, 15, 16, 19, 27, 28, 30)
AKS	nœuds seulement (plan de contrôle gratuit en tier Free)	projets 07, 17, 29

```
# kind : un cluster Kubernetes dans des conteneurs Docker, en 1 minute
curl -Lo ./kind https://kind.sigs.k8s.io/dl/latest/kind-linux-amd64 && chmod +x kind && sudo mv kind /usr/local/bin/
kind create cluster --name labo
kubectl get nodes
kind delete cluster --name labo
```

BASH

Les spécificités d'EKS

- **Plan de contrôle** géré par AWS (0,10 \$/h). **Nœuds** : des *managed node groups* (EC2 gérées par AWS), ou **Fargate**, ou **Karpenter** (autoscaling intelligent).
- **Authentification** : ton identité IAM est traduite en identité Kubernetes. Méthode moderne : les **access entries** ([aws eks create-access-entry](#)) ; méthode historique : la ConfigMap [aws-auth](#). La commande [aws eks update-kubeconfig](#) écrit la configuration de [kubectl](#).
- **Réseau** : le plugin **VPC CNI** donne à chaque pod une vraie IP du VPC.

- **Droits IAM pour les pods** : **IRSA** (*IAM Roles for Service Accounts*, via un fournisseur OIDC) ou **EKS Pod Identity** (plus récent et plus simple). Indispensables pour l'AWS Load Balancer Controller et le pilote EBS CSI (projet 15).
- **Add-ons** : VPC CNI, CoreDNS, kube-proxy, EBS CSI driver, installables et mis à jour par AWS.

● LE COÛT D'UN CLUSTER EKS

Plan de contrôle 73 \$/mois + 2 nœuds [t3.medium](#) 60 \$/mois + NAT 35 \$/mois + chaque [LoadBalancer](#) / [Ingress](#) 18 \$/mois. **Un cluster d'apprentissage se crée en début de session et se détruit en fin de session.** Garde tes manifests et tes commandes dans Git pour tout recréer en 20 minutes.

Leçon 10 — Terraform

L'idée

Terraform (HashiCorp) décrit l'infrastructure dans des fichiers `.tf` écrits en **HCL**. On déclare **ce qu'on veut**, Terraform calcule **ce qu'il faut faire** pour y arriver (créer, modifier, supprimer) et le fait en appelant les API du cloud. Il fonctionne avec des centaines de fournisseurs (*providers*) : AWS, Azure, Google, Kubernetes, GitHub, Cloudflare...

(OpenTofu est un fork open source de Terraform, compatible ; tout ce qui suit s'y applique.)

Le cycle de travail

```
terraform init      # télécharge les providers et modules, configure le backend
terraform fmt       # formate les fichiers
terraform validate  # vérifie la syntaxe
terraform plan      # calcule et AFFICHE les changements (ne modifie rien)
terraform apply     # applique (demande confirmation)
terraform destroy   # détruit tout ce que gère cette configuration
terraform output    # affiche les sorties
terraform state list # liste les ressources suivies
```

BASH

💡 LIRE UN PLAN

Dans la sortie de `plan` : `+` création, `-` destruction, `~` modification sur place, `-/+` **destruction puis recréation** (attention : un serveur ou une base recréés = données perdues). Lis toujours le résumé final `Plan: 3 to add, 1 to change, 0 to destroy` avant de taper `yes`.

Les blocs du langage

```
# versions.tf - quels providers, quelles versions
terraform {
  required_version = "≥ 1.10"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" } # ~> 5.0 = toute 5.x
  }
}

# provider : comment se connecter
provider "aws" {
  region = var.region
}

# variable : un paramètre d'entrée
variable "region" {
  type    = string
  default = "eu-west-3"
}

variable "instance_count" {
  type        = number
  description = "Nombre de serveurs web"
  default     = 2
}

# data : LIRE quelque chose qui existe déjà
data "aws_availability_zones" "available" { state = "available" }

# resource : CRÉER quelque chose
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
  tags       = { Name = "demo-vpc" }
}

resource "aws_subnet" "public" {
  count          = 2                                # 2 subnets
  vpc_id        = aws_vpc.main.id                  # RÉFÉRENCE à une autre ressource
  cidr_block     = cidrsubnet("10.0.0.0/16", 8, count.index) # 10.0.0.0/24, 10.0.1.0/24
  availability_zone = data.aws_availability_zones.available.names[count.index]
}
```

HCL

```
# locals : des valeurs calculées, pour éviter les répétitions
locals {
  common_tags = { Project = "io", ManagedBy = "terraform" }
}

# output : une valeur à afficher ou à transmettre
output "vpc_id" { value = aws_vpc.main.id }
```

🐛 LE GRAPHE DE DÉPENDANCES

`vpc_id = aws_vpc.main.id` crée une **dépendance** : Terraform sait qu'il doit créer le VPC avant le subnet, et le détruire après. Il construit un graphe de toutes les ressources et crée en parallèle tout ce qui peut l'être. Tu n'écris jamais l'ordre ; seulement les liens. (`depends_on` existe pour les rares dépendances invisibles.)

Fournir les valeurs des variables : fichier `terraform.tfvars` (chargé automatiquement), `-var-file=prod.tfvars`, `-var="region=eu-west-1"`, ou variables d'environnement `TF_VAR_region`. Les variables sensibles se déclarent avec `sensitive = true` et ne se commitent jamais.

L'état (state)

Terraform enregistre dans `terraform.tfstate` la correspondance entre ton code et les ressources réelles (« `aws_vpc.main = vpc-0abc123` »). Sans état, il ne saurait pas ce qu'il a créé.

- Il contient souvent des **secrets en clair** (mots de passe de bases...) : ne jamais le commiter.
- En équipe ou en CI, il doit être **distant** (*remote backend*) et **verrouillé** pendant un `apply` pour empêcher deux modifications simultanées.

```
# backend.tf - état dans S3, verrouillage natif (Terraform ≥ 1.10)
terraform {
  backend "s3" {
    bucket      = "io-tfstate-<prenom>"
    key         = "projet-11/terraform.tfstate"
    region     = "eu-west-3"
    encrypt     = true
    use_lockfile = true      # verrou via un fichier .tflock dans S3 (remplace la table DynamoDB)
  }
}
```

HCL

```
# Créer le bucket d'état une fois pour toutes (versionné : on peut revenir à un état précédent)
aws s3api create-bucket --bucket io-tfstate-<prenom> --region eu-west-3 \
  --create-bucket-configuration LocationConstraint=eu-west-3
aws s3api put-bucket-versioning --bucket io-tfstate-<prenom> --versioning-configuration Status=Enabled
```

BASH

i DYNAMODB POUR LE VERROU

Les projets du dépôt d'origine (11, 19, 26) utilisent une table **DynamoDB** pour le verrou (`dynamodb_table = "... "`). C'était la seule méthode avant Terraform 1.10 ; elle fonctionne toujours mais est dépréciée. Si tu suis le code d'origine, crée la table avec une clé de partition `LockID` (type String).

Les modules

Un **module** est un dossier de fichiers `.tf` réutilisable, avec ses variables (entrées) et ses outputs (sorties). On l'appelle avec un bloc `module` :

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"      # module public du Terraform Registry
  version = "~> 5.0"

  name           = "io-vpc"
  cidr           = "10.0.0.0/16"
  azs            = ["eu-west-3a", "eu-west-3b"]
  public_subnets = ["10.0.1.0/24", "10.0.2.0/24"]
  private_subnets = ["10.0.11.0/24", "10.0.12.0/24"]
  enable_nat_gateway = true
  single_nat_gateway = true
}

resource "aws_instance" "web" {
  subnet_id = module.vpc.private_subnets[0]    # on utilise une SORTIE du module
  # ...
}
```

HCL

Les modules communautaires `terraform-aws-modules/vpc`, `/eks`, `/rds` ... sont très utilisés en entreprise (projet 19). Tes propres modules vivent dans un dossier `modules/` (projet 11).

Les bonnes pratiques

- Structure : `versions.tf`, `providers.tf`, `variables.tf`, `main.tf` (ou un fichier par thème), `outputs.tf`, `terraform.tfvars` (non commité si sensible).
- **Épingle les versions** des providers et des modules.
- **Toujours `plan` avant `apply`**, et en CI : `fmt -check`, `validate`, `plan` sur la PR, `apply` après fusion.
- Ne modifie **jamais à la main** une ressource gérée par Terraform (sinon *drift* : l'état ne correspond plus à la réalité ; `terraform plan` le détecte).
- Analyse de sécurité : **tflint**, **Checkov**, **tfsec**/**Trivy config**.
- `terraform import` (ou un bloc `import`) pour reprendre sous contrôle une ressource créée à la main.

Leçon 11 — Helm

Déployer une application dans Kubernetes demande souvent 5 à 15 fichiers YAML (Deployment, Service, Ingress, ConfigMap, Secret, HPA, ServiceAccount...), avec des valeurs qui changent selon l'environnement (nombre de replicas, nom de domaine, tag d'image). **Helm** est le **gestionnaire de paquets de Kubernetes** : il regroupe ces fichiers en un **chart** paramétrable.

```
mon-chart/
├─ Chart.yaml      # nom, version du chart, version de l'application
├─ values.yaml     # les valeurs par défaut
└─ templates/      # les manifests, avec des trous {{ ... }}
  ├─ deployment.yaml
  ├─ service.yaml
  └─ _helpers.tpl
```

```
# templates/deployment.yaml (extrait)
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
      - name: app
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

```
# values.yaml
replicaCount: 2
image:
  repository: moncompte/monapp
  tag: "1.0.0"
```

```
helm repo add bitnami https://charts.bitnami.com/bitnami # ajouter un dépôt de charts
helm repo update
helm search repo nginx
helm install monapp ./mon-chart -n demo --create-namespace # installer une "release"
helm upgrade --install monapp ./mon-chart -n demo --set image.tag=1.1.0 # installer OU mettre à jour
helm upgrade monapp ./mon-chart -f values-prod.yaml # valeurs d'un autre environnement
helm list -A # les releases installées
helm history monapp -n demo / helm rollback monapp 1 -n demo # historique et retour arrière
helm template ./mon-chart # afficher le YAML généré sans rien installer
helm uninstall monapp -n demo
helm create mon-chart # générer un chart d'exemple complet
```

Tu utiliseras Helm surtout pour installer des **outils tiers** prêts à l'emploi (AWS Load Balancer Controller, kube-prometheus-stack, Argo CD, NGINX Ingress) et pour empaqueter tes propres applications (projets 06, 15).

Leçon 12 — GitOps et Argo CD

Le principe

Dans un pipeline « classique » (push), c'est le serveur de CI qui exécute `kubectl apply` sur le cluster : il doit détenir les identifiants du cluster, et si quelqu'un modifie le cluster à la main, personne ne le remarque.

Le **GitOps** inverse le sens (pull) :

1. L'**état désiré** du cluster (les manifests ou charts) est stocké dans **un dépôt Git**.
2. Un **agent installé dans le cluster** (Argo CD, ou Flux) surveille ce dépôt en permanence.
3. Dès que Git change, l'agent applique le changement. Si le cluster dérive (modification manuelle), l'agent le signale et peut le **corriger automatiquement** (*self-heal*).



LES BÉNÉFICES

- **Git est la source de vérité** : l'historique Git est l'historique des déploiements. Revenir en arrière = `git revert`.
- **Sécurité** : la CI n'a plus besoin d'accès au cluster ; c'est le cluster qui tire les changements.
- **Détection de dérive** : ce qui est dans le cluster correspond toujours à Git.
- **Revue** : un changement de production passe par une pull request.

Argo CD en pratique

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl -n argocd get pods # attendre que tout soit Running
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d; echo
kubectl -n argocd port-forward svc/argocd-server 8443:443 # https://localhost:8443 , utilisateur admin
```

Une **Application** Argo CD relie un dépôt Git (et un chemin) à un namespace du cluster :

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: monapp
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/moi/monapp-manifests.git
    targetRevision: main
    path: k8s # dossier contenant les manifests (ou un chart Helm)
  destination:
    server: https://kubernetes.default.svc
    namespace: monapp
  syncPolicy:
    automated:
      prune: true # supprimer du cluster ce qui a été supprimé de Git
      selfHeal: true # annuler les modifications manuelles
    syncOptions: [CreateNamespace=true]
```

Les états affichés : **Synced** / **OutOfSync** (le cluster correspond-il à Git ?) et **Healthy** / **Progressing** / **Degraded** (les ressources fonctionnent-elles ?).

QUESTIONS D'ENTRETIEN

- Différence entre un Deployment et un StatefulSet ? entre un Service ClusterIP, NodePort et LoadBalancer ?
- Un pod est en `CrashLoopBackOff` : que fais-tu ? (`kubectl describe` , `kubectl logs --previous` , vérifier configuration, probes, ressources.)
- Qu'est-ce que l'état Terraform, pourquoi le stocker à distance et le verrouiller ?
- Qu'apporte Helm par rapport à des fichiers YAML bruts ?
- Explique le GitOps. Push ou pull : quels avantages ?

Projet 08 — Kubernetes de bout en bout sur EKS : le jeu 2048

FICHE PROJET

Niveau	★★★☆☆ Intermédiaire
Durée	3 à 5 heures
Coût	~0,25 \$/h (plan de contrôle + 1 à 2 nœuds + load balancer). Compte 1 à 2 \$ en une session.
Compétences	créer un cluster EKS (console puis eksctl), rôles IAM du cluster et des nœuds, node groups, kubeconfig, Pod, Deployment, Service LoadBalancer, mise à l'échelle
Prérequis	Leçon 9 (Kubernetes) ; idéalement, avoir joué avec kind en local
Dépôt d'origine	DevOps-Project-08/README.md

Ce qu'on construit



On va le faire **deux fois** : d'abord à la main dans la console (comme le dépôt d'origine), pour voir chaque brique ; puis en **une commande** avec `eksctl`, comme on le fait en vrai.

Partie 1 — La méthode console (pour comprendre)

Étape 1 — Les rôles IAM

Un cluster EKS a besoin de **deux** rôles :

Rôle du cluster (`eks-cluster-role`) : permet au plan de contrôle EKS de gérer des ressources AWS pour toi (interfaces réseau, load balancers). IAM → *Roles* → *Create role* → *AWS service* → **EKS** → cas d'usage **EKS - Cluster** → politique `AmazonEKSClusterPolicy`.

Rôle des nœuds (`eks-node-grp-role`) : permet aux EC2 des nœuds de rejoindre le cluster, de configurer le réseau des pods et de télécharger des images. *AWS service* → **EC2** → politiques :

- `AmazonEKSWorkerNodePolicy` (rejoindre le cluster),
- `AmazonEKS_CNI_Policy` (attribuer des IP du VPC aux pods),
- `AmazonEC2ContainerRegistryReadOnly` (télécharger depuis ECR).

Étape 2 — Le cluster

EKS → *Clusters* → *Create cluster* → **Custom configuration** (désactive *EKS Auto Mode* pour ce premier essai, afin de voir les briques une à une) :

- Nom `io-eks-1`, rôle `eks-cluster-role`, version Kubernetes **par défaut proposée**.
- *Cluster access* : **EKS API** (access entries) ; coche *Allow cluster administrator access* pour ton identité.
- Réseau : VPC **par défaut**, 2 ou 3 subnets, Security Group par défaut ; *Endpoint access* : **Public**.
- Add-ons : garde les versions par défaut de **VPC CNI**, **CoreDNS**, **kube-proxy** (et *EKS Pod Identity Agent*).
- *Create*. Compte **10 à 15 minutes**, jusqu'à l'état **Active**.

Étape 3 — Le node group

Cluster → onglet *Compute* → **Add node group** :

- Nom `io-eks-nodegrp-1`, rôle `eks-node-grp-role`.
- AMI : **Amazon Linux 2023** (par défaut), type `t3.medium`, disque 20 Go.
- Scaling : *desired* **1**, *min* **1**, *max* **2**.
- Subnets : ceux du cluster. (*L'accès SSH aux nœuds est optionnel ; inutile ici.*)
- *Create* : 2 à 3 minutes.

⚠ POURQUOI PAS `T3.MICRO` ?

Chaque nœud EKS a un nombre maximal de pods, lié au nombre d'adresses IP que son type d'instance peut porter. Un `t3.micro` n'en accepte que 4, et les pods système (CoreDNS, kube-proxy, CNI...) en occupent déjà la plupart. Résultat : tes pods restent en `Pending`. `t3.medium` (17 pods) est le minimum confortable.

Étape 4 — S'authentifier auprès du cluster

Ouvre **CloudShell** (icône  en haut de la console AWS : un terminal avec AWS CLI et kubectl déjà installés et authentifiés) ou ton terminal local :

```
aws sts get-caller-identity           # qui suis-je pour AWS ?
aws eks update-kubeconfig --region eu-west-3 --name io-eks-1
# → écrit la configuration de connexion dans ~/.kube/config
kubectl get nodes
# NAME                                STATUS    ROLES    AGE   VERSION
# ip-172-31-22-18.eu-west-3.compute.internal  Ready    <none>   3m    v1.3x.x
```

BASH

CE QUE FAIT 'UPDATE-KUBECONFIG'

Il ajoute à `~/.kube/config` l'adresse de l'API du cluster, son certificat, et une commande (`aws eks get-token`) que kubectl appellera pour obtenir un jeton d'authentification à partir de ton identité IAM. EKS vérifie ensuite que cette identité a une **access entry** dans le cluster. Si tu as créé le cluster avec un autre utilisateur/rôle que celui de ton terminal, tu obtiendras `Unauthorized` : il faut alors ajouter une access entry pour ton identité (cluster → onglet Access).

Étape 5 — Un premier Pod

```
# 2048-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: game-2048-pod           # (le nom "2048-pod" du dépôt d'origine est accepté, mais un nom commençant par une lettre est plus sûr)
  labels:
    app: 2048-ws
spec:
  containers:
    - name: game-2048
      image: blackicebird/2048
      ports:
        - containerPort: 80
```

YAML

```
kubectl apply -f 2048-pod.yaml
kubectl get pods -o wide
kubectl describe pod game-2048-pod    # lis la section Events : Scheduled → Pulling → Pulled → Created → Started
```

BASH

Étape 6 — L'exposer avec un Service LoadBalancer

```
# mygame-svc.yaml
apiVersion: v1
kind: Service
metadata:
  name: mygame-svc
spec:
  selector:
    app: 2048-ws               # doit correspondre EXACTEMENT au label du pod
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: LoadBalancer
```

YAML

```
kubectl apply -f mygame-svc.yaml
kubectl get svc mygame-svc -w    # attends qu'EXTERNAL-IP passe de <pending> à un nom DNS (Ctrl+C pour arrêter)
kubectl describe svc mygame-svc  # "LoadBalancer Ingress: a06aa...elb.eu-west-3.amazonaws.com"
```

BASH

Ouvre `http://<nom-DNS>` dans ton navigateur (2 à 3 minutes de propagation) : **le jeu 2048 s'affiche**.

QUI A CRÉÉ CE LOAD BALANCER ?

Tu n'as rien créé dans EC2. C'est le **contrôleur de service intégré** à EKS qui, voyant un Service de type `LoadBalancer`, a appelé l'API AWS pour créer un load balancer (Classic ou NLB) et y enregistrer les nœuds. Kubernetes pilote le cloud : c'est sa force... et la raison pour laquelle il faut **supprimer le Service avant le cluster**, sinon le load balancer reste orphelin et facturé.

Étape 7 — Passer du Pod au Deployment (et monter en charge)

Un Pod seul n'est **jamais** recréé s'il meurt. En vrai, on utilise un Deployment :

```
kubectl delete pod game-2048-pod
```

BASH

```
# 2048-deploy.yaml
apiVersion: apps/v1
```

YAML

```
kind: Deployment
metadata:
  name: game-2048
spec:
  replicas: 3
  selector:
    matchLabels: { app: 2048-ws }
  template:
    metadata:
      labels: { app: 2048-ws }
    spec:
      containers:
        - name: game-2048
          image: blackicebird/2048
          ports: [{ containerPort: 80 }]
          resources:
            requests: { cpu: 50m, memory: 64Mi }
            limits: { cpu: 200m, memory: 128Mi }
```

```
kubectl apply -f 2048-deploy.yaml
kubectl get pods -l app=2048-ws      # 3 pods ; le Service les a trouvés tout seul grâce au label
kubectl get endpoints mygame-svc     # les 3 IP de pods derrière le Service

# Expérience 1 : auto-réparation
kubectl delete pod <un-des-pods>
kubectl get pods -l app=2048-ws -w  # un nouveau pod apparaît aussitôt

# Expérience 2 : mise à l'échelle
kubectl scale deployment game-2048 --replicas=6
kubectl get pods -o wide             # si un nœud ne suffit pas : des pods en Pending → augmente le node group à 2
```

BASH

✓ VÉRIFICATION

- `kubectl get nodes` montre tes nœuds `Ready`.
- Le jeu est accessible par le nom DNS du load balancer.
- Supprimer un pod ne coupe pas le jeu, et un nouveau pod le remplace.

Partie 2 — La même chose en une commande avec eksctl

Supprime d'abord tout ce qui précède (voir *Nettoyage*), puis :

```
eksctl create cluster \
  --name io-eks-2 --region eu-west-3 \
  --nodegroup-name workers --node-type t3.medium \
  --nodes 1 --nodes-min 1 --nodes-max 2 --managed
# 15 à 20 minutes : eksctl crée un VPC dédié, les rôles, le cluster, le node group, et configure kubectl

kubectl get nodes
kubectl apply -f 2048-deploy.yaml -f mygame-svc.yaml
```

BASH

🗨️ CE QU'EKSCCTL FAIT POUR TOI

Il génère et déploie des piles **CloudFormation** (visibles dans la console CloudFormation : `eksctl-io-eks-2-cluster`, `eksctl-io-eks-2-nodegroup-workers`) : un VPC propre avec subnets publics et privés, une NAT Gateway, les rôles IAM, le cluster et les nœuds. Et `eksctl delete cluster` supprime tout dans le bon ordre. Pour un cluster d'apprentissage, c'est la méthode recommandée.

(En production, on décrit plutôt le cluster en **Terraform** (projet 19), ou avec un fichier de configuration `eksctl` versionné :

`eksctl create cluster -f cluster.yaml`.)

Dépannage

🔧 DÉPANNAGE

- **error: You must be logged in to the server (Unauthorized)** : ton identité IAM n'a pas d'access entry. Cluster → Access → Create access entry → ton utilisateur/rôle → politique `AmazonEKSClusterAdminPolicy`.
- **Nœuds absents de `kubectl get nodes`** : le node group est en échec (onglet *Compute* → *Health issues*) ; souvent un rôle de nœud sans les 3 politiques, ou des subnets sans accès Internet (ni IP publique, ni NAT) qui empêchent les nœuds de joindre l'API.
- **Pods en Pending** : `kubectl describe pod` → Too many pods ou Insufficient cpu : augmente le node group ou le type d'instance.
- **EXTERNAL-IP reste <pending>** : les subnets publics doivent porter le tag `kubernetes.io/role/elb=1` pour que Kubernetes sache où créer le load balancer (eksctl les tague automatiquement ; le VPC par défaut parfois non). `kubectl describe svc` donne l'erreur exacte.
- **exec format error** : image construite pour ARM sur des nœuds x86 (ou l'inverse).

Nettoyage

🔧 NETTOYAGE OBLIGATOIRE

L'ordre est important.

1. **Supprime le Service** d'abord : `kubectl delete svc mygame-svc` (supprime le load balancer AWS). Vérifie dans EC2 → *Load Balancers* qu'il a disparu.
2. `kubectl delete deployment game-2048`.
3. **Cluster console** : supprime le node group (*Compute* → *Delete*), attends, puis supprime le cluster. **Cluster eksctl** :
`eksctl delete cluster --name io-eks-2 --region eu-west-3`.
4. Vérifie CloudFormation (aucune pile `eksctl-...` restante), EC2 (instances, volumes), VPC (NAT Gateway, Elastic IP).
5. Les rôles IAM peuvent rester (gratuits) pour les projets suivants.

Pour aller plus loin

- Remplace le Service `LoadBalancer` par un **Ingress** avec l'AWS Load Balancer Controller (projet 15).
- Ajoute un **HorizontalPodAutoscaler** (`kubectl autoscale deployment game-2048 --cpu-percent=50 --min=2 --max=10`) : il nécessite le *metrics-server* (`kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml`).
- Essaie **EKS Auto Mode** : AWS gère aussi les nœuds, le stockage et les load balancers.
- Construis ta propre image du jeu (le code est open source) et pousse-la dans ECR.

🔑 QUESTIONS D'ENTRETIEN

- Quels rôles IAM faut-il pour EKS et pourquoi deux ?
- Que fait `aws eks update-kubeconfig` ?
- Différence entre un Pod et un Deployment ? Pourquoi ne jamais déployer un Pod seul ?
- Comment Kubernetes sait-il vers quels pods envoyer le trafic d'un Service ?
- Pourquoi faut-il supprimer les Services LoadBalancer avant le cluster ?

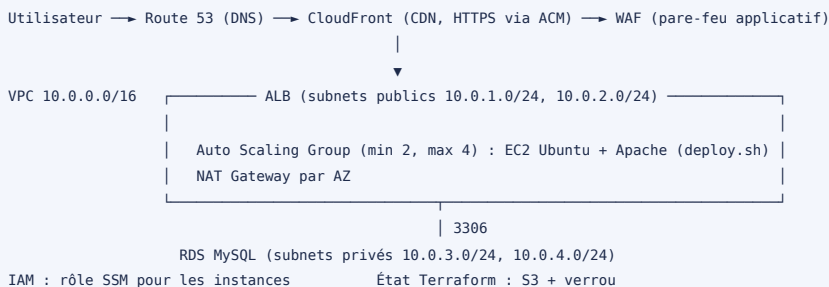
Projet 11 — Infrastructure 2-tiers sur AWS avec Terraform

FICHE PROJET

Niveau	★★★★☆ Intermédiaire
Durée	5 à 8 heures
Coût	~0,15 \$/h (2 NAT Gateways, ALB, RDS, 2 instances). Fais <code>terraform destroy</code> à chaque fin de session. CloudFront/WAF/Route 53 : quelques centimes + domaine (optionnel).
Compétences	Terraform modulaire, backend S3 + verrou, variables et <code>tfvars</code> , sources de données, VPC multi-AZ, ALB, Auto Scaling, RDS, IAM, ACM, CloudFront, WAF, Route 53
Prérequis	Leçon 10 (Terraform) ; projets 02 et 01 (tu reconnaîtras les briques)
Dépôt d'origine	DevOps-Project-11 (<code>main.tf</code> , <code>backend.tf</code> , <code>variables.tfvars</code> , dossier <code>modules/</code>)

Ce qu'on construit

Une architecture web **2-tiers** (serveurs web + base de données) complète, sécurisée et scalable, **entièrement décrite en Terraform** et découpée en modules.



Module	Contenu
<code>aws-vpc</code>	VPC, IGW, 2 subnets publics, 2 privés, 2 EIP, 2 NAT Gateways, 4 tables de routage
<code>security-group</code>	SG de l'ALB (80/443 depuis Internet), du web (80 depuis l'ALB), de la DB (3306 depuis le web)
<code>aws-rds</code>	subnet group + instance MySQL
<code>alb-tg</code>	Application Load Balancer, target group, listener
<code>aws-iam</code>	rôle, politique et instance profile pour SSM
<code>aws-autoscaling</code>	Launch Template (Ubuntu, user data <code>deploy.sh</code>), ASG, politiques de scaling + alarmes CPU
<code>aws-waf-cdn-acm-route53</code>	certificat ACM, distribution CloudFront, Web ACL WAF, enregistrements Route 53

Étape 1 — Récupérer et lire le code

```
cd ~/IO/projects && mkdir 11-terraform-2tier && cd 11-terraform-2tier
cp -r /tmp/dp/DevOps-Project-11/* .
tree -L 2
```

BASH

Lis chaque module avant de lancer quoi que ce soit. Pour chaque fichier, identifie : les `variable` (entrées), les `resource` (ce qui est créé), les `data` (ce qui est lu), les `output`. Ce travail de lecture est exactement ce que tu feras en entreprise en arrivant sur un projet existant.

⚠ TROIS PIÈGES DANS LE CODE D'ORIGINE

- Les chemins des modules.** `main.tf` appelle `source = "../modules/aws-vpc"` et `deploy.sh` est chargé avec `filebase64("../modules/...")`. Ces chemins sont relatifs **au dossier où l'on lance Terraform**. Ils supposent que `main.tf` est dans un **sous-dossier** (par exemple `main/`). Avec la structure du dépôt (tout à la racine), `terraform init` échoue avec `Unreadable module directory`. Solution : crée un dossier `main/` et déplaces-y `main.tf`, `variables.tf`, `backend.tf` et `variables.tfvars`. Tu lanceras Terraform depuis `main/`.
- Le mot de passe de la base** est en clair dans `variables.tfvars` (`RDS-PWD = "Admin1234"`). Retire-le du fichier et fournis-le autrement (étape 3).
- Les modules se trouvent mutuellement par leurs tags** (`data "aws_subnet"` avec un filtre `tag:Name`) et enchaînent des `depends_on`. Ça marche, mais c'est fragile. La bonne pratique est de **passer les sorties d'un module en entrée du suivant** (`subnet_id = module.vpc.public_subnet_ids[0]`). Bon exercice de refactoring une fois que tout fonctionne.

Étape 2 — Préparer le backend (état distant)

`backend.tf` stocke l'état dans S3 avec une table DynamoDB pour le verrou. Mets à jour avec `ton` bucket, et modernise le verrou :

```
terraform {
  backend "s3" {
    bucket     = "io-tfstate-<prenom>"           # bucket créé à la leçon 10
    key        = "projet-11/terraform.tfstate"
    region     = "eu-west-3"
    encrypt    = true
    use_lockfile = true                        # remplace dynamodb_table (Terraform ≥ 1.10)
  }
  required_version = "≥ 1.10"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
}
```

(Si tu préfères garder `dynamodb_table = "Lock-Files"` comme l'original : crée la table

```
aws dynamodb create-table --table-name Lock-Files --attribute-definitions AttributeName=LockID,AttributeType=S --key-schema AttributeName=LockID,KeyType=HASH
```

Vérifie aussi le bloc `provider "aws"` (région). **Attention** : pour CloudFront, le certificat ACM et la Web ACL WAF doivent être créés en `us-east-1` (exigence d'AWS pour les services « globaux »). Regarde comment le module `aws-waf-cdn-acm-route53` gère la région ; le plus simple pour ce projet est de tout déployer en `us-east-1`.

Étape 3 — Les variables

Ouvre `variables.tfvars` et adapte : noms, CIDR (garde-les), **domaine** (`DOMAIN-NAME`). Puis le mot de passe :

Terraform lit automatiquement toute variable d'environnement `TF_VAR_<nom>` comme valeur de la variable `<nom>`. Problème : la variable s'appelle `RDS-PWD`, et Bash n'accepte pas de tiret dans un nom de variable d'environnement. Renomme-la donc `RDS_PWD` (dans `variables.tf`, dans l'appel du module dans `main.tf`) et ajoute `sensitive = true` à sa déclaration pour qu'elle n'apparaisse jamais dans les sorties de `plan` :

```
variable "RDS_PWD" {
  type        = string
  sensitive   = true
}
```

```
# Jamais dans un fichier commité : saisie masquée, puis variable d'environnement
read -s TF_VAR_RDS_PWD && export TF_VAR_RDS_PWD
```

PAS DE NOM DE DOMAINE ?

Les modules ACM, CloudFront et Route 53 exigent un domaine que tu contrôles (un `.com` coûte ~15 \$/an dans Route 53, et la *hosted zone* 0,50 \$/mois). Sans domaine, **commente le bloc** `module "route53"` dans `main.tf` : tu auras toute l'architecture sauf la couche CDN/DNS, et tu accèderas au site par le nom DNS de l'ALB.

Étape 4 — Init, plan, apply

```
cd main
terraform init                # télécharge le provider, initialise le backend S3
terraform fmt -recursive ..   # formatage
terraform validate
terraform plan -var-file=variables.tfvars -out=plan.tfplan
terraform apply plan.tfplan
```

'-OUT=PLAN.TFPLAN'

Sauvegarder le plan puis appliquer **ce fichier** garantit que ce qui est appliqué est exactement ce que tu as relu, même si quelque chose a changé entre-temps. C'est la pratique standard en CI (le plan est relu sur la pull request, puis appliqué tel quel).

Lis le plan : combien de ressources ? Repère les NAT Gateways, l'instance RDS, la distribution CloudFront. L' `apply` prend **15 à 25 minutes** (RDS et CloudFront sont les plus lents).

Étape 5 — Vérifier

```
terraform state list          # toutes les ressources gérées
terraform output              # si des outputs sont définis
aws elbv2 describe-load-balancers --query "LoadBalancers[].DNSName" --output text
```

✓ VÉRIFICATION

- `http://<DNS-de-l'ALB>` affiche la page installée par `deploy.sh` (Apache).
- EC2 → *Target Groups* → *Web-TG* : 2 cibles *healthy*.
- Connecte-toi à une instance par **Session Manager** (le module IAM lui donne le rôle SSM) et teste l'accès à la base :
`sudo apt install -y mysql-client && mysql -h <endpoint-rds> -u admin -p`.
- Avec domaine : `https://<ton-domaine>` passe par CloudFront (en-tête *x-cache* dans les outils développeur du navigateur), le certificat est valide.
- **Test de dérive** : dans la console, change à la main la capacité désirée de l'ASG, puis lance `terraform plan` : Terraform détecte l'écart et propose de le corriger.

Étape 6 — Comprendre les briques « edge »

🔑 CLOUDFRONT, WAF ET ACM

- **CloudFront** est le **CDN** d'AWS : des centaines de points de présence dans le monde gardent une copie des contenus et servent les utilisateurs au plus près, ce qui réduit la latence et la charge des serveurs.
- **ACM** (*Certificate Manager*) fournit gratuitement des certificats TLS, renouvelés automatiquement. La validation se fait par un enregistrement DNS (créé ici automatiquement dans Route 53).
- **WAF** (*Web Application Firewall*) filtre les requêtes HTTP : injections SQL, XSS, robots, limitation de débit par IP... grâce à des règles gérées par AWS (*AWS Managed Rules*).
- **Route 53** fait pointer ton domaine vers CloudFront avec un enregistrement *Alias*.

Dépannage

🔧 DÉPANNAGE

- `Unreadable module directory` : voir le piège n°1 (chemins relatifs).
- `Error acquiring the state lock` : un `apply` précédent a été interrompu. Vérifie qu'aucun autre n'est en cours, puis `terraform force-unlock <LOCK_ID>`.
- `InvalidParameterValue: ... password` pour RDS : le mot de passe ne respecte pas les règles (8 caractères minimum, sans `/`, `"`, `@`, espace).
- **ACM reste en Pending validation** : la zone Route 53 n'est pas celle qui fait autorité pour ton domaine (vérifie les serveurs de noms chez ton registrar).
- `WAFUnavailableEntityException` / **certificat introuvable pour CloudFront** : la Web ACL ou le certificat ne sont pas en `us-east-1`.
- **L'ASG crée des instances qui ne deviennent jamais healthy** : `deploy.sh` a échoué. Session Manager → `sudo cat /var/log/cloud-init-output.log`.

Nettoyage

🔑 NETTOYAGE OBLIGATOIRE

```
cd main
terraform destroy -var-file=variables.tfvars
```

BASH

Vérifie ensuite dans la console : NAT Gateways et Elastic IPs (supprimées ?), snapshots RDS finaux éventuels, distribution CloudFront (sa suppression peut prendre 15 minutes). Garde le bucket d'état et la *hosted zone* si tu as un domaine.

Pour aller plus loin

- **Refactoring** : supprime toutes les recherches par tag entre modules et remplace-les par des `output` → `variable`. Supprime les `depends_on` devenus inutiles.
- Ajoute des `output` utiles dans `main/` (DNS de l'ALB, endpoint RDS).
- Génère le mot de passe RDS avec `manage_master_user_password = true` (RDS le crée et le range dans Secrets Manager : plus aucun mot de passe dans Terraform).
- Ajoute un pipeline GitHub Actions : `fmt -check` + `validate` + `plan` sur chaque PR, `apply` sur `main` (avec OIDC, voir projet 14).
- Analyse le code avec **Checkov** (`pip install checkov && checkov -d ..`) et corrige les alertes.

🔑 QUESTIONS D'ENTRETIEN

- Comment organises-tu un projet Terraform en modules ? Comment les modules communiquent-ils ?
- Où et comment stockes-tu l'état ? Que se passe-t-il si deux personnes lancent `apply` en même temps ?
- Comment gères-tu les secrets (mot de passe de base) dans Terraform ?
- Qu'est-ce que la dérive (*drift*) et comment la détectes-tu ?
- Pourquoi le certificat ACM d'une distribution CloudFront doit-il être en `us-east-1` ?

Projet 12 — Super Mario sur Kubernetes, avec Terraform

FICHE PROJET

Niveau	★★★★☆ Intermédiaire
Durée	2 à 4 heures
Coût	~0,20 \$/h (EKS + 1 nœud + load balancer + petite EC2)
Compétences	EC2 « bootstrap » avec rôle IAM, script d'installation, EKS en Terraform (ressources <code>aws_eks_cluster</code> , <code>aws_eks_node_group</code> , rôles IAM), backend S3, déploiement Kubernetes
Prérequis	Projet 08 (EKS à la main), leçon 10 (Terraform)
Dépôt d'origine	DevOps-Project-12/README.md → code source github.com/Aj7Ay/k8s-mario (dossier <code>EKS-TF/</code> , <code>deployment.yaml</code> , <code>service.yaml</code> , <code>script.sh</code>)

Ce qu'on construit

Le même type de résultat que le projet 08 (un jeu dans EKS), mais le cluster est cette fois **créé par Terraform**, depuis une machine d'administration dédiée.

```
Toi —SSH→ EC2 "bootstrap" (Ubuntu) — rôle IAM → API AWS
| script.sh : Terraform + kubectl + AWS CLI
| terraform apply (état dans S3)
▼
EKS "EKS_CLOUD" (VPC par défaut)
node group 1 × t3.medium
Deployment mario (2 replicas, image sevenajay/mario) — Service LoadBalancer → navigateur
```

POURQUOI UNE MACHINE « BOOTSTRAP » ?

On centralise les outils et les droits sur une machine d'administration, avec un **rôle IAM** plutôt que des clés sur ton portable. C'est une pratique courante (on parle aussi de *jump box* ou de *bastion d'administration*). En entreprise, on remplace souvent cette machine par un pipeline CI.

Étape 1 — La machine bootstrap

- Rôle IAM** : IAM → Create role → AWS service → EC2 → politique `AdministratorAccess` → nom `io-bootstrap-role`. (Acceptable pour un lab jetable ; en entreprise, on limiterait aux droits EKS/EC2/IAM nécessaires.)
- Instance** : Ubuntu 24.04, `t3.micro`, SG SSH depuis My IP, rôle `io-bootstrap-role`, nom `mario-bootstrap`.
- Connecte-toi et installe les outils :

```
ssh -i ~/.ssh/io-key.pem ubuntu@<IP>
git clone https://github.com/Aj7Ay/k8s-mario.git && cd k8s-mario
cat script.sh # lis TOUJOURS un script avant de l'exécuter
sudo chmod +x script.sh && ./script.sh
terraform -version && kubectl version --client && aws --version
aws sts get-caller-identity # doit afficher le rôle io-bootstrap-role (assumed-role/...)
```

BASH

✓ VÉRIFICATION

`aws sts get-caller-identity` répond **sans** que tu aies tapé `aws configure` : la CLI a trouvé seule les identifiants temporaires du rôle via le service de métadonnées de l'instance.

Étape 2 — Lire le code Terraform

```
cd EKS-TF && ls
# backend.tf main.tf provider.tf
```

BASH

Ce que fait `main.tf`, bloc par bloc :

Bloc	Rôle
<code>data "aws_iam_policy_document" "assume_role"</code>	la politique de confiance : « le service <code>eks.amazonaws.com</code> peut endosser ce rôle »
<code>aws_iam_role.example</code> + attachement <code>AmazonEKSClusterPolicy</code>	le rôle du cluster (comme au projet 08, mais en code)
<code>data "aws_vpc" "default"</code> / <code>data "aws_subnets" "public"</code>	lit le VPC par défaut et ses subnets (rien n'est créé)

Bloc	Rôle
<code>aws_eks_cluster.example</code>	le cluster <code>EKS_CLOUD</code> dans ces subnets
<code>aws_iam_role.example1</code> + 3 attachements	le rôle des nœuds (WorkerNode, CNI, ECR ReadOnly)
<code>aws_eks_node_group.example</code>	le node group : 1 nœud (min 1, max 2)

Les `depends_on` garantissent que les politiques IAM sont attachées **avant** la création du cluster et des nœuds, et détachées **après** leur destruction (sinon EKS ne pourrait plus nettoyer ses propres ressources).

Étape 3 — Adapter le code

1. `backend.tf` : mets ton bucket et ta région :

```
terraform {
  backend "s3" {
    bucket     = "io-tfstate-<prenom>"
    key        = "EKS/terraform.tfstate"
    region     = "eu-west-3"
    use_lockfile = true
  }
}
```

2. `provider.tf` : `region = "eu-west-3"`.
3. `main.tf` : remplace `instance_types = ["t2.medium"]` par `["t3.medium"]` (génération plus récente et moins chère).

⚠ LES SUBNETS DU VPC PAR DÉFAUT

Le code prend **tous** les subnets du VPC par défaut. Dans certaines régions (notamment `us-east-1`), une AZ n'est pas supportée par EKS (`us-east-1e`), et la création échoue avec `UnsupportedAvailabilityZoneException`. Si ça t'arrive, filtre les subnets par AZ :

```
data "aws_subnets" "public" {
  filter { name = "vpc-id",          values = [data.aws_vpc.default.id] }
  filter { name = "availability-zone", values = ["eu-west-3a", "eu-west-3b", "eu-west-3c"] }
}
```

Étape 4 — Créer le cluster

```
terraform init
terraform validate
terraform plan
terraform apply --auto-approve    # 10 à 15 minutes
```

Puis configure kubectl :

```
aws eks update-kubeconfig --name EKS_CLOUD --region eu-west-3
kubectl get nodes
```

⚠ 'UNAUTHORIZED' AVEC KUBECTL ?

Avec l'API des access entries (mode par défaut des clusters récents créés par Terraform), l'**identité qui crée le cluster** n'a pas toujours automatiquement les droits d'administration Kubernetes. Ajoute dans `aws_eks_cluster` :

```
access_config {
  authentication_mode           = "API_AND_CONFIG_MAP"
  bootstrap_cluster_creator_admin_permissions = true
}
```

Ici, c'est le rôle `io-bootstrap-role` qui crée le cluster et qui utilise kubectl : il sera administrateur.

Étape 5 — Déployer Mario

```
cd ..                                # retour à la racine du dépôt
cat deployment.yaml service.yaml     # 2 replicas de sevenajay/mario, Service LoadBalancer port 80
kubectl apply -f deployment.yaml
kubectl get deploy,pods
kubectl apply -f service.yaml
kubectl get svc mario-service -w      # attends l'EXTERNAL-IP
kubectl describe svc mario-service | grep "LoadBalancer Ingress"
```

Ouvre `http://<LoadBalancer Ingress>` : Super Mario se lance dans ton navigateur. 🍄

Dépannage

🔧 DÉPANNAGE

- `Error: creating EKS Cluster ... AccessDeniedException` : le rôle de l'instance n'a pas les droits (vérifie qu'il est bien attaché à l'instance).
- `Error acquiring the state lock / NoSuchBucket` : bucket du backend inexistant ou dans une autre région.
- **Node group en `CREATE_FAILED` (« Instances failed to join the kubernetes cluster »)** : subnets sans IP publique automatique et sans NAT. Les subnets du VPC par défaut ont l'IP publique automatique activée ; si tu as modifié ce réglage, réactive-le.
- **Pods Pending** : un seul `t3.medium` peut manquer de place ; passe `desired_size` à 2.

Nettoyage

🔧 NETTOYAGE OBLIGATOIRE

```
kubectl delete -f service.yaml          # D'ABORD le Service (supprime le load balancer)
kubectl delete -f deployment.yaml
cd EKS-TF && terraform destroy --auto-approve  # 10 minutes
```

BASH

Puis *terminate* l'instance `mario-bootstrap` et supprime le rôle `io-bootstrap-role` (ou garde-le pour le projet 19). Vérifie EC2 → *Load Balancers* : rien ne doit rester.

💡 POURQUOI SUPPRIMER LE SERVICE AVANT 'TERRAFORM DESTROY' ?

Le load balancer a été créé par **Kubernetes**, pas par Terraform : Terraform ne le connaît pas et ne le supprimera pas. Pire, ce load balancer et son Security Group occupent le VPC et peuvent bloquer la suppression d'autres ressources. Règle générale : **ce qu'un outil a créé, c'est à cet outil de le supprimer.**

Pour aller plus loin

- Ajoute des `output` (nom du cluster, endpoint) et une `variable` pour la région et le type d'instance.
- Remplace le VPC par défaut par le module `terraform-aws-modules/vpc` avec subnets privés (c'est ce que fait le projet 19).
- Déploie Mario **avec Terraform aussi**, grâce au provider `kubernetes` (ressources `kubernetes_deployment` et `kubernetes_service`). Discute : est-ce une bonne idée de gérer les applications avec le même outil que l'infrastructure ? (*Souvent non : on préfère Helm/Argo CD pour les applications.*)

🔧 QUESTIONS D'ENTRETIEN

- Comment crées-tu un cluster EKS en Terraform ? Quelles ressources sont indispensables ?
- Pourquoi les `depends_on` sur les attachements de politiques IAM ?
- Comment une EC2 obtient-elle des droits AWS sans clé d'accès ?
- Pourquoi un `terraform destroy` peut-il échouer quand des Services LoadBalancer existent encore ?

Projet 15 — Application e-commerce 3-tiers sur EKS avec Helm

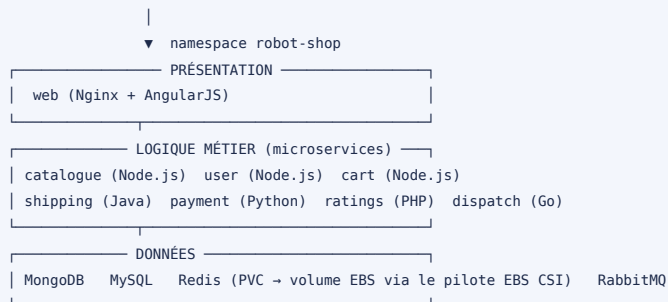
FICHE PROJET

Niveau	★★★★☆ Avancé
Durée	4 à 6 heures
Coût	~0,30 \$/h (EKS + 2 à 3 nœuds + ALB + volumes EBS)
Compétences	eksctl, fournisseur OIDC du cluster, IRSA (rôles IAM pour les pods), AWS Load Balancer Controller , Ingress → ALB, pilote EBS CSI et volumes persistants, chart Helm d'une application microservices
Prérequis	Projet 08, leçons 9 et 11
Dépôt d'origine	DevOps-Project-15/README.md → application github.com/uniqueSreedhar/RobotShop-Project (dossier <code>EKS/helm</code>)

Ce qu'on construit

Robot Shop, une boutique en ligne de démonstration composée d'une dizaine de **microservices** écrits dans des langages différents, déployée en une commande avec **Helm**, exposée par un **ALB** créé automatiquement à partir d'un objet **Ingress**, et dont les bases de données stockent leurs données sur des **volumes EBS**.

Navigateur → ALB (créé par l'AWS Load Balancer Controller à partir de l'Ingress)



Les concepts nouveaux

🧠 IRSA : DONNER DES DROITS AWS À UN POD

L'**AWS Load Balancer Controller** est un pod qui doit appeler l'API AWS pour créer des ALB. Le **pilote EBS CSI** est un pod qui doit créer et attacher des volumes EBS. Comment leur donner des droits sans clés d'accès ?

- Le cluster expose un **fournisseur d'identité OIDC**, qu'on enregistre dans IAM.
- On crée un **rôle IAM** dont la politique de confiance dit : « le ServiceAccount Kubernetes `kube-system/aws-load-balancer-controller` de ce cluster peut m'endosser ».
- Le ServiceAccount porte une annotation avec l'ARN du rôle ; le pod obtient automatiquement des identifiants temporaires.

C'est l'équivalent, pour un pod, du rôle IAM d'une instance EC2. (EKS propose aussi **Pod Identity**, plus récent et plus simple ; IRSA reste très répandu et c'est ce que montre le projet.)

🧠 INGRESS + AWS LOAD BALANCER CONTROLLER

Au projet 08, chaque Service `LoadBalancer` créait **son** load balancer. Ici, un seul objet **Ingress** décrit les règles HTTP, et le **contrôleur** traduit cet Ingress en un **ALB** configuré (listeners, règles, target groups). Un ALB pour toute l'application, routage par chemin ou par nom de domaine, HTTPS avec ACM : c'est la méthode standard sur EKS.

🧠 LE PILOTE EBS CSI

Les bases de données (MongoDB, MySQL, Redis) demandent du stockage persistant via des **PersistentVolumeClaims**. Sur EKS, c'est le **pilote EBS CSI** (un add-on) qui crée les volumes EBS correspondants et les attache au bon nœud. Sans lui, les PVC restent en `Pending` et les pods de bases de données ne démarrent jamais.

Étape 1 — Le cluster

```
export CLUSTER=io-robotshop
export REGION=eu-west-3
eksctl create cluster --name $CLUSTER --region $REGION \
  --nodegroup-name workers --node-type t3.medium \
  --nodes 2 --nodes-min 1 --nodes-max 3 --managed
kubectl get nodes
```

BASH

(Le dépôt d'origine utilise des `t3.small` : avec une dizaine de microservices plus les contrôleurs, c'est trop juste. `t3.medium` × 2 est un minimum confortable.)

Étape 2 — Le fournisseur OIDC

```
oidc_id=$(aws eks describe-cluster --name $CLUSTER --query "cluster.identity.oidc.issuer" --output text | cut -d '/' -f 5)
aws iam list-open-id-connect-providers | grep $oidc_id # déjà là ? (eksctl l'a peut-être créé)
eksctl utils associate-iam-oidc-provider --cluster $CLUSTER --region $REGION --approve
```

BASH

Étape 3 — Les droits IAM de l'AWS Load Balancer Controller

```
# La politique officielle, à jour (le dépôt d'origine épingle la v2.5.4, ancienne)
curl -o iam_policy.json https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/main/docs/install/iam_policy.json
aws iam create-policy --policy-name AWSLoadBalancerControllerIAMPolicy --policy-document file://iam_policy.json

ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
eksctl create iamserviceaccount \
  --cluster=$CLUSTER --region $REGION \
  --namespace=kube-system \
  --name=aws-load-balancer-controller \
  --role-name AmazonEKSLoadBalancerControllerRole \
  --attach-policy-arn=arn:aws:iam::$ACCOUNT_ID:policy/AWSLoadBalancerControllerIAMPolicy \
  --approve
kubectl get sa aws-load-balancer-controller -n kube-system -o yaml | grep role-arn # l'annotation IRSA
```

BASH

Étape 4 — Installer le contrôleur avec Helm

```
VPC_ID=$(aws eks describe-cluster --name $CLUSTER --query "cluster.resourcesVpcConfig.vpcId" --output text)
helm repo add eks https://aws.github.io/eks-charts && helm repo update eks
helm install aws-load-balancer-controller eks/aws-load-balancer-controller -n kube-system \
  --set clusterName=$CLUSTER \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller \
  --set region=$REGION \
  --set vpcId=$VPC_ID
kubectl get deployment -n kube-system aws-load-balancer-controller # READY 2/2
```

BASH

`serviceAccount.create=false` : on réutilise le ServiceAccount créé à l'étape 3 (celui qui porte le rôle IAM).

Étape 5 — Le pilote EBS CSI

```
eksctl create iamserviceaccount \
  --name ebs-csi-controller-sa --namespace kube-system \
  --cluster $CLUSTER --region $REGION \
  --role-name AmazonEKS_EBS_CSI_DriverRole --role-only \
  --attach-policy-arn arn:aws:iam::aws:policy/service-role/AmazonEBSCSIDriverPolicy \
  --approve

eksctl create addon --name aws-ebs-csi-driver --cluster $CLUSTER --region $REGION \
  --service-account-role-arn arn:aws:iam::$ACCOUNT_ID:role/AmazonEKS_EBS_CSI_DriverRole --force

kubectl get pods -n kube-system | grep ebs-csi
kubectl get storageclass # une StorageClass gp2 (ou gp3) doit exister
```

BASH

'--ROLE-ONLY'

Ici, eksctl crée seulement le **rôle** IAM (pas le ServiceAccount), parce que c'est l'**add-on** EKS qui crée et gère son propre ServiceAccount ; on lui passe l'ARN du rôle à utiliser.

Étape 6 — Installer Robot Shop avec Helm

```
git clone https://github.com/uniquesreedhar/RobotShop-Project.git
cd RobotShop-Project/EKS/helm
ls # Chart.yaml values.yaml templates/ ingress.yaml
cat values.yaml # options : StorageClass, images, ressources...
kubectl create ns robot-shop
helm install robot-shop --namespace robot-shop .
kubectl get pods -n robot-shop -w # attends que tout soit Running (2 à 5 minutes)
kubectl get pvc -n robot-shop # les PVC doivent être Bound
```

BASH

⚠ PVC EN 'PENDING' ?

`kubectl describe pvc <nom> -n robot-shop`. Si l'événement dit *waiting for first consumer*, c'est normal : le volume est créé quand le pod est placé. Si la StorageClass demandée n'existe pas (par exemple `gp2` absente sur un cluster récent qui n'a que `gp3`), adapte la StorageClass dans `values.yaml` ou crée-la.

Étape 7 — L'Ingress

```
cat ingress.yaml
```

BASH

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: robot-shop
  namespace: robot-shop
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing # ALB public
    alb.ingress.kubernetes.io/target-type: ip # l'ALB envoie directement aux IP des pods
spec:
  ingressClassName: alb
  rules:
  - http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service: { name: web, port: { number: 8080 } }
```

YAML

```
kubectl apply -f ingress.yaml
kubectl get ingress -n robot-shop -w # ADDRESS : le nom DNS de l'ALB (2 à 3 minutes)
```

BASH

✓ VÉRIFICATION

- Ouvre `http://<ADDRESS>` : la boutique Robot Shop s'affiche. Crée un compte, connecte-toi, ajoute des robots au panier, note un robot (*Rate the Artificial Intelligence*), passe commande.
- EC2 → *Load Balancers* : un ALB `k8s-robotshop-...` ; ses *target groups* contiennent des **IP de pods**.
- EC2 → *Volumes* : des volumes EBS créés pour les bases de données.
- Supprime le pod MySQL (`kubectl delete pod -n robot-shop -l service=mysql`) : il revient avec **les mêmes données** (le volume persiste).

Dépannage

🔧 DÉPANNAGE

- L'Ingress n'a jamais d'ADDRESS** : `kubectl logs -n kube-system deploy/aws-load-balancer-controller`. Causes classiques : `AccessDenied` (rôle IRSA mal configuré ou politique ancienne), subnets publics non tagués `kubernetes.io/role/elb=1` (eksctl les tague), `ingressClassName` absent.
- helm install du contrôleur : webhook errors** à l'installation de l'Ingress → le contrôleur n'est pas encore prêt ; attends qu'il soit `2/2`.
- Pods Pending (Insufficient cpu/memory)** : ajoute un nœud (`eksctl scale nodegroup --cluster $CLUSTER --name workers --nodes 3`).
- Pods CrashLoopBackOff sur un microservice** : souvent sa base n'est pas encore prête ; patiente ou supprime le pod pour le relancer.

Nettoyage

🔧 NETTOYAGE OBLIGATOIRE

```
kubectl delete -f ingress.yaml           # D'ABORD : supprime l'ALB
helm uninstall robot-shop -n robot-shop   # supprime les pods (les PVC peuvent rester :)
kubectl delete pvc --all -n robot-shop    # supprime les volumes EBS associés
helm uninstall aws-load-balancer-controller -n kube-system
eksctl delete cluster --name $CLUSTER --region $REGION
```

BASH

Vérifie EC2 → *Load Balancers* et *Volumes* (aucun volume `available` orphelin), CloudFormation (piles `eksctl-...`). Les politiques/rôles IAM peuvent être supprimés ou gardés.

Pour aller plus loin

- Ajoute **HTTPS** : certificat ACM + annotations `alb.ingress.kubernetes.io/certificate-arn` et `listen-ports`.
- Remplace IRSA par **EKS Pod Identity** (`eksctl create podidentityassociation`).
- Installe la pile de monitoring (annexe A.10) et observe la consommation de chaque microservice.
- Lis les templates du chart Robot Shop : repère `{{ .Values... }}`, les `if`, les `range`.

🔑 QUESTIONS D'ENTRETIEN

- Qu'est-ce qu'IRSA et quel problème résout-il ?
- Différence entre un Service LoadBalancer et un Ingress sur EKS ?
- Que fait le pilote EBS CSI ? Que se passe-t-il pour les données quand un pod de base de données est recréé ?
- `target-type: ip` vs `instance` pour l'ALB ?

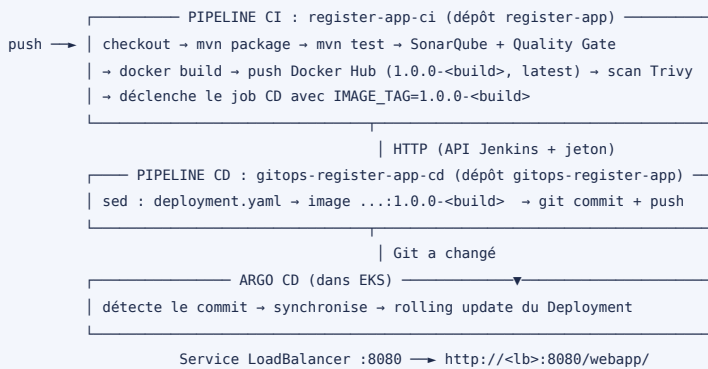
Projet 16 — Projet DevOps temps réel : de Jenkins à Kubernetes avec Argo CD

FICHE PROJET

Niveau	★★★★☆ Avancé
Durée	10 à 15 heures
Coût	~0,45 \$/h quand tout tourne (Jenkins master + agent + SonarQube t3.medium + bootstrap + EKS 2-3 nœuds + 2 load balancers)
Compétences	Jenkins master/agent par SSH, SonarQube auto-hébergé (PostgreSQL), Docker Hub, Trivy, eksctl, Argo CD (installation, CLI, ajout de cluster, Application), deux pipelines CI et CD reliés, dépôt GitOps
Prérequis	Projets 05 et 08, leçon 12 (GitOps)
Dépôt d'origine	DevOps-Project-16/README.md ; code : github.com/Ashfaque-9x/register-app (CI) et github.com/MSFaizi/gitops-register-app (manifests + pipeline CD)

Ce qu'on construit

Le modèle « CI Jenkins + CD GitOps » qu'on retrouve dans énormément d'entreprises :



Serveurs : [Jenkins-Master](#) (t3.small), [Jenkins-Agent](#) (t3.medium, Docker), [SonarQube](#) (t3.medium), [EKS-Bootstrap-Server](#) (t3.micro), plus le cluster EKS.

Étape 1 — Jenkins master et agent

1.1 — Deux instances Ubuntu 24.04, 15 à 20 Go de disque : [Jenkins-Master](#) ([t3.small](#), port 8080 ouvert depuis ton IP et GitHub) et [Jenkins-Agent](#) ([t3.medium](#)). Même Security Group ou SG qui autorise le SSH entre elles.

1.2 — Sur les deux machines :

```
sudo apt update && sudo apt upgrade -y
sudo hostnamectl set-hostname Jenkins-Master # (Jenkins-Agent sur la seconde ! le tutoriel d'origine a une coquille)
exec bash
sudo apt install -y openjdk-21-jre
```

1.3 — Sur le master : installe Jenkins (annexe A.3). **Sur l'agent :** installe Docker et donne-le à ubuntu :

```
sudo apt install -y docker.io
sudo usermod -aG docker ubuntu
sudo reboot # pour appliquer le groupe ; reconnecte-toi ensuite
```

1.4 — Clé SSH master → agent. Sur le **master** :

```
ssh-keygen -t ed25519 -N "" -f ~/.ssh/jenkins_agent
cat ~/.ssh/jenkins_agent.pub # copie
```

Sur l'**agent** : colle la ligne dans [~/.ssh/authorized_keys](#) (le fichier s'appelle [authorized_keys](#), avec un **s** : le tutoriel d'origine écrit [authorized_key](#), ce qui ne fonctionne pas).

```
echo "<clé publique>" >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys
```

i PAS BESOIN DE TOUCHER À `SSHD\_CONFIG`

Le tutoriel d'origine modifie `/etc/ssh/sshd_config` (décommenter `PubkeyAuthentication` et `AuthorizedKeysFile`). Sur Ubuntu, ces options sont **déjà actives par défaut** : ce n'est pas nécessaire.

1.5 — Configurer Jenkins (`http://<IP_MASTER>:8080`) :

- Débloque, installe les plugins suggérés, crée l'admin.
- *Manage Jenkins* → *Nodes* → *Built-In Node* → *Configure* : **O exécuteur**.
- *Credentials* : *SSH Username with private key*, ID `Jenkins-Agent`, user `ubuntu`, clé = contenu de `~/.ssh/jenkins_agent` (la privée).
- *Nodes* → *New node Jenkins-Agent (Permanent)* : 2 exécuteurs, *Remote root* `/home/ubuntu`, *label* `Jenkins-Agent`, *Launch agents via SSH*, *Host* = IP **privée** de l'agent, credential `Jenkins-Agent`, *Non verifying*.
- Vérifie que l'agent est en ligne.

1.6 — Outils et credentials pour le pipeline :

- *Plugins* : **Maven Integration, Pipeline Maven Integration, Eclipse Temurin installer, SonarQube Scanner, Sonar Quality Gates, Docker, Docker Pipeline, Docker Commons, docker-build-step, CloudBees Docker Build and Publish**.
- *Tools* : Maven `Maven3` (auto-install) ; JDK `Java17` (auto-install depuis adoptium.net, version 17).
- *Credentials* : `github` (*Username with password* : ton pseudo + **token** GitHub ; GitHub refuse les mots de passe pour Git depuis 2021), `dockerhub` (*Username with password* : pseudo Docker Hub + **access token** Docker Hub).

Étape 2 — Le pipeline CI

1. Fork `github.com/Ashfaque-9x/register-app` dans ton compte GitHub.
2. Dans **ton** fork, modifie le `Jenkinsfile` :
 - L'URL du `git` (ton fork) ;
 - `DOCKER_USER` = ton pseudo Docker Hub ;
 - le nom d'image de l'étape Trivy (`<toi>/register-app-pipeline:latest`) ;
 - L'URL du job CD et l'utilisateur Jenkins de l'étape *Trigger CD Pipeline* (étape 6) ;
 - l'adresse e-mail du bloc `post` (ou supprime ce bloc si tu ne configures pas l'e-mail, voir projet 09).
3. Remplace `FROM tomcat:latest` dans le `Dockerfile` par `FROM tomcat:9.0-jdk17` (l'application utilise `javax.*` : voir projet 01).
4. Jenkins → *New Item register-app-ci* → **Pipeline** → *Discard old builds* (max 2) → *Pipeline script from SCM* → Git → URL de ton fork → credential `github` → branche `*/main` → *Save*.

Étape 3 — SonarQube auto-hébergé

Instance Ubuntu 24.04 `t3.medium` (SonarQube embarque Elasticsearch : 4 Go de RAM minimum), 20 Go, port **9000** ouvert depuis ton IP et le SG de Jenkins.

Le tutoriel d'origine installe PostgreSQL et SonarQube « à la main » (zip + service systemd). La méthode la plus simple et la plus fiable aujourd'hui est **Docker Compose** (détails en annexe A.4) :

```
sudo apt update && sudo apt install -y docker.io docker-compose-v2
# Réglages noyau exigés par Elasticsearch (embarqué dans SonarQube)
echo "vm.max_map_count=524288" | sudo tee /etc/sysctl.d/99-sonarqube.conf
echo "fs.file-max=131072" | sudo tee -a /etc/sysctl.d/99-sonarqube.conf
sudo sysctl --system

mkdir ~/sonarqube && cd ~/sonarqube
cat > compose.yaml <<'EOF'
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar_password_a_changer
      POSTGRES_DB: sonarqube
    volumes: [pgdata:/var/lib/postgresql/data]
    restart: unless-stopped
  sonarqube:
    image: sonarqube:community
    depends_on: [db]
    ports: ["9000:9000"]
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://db:5432/sonarqube
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar_password_a_changer
    volumes:
      - sonar_data:/opt/sonarqube/data
      - sonar_extensions:/opt/sonarqube/extensions
      - sonar_logs:/opt/sonarqube/logs
    ulimits:
```

```

    nofile: { soft: 131072, hard: 131072 }
    restart: unless-stopped
volumes: { pgdata: {}, sonar_data: {}, sonar_extensions: {}, sonar_logs: {} }
EOF
sudo docker compose up -d
sudo docker compose logs -f sonarqube      # attends "SonarQube is operational"

```

🧠 'VM.MAX_MAP_COUNT'

Elasticsearch a besoin de nombreuses zones de mémoire mappée. Avec la valeur par défaut de Linux (65 530), SonarQube s'arrête au démarrage avec `max virtual memory areas vm.max_map_count [65530] is too low`. C'est l'équivalent des modifications de `/etc/sysctl.conf` et `/etc/security/limits.conf` du tutoriel d'origine.

Ouvre `http://<IP_SONAR>:9000` → `admin` / `admin` → change le mot de passe.

Relier SonarQube à Jenkins :

1. SonarQube → *My Account* → *Security* → token **Global Analysis Token** `jenkins-sonarqube-token` → copie-le.
2. Jenkins → *Credentials* → *Secret text*, ID `jenkins-sonarqube-token`.
3. Jenkins → *System* → *SonarQube servers* : nom `sonarqube-server`, URL `http://<IP_PRIVÉE_SONAR>:9000`, token `jenkins-sonarqube-token`.
4. Jenkins → *Tools* → *SonarQube Scanner installations* : `sonarqube-scanner` (auto-install).
5. SonarQube → *Administration* → *Configuration* → *Webhooks* → *Create* : `sonarqube-webhook`, URL `http://<IP_PRIVÉE_JENKINS>:8080/sonarqube-webhook/`.

Étape 4 — Le cluster EKS et le serveur bootstrap

1. Instance `EKS-Bootstrap-Server` (Ubuntu, `t3.micro`) avec un rôle IAM `eksctl_role` (*AdministratorAccess* pour ce lab).
2. Installe AWS CLI, kubectl et eksctl (partie 0, étape 5).
3. Crée le cluster :

```

eksctl create cluster --name io-register --region eu-west-3 \
  --node-type t3.medium --nodes 2 --nodes-min 2 --nodes-max 3 --managed
kubectl get nodes

```

BASH

(Le tutoriel demande 3 nœuds `t2.small` ; 2 `t3.medium` offrent plus de capacité pour un coût proche.)

Étape 5 — Argo CD

```

kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl get pods -n argocd -w

# CLI argocd (dernière version)
curl -sSL -o argocd https://github.com/argoproj/argo-cd/releases/latest/download/argocd-linux-amd64
sudo install -m 555 argocd /usr/local/bin/argocd && rm argocd

# Exposer l'interface (le tutoriel utilise un LoadBalancer ; port-forward est l'alternative gratuite)
kubectl patch svc argocd-server -n argocd -p '{"spec": {"type": "LoadBalancer"}}'
kubectl get svc argocd-server -n argocd      # EXTERNAL-IP au bout de 2 minutes

# Mot de passe initial de l'utilisateur admin
argocd admin initial-password -n argocd

```

BASH

Ouvre `https://<EXTERNAL-IP>` (accepte le certificat auto-signé), connecte-toi en `admin`, puis **change le mot de passe** (*User Info* → *Update password*).

```

argocd login <EXTERNAL-IP> --username admin --insecure
argocd cluster list

```

BASH

📌 AJOUTER LE CLUSTER EKS À ARGO CD : NÉCESSAIRE ?

Le tutoriel ajoute le cluster EKS avec `argocd cluster add <contexte>`. **Argo CD tourne déjà dans ce cluster** et le connaît sous le nom `in-cluster` (`https://kubernetes.default.svc`) : on peut l'utiliser directement. `argocd cluster add` sert quand un Argo CD central pilote **plusieurs** clusters (dev, prod...). Fais-le pour t'entraîner :

```

kubectl config get-contexts
argocd cluster add <nom-du-contexte-eks> --name io-register-eks
argocd cluster list

```

BASH

Étape 6 — Le dépôt GitOps et le pipeline CD

1. Fork `github.com/MSFaizi/gitops-register-app`. Il contient `deployment.yaml`, `service.yaml` et un `Jenkinsfile`.

2. Dans ton fork : `deployment.yaml` → image `<toi>/register-app-pipeline:1.0.0-1` ; `Jenkinsfile` → URL de ton dépôt dans le `git` et le `git push`, ton nom et ton e-mail dans `git config`.

Le Jenkinsfile CD, expliqué :

```
pipeline {
  agent { label "Jenkins-Agent" }
  environment { APP_NAME = "register-app-pipeline" }
  stages {
    stage("Cleanup Workspace") { steps { cleanWs() } }
    stage("Checkout from SCM") {
      steps { git branch: 'main', credentialsId: 'github', url: 'https://github.com/<toi>/gitops-register-app' }
    }
    stage("Update the Deployment Tags") {
      steps {
        // remplace "register-app-pipeline:<ancien tag>" par "register-app-pipeline:<IMAGE_TAG>"
        sh """
          sed -i 's/${APP_NAME}.${APP_NAME}:${IMAGE_TAG}/g' deployment.yaml
          cat deployment.yaml
        """
      }
    }
    stage("Push the changed deployment file to git") {
      steps {
        sh """
          git config --global user.name "<ton nom>"
          git config --global user.email "<ton e-mail>"
          git add deployment.yaml
          git commit -m "Updated Deployment manifest to ${IMAGE_TAG}"
        """
        withCredentials([gitUsernamePassword(credentialsId: 'github', gitToolName: 'Default')]) {
          sh "git push https://github.com/<toi>/gitops-register-app main"
        }
      }
    }
  }
}
```

3. Connecter le dépôt à Argo CD : *Settings* → *Repositories* → *Connect repo* → HTTPS → URL de ton fork (dépôt public : pas d'identifiants ; privé : ton pseudo + un **token**, jamais ton mot de passe).

4. Créer l'Application : *New App* → nom `register-app`, projet `default`, *Sync policy* **Automatic** + **Prune resources** + **Self heal**, dépôt = ton fork, révision `HEAD`, chemin `./`, cluster `in-cluster` (ou celui ajouté), namespace `default` → *Create*.

```
kubectl get pods
kubectl get svc register-app-service      # EXTERNAL-IP
```

Ouvre `http://<EXTERNAL-IP>:8080/webapp/` : l'application d'inscription s'affiche.

5. Le job CD dans Jenkins : *New Item* `gitops-register-app-cd` → Pipeline → *Discard old builds* → **This project is parameterized** → *String Parameter* `IMAGE_TAG` → *Triggers*: **Trigger builds remotely** → token `gitops-token` → *Pipeline script from SCM* → ton fork gitops, branche `*/main` → *Save*.

6. Le jeton d'API Jenkins (pour que la CI puisse déclencher la CD) : ton profil Jenkins → *Security* → *API Token* → *Add new token* `JENKINS_API_TOKEN` → copie. Puis *Credentials* → *Secret text*, ID `JENKINS_API_TOKEN`.

7. L'étape qui relie CI et CD, dans le Jenkinsfile de `register-app` :

```
stage("Trigger CD Pipeline") {
  steps {
    sh """
      curl -s -f --user <ton-user-jenkins>:${JENKINS_API_TOKEN} -X POST \
        --data 'IMAGE_TAG=${IMAGE_TAG}' \
        'http://<IP_PRIVÉE_MASTER>:8080/job/gitops-register-app-cd/buildWithParameters?token=gitops-token'
    """
  }
}
```

POURQUOI SÉPARER CI ET CD EN DEUX DÉPÔTS ET DEUX JOBS

- Le dépôt **applicatif** contient le code ; le dépôt **GitOps** contient l'état désiré du cluster. Les droits peuvent différer (les développeurs poussent le code ; seule la CI modifie les manifests).
- Chaque déploiement est un **commit** dans le dépôt GitOps : historique complet, retour arrière par `git revert`.
- La CI **n'a aucun accès au cluster** : c'est Argo CD qui tire les changements. Moins de secrets à distribuer, moins de surface d'attaque.

Étape 7 — Le test de bout en bout

Modifie `webapp/src/main/webapp/index.jsp` dans ton fork `register-app`, commit, push. (Pour un déclenchement automatique : *webhook GitHub* comme au projet 05, ou *Poll SCM* `H/2 * * * *` comme le propose le tutoriel. Le tutoriel indique `*`, c'est-à-dire chaque minute : trop agressif.)

✓ VÉRIFICATION

1. `register-app-ci` démarre : build, tests, analyse SonarQube (visible sur le tableau de bord SonarQube), image poussée sur Docker Hub avec le tag `1.0.0-<build>`, rapport Trivy dans la console.
2. `gitops-register-app-cd` démarre tout seul avec `IMAGE_TAG=1.0.0-<build>`.
3. Un nouveau commit « Updated Deployment manifest to 1.0.0- » apparaît dans le dépôt GitOps.
4. Argo CD passe brièvement *OutOfSync* puis *Synced* ; les pods sont remplacés progressivement (`kubectl get pods -w`).
5. La page `http://<EXTERNAL-IP>:8080/webapp/` affiche ta modification.
6. **Self-heal** : `kubectl scale deploy register-app-deployment --replicas=5`. Quelques secondes plus tard, Argo CD remet 2 replicas (la valeur de Git).

Dépannage

🔧 DÉPANNAGE

- **Le job CD ne démarre pas** : `curl` avec `-f` fait échouer l'étape en cas d'erreur HTTP. 401 → utilisateur ou jeton faux ; 403 → *Trigger builds remotely* non coché ou token `gitops-token` différent ; *Connection refused* → mauvaise IP/port.
- **git push refusé dans le job CD** : le credential `github` doit contenir un **token** avec le droit *Contents: Read and write* sur le dépôt GitOps.
- **sed ne remplace rien** : la ligne `image:` doit contenir exactement `register-app-pipeline` (variable `APP_NAME`).
- **Argo CD reste OutOfSync** : pas de synchro automatique (vérifie *Sync Policy*), ou erreur YAML : clique sur l'application pour lire le message.
- **SonarQube ne démarre pas** : `docker compose logs sonarqube` ; presque toujours `vm.max_map_count` ou manque de mémoire.
- **Trivy en Docker** : *permission denied sur docker.sock* : l'utilisateur de l'agent n'est pas dans le groupe docker (redémarre l'agent après l'ajout).

Nettoyage

👉 NETTOYAGE OBLIGATOIRE

1. Supprime l'Application dans Argo CD (avec *cascade* : supprime aussi le Deployment et le Service, donc le load balancer de l'appli).
2. `kubectl delete svc argocd-server -n argocd` (ou repasse-le en ClusterIP) pour supprimer son load balancer.
3. `eksctl delete cluster --name io-register --region eu-west-3`.
4. *Terminate* les 4 instances EC2 ; supprime le rôle `eksctl_role`, les SG.
5. Supprime les webhooks GitHub ; garde tes forks et ton image Docker Hub.

🔑 QUESTIONS D'ENTRETIEN

- Décris un flux complet du commit à la production avec Jenkins et Argo CD.
- Pourquoi la CI ne fait-elle pas directement `kubectl apply` ?
- Comment un job Jenkins en déclenche-t-il un autre avec un paramètre ?
- Que font *prune* et *self-heal* dans Argo CD ?
- Comment revenir à la version précédente de l'application dans ce modèle ? (`git revert` du dernier commit du dépôt GitOps.)

Projet 17 — Déployer une application sur AKS avec Azure DevOps et Cloud Shell

FICHE PROJET

Niveau	★☆☆☆☆ Débutant+
Durée	2 à 4 heures
Coût	~0,10 \$/h (AKS 1 nœud + ACR Basic + IP publique)
Compétences	Azure Cloud Shell, extension azure-devops de la CLI, AKS en ligne de commande, ACR, rôle AcrPull, pipeline Azure DevOps généré pour AKS (environnement Kubernetes, imagePullSecret), GitHub comme source
Prérequis	Projet 07 (ou au moins un compte Azure et une organisation Azure DevOps)
Dépôt d'origine	DevOps-Project-17/README.md ; application github.com/ghostinthewires/k8s-application (Node.js)

Ce qu'on construit

```
GitHub (fork de k8s-application) → push → Azure Pipelines
├─ Build : docker build/push → ACR (ksapplication:<BuildId>)
└─ Deploy : environnement « k8s-application.default »
    kubectl apply manifests/ → AKS « myakscluster »
    Service LoadBalancer :8888
```

Ce projet montre la voie **rapide** : tout se fait depuis **Azure Cloud Shell** (un terminal dans le navigateur, déjà authentifié, avec `az`, `kubectl`, `git` et `helm`), et Azure DevOps **génère** le pipeline pour toi. Tu le liras ensuite ligne par ligne.

Étape 1 — Cloud Shell et Azure DevOps CLI

Ouvre <https://shell.azure.com> → **Bash**. Puis :

```
az extension add --name azure-devops
az devops configure --defaults organization=https://dev.azure.com/<ton-organisation>/
az devops login
# colle un Personal Access Token Azure DevOps (User settings → Personal access tokens)
az devops project create --name k8s-project
az devops configure --defaults project=k8s-project
```

💡 LE PAT AZURE DEVOPS

Le tutoriel d'origine exporte le jeton dans `AZURE_DEVOPS_EXT_PAT`. `az devops login` fait la même chose en stockant le jeton de façon plus sûre. Donne au PAT une durée courte (7 à 30 jours) et seulement les droits nécessaires (*Build: Read & execute, Project and Team, Service Connections, Code*).

Étape 2 — L'infrastructure : AKS et ACR

```
RG=my-aks-rg
ACR=ioacr$RANDOM # unique au monde, minuscules et chiffres
az group create --location francecentral --resource-group $RG
az acr create -g $RG -n $ACR --sku Basic
az aks create -g $RG -n myakscluster \
  --node-count 1 --node-vm-size Standard_B2s \
  --generate-ssh-keys \
  --attach-acr $ACR # donne aux nœuds le rôle AcrPull sur ce registre
az aks get-credentials -g $RG -n myakscluster
kubectl get nodes
```

i CE QUI CHANGE PAR RAPPORT AU DÉPÔT D'ORIGINE

Le tutoriel (2020) crée un **service principal** avec un mot de passe (`az ad sp create-for-rbac`), le donne au cluster, active le compte admin de l'ACR, puis attribue le rôle `acrpull` à la main. Aujourd'hui, AKS utilise par défaut une **identité managée** (aucun secret à gérer) et `--attach-acr` attribue le rôle `AcrPull` en une option. Résultat identique, sans mot de passe. L'erreur « 400 Client Error ... localhost:50342 » citée par le tutoriel concernait cette ancienne méthode.

Étape 3 — Le code de l'application

1. Fork github.com/ghostinthewires/k8s-application sur ton GitHub.
2. Dans Cloud Shell :

```
git clone https://github.com/<toi>/k8s-application.git && cd k8s-application
ls
# Dockerfile app.js package.json views/ manifests/ azure-pipelines.yml
cat Dockerfile manifests/*.yaml
```

BASH

C'est une petite application Node.js (Express) qui écoute sur le port **8888**. Le dossier `manifests/` contient un `deployment.yml` et un `service.yml` (type `LoadBalancer`). Le fichier `azure-pipelines.yml` existant est celui de l'auteur : **supprime-le** pour que l'assistant en génère un pour **ton** ACR et **ton** cluster.

```
git rm azure-pipelines.yml && git commit -m "Retire le pipeline de l'auteur" && git push
```

BASH

Étape 4 — Créer le pipeline

Méthode recommandée (interface web) : Azure DevOps → projet `k8s-project` → *Pipelines* → *New pipeline* → **GitHub** (autorise l'application *Azure Pipelines* sur ton compte GitHub, limite-la à ce dépôt) → choisis `k8s-application` → modèle **Deploy to Azure Kubernetes Service** → abonnement → cluster `myaksccluster` → namespace **Existing : default** → registre `$ACR` → nom d'image `ksapplication` → port **8888** → *Validate and configure* → relis le YAML → *Save and run* (commit direct sur `master` / `main`).

Méthode CLI (celle du tutoriel) : `az pipelines create --name "k8s-application-pipeline"` lance le même assistant dans le terminal. (*GitHub n'accepte plus les mots de passe : l'assistant te demandera un **token GitHub**, pas ton mot de passe.*)

Étape 5 — Lire le pipeline généré

```
trigger:
- master
# chaque commit sur master lance le pipeline

variables:
  dockerRegistryServiceConnection: '<id>' # connexion à ton ACR, créée par l'assistant
  imageRepository: 'ksapplication'
  containerRegistry: '<acr>.azurecr.io'
  dockerfilePath: '**/Dockerfile'
  tag: '$(Build.BuildId)' # chaque build produit un tag unique
  imagePullSecret: '<acr>—auth' # secret créé dans le cluster pour tirer l'image
  vmImageName: 'ubuntu-latest'

stages:
- stage: Build
  jobs:
  - job: Build
    pool: { vmImage: $(vmImageName) }
    steps:
    - task: Docker@2 # build + push vers ACR
      inputs:
        command: buildAndPush
        repository: $(imageRepository)
        dockerfile: $(dockerfilePath)
        containerRegistry: $(dockerRegistryServiceConnection)
        tags: $(tag)
    - upload: manifests # publie le dossier manifests comme artefact
      artifact: manifests

- stage: Deploy
  dependsOn: Build
  jobs:
  - deployment: Deploy
    environment: 'k8s-application.default' # environnement Azure DevOps lié au namespace "default" du cluster
    strategy:
      runOnce:
        deploy:
          steps:
          - task: KubernetesManifest@1 # crée/maj le secret de pull de l'image
            inputs:
              action: createSecret
              secretName: $(imagePullSecret)
              dockerRegistryEndpoint: $(dockerRegistryServiceConnection)
          - task: KubernetesManifest@1 # applique les manifests en remplaçant l'image par <acr>/ksapplication:<tag>
            inputs:
              action: deploy
              manifests: |
```

YAML

```
$(Pipeline.Workspace)/manifests/deployment.yml
$(Pipeline.Workspace)/manifests/service.yml
imagePullSecrets: $(imagePullSecret)
containers: $(containerRegistry)/$(imageRepository):$(tag)
```

🗨️ CE QUE FAIT LA TÂCHE 'KUBERNETESMANIFEST'

L'option `containers:` réécrit à la volée l'image du `deployment.yml` avec le tag du build : le fichier dans Git n'a pas besoin de connaître le tag. C'est une autre réponse au problème « comment déployer exactement l'image que je viens de construire », différente du `sed` + commit du GitOps (projet 16). L'environnement `k8s-application.default` garde l'historique des déploiements et affiche les ressources Kubernetes dans Azure DevOps (*Pipelines → Environments*).

📖 L'IMAGEPULLSECRET EST-IL NÉCESSAIRE ?

Avec `--attach-acr`, les nœuds ont déjà le droit de tirer l'image : le secret est redondant (mais inoffensif). Le modèle le crée par défaut pour fonctionner aussi avec un cluster qui n'a pas ce rôle.

Étape 6 — Vérifier

```
kubectl get all
kubectl get svc                # EXTERNAL-IP du service
```

BASH

Ouvre <http://<EXTERNAL-IP>:8888> : la page de l'application s'affiche.

✅ VÉRIFICATION

- Le pipeline montre deux stages verts ; *Environments → k8s-application.default* liste le déploiement.
- ACR → *Repositories → ksapplication* avec un tag par build.
- Modifie `views/index.html` (ou le texte dans `app.js`) sur GitHub → commit → nouveau build → la page change.

Dépannage

🔧 DÉPANNAGE

- `No hosted parallelism has been purchased or granted` : voir le projet 07 (demande de gratuité ou agent auto-hébergé).
- `Pod ImagePullBackOff` : nom d'ACR erroné ou rôle `AcrPull` absent (`az aks check-acr -g $RG -n myakscluster --acr $ACR.azurecr.io`).
- `EXTERNAL-IP en <pending> longtemps` : quota d'IP publiques de l'abonnement d'essai atteint ; `kubectl describe svc` donne l'erreur.
- `Quota de vCPU : Standard_B2s` (2 vCPU) suffit ; les abonnements d'essai sont limités à quelques vCPU par région.

Nettoyage

🔧 NETTOYAGE OBLIGATOIRE

```
az group delete -n my-aks-rg --yes --no-wait    # supprime AKS, ACR, IP, disques... (et le groupe MC_... des nœuds)
```

BASH

Dans Azure DevOps, tu peux supprimer le projet `k8s-project` (*Project settings → Overview → Delete*). Révoque le PAT et l'autorisation GitHub si tu ne t'en sers plus.

🔧 QUESTIONS D'ENTRETIEN

- Comment donnes-tu à AKS le droit de tirer des images depuis ACR ?
- Qu'est-ce qu'un environnement Kubernetes dans Azure DevOps ?
- Comment le pipeline déploie-t-il le tag d'image qu'il vient de construire ?
- Service principal avec secret ou identité managée : lequel et pourquoi ?

Projet 18 — Pipeline Jenkins GitOps pour une application Java : Maven, SonarQube, Docker, Argo CD

FICHE PROJET

Niveau	★★★☆☆ Intermédiaire
Durée	5 à 8 heures
Coût	~0,05 \$/h (une EC2 t3.large pour Jenkins + SonarQube). Kubernetes en local (kind/minikube) : gratuit
Compétences	agent Jenkins Docker (le build tourne dans un conteneur), Maven, SonarQube, build/push Docker Hub, mise à jour automatique du manifest dans Git, Argo CD sur un cluster local
Prérequis	Projet 16 (même logique, en plus simple), leçon 12
Dépôt d'origine	DevOps-Project-18 (spring-boot-app/ avec JenkinsFile et Dockerfile , spring-boot-app-manifests/) — inspiré de <i>Jenkins-Zero-To-Hero</i> d'Abhishek Veeramalla

Ce qu'on construit

```
git push → Jenkins (EC2)
  agent = conteneur Docker "maven + docker CLI" (socket Docker de l'hôte monté)
  1. mvn clean package
  2. mvn sonar:sonar → SonarQube (conteneur sur la même EC2)
  3. docker build + push → Docker Hub <toi>/ultimate-cicd:<BUILD_NUMBER>
  4. sed replaceImageTag → <BUILD_NUMBER> dans deployment.yml, git commit + push
      |
      ▼ dépôt Git (dossier spring-boot-app-manifests)
Ton poste : cluster kind ← Argo CD surveille ce dossier, synchronise → Deployment + Service
```

La grande différence avec le projet 16 : **un seul pipeline** fait la CI et met à jour le manifest (pas de job CD séparé), et l'**agent Jenkins est un conteneur** jetable.

Les concepts

UN AGENT JENKINS DOCKER

```
agent {
  docker {
    image 'abhishekf5/maven-abhishek-docker-agent:v1'
    args '--user root -v /var/run/docker.sock:/var/run/docker.sock'
  }
}
```

GROOVY

Jenkins lance un **conteneur** à partir de cette image (qui contient JDK, Maven et le client Docker), exécute toutes les étapes dedans, puis le supprime. Avantages : pas d'outils à installer sur le serveur Jenkins, environnement de build identique à chaque fois, versions faciles à changer. Le montage de `/var/run/docker.sock` permet au conteneur d'utiliser le **Docker de l'hôte** pour construire l'image de l'application (on parle de *Docker-outside-of-Docker*). Attention : cela donne au build un accès équivalent à root sur l'hôte.

LE MARQUEUR 'REPLACEIMAGETAG'

Dans `spring-boot-app-manifests/deployment.yml`, l'image est écrite `.../ultimate-cicd:replaceImageTag`. Le pipeline remplace ce mot par le numéro de build avec `sed`, puis commit. **Problème** : au build suivant, le mot `replaceImageTag` n'existe plus (il a été remplacé par `1`), donc `sed` ne remplace plus rien ! Le dépôt d'origine a ce défaut. On le corrige à l'étape 4 avec une expression régulière qui remplace **n'importe quel** tag.

Étape 1 — Le serveur Jenkins + SonarQube

Instance Ubuntu 24.04 [t3.large](#) (8 Go : Jenkins + SonarQube + builds), 30 Go, ports 22, 8080 et 9000 depuis ton IP (et 8080 depuis GitHub pour le webhook).

```
# Java + Jenkins : annexe A.3
# Docker, et droits pour l'utilisateur jenkins (il doit lancer des conteneurs d'agent)
sudo apt install -y docker.io
sudo usermod -aG docker jenkins
sudo usermod -aG docker ubuntu
sudo systemctl restart jenkins
```

BASH

```
# SonarQube en conteneur (rapide, base embarquée : suffisant pour un lab)
echo "vm.max_map_count=524288" | sudo tee /etc/sysctl.d/99-sonar.conf && sudo sysctl --system
sudo docker run -d --name sonarqube --restart unless-stopped -p 9000:9000 \
-v sonar_data:/opt/sonarqube/data sonarqube:community
```

Plugins Jenkins : **Docker Pipeline**, **SonarQube Scanner** (pas indispensable ici car le pipeline appelle `mvn sonar:sonar` directement).

Étape 2 — Préparer ton dépôt

Crée un dépôt GitHub `java-gitops` et copies-y les dossiers `spring-boot-app/` et `spring-boot-app-manifests/` du projet 18.

```
java-gitops/
├── spring-boot-app/
│   ├── JenkinsFile
│   ├── Dockerfile
│   ├── pom.xml
│   └── src/
└── spring-boot-app-manifests/
    ├── deployment.yml
    └── service.yml
```

Dans `spring-boot-app-manifests/deployment.yml`, remplace l'image par `<toi-dockerhub>/ultimate-cicd:replaceImageTag`.

Dans `spring-boot-app/Dockerfile`, remplace `FROM adoptopenjdk/openjdk11:alpine-jre` (projet AdoptOpenJDK arrêté, images plus maintenues) par `FROM eclipse-temurin:11-jre`, son successeur officiel.

Étape 3 — Les credentials Jenkins

ID	Type	Contenu
<code>sonarqube</code>	Secret text	token généré dans SonarQube (<i>My Account</i> → <i>Security</i>)
<code>docker-cred</code>	Username with password	pseudo Docker Hub + access token
<code>github</code>	Secret text	token GitHub avec <i>Contents: Read and write</i> sur <code>java-gitops</code>

Étape 4 — Le Jenkinsfile, adapté

Le Jenkinsfile d'origine contient des chemins vers le dépôt de l'auteur (`java-maven-sonar-argocd-helm-k8s/...`), son IP SonarQube, son pseudo Docker Hub et son identité Git. Voici la version adaptée à ta structure :

```
pipeline {
  agent {
    docker {
      image 'abhishekf5/maven-abhishek-docker-agent:v1'
      args '--user root -v /var/run/docker.sock:/var/run/docker.sock'
    }
  }
  environment {
    SONAR_URL = "http://<IP_PRIVÉE_EC2>:9000"
    DOCKER_IMAGE = "<toi-dockerhub>/ultimate-cicd:${BUILD_NUMBER}"
    GIT_REPO = "github.com/<toi>/java-gitops"
  }
  stages {
    stage('Build et tests') {
      steps {
        sh 'cd spring-boot-app && mvn -B clean package'
      }
    }
    stage('Analyse SonarQube') {
      steps {
        withCredentials([string(credentialsId: 'sonarqube', variable: 'SONAR_AUTH_TOKEN')]) {
          sh 'cd spring-boot-app && mvn -B sonar:sonar -Dsonar.token=$SONAR_AUTH_TOKEN -Dsonar.host.url=$SONAR_URL'
        }
      }
    }
    stage('Image Docker') {
      steps {
        script {
          sh 'cd spring-boot-app && docker build -t $DOCKER_IMAGE .'
          docker.withRegistry('https://index.docker.io/v1/', 'docker-cred') {
            docker.image("${DOCKER_IMAGE}").push()
          }
        }
      }
    }
  }
}
```

```

    }
  }
}
stage('Mise à jour du manifest') {
  steps {
    withCredentials([string(credentialsId: 'github', variable: 'GITHUB_TOKEN')]) {
      sh '''
        git config user.email "jenkins@exemple.com"
        git config user.name "Jenkins"
        # remplace le tag quel qu'il soit (replaceImageTag, 12, 13...) par le numéro de build
        sed -i -E "s#(ultimate-cicd:).*#\1${BUILD_NUMBER}#" spring-boot-app-manifests/deployment.yml
        git add spring-boot-app-manifests/deployment.yml
        git commit -m "ci: image ultimate-cicd:${BUILD_NUMBER} [skip ci]"
        git push https://${GITHUB_TOKEN}@${GIT_REPO} HEAD:main
      '''
    }
  }
}
}
}
}
}

```

⚠ LA BOUCLE INFINIE

Le pipeline pousse un commit dans **le même dépôt** que celui qui le déclenche. Avec un webhook, ce commit relance le pipeline, qui pousse un commit, qui relance... Solutions : déclencher uniquement sur les changements de [spring-boot-app/](#) (option *Additional Behaviours* → *Polling ignores commits in certain paths / Included Regions*), ignorer les commits de l'utilisateur Jenkins, ou — le plus propre — **mettre les manifests dans un dépôt séparé** (comme au projet 16). Le `[skip ci]` dans le message est respecté par certains outils (GitHub Actions, GitLab), mais pas nativement par Jenkins.

Jenkins → *New Item* → `java-gitops` → **Pipeline** → *Pipeline script from SCM* → Git → ton dépôt → *Script Path* : `spring-boot-app/JenkinsFile` → *Save* → *Build Now*.

✓ VÉRIFICATION

- Le build passe ; SonarQube (`http://<IP>:9000`) montre le projet `spring-boot-demo` analysé.
- Docker Hub contient `ultimate-cicd:1`.
- Dans GitHub, un commit « ci: image ultimate-cicd:1 » a modifié `deployment.yml`.

Étape 5 — Argo CD sur un cluster local

Sur **ton poste** (Docker Desktop ou Docker dans WSL) :

```

kind create cluster --name gitops          # (ou : minikube start)
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl -n argocd rollout status deploy/argocd-server
argocd admin initial-password -n argocd    # (CLI argocd : voir projet 16) ou la commande kubectl de la leçon 12
kubectl -n argocd port-forward svc/argocd-server 8443:443

```

Crée l'Application (dans un fichier `app.yaml`, c'est plus reproductible que les clics) :

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: spring-boot-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/<toi>/java-gitops.git
    targetRevision: main
    path: spring-boot-app-manifests
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
    automated: { prune: true, selfHeal: true }

```

```

kubectl apply -f app.yaml
kubectl get pods
kubectl port-forward svc/spring-boot-app-service 8080:80 # nom et port : voir service.yaml

```

Ouvre `http://localhost:8080` : l'application Spring Boot répond.

🧠 ARGO CD N'A PAS BESOIN D'ÊTRE JOIGNABLE DEPUIS INTERNET

Ton cluster kind est sur ton poste, invisible depuis Internet, et ça fonctionne quand même : c'est **Argo CD qui va chercher** les changements sur GitHub (pull), et les nœuds qui vont chercher l'image sur Docker Hub. C'est l'un des grands avantages du modèle GitOps : on peut déployer dans des clusters privés sans ouvrir aucun port entrant.

Étape 6 — Boucle complète

Modifie un texte de `spring-boot-app/src/main/resources/templates/index.html`, commit, push → Jenkins construit `ultimate-cicd:2` et met à jour le manifest → Argo CD (synchro toutes les 3 minutes par défaut, ou bouton *Refresh*) déploie la nouvelle version.

Dépannage

🔧 DÉPANNAGE

- `docker: permission denied` **au lancement de l'agent** : l'utilisateur `jenkins` n'est pas dans le groupe `docker`, ou Jenkins n'a pas été redémarré après l'ajout.
- `mvn sonar:sonar : Not authorized` : token invalide ; `Connection refused` : utilise l'IP **privée** de l'EC2 (depuis un conteneur, `localhost` désigne le conteneur lui-même, pas l'hôte !).
- `git push : 403` : token GitHub sans droit d'écriture ; `nothing to commit` : le `sed` n'a rien remplacé (vérifie le nom d'image dans `deployment.yml`).
- **Pods ImagePullBackOff dans kind** : nom d'image erroné, ou image construite en ARM sur un Mac M1 et nœud x86 (rare avec kind, qui utilise l'architecture de ton poste).

Nettoyage

🧹 NETTOYAGE OBLIGATOIRE

`kind delete cluster --name gitops` ; `terminate` l'EC2 Jenkins ; supprime le webhook GitHub. Garde le dépôt et l'image.

🗣️ QUESTIONS D'ENTRETIEN

- Quels sont les avantages d'un agent Jenkins Docker ? Ses risques (socket Docker) ?
- Comment évites-tu qu'un pipeline qui commit dans son propre dépôt ne se relance à l'infini ?
- Pourquoi Argo CD peut-il déployer dans un cluster sans IP publique ?
- Un seul dépôt (code + manifests) ou deux dépôts : avantages et inconvénients ?

Projet 19 — De zéro à la production : EKS et CI/CD avec Jenkins et Terraform

FICHE PROJET

Niveau	★★★★☆ Avancé
Durée	6 à 10 heures
Coût	Jenkins t3.large ~0,09 \$/h + EKS (~0,10 \$/h) + 2 nœuds + NAT (~0,05 \$/h) + load balancer. ~0,35 \$/h quand tout tourne
Compétences	Terraform avec modules publics (vpc , security-group , ec2-instance , eks), backends S3 séparés, tfvars par environnement, Jenkins qui exécute Terraform (paramètre apply/destroy, validation manuelle), déploiement sur EKS depuis Jenkins
Prérequis	Projets 11 et 12, leçon 10
Dépôt d'origine	DevOps-Project-19 : jenkins_server/ (Terraform du serveur + script d'installation), tf-aws-eks/ (Terraform du cluster), manifest/ , Jenkinsfile

Ce qu'on construit

Deux couches d'infrastructure, toutes deux en Terraform, la seconde pilotée par un pipeline :

COUCHE 1 (depuis ton poste)	COUCHE 2 (depuis Jenkins)
terraform apply jenkins_server/tf-aws-ec2	pipeline Jenkins (paramètre action = apply destroy)
VPC jenkins-vpc (module vpc)	checkout → terraform init → validate → plan
SG 8080/22/9000 (module security-group)	→ [validation manuelle] → terraform \$action
EC2 Jenkins-Build-Server (module ec2-instance)	VPC my-eks-vpc (module vpc, subnets privés + NAT)
user data : Jenkins, Docker, SonarQube,	EKS my-eks-cluster (module eks, managed node group)
AWS CLI, Terraform, kubectl, Helm, Trivy	→ aws eks update-kubeconfig → kubectl apply nginx
	Service LoadBalancer → navigateur
État : s3://<bucket>/jenkins/terraform.tfstate	État : s3://<bucket>/eks/terraform.tfstate

POURQUOI SÉPARER EN DEUX COUCHES AVEC DEUX ÉTATS ?

Le serveur Jenkins vit longtemps ; le cluster EKS peut être créé et détruit souvent. Deux **états Terraform séparés** permettent de détruire l'un sans risquer l'autre, de limiter l'impact d'une erreur (*blast radius*) et d'accélérer les [plan](#). En entreprise, on découpe ainsi l'infrastructure en couches : réseau, données, plateformes, applications.

Étape 1 — Lire et adapter le Terraform du serveur Jenkins

```
cd ~/IO/projects && mkdir 19-eks-terraform-jenkins && cd 19-eks-terraform-jenkins
cp -r /tmp/dp/DevOps-Project-19/* .
cd jenkins_server/tf-aws-ec2 && ls
# backend.tf data.tf main.tf outputs.tf provider.tf variables.tf variables/{dev,test,prod}.tfvars
```

BASH

`main.tf` utilise trois **modules du Terraform Registry** ([terraform-aws-modules/...](#)) : un module crée en quelques lignes ce qui demanderait des dizaines de ressources.

À corriger avant de lancer :

1. `backend.tf` : ton bucket (`io-tfstate-<prenom>`), clé `jenkins/terraform.tfstate`, région `eu-west-3`, `use_lockfile = true`.
2. `variables/dev.tfvars` : `aws_region = "eu-west-3"`, ton `aws_account_id`, `instance_type = "t3.large"`.
3. L'AMI codée en dur : `ami = "ami-0e8a34246278c21e4"` est un identifiant d'Amazon Linux 2 **dans us-east-1** uniquement. Un ID d'AMI est **propre à une région** : dans `eu-west-3`, il n'existe pas. Remplace-le par une recherche dynamique dans `data.tf` :

```
data "aws_ami" "al2023" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name = "name"
    values = ["al2023-ami-2023.*-x86_64"]
  }
}
```

HCL

puis dans `main.tf` : `ami = data.aws_ami.al2023.id`. (Le script `install_build_tools.sh` utilise `yum` : il fonctionne sur Amazon Linux 2023, où `yum` est un alias de `dnf`. Le paquet `docker` et `java-17-amazon-corretto-devel` existent sur AL2023.)

4. **Security Group** : le module ouvre 22, 8080 et 9000 à `0.0.0.0/0`. Restreins au moins **22** à ton IP.

5. **Rôle IAM de l'instance** : le pipeline exécutera Terraform et kubectl **depuis Jenkins**. Plutôt que de stocker des clés AWS dans Jenkins (ce que fait le Jenkinsfile d'origine), donne un **rôle** à l'instance. Ajoute au module `ec2_instance` :

```
create_iam_instance_profile = true
iam_role_policies = {
  admin = "arn:aws:iam::aws:policy/AdministratorAccess" # lab uniquement ; restreindre en entreprise
}
```

Étape 2 — Créer le serveur Jenkins

```
terraform init
terraform plan -var-file=variables/dev.tfvars
terraform apply -var-file=variables/dev.tfvars
terraform output # IP publique du serveur
```

Le user data (`../scripts/install_build_tools.sh`) installe Jenkins, Git, Docker, un conteneur SonarQube, AWS CLI, Terraform, kubectl, Helm et Trivy. Patiente 5 minutes, puis :

```
ssh -i ~/.ssh/io-key.pem ec2-user@<IP>
sudo cat /var/log/cloud-init-output.log | tail -30 # le script s'est-il bien terminé ?
jenkins --version; docker ps; terraform -version; kubectl version --client; aws sts get-caller-identity
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Configure Jenkins (`http://<IP>:8080`) : plugins suggérés + **Pipeline: Stage View**.

Étape 3 — Le Terraform du cluster EKS

```
cd ../../tf-aws-eks && ls
# backend.tf data.tf eks.tf provider.tf variables.tf vpc.tf variables/{dev,test,prod}.tfvars
```

`vpc.tf` crée un VPC avec subnets **publics et privés**, une NAT Gateway unique, et surtout les **tags** dont EKS a besoin :

```
public_subnet_tags = { "kubernetes.io/role/elb" = 1 } # où créer les load balancers publics
private_subnet_tags = { "kubernetes.io/role/internal-elb" = 1 } # où créer les load balancers internes
```

`eks.tf` appelle le module `terraform-aws-modules/eks/aws` : cluster dans les subnets **privés** (les nœuds n'ont pas d'IP publique), endpoint d'API public, un *managed node group*.

À corriger :

1. `backend.tf` : ton bucket, clé `eks/terraform.tfstate`, `eu-west-3`.
2. `variables/dev.tfvars` : région et CIDR.
3. `eks.tf` :
 - `cluster_version` : supprime la ligne ou mets une version **actuellement supportée** par EKS (la 1.29 du dépôt est hors support standard, donc facturée en *extended support*, 6 fois plus cher !);
 - `instance_types = ["t3.medium"]` (le `t2.small` d'origine est trop petit) ; tu peux garder `capacity_type = "SPOT"` (instances jusqu'à 70 % moins chères, qu'AWS peut reprendre) ;
 - ajoute `enable_cluster_creator_admin_permissions = true`.

⚠ 'ENABLE_CLUSTER_CREATOR_ADMIN_PERMISSIONS'

Depuis la version 20 du module EKS, l'identité qui crée le cluster **n'est plus administratrice Kubernetes par défaut**. Sans cette option, Terraform crée le cluster, mais l'étape `kubectl apply` du pipeline échoue avec `Unauthorized`. C'est l'erreur la plus fréquente de ce projet.

Pousse le tout dans ton dépôt GitHub (`terraform-eks-cicd`), avec `tf-aws-eks/`, `manifest/` et le `Jenkinsfile` à la racine.

Étape 4 — Le pipeline Jenkins

Le Jenkinsfile d'origine lit `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` depuis les credentials Jenkins. Avec le rôle IAM de l'instance (étape 1), c'est **inutile** : supprime ce bloc. Version corrigée et rejouable :

```
pipeline {
  agent any
  parameters {
    choice(name: 'action', choices: ['apply', 'destroy'], description: 'Créer ou détruire le cluster')
  }
}
```

```

environment {
    AWS_DEFAULT_REGION = 'eu-west-3'
}
stages {
    stage('Checkout') {
        steps { git branch: 'main', url: 'https://github.com/<toi>/terraform-eks-cicd.git' }
    }
    stage('Terraform init') {
        steps { dir('tf-aws-eks') { sh 'terraform init -input=false' } }
    }
    stage('Terraform validate') {
        steps { dir('tf-aws-eks') { sh 'terraform validate' } }
    }
    stage('Terraform plan') {
        steps {
            dir('tf-aws-eks') {
                script {
                    def destroyFlag = params.action == 'destroy' ? '-destroy' : ''
                    sh "terraform plan -input=false ${destroyFlag} -var-file=variables/dev.tfvars -out=tfplan"
                }
            }
            input(message: "Appliquer ce plan (${params.action}) ?", ok: 'Confirmer')
        }
    }
    stage('Terraform apply/destroy') {
        steps { dir('tf-aws-eks') { sh 'terraform apply -input=false tfplan' } }
    }
    stage('Déployer Nginx') {
        when { expression { params.action == 'apply' } } // pas de déploiement après un destroy
        steps {
            dir('manifest') {
                sh '''
                    aws eks update-kubeconfig --name my-eks-cluster
                    kubectl create namespace eks-nginx-app --dry-run=client -o yaml | kubectl apply -f -
                    kubectl apply -f deployment.yaml -n eks-nginx-app
                    kubectl apply -f service.yaml -n eks-nginx-app
                    kubectl rollout status deploy/nginx -n eks-nginx-app --timeout=180s
                '''
            }
        }
    }
}
}

```

🔴 LES AMÉLIORATIONS PAR RAPPORT À L'ORIGINAL

- **input(...)** : le pipeline **s'arrête** et attend qu'un humain lise le plan et clique *Confirmer*. C'est la validation manuelle avant de modifier l'infrastructure.
- **plan -out=tfplan puis apply tfplan** : on applique exactement le plan validé.
- **plan -destroy** : avec **action=destroy**, le plan affiché est celui de la destruction (dans l'original, on validait un plan de création avant de détruire !).
- **when { ... 'apply' }** : on ne déploie pas l'application juste après avoir détruit le cluster (l'original le faisait, et échouait).
- **create namespace ... | kubectl apply** : rejouable (l'original échouait au 2^e lancement car le namespace existait déjà).

Jenkins → *New Item* **eks-cicd** → **Pipeline** → *Pipeline script from SCM* → ton dépôt → **Jenkinsfile** → *Save*. Le premier lancement découvre le paramètre ; les suivants proposent **Build with Parameters**.

Étape 5 — Lancer

Build with Parameters → **action = apply** → au stage *Terraform plan*, survole l'étape et clique **Confirmer** après avoir lu le plan dans la console. 15 à 20 minutes plus tard :

```

# sur le serveur Jenkins (ou ton poste avec les bons droits)
aws eks update-kubeconfig --name my-eks-cluster --region eu-west-3
kubectl get nodes
kubectl get all -n eks-nginx-app
kubectl get svc nginx -n eks-nginx-app      # EXTERNAL-IP → page "Welcome to nginx!"

```

BASH

✅ VÉRIFICATION

- Les 6 stages sont verts ; la console montre le plan complet (VPC, subnets, NAT, EKS, node group, rôles IAM...).
- La page Nginx s'affiche par le nom DNS du load balancer.
- Deux fichiers d'état existent dans S3 : **jenkins/terraform.tfstate** et **eks/terraform.tfstate**.

Dépannage

🔑 DÉPANNAGE

- `InvalidAMIID.NotFound` : l'AMI codée en dur (étape 1, point 3).
- `Unauthorized` au `kubectl apply` : `enable_cluster_creator_admin_permissions` manquant, ou Jenkins utilise une autre identité que celle qui a créé le cluster.
- `terraform: command not found dans Jenkins` : le user data n'a pas fini ou a échoué (`/var/log/cloud-init-output.log`).
- `Error acquiring the state lock` : un build interrompu. Après vérification : `terraform force-unlock <ID>` depuis le serveur.
- **Le destroy échoue sur le VPC (`DependencyViolation`)** : le load balancer créé par le Service Kubernetes existe encore. Supprime d'abord `kubectl delete svc nginx -n eks-nginx-app`. Ajoute idéalement cette commande en tête d'un stage « pré-destroy ».
- **Nœuds Spot non disponibles** : passe `capacity_type` à `ON_DEMAND`.

Nettoyage

🔑 NETTOYAGE OBLIGATOIRE

1. `kubectl delete svc nginx -n eks-nginx-app` (supprime le load balancer).
2. Pipeline avec `action = destroy` (ou `terraform destroy -var-file=variables/dev.tfvars` dans `tf-aws-eks/`).
3. Depuis ton poste : `terraform destroy -var-file=variables/dev.tfvars` dans `jenkins_server/tf-aws-ec2/`.
4. Vérifie NAT Gateways, Elastic IPs, load balancers, volumes EBS.

🔑 QUESTIONS D'ENTRETIEN

- Pourquoi utiliser les modules publics `terraform-aws-modules` ? Quels risques ? (*Gain de temps, bonnes pratiques intégrées ; il faut épingler les versions et lire les changements entre versions majeures.*)
- Pourquoi séparer l'état du serveur Jenkins et celui du cluster ?
- Comment ajoutes-tu une validation humaine dans un pipeline Terraform ?
- Pourquoi un ID d'AMI ne fonctionne-t-il que dans une région ?
- À quoi servent les tags `kubernetes.io/role/elb` sur les subnets ?

Projet 20 — Déployer du Terraform sur Azure avec un pipeline Azure DevOps

FICHE PROJET

Niveau	★★★☆☆ Intermédiaire
Durée	3 à 5 heures
Coût	quelques centimes (Service Bus Standard ~0,013 \$/h, stockage de l'état)
Compétences	service principal (ou fédération d'identité), RBAC Azure, backend Terraform <code>azurerm</code> , variable groups et secrets, templates de pipeline réutilisables, environnements et approbations , déploiement multi-environnements
Prérequis	Projet 07
Dépôt d'origine	DevOps-Project-20 : <code>terraform/</code> (Service Bus + 2 files), <code>deploy/tfdemo-env01-terraform.yml</code> , <code>deploy/templates/terraform-template.yml</code>

Ce qu'on construit

Un pipeline Terraform **propre et réutilisable** sur Azure, qui déploie un **Azure Service Bus** (un service de files de messages) avec deux files. L'accent est mis sur les points que les tutoriels survolent : les **droits** de l'identité de déploiement, la gestion des **secrets**, l'**approbation** avant `apply`, et le **passage à plusieurs environnements** sans copier-coller.

```
Azure Repos
terraform/      (main.tf, servicebus.tf, variables.tf, providers.tf, env01.tfvars, env02.tfvars)
deploy/
  templates/terraform-template.yml  ← la logique plan/apply, écris UNE fois
  tfdemo-env01-terraform.yml        ← pipeline env01 : appelle le template avec ses paramètres
  tfdemo-env02-terraform.yml        ← pipeline env02 : idem
  |
  ▼ agent hébergé (ubuntu-latest)
Stage Plan → Stage Apply (job "deployment" lié à l'environnement env01 → APPROBATION)
  | identité : tfdemo-spn (Contributor sur l'abonnement, Storage Blob Data Contributor sur l'état)
  ▼
tfstate-tfdemo-rg / tfstatetfdemostg / tfstate / tfdemo.env01.tfstate
tfdemo-env01-rg : Service Bus namespace « tfdemo-env01-sbns » + queue01 + queue02
```

AZURE SERVICE BUS EN DEUX MOTS

Une **file de messages** permet à deux applications de communiquer de façon **asynchrone** : l'une dépose des messages (une commande à traiter), l'autre les consomme à son rythme. Si le consommateur tombe, les messages attendent dans la file au lieu d'être perdus. C'est l'équivalent d'**Amazon SQS** ou de **RabbitMQ** (que tu as vu dans Robot Shop).

Étape 1 — L'identité de déploiement

Le tutoriel d'origine crée une **App registration** `tfdemo-spn` avec un **secret client**. Méthode :

- Portail Azure → **Microsoft Entra ID** → *App registrations* → *New registration* → `tfdemo-spn`.
- Note l'**Application (client) ID** et le **Directory (tenant) ID**.
- Certificates & secrets* → *New client secret* → description `ADO`, expiration courte → **note la valeur** (affichée une seule fois).

💡 MIEUX : LA FÉDÉRATION D'IDENTITÉ, SANS SECRET

Comme au projet 07, tu peux créer une *service connection* Azure Resource Manager en **Workload Identity Federation** et utiliser la tâche `AzureCLI@2` avec `addSpnToEnvironment: true` pour fournir à Terraform des identifiants **temporaires** (variables `ARM_USE_OIDC=true`, `ARM_OIDC_TOKEN` ...). Plus de secret qui expire ou qui fuit. On suit ici la méthode du tutoriel (plus simple à comprendre), puis on te montre la variante en fin de projet.

Étape 2 — Le stockage de l'état et les droits

```
az group create -n tfstate-tfdemo-rg -l francecentral
az storage account create -n tfstatetfdemo$RANDOM -g tfstate-tfdemo-rg -l francecentral --sku Standard_LRS
az storage container create --name tfstate --account-name <nom-du-storage> --auth-mode login
```

BASH

Puis deux **attributions de rôles** pour `tfdemo-spn` :

Rôle	Scope	Pourquoi
Storage Blob Data Contributor	le storage account de l'état	lire/écrire le fichier d'état
Contributor	l'abonnement	créer les groupes de ressources et le Service Bus

```
SPN_ID=<application-client-id>
SUB=$(az account show --query id -o tsv)
az role assignment create --assignee $SPN_ID --role "Storage Blob Data Contributor" \
  --scope $(az storage account show -n <nom-du-storage> --query id -o tsv)
az role assignment create --assignee $SPN_ID --role "Contributor" --scope /subscriptions/$SUB
```

BASH

🗯 LE MOINDRE PRIVILÈGE, VERSION AZURE

Contributor sur tout l'abonnement, c'est large. Alternative proposée par le tutoriel : **créer à l'avance** le groupe de ressources cible et donner *Contributor* **uniquement sur ce groupe**. Terraform ne crée alors plus le groupe (on le lit avec un bloc `data "azurerm_resource_group"`). Plus de travail initial, beaucoup moins de risque.

Étape 3 — Le code Terraform

Importe le dossier `DevOps-Project-20` dans un dépôt Azure Repos. Points à adapter :

`terraform/providers.tf` — le backend pointe vers le storage de l'état ; ajoute `use_azuread_auth = true` pour s'authentifier par Entra ID plutôt que par clé de stockage :

```
terraform {
  required_providers {
    azurerm = { source = "hashicorp/azurerm", version = "~> 4.0" }
    random  = { source = "hashicorp/random",  version = "~> 3.0" }
  }
  backend "azurerm" {
    resource_group_name = "tfstate-tfdemo-rg"
    storage_account_name = "<nom-du-storage>"
    container_name       = "tfstate"
    key                  = "tfdemo.env01.tfstate"
    use_azuread_auth     = true
  }
}
provider "azurerm" {
  features {}
  # subscription_id est obligatoire avec azurerm v4 : fourni par la variable d'environnement ARM_SUBSCRIPTION_ID
}
```

HCL

⚠ AZURERM V3 → V4

Le dépôt utilise `azurerm ~> 3.0`. Avec la v4 (actuelle), quelques attributs ont été renommés : dans `azurerm_servicebus_queue`, `enable_batched_operations` → `batched_operations_enabled`, `enable_partitioning` → `partitioning_enabled`, `enable_express` → `express_enabled`. Si `terraform validate` signale un argument inconnu, c'est presque toujours un renommage de ce type : consulte la page de la ressource dans la documentation du provider (section *Upgrade guide*). Tu peux aussi garder `~> 3.0` pour coller au tutoriel.

Le fichier de variables : le pipeline attend `tfdemo.env01.tfvars`, **absent du dépôt**. Crée-le dans `terraform/` :

```
project      = "tfdemo"
environment  = "env01"
location     = "francecentral"
```

HCL

(Les noms exacts des variables sont dans `variables.tf`.)

Étape 4 — Le variable group des secrets

Azure DevOps → *Pipelines* → *Library* → + *Variable group* → nom `Terraform-SPN` :

Variable	Valeur	Secret ?
<code>ARM_CLIENT_ID</code>	Application (client) ID	non
<code>ARM_TENANT_ID</code>	Directory (tenant) ID	non
<code>ARM_SUBSCRIPTION_ID</code>	ID de l'abonnement	non
<code>ARM_CLIENT_SECRET</code>	la valeur du secret	oui (cadenas)

Le provider `azurerm` lit **automatiquement** ces variables d'environnement `ARM_*` pour s'authentifier.

⚠ LES VARIABLES SECRÈTES NE SONT PAS DES VARIABLES D'ENVIRONNEMENT

Azure DevOps expose automatiquement les variables **non secrètes** comme variables d'environnement des scripts. Les variables **secrètes**, non : il faut les passer explicitement avec `env:`. C'est pour cela que le template contient :

```
env:
  ARM_CLIENT_SECRET: $(ARM_CLIENT_SECRET)  # this needs to be explicitly set as it's a sensitive value
```

Oublier ce bloc donne une erreur d'authentification déroutante, alors que la variable « existe » bien.

Étape 5 — Les pipelines : un template, plusieurs environnements

`deploy/templates/terraform-template.yml` — la logique commune, paramétrée :

```
parameters:
- name: rootFolder
  type: string
- name: tfvarsFile
  type: string
- name: adoEnvironment
  type: string

stages:
- stage: Terraform_Plan
  jobs:
  - job: Terraform_Plan
    pool: { vmImage: ubuntu-latest }
    steps:
    - script: |
        terraform init -input=false
        terraform plan -input=false -var-file ${ parameters.tfvarsFile } -out=tfplan
      displayName: Terraform plan
      workingDirectory: ${ parameters.rootFolder }
      env:
        ARM_CLIENT_SECRET: $(ARM_CLIENT_SECRET)
    - publish: ${ parameters.rootFolder }/tfplan  # amélioration : on garde le plan validé
      artifact: tfplan

- stage: Terraform_Apply
  dependsOn: Terraform_Plan
  condition: succeeded()
  jobs:
  - deployment: Terraform_Apply
    pool: { vmImage: ubuntu-latest }
    environment: ${ parameters.adoEnvironment }  # ← l'approbation se configure sur cet environnement
    strategy:
      runOnce:
        deploy:
          steps:
          - checkout: self
          - download: current
            artifact: tfplan
          - script: |
              terraform init -input=false
              terraform apply -input=false $(Pipeline.Workspace)/tfplan/tfplan
            displayName: Terraform apply
            workingDirectory: ${ parameters.rootFolder }
            env:
              ARM_CLIENT_SECRET: $(ARM_CLIENT_SECRET)
```

`deploy/tfdemo-env01-terraform.yml` — le pipeline de l'environnement env01 :

```
name: Terraform - deploy Service Bus to env01
trigger: none

variables:
- group: Terraform_SPN
- name: rootFolder
  value: 'terraform/'
- name: tfvarsFile
  value: 'tfdemo.env01.tfvars'
- name: adoEnvironment
  value: 'env01'
```

```

stages:
- template: templates/terraform-template.yml
  parameters:
    rootFolder: ${rootFolder}
    tfvarsFile: ${tfvarsFile}
    adoEnvironment: ${adoEnvironment}

```

🧠 LE TEMPLATE, C'EST LE « DRY » DES PIPELINES

Don't Repeat Yourself. La logique plan/apply est écrite **une seule fois**. Chaque environnement n'est qu'un petit fichier de paramètres. Corriger ou améliorer le template profite à tous les environnements. Jenkins a l'équivalent avec les *shared libraries*, GitHub Actions avec les *reusable workflows*, GitLab avec `include: .`

📌 AMÉLIORATION PAR RAPPORT À L'ORIGINAL

Dans le template d'origine, le stage *Apply* relance un `terraform apply` complet : le plan qui est appliqué n'est **pas forcément** celui qu'on a approuvé (si quelque chose a changé entre-temps). Publier le plan comme artefact et appliquer **ce fichier** garantit que l'on applique exactement ce qui a été relu.

Étape 6 — Créer l'environnement et l'approbation

1. Pipelines → Environments → New environment → nom `env01`, ressource : *None*.
2. `env01` → : → Approvals and checks → Approvals → ajoute-toi comme approbateur.
3. Pipelines → New pipeline → Azure Repos Git → ton dépôt → Existing YAML → `deploy/tfdemo-env01-terraform.yml` → Run.

Le stage *Plan* s'exécute ; lis le plan dans les logs. Le stage *Apply* affiche « **Waiting for approval** » : clique *Review* → *Approve*.

✅ VÉRIFICATION

- Portail Azure : groupe `tfdemo-env01-rg` contenant le namespace Service Bus `tfdemo-env01-sbns` avec `queue01` et `queue02`.
- Le conteneur `tfstate` contient `tfdemo.env01.tfstate`.
- *Environments* → `env01` : l'historique des déploiements.

Étape 7 — Passer à l'échelle : un deuxième environnement

1. Copie `tfdemo.env01.tfvars` en `tfdemo.env02.tfvars` (`environment = "env02"`).
2. Copie `tfdemo-env01-terraform.yml` en `tfdemo-env02-terraform.yml` et change les **trois** valeurs (`tfvarsFile`, `adoEnvironment`, et la **clé d'état**).
3. La **clé d'état doit être différente** par environnement, sinon `env02` écraserait l'état d'`env01`. Comme la clé est écrite dans `providers.tf`, le plus propre est de la **passer à l'init** : `terraform init -backend-config="key=tfdemo.env02.tfstate"`. Ajoute un paramètre `stateKey` au template.
4. Crée l'environnement `env02` (avec ou sans approbation : on approuve souvent la production, pas les environnements de test).

Variante — sans aucun secret (Workload Identity Federation)

Remplace les étapes `script` par une tâche `AzureCLI@2` utilisant la service connection fédérée du projet 07 :

```

- task: AzureCLI@2
  inputs:
    azureSubscription: 'azure-devops-journey-service-connection'
    scriptType: bash
    scriptLocation: inlineScript
    addSpnToEnvironment: true # expose servicePrincipalId, tenantId, idToken au script
    workingDirectory: ${ parameters.rootFolder }
    inlineScript: |
      export ARM_CLIENT_ID=$servicePrincipalId
      export ARM_TENANT_ID=$tenantId
      export ARM_SUBSCRIPTION_ID=$(az account show --query id -o tsv)
      export ARM_USE_OIDC=true
      export ARM_OIDC_TOKEN=$idToken
      terraform init -input=false
      terraform plan -input=false -var-file ${ parameters.tfvarsFile } -out=tfplan

```

Plus de variable `ARM_CLIENT_SECRET`, plus de secret à faire tourner.

Dépannage

🔧 DÉPANNAGE

- **AuthorizationFailed** lors du **init** (accès au blob) : rôle *Storage Blob Data Contributor* manquant (les rôles de **données** sont distincts des rôles de **gestion** comme *Contributor*). La propagation d'un rôle peut prendre quelques minutes.
- **subscription ID could not be determined** : azurearm v4 exige `ARM_SUBSCRIPTION_ID`.
- **Invalid client secret provided** : secret expiré ou mal copié ; ou `ARM_CLIENT_SECRET` non passé en `env` :
- **Unsupported argument** : renommages azurearm v3 → v4 (voir l'encadré de l'étape 3).
- **Saved plan is stale** : un autre `apply` a modifié l'état entre le plan et l'apply ; relance le pipeline.

Nettoyage

👉 NETTOYAGE OBLIGATOIRE

```
az group delete -n tfdemo-env01-rg --yes --no-wait
az group delete -n tfdemo-env02-rg --yes --no-wait    # si créé
az group delete -n tfstate-tfdemo-rg --yes --no-wait  # en dernier
az ad app delete --id <application-client-id>         # supprime tfdemo-spn
```

BASH

(Mieux : ajoute un pipeline `terraform destroy` qui utilise le même template.)

🔑 QUESTIONS D'ENTRETIEN

- Quels droits donnes-tu à l'identité qui exécute Terraform sur Azure ? Pourquoi deux rôles ?
- Comment passes-tu un secret à un script dans Azure Pipelines ?
- Comment déploies-tu le même code Terraform dans plusieurs environnements sans dupliquer la logique ? (*Template de pipeline + tfvars + clé d'état par environnement, ou workspaces.*)
- Pourquoi appliquer un plan sauvegardé plutôt que relancer `terraform apply` ?

PARTIE 4

Cloud natif AWS & Serverless

Les outils de CI/CD intégrés à AWS, l'architecture serverless et GitLab CI. Trois projets : un pipeline CodePipeline complet, une API 100 % serverless en Terraform, et l'automatisation d'infrastructure avec GitLab.

Leçon 13 — La CI/CD « maison » d'AWS et le serverless

Les AWS Developer Tools

Service	Rôle	Équivalent
CodeCommit	dépôt Git hébergé	GitHub — <i>fermé aux nouveaux clients depuis juillet 2024</i>
CodeConnections (ex-CodeStar Connections)	relie AWS à GitHub, GitLab ou Bitbucket	—
CodeBuild	exécute des builds dans des conteneurs éphémères, décrits par un <code>buildspec.yml</code>	un runner GitHub Actions / un agent Jenkins
CodeDeploy	déploie sur EC2, sur des serveurs sur site, sur Lambda ou sur ECS (avec blue/green), décrit par un <code>appspec.yml</code>	un script de déploiement + des stratégies
CodePipeline	enchaîne les étapes Source → Build → Deploy (avec approbations)	le Jenkinsfile / le workflow
CodeArtifact	dépôt d'artefacts (Maven, npm, PyPI...)	JFrog, Nexus
ECR	registre d'images	Docker Hub
Systems Manager Parameter Store / Secrets Manager	stockage de paramètres et de secrets	Jenkins Credentials, Key Vault

Le `buildspec.yml` (CodeBuild)

```
version: 0.2
env:
  parameter-store:           # lit des valeurs dans SSM Parameter Store
    DOCKER_USER: /myapp/docker/username
    DOCKER_PASS: /myapp/docker/password
phases:
  install:
    runtime-versions: { nodejs: 20 }
  pre_build:
    commands:
      - echo "$DOCKER_PASS" | docker login -u "$DOCKER_USER" --password-stdin
  build:
    commands:
      - docker build -t monimage:$CODEBUILD_RESOLVED_SOURCE_VERSION .
  post_build:
    commands:
      - docker push monimage:$CODEBUILD_RESOLVED_SOURCE_VERSION
artifacts:
  # ce qui est transmis à l'étape suivante du pipeline
  files: ['appspec.yml', 'scripts/**/*']
```

Le `appspec.yml` (CodeDeploy sur EC2)

```
version: 0.0
os: linux
files:
  - source: /
    destination: /opt/monapp
hooks:
  # scripts exécutés à chaque phase du déploiement
  ApplicationStop: [{ location: scripts/stop.sh, timeout: 300, runas: root }]
  AfterInstall:    [{ location: scripts/start.sh, timeout: 300, runas: root }]
  ValidateService: [{ location: scripts/check.sh, timeout: 120, runas: root }]
```

L'agent **CodeDeploy** installé sur chaque instance récupère le paquet de déploiement et exécute les *hooks* dans l'ordre : `ApplicationStop` → `DownloadBundle` → `BeforeInstall` → `Install` → `AfterInstall` → `ApplicationStart` → `ValidateService` .

Le serverless

Serverless ne veut pas dire « sans serveur », mais « **sans serveur à gérer** » : tu fournis du code ou une configuration, le fournisseur gère entièrement l'exécution, la mise à l'échelle (y compris jusqu'à zéro) et la disponibilité. Tu paies à l'**usage réel** (à la requête, à la milliseconde), pas à l'heure.

Brique	Service AWS	Remarque
Calcul	Lambda	une fonction exécutée à chaque événement ; 15 minutes maximum par exécution

Brique	Service AWS	Remarque
API HTTP	API Gateway (REST ou HTTP API)	reçoit les requêtes et déclenche les Lambda
Base relationnelle	Aurora Serverless v2	s'adapte à la charge (en ACU) ; la v2 peut descendre à 0 ACU selon les versions
Base NoSQL	DynamoDB	entièrement serverless, paiement à la requête
Stockage	S3	
Files / événements	SQS, SNS, EventBridge	
Conteneurs	Fargate, App Runner	(projet 04)

🔗 LES AVANTAGES ET LES PIÈGES

Avantages : aucun serveur à patcher, mise à l'échelle automatique, coût nul quand rien ne se passe, haute disponibilité intégrée.

Pièges : *cold start* (la première invocation après une période d'inactivité est plus lente), limites (durée, taille du paquet), débogage moins direct, coûts qui peuvent dépasser des serveurs classiques à très fort trafic constant, et dépendance forte au fournisseur (*vendor lock-in*).

GitLab CI en bref

GitLab intègre la CI/CD dans le même produit que le dépôt. Le pipeline se décrit dans `.gitlab-ci.yml` à la racine :

```

stages: [build, test, deploy]           # ordre des étapes

variables:
  APP_ENV: staging

build-job:
  stage: build
  image: node:20                         # chaque job tourne dans un conteneur
  script:
    - npm ci
    - npm run build
  artifacts:
    paths: [dist/]                       # transmis aux jobs suivants

test-job:
  stage: test
  image: node:20
  script: [npm test]

deploy-job:
  stage: deploy
  script: [./deploy.sh]
  rules:
    - if: $CI_COMMIT_BRANCH == "main"    # seulement sur main
  when: manual                            # bouton manuel
  environment: production

```

Les jobs d'un même *stage* tournent en parallèle ; les *stages* s'enchaînent. Les **variables CI/CD** (secrets) se définissent dans *Settings* → *CI/CD* → *Variables* (cocher *Masked* et *Protected*). Les **runners** exécutent les jobs : partagés (fournis par GitLab.com, avec un quota de minutes gratuites) ou auto-hébergés.

🔑 QUESTIONS D'ENTRETIEN

- Quel est le rôle de chaque service : CodeBuild, CodeDeploy, CodePipeline ?
- Que contiennent un `buildspec.yml` et un `appspec.yml` ?
- Qu'est-ce que le serverless ? Cite deux avantages et deux inconvénients.
- Qu'est-ce qu'un *cold start* Lambda et comment le limiter ? (*Paquet léger, runtime rapide, provisioned concurrency, SnapStart pour Java.*)

Projet 21 — Pipeline CI/CD avec les outils DevOps d'AWS

FICHE PROJET

Niveau	★★★★☆ Intermédiaire
Durée	4 à 6 heures
Coût	quelques centimes (CodeBuild à la minute, CodePipeline 1 pipeline gratuit/mois en V1 ou ~0,002 \$/minute d'action en V2, une t3.micro)
Compétences	CodeConnections (GitHub), CodeBuild + buildspec.yml , Parameter Store pour les secrets, Docker Hub, artefacts S3, agent et application CodeDeploy, appspec.yml et hooks, CodePipeline
Prérequis	Leçon 13, projet 04 (Docker)
Dépôt d'origine	DevOps-Project-21 : application Netflix-clone (React/Vite), Dockerfile , buildspec.yml , appspec.yml , scripts/start.sh , scripts/stop.sh

Ce qu'on construit

```
GitHub —(CodeConnections)—> CodePipeline
├─ Source : récupère le code
├─ Build  : CodeBuild (buildspec.yml)
│         docker build (clé TMDB lue dans Parameter Store)
│         docker push → Docker Hub <toi>/netflix-react-app:latest
│         artefact (appspec.yml + scripts/) → S3
└─ Deploy : CodeDeploy → EC2 (agent CodeDeploy + Docker)
      ApplicationStop : scripts/stop.sh (arrête l'ancien conteneur)
      AfterInstall    : scripts/start.sh (docker run -p 8080:80)
                      → http://<IP>:8080
```

i CODECOMMIT N'EST PLUS DISPONIBLE

Le dépôt d'origine utilise **CodeCommit** (avec une clé SSH d'utilisateur IAM). AWS a **fermé CodeCommit aux nouveaux clients** en juillet 2024. On utilise donc **GitHub** relié à AWS par **CodeConnections**. Le reste du pipeline est identique, et c'est d'ailleurs la configuration la plus courante en entreprise.

Étape 1 — Le code

1. Crée un dépôt GitHub [netflix-aws-cicd](#) et copies-y le contenu de [DevOps-Project-21](#).
2. Ouvre [scripts/start.sh](#) et [scripts/stop.sh](#) : ils contiennent l'image [dhruvdarji123/netflix-react-app](#) (l'auteur). Remplace par [<toi-dockerhub>/netflix-react-app](#). Fais de même dans [buildspec.yml](#).
3. Crée une **clé d'API TMDB** (compte gratuit sur [themoviedb.org](#) → *Settings* → *API*). L'application l'utilise pour afficher les films.

Rends les scripts plus robustes (le [stop.sh](#) d'origine échoue au tout premier déploiement, quand aucun conteneur n'existe) :

```
#!/bin/bash
# scripts/stop.sh
docker rm -f netflix 2>/dev/null || true
docker image rm <toi-dockerhub>/netflix-react-app:latest 2>/dev/null || true
```

BASH

```
#!/bin/bash
# scripts/start.sh
docker pull <toi-dockerhub>/netflix-react-app:latest
docker run -d --name netflix --restart unless-stopped -p 8080:80 <toi-dockerhub>/netflix-react-app:latest
```

BASH

Étape 2 — Les secrets dans Parameter Store

Systems Manager → Parameter Store → Create parameter (ou en CLI) :

```
aws ssm put-parameter --name /myapp/docker-credentials/username --type String --value "<toi-dockerhub>"
aws ssm put-parameter --name /myapp/docker-credentials/password --type SecureString --value "<token-dockerhub>"
aws ssm put-parameter --name /myapp/api/key --type SecureString --value "<cle-tmdb>"
```

BASH

Le [buildspec.yml](#) les lit dans sa section [env.parameter-store](#) : CodeBuild les injecte comme variables d'environnement, masquées dans les logs.

Étape 3 — Le projet CodeBuild

1. *Developer Tools* → *Settings* → *Connections* → *Create connection* → **GitHub** → autorise l'application AWS sur ton compte, limitée au dépôt `netflix-aws-cicd` → la connexion doit être *Available*.
2. *CodeBuild* → *Create build project* :
 - nom `netflix-build` ; *Source* : **GitHub** via la connexion, dépôt `netflix-aws-cicd` ;
 - *Environment* : image managée **Amazon Linux**, runtime *Standard*, dernière image ; **coche « Privileged »** (*Enable this flag if you want to build Docker images*) ;
 - *Service role* : *New service role* (`codebuild-netflix-build-service-role`) ;
 - *Buildspec* : *Use a buildspec file*, nom `buildspec.yaml` (l'extension `.yaml` n'est pas celle par défaut, `.yml` : précise-la !) ;
 - *Artifacts* : **Amazon S3**, bucket `io-21-artifacts-<prenom>` (à créer), *packaging* *Zip*.
3. **Donne au rôle CodeBuild le droit de lire les paramètres** : IAM → le rôle → *Add permissions* → politique inline autorisant `ssm:GetParameters` sur `arn:aws:ssm:eu-west-3:<compte>:parameter/myapp/*` (et `kms:Decrypt` si clé KMS personnalisée).

🔧 LE MODE « PRIVILEGED »

CodeBuild exécute ton build dans un conteneur. Construire une image Docker **depuis** un conteneur nécessite des privilèges supplémentaires (démarrer un démon Docker interne). Sans ce drapeau, `docker build` échoue avec `Cannot connect to the Docker daemon`.

⚠️ 'RUNTIME-VERSIONS: NODEJS: LATEST'

Le `buildspec.yaml` d'origine demande `nodejs: latest`. Certaines images CodeBuild refusent `latest` ; mets une version explicite supportée (par exemple `nodejs: 20`). La construction de l'application se fait de toute façon **dans** le Dockerfile (étape `node:...-alpine`).

Lance un premier build manuel (*Start build*) : il doit pousser l'image sur Docker Hub et déposer le zip d'artefacts dans S3.

Étape 4 — L'instance de déploiement

1. **Rôle IAM** pour l'instance : `AmazonEC2RoleforAWSCodeDeploy` (lecture des artefacts S3) + `AmazonSSMManagedInstanceCore`. Nom `io-21-ec2-role`.
2. **Instance** : Ubuntu 24.04, `t3.micro`, rôle `io-21-ec2-role`, SG : 22 depuis ton IP, **8080 depuis ton IP**. Tag `Name=netflix-codedeploy` (CodeDeploy trouvera l'instance par ce tag).
3. **Installer Docker et l'agent CodeDeploy** (user data ou SSH) :

```
sudo apt update && sudo apt install -y docker.io ruby-full wget
sudo systemctl enable --now docker
cd /tmp
wget https://aws-codedeploy-eu-west-3.s3.eu-west-3.amazonaws.com/latest/install # adapter la région (2 fois)
chmod +x install && sudo ./install auto
sudo systemctl status codedeploy-agent
```

💡 LE FICHIER 'INSTALL' DU DÉPÔT D'ORIGINE

Le dossier contient un fichier `install` vide (0 octet) : c'est un reste du script d'installation de l'agent. Télécharge la version officielle comme ci-dessus, depuis le bucket de ta région.

Étape 5 — L'application CodeDeploy

1. **Rôle de service CodeDeploy** : IAM → *Create role* → *AWS service* → **CodeDeploy** → cas d'usage *CodeDeploy* (politique `AWSCodeDeployRole`) → `io-21-codedeploy-role`.
2. *CodeDeploy* → *Applications* → *Create application* → nom `netflix-app`, plateforme **EC2/On-premises**.
3. *Create deployment group* → `netflix-dg`, rôle `io-21-codedeploy-role`, type *In-place*, *Environment configuration* : **Amazon EC2 instances** → tag `Name` = `netflix-codedeploy` ; *Install AWS CodeDeploy Agent* : *Never* (déjà installé) ; *Load balancer* : décoché.

Étape 6 — Le pipeline

CodePipeline → *Create pipeline* → *Build custom pipeline* :

- nom `netflix-pipeline`, type *V2*, *New service role* ;
- **Source** : GitHub (via la connexion), dépôt, branche `main`, déclenchement sur push ;
- **Build** : AWS CodeBuild, projet `netflix-build` ;
- **Deploy** : AWS CodeDeploy, application `netflix-app`, groupe `netflix-dg`.

À la création, le pipeline s'exécute une première fois.

🔧 LE CHEMIN DES ARTEFACTS ENTRE LES ÉTAPES

CodePipeline stocke la sortie de chaque étape dans son **bucket d'artefacts** S3. L'étape Build reçoit le code source, produit un artefact (ici tout le dépôt, qui contient `appspec.yaml` et `scripts/`), que l'étape Deploy remet à CodeDeploy. L'agent sur l'instance télécharge ce zip, lit `appspec.yaml` à la racine, et exécute les hooks. Si `appspec.yaml` n'est pas à la racine de l'artefact, le déploiement échoue.

✓ VÉRIFICATION

- Les trois étapes du pipeline sont vertes.
- `CodeBuild` → `Build history` : logs du build, image poussée sur Docker Hub.
- `CodeDeploy` → `Deployments` : détail des événements de cycle de vie (`ApplicationStop` , `AfterInstall` ...) avec leur durée.
- `http://<IP_PUBLIQUE_EC2>:8080` affiche le clone de Netflix avec des affiches de films.
- Modifie un texte dans `src/` , push sur GitHub : le pipeline repart seul et la page change.

Dépannage

🔧 DÉPANNAGE

- `CodeBuild` : `AccessDeniedException` sur `ssm:GetParameters` : la politique inline de l'étape 3.
- `CodeBuild` : `YAML_FILE_ERROR` / `buildspec` introuvable : nom du fichier (`buildspec.yaml` vs `.yaml`).
- `CodeBuild` : `Cannot connect to the Docker daemon` : mode `Privileged` non coché.
- `CodeBuild` : `toomanyrequests` de `Docker Hub` : limite de téléchargements anonymes atteinte ; le `docker login` avant le build (déplace-le en `pre_build`) ou une image de base depuis `public.ecr.aws` règle le problème.
- `CodeDeploy` : `déploiement bloqué en In progress puis échec « agent not responding »` : l'agent ne tourne pas, ou l'instance n'a pas de rôle permettant d'accéder à S3 (`/var/log/aws/codedeploy-agent/codedeploy-agent.log`).
- `CodeDeploy` : `ScriptFailed` sur `ApplicationStop` au premier déploiement : le `stop.sh` d'origine échoue car le conteneur n'existe pas encore (d'où notre version avec `|| true`).
- `Page blanche ou sans films` : la clé TMDb n'a pas été passée au build (`--build-arg TMDb_V3_API_KEY=$KEY`).

Nettoyage

🧹 NETTOYAGE OBLIGATOIRE

Supprime le pipeline, le projet CodeBuild, l'application CodeDeploy, l'instance EC2, les buckets S3 (artefacts du pipeline et `io-21-artifacts-...`), les paramètres SSM, la connexion GitHub et les rôles IAM créés.

🔑 QUESTIONS D'ENTRETIEN

- Décris le rôle de CodeBuild, CodeDeploy et CodePipeline dans ce projet.
- Où sont stockés les secrets et comment CodeBuild y accède-t-il ?
- Quels sont les hooks d'un déploiement CodeDeploy sur EC2 ?
- Pourquoi un script de déploiement doit-il être idempotent ?

Projet 22 — Architecture 100 % serverless sur AWS avec CI/CD

FICHE PROJET

Niveau	★★★★☆ Avancé
Durée	6 à 10 heures
Coût	Aurora Serverless v2 : ~0,06 \$/h au minimum (0,5 ACU) — c'est le poste principal, détruis après chaque session. Lambda, API Gateway, S3, endpoints : quelques centimes. VPC endpoint Secrets Manager : ~0,01 \$/h par AZ
Compétences	Lambda dans un VPC, API Gateway (REST, proxy), Aurora Serverless v2, Secrets Manager (mot de passe géré et rotation), KMS, VPC endpoints (Gateway et Interface), IAM fin, Terraform, GitHub Actions avec OIDC
Prérequis	Leçons 10 et 13, projet 14 (GitHub Actions)
Dépôt d'origine	DevOps-Project-22 : main.tf , variables.tf , provider.tf , versions.tf , API Node.js dans serverless-api/

Ce qu'on construit

Une API REST (utilisateurs, produits, images de produits) **sans aucun serveur à gérer** :



Les points d'architecture à comprendre

🔴 UNE LAMBDA DANS UN VPC

Pour joindre Aurora (qui est dans des subnets privés), la Lambda doit être **dans le VPC** : AWS lui crée des interfaces réseau (ENI) dans les subnets indiqués. Conséquence : elle n'a **plus accès à Internet**, ni aux services AWS publics (S3, Secrets Manager...), sauf si on lui en donne le moyen.

🔴 LES VPC ENDPOINTS AU LIEU D'UNE NAT GATEWAY

Plutôt qu'une NAT Gateway (~35 \$/mois), le projet crée :

- un **endpoint de type Gateway pour S3** (gratuit) : une route dans la table de routage privée envoie le trafic S3 directement au service, sans passer par Internet ;
- un **endpoint de type Interface pour Secrets Manager** (une ENI privée dans le VPC, ~7 \$/mois par AZ) : la Lambda appelle Secrets Manager via une IP privée.

Bonus sécurité : le trafic ne quitte jamais le réseau AWS.

🔴 'MANAGE_MASTER_USER_PASSWORD = TRUE'

Aurora **génère lui-même** le mot de passe administrateur et le range dans **Secrets Manager**, chiffré par une clé **KMS** dédiée, avec **rotation automatique** (tous les 7 jours par défaut). Le mot de passe n'apparaît **nulle part** : ni dans le code, ni dans Terraform, ni dans les variables de la Lambda. La Lambda reçoit seulement l'**ARN du secret** (`SECRET_ID`) et le lit à l'exécution grâce à son rôle IAM.

Étape 1 — Lire et moderniser le code Terraform

```
cd ~/IO/projects && mkdir 22-serverless && cd 22-serverless
cp -r /tmp/dp/DevOps-Project-22/* .
grep -n 'resource "' main.tf | wc -l          # environ 80 ressources
```

BASH

Corrections indispensables :

1. **Runtime Lambda** : `runtime = "nodejs16.x"` n'est plus accepté par AWS (runtime retiré : on ne peut plus créer de fonction avec). Mets `nodejs20.x` ou `nodejs22.x`, et vérifie que les dépendances de `serverless-api/package.json` s'installent avec cette version.
2. **Version d'Aurora** : `engine_version = "8.0.mysql_aurora.3.02.0"` est trop ancienne. Supprime la ligne (Aurora choisira la version par défaut) dans `aws_rds_cluster` et `aws_rds_cluster_instance` (où elle référence celle du cluster), ou mets une version 3.x actuelle (`aws_rds_describe_db_engine_versions --engine aurora-mysql --query "DBEngineVersions[].EngineVersion"`).
3. **Domaine personnalisé** : les ressources `aws_api_gateway_domain_name`, `aws_route53_record` et `aws_api_gateway_base_path_mapping` exigent un domaine et un certificat ACM. Sans domaine, **commente-les** : tu utiliseras l'URL fournie par API Gateway.
4. **Variables** : renseigne un fichier `terraform.tfvars` (région, profil, CIDR, nombre de subnets privés, `artifact_location`, `api_stage` ...) selon `variables.tf`. Ajoute un **backend S3** (leçon 10).

Étape 2 — Empaqueter la Lambda

```
cd serverless-api
npm ci --omit=dev           # dépendances de production uniquement
zip -r ../serverless-api.zip . -x "*.git*" "test/*"
cd ..
ls -lh serverless-api.zip    # artifact_location = "serverless-api.zip"
```

Lis `index.js` : en mode `ENVIRONMENT=lambda`, l'application Express est enveloppée par `serverless-http` et exportée comme `handler` (d'où `handler = "index.handler"` dans Terraform). En local (`node index.js`), la même application écoute sur le port 3000. **Une seule base de code pour les deux modes** : pratique pour tester.

🔑 LE ZIP, TERRAFORM ET 'SOURCE_CODE_HASH'

`source_code_hash = filebase64sha256(var.artifact_location)` : Terraform calcule une empreinte du zip. Si le code change, l'empreinte change, et Terraform met à jour la fonction. Sans cet attribut, Terraform ne verrait pas qu'il faut redéployer le code.

Étape 3 — Déployer à la main une première fois

```
terraform init
terraform plan -out=tfplan
terraform apply tfplan      # 10 à 15 minutes (Aurora est le plus long)
```

Récupère l'URL de l'API : `API Gateway` → `serverless API` → `Stages` → → **Invoke URL** (ou ajoute un `output`).

```
API=https://xxxxxxx.execute-api.eu-west-3.amazonaws.com/<stage>
curl $API/healthz
curl -X POST $API/v1/user -H 'Content-Type: application/json' \
  -d '{"first_name":"Ada","last_name":"Lovelace","username":"ada@exemple.com","password":"Motdepasse123!"}'
curl $API/v1/user/<id renvoyé>      # voir le README de serverless-api pour les champs et l'authentification
```

✅ VÉRIFICATION

- `/healthz` répond 200.
- CloudWatch → `Log groups` → `/aws/lambda/serverless_api` : les logs de chaque invocation, dont « Tables are created successfully! » (Sequelize crée les tables au premier appel).
- Secrets Manager : un secret `rds!cluster-...` géré par RDS, avec la rotation activée.
- Le premier appel est plus lent (*cold start* : création de l'environnement Lambda, connexion à Aurora qui peut se « réveiller »), les suivants sont rapides.

Étape 4 — La CI/CD avec GitHub Actions

Le dépôt d'origine décrit la CI/CD mais ne fournit pas le workflow. Voici une version complète, avec **OIDC** (rôle IAM `github-serverless-deploy` créé comme au projet 14, avec des droits suffisants pour Terraform sur ces services, et la condition `sub` limitée à ton dépôt) :

```
# .github/workflows/deploy.yml
name: Serverless API

on:
  pull_request:
    branches: [ main ]
  push:
    branches: [ main ]

permissions:
  contents: read
  id-token: write
  pull-requests: write

jobs:
```

```

terraform:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4

    - uses: actions/setup-node@v4
      with: { node-version: '20', cache: npm, cache-dependency-path: serverless-api/package-lock.json }

    - name: Tests et paquet Lambda
      working-directory: serverless-api
      run: |
        npm ci
        npm test --if-present
        npm ci --omit=dev
        zip -qr ../serverless-api.zip . -x "*.git*"

    - uses: aws-actions/configure-aws-credentials@v4
      with:
        role-to-assume: ${ secrets.AWS_ROLE_ARN }
        aws-region: eu-west-3

    - uses: hashicorp/setup-terraform@v3

    - run: terraform fmt -check -recursive
    - run: terraform init -input=false
    - run: terraform validate
    - run: terraform plan -input=false -out=tfplan

    - name: Apply (uniquement sur main)
      if: github.event_name == 'push' && github.ref == 'refs/heads/main'
      run: terraform apply -input=false tfplan

```

🔗 PLAN SUR LA PR, APPLY SUR MAIN

Sur une **pull request**, le workflow construit, teste et affiche le **plan** : le relecteur voit exactement ce qui changera dans l'infrastructure. Après **fusion** dans `main`, le même workflow **applique**. Pour aller plus loin, publie le plan en commentaire de la PR (actions comme `terraform-plan-comment`) et protège `main` avec un *environment* GitHub exigeant une approbation.

Dépannage

🔧 DÉPANNAGE

- **InvalidParameterValueException: The runtime parameter of nodejs16.x is no longer supported** : étape 1, point 1.
- **La Lambda dépasse le timeout (30 s) à la première requête** : Aurora Serverless se réveille, ou la Lambda n'arrive pas à joindre Secrets Manager (endpoint Interface absent, ou son Security Group n'autorise pas le SG de la Lambda en 443, ou `private_dns_enabled` désactivé).
- **AccessDeniedException sur secretsmanager:GetSecretValue** ou `kms:Decrypt` : politique `lambda-rds-secret-policy` ; le rôle doit aussi pouvoir déchiffrer avec la clé KMS du secret.
- **Missing Authentication Token** d'API Gateway : mauvais chemin ou stage non déployé (`aws_api_gateway_deployment` doit être recréé quand les routes changent).
- **ENI / suppression lente** : au `destroy`, AWS met parfois 20 à 40 minutes à libérer les interfaces réseau des Lambda dans le VPC ; c'est normal, laisse Terraform attendre.

Nettoyage

🔧 NETTOYAGE OBLIGATOIRE

```
terraform destroy
```

BASH

Vérifie : cluster Aurora supprimé (et pas de snapshot final si `skip_final_snapshot = true`), endpoints VPC, bucket S3 (vide-le s'il contient des images : `aws s3 rm s3://<bucket> --recursive`), clé KMS (elle passe en *pending deletion* pendant 7 à 30 jours, sans coût), secret Secrets Manager (supprimé avec le cluster), log groups CloudWatch.

🔧 QUESTIONS D'ENTRETIEN

- Pourquoi une Lambda dans un VPC perd-elle l'accès à Internet ? Quelles solutions ?
- Différence entre un VPC endpoint Gateway et Interface ?
- Comment la Lambda obtient-elle le mot de passe de la base sans qu'il apparaisse nulle part ?
- Serverless ou conteneurs : comment choisis-tu ?
- Pourquoi Terraform a-t-il besoin de `source_code_hash` pour une Lambda ?

Projet 26 — Automatiser l'infrastructure AWS avec Terraform et GitLab CI/CD

FICHE PROJET

Niveau	★★★☆☆ Intermédiaire
Durée	3 à 5 heures
Coût	~0 € (une <code>t3.micro</code> quelques minutes ; minutes GitLab gratuites)
Compétences	modules Terraform locaux et sorties, backend S3 configuré à l'init (<code>-backend-config</code>), GitLab (projet, variables CI/CD masquées et protégées, runners), <code>.gitlab-ci.yml</code> (stages, artefacts, cache, jobs manuels), OIDC GitLab → AWS
Prérequis	Leçons 10 et 13
Dépôt d'origine	<code>DevOps-Project-26</code> : <code>main.tf</code> , <code>backend.tf</code> , <code>provider.tf</code> , <code>tfstate.config</code> , modules <code>vpc/</code> et <code>web/</code> , <code>.gitlab-ci.yml</code>

Ce qu'on construit

Partie 1 (poste local)	Partie 2 (GitLab CI)
terraform init/plan/apply	git push → pipeline GitLab (runner partagé, image Terraform)
module vpc : VPC, subnet public, SG	validate → plan (artefact planfile) → apply (MANUEL) → destroy (MANUEL)
module web : instance EC2	identifiants AWS : variables CI/CD masquées (ou OIDC)
état partagé : s3://<bucket>/terraform/staging/terraform.tfstate	

Le but : passer d'une infrastructure lancée **depuis ton poste** à une infrastructure lancée **par un pipeline**, avec le même code et le même état.

Étape 1 — Le code et ses défauts

```
.
├─ main.tf      # appelle les modules vpc et web
├─ provider.tf  # région
├─ backend.tf   # backend S3
├─ tfstate.config # paramètres du backend passés à l'init
├─ vpc/ (main.tf, variables.tf, outputs.tf) # VPC 10.0.0.0/16, subnet 10.0.1.0/24, SG port 22
└─ web/ (main.tf, variables.tf, outputs.tf) # aws_instance
```

`main.tf` montre la bonne façon de relier des modules : les **sorties** du module `vpc` deviennent les **entrées** du module `web`.

```
module "vpc" { source = "../vpc" }
module "ec2" {
  source = "../web"
  sn      = module.vpc.pb_sn      # sortie "pb_sn" du module vpc
  sg      = module.vpc.sg
}
```

À corriger (le code d'origine ne fonctionne pas tel quel) :

1. `backend.tf` : l'argument `aws_dynamodb_table` **n'existe pas** (c'est `dynamodb_table`). Modernise et ne garde que la structure, les valeurs viennent de `tfstate.config` :

```
terraform {
  backend "s3" {}          # configuration partielle : complétée par -backend-config à l'init
}
```

```
# tfstate.config
bucket      = "io-tfstate-<prenom>"
key          = "terraform/staging/terraform.tfstate"
region      = "eu-west-3"
encrypt     = true
use_lockfile = true
```

2. `provider.tf` : `region = "eu-west-3"` ; et dans `vpc/main.tf` : `availability_zone = "eu-west-3a"`.

3. `web/main.tf` : l'AMI `ami-04a81a99f5ec58529` est une Ubuntu de `us-east-1` (un ID d'AMI est propre à une région). Remplace par une recherche (voir le bloc `data "aws_ami" "ubuntu"` du projet 06). Et remplace `security_groups = [var.sg]` par `vpc_security_group_ids = [var.sg]` : dans un VPC personnalisé, `security_groups` attend des **noms** et provoque une recréation de l'instance à chaque `apply`.
4. **Le réseau** : le module `vpc` crée un subnet « public » (IP publique automatique) mais **ni Internet Gateway, ni route** : l'instance n'est pas joignable. Ajoute une `aws_internet_gateway`, une `aws_route_table` avec `0.0.0.0/0 → IGW` et l'association au subnet (comme au projet 06). Restreins le port 22 à ton IP.

🧠 LA CONFIGURATION PARTIELLE DU BACKEND

Un bloc `backend` ne peut pas utiliser de variables Terraform. Pour garder le code identique entre environnements (staging, prod) tout en changeant de bucket ou de clé, on laisse le bloc vide et on fournit les valeurs à l'initialisation : `terraform init -backend-config=tfstate.config` (ou `-backend-config="key=prod/terraform.tfstate"`). C'est la technique utilisée par le pipeline GitLab.

Étape 2 — Partie 1 : en local

```
terraform init -backend-config=tfstate.config
terraform fmt -recursive && terraform validate
terraform plan -out=planfile
terraform apply planfile
terraform output
terraform destroy      # on détruit : la suite se fera par le pipeline, avec le même état S3
```

BASH

Étape 3 — Le projet GitLab

1. Crée un compte sur gitlab.com → *New project* → *Create blank project* → `aws-terraform-gitlab` (privé).
2. Pousse le code :

```
git remote add gitlab git@gitlab.com:<toi>/aws-terraform-gitlab.git      # (ajoute ta clé SSH dans GitLab → Preferences → SSH Keys)
git push -u gitlab main
```

BASH

3. **Variables CI/CD** : *Settings* → *CI/CD* → *Variables* → *Add variable* :

Clé	Valeur	Options
<code>MY_AWS_KEY</code>	access key ID d'un utilisateur IAM dédié (droits limités à EC2/VPC + bucket d'état)	<i>Masked, Protected</i>
<code>MY_AWS_ACCESS_KEY</code>	secret access key	<i>Masked, Protected</i>

⚠ « PROTECTED » ET LES BRANCHES

Une variable **protégée** n'est disponible que dans les pipelines des branches **protégées** (`main` par défaut). Si ton pipeline tourne sur une autre branche, les variables sont vides et Terraform échoue avec `No valid credential sources found`. C'est voulu : seules les branches relues peuvent toucher à la production.

Étape 4 — Le `.gitlab-ci.yml`, modernisé

Le fichier d'origine utilise une image interne de GitLab et `TF_LOG: DEBUG` (très verbeux, peut afficher des informations sensibles). Version propre :

```
image:
  name: hashicorp/terraform:1.10      # image officielle, version épinglée
  entrypoint: [""]                     # l'image a terraform comme entrypoint : on le neutralise pour exécuter des scripts

variables:
  TF_IN_AUTOMATION: "true"
  AWS_ACCESS_KEY_ID: ${MY_AWS_KEY}
  AWS_SECRET_ACCESS_KEY: ${MY_AWS_ACCESS_KEY}
  AWS_DEFAULT_REGION: "eu-west-3"

cache:
  key: terraform
  paths: [.terraform]                  # garder les providers téléchargés entre les jobs

before_script:
  - terraform --version
  - terraform init -input=false -backend-config=tfstate.config

stages: [validate, plan, apply, destroy]

validate:
  stage: validate
  script:
    - terraform fmt -check -recursive
    - terraform validate
```

YAML

```

plan:
  stage: plan
  script:
    - terraform plan -input=false -out=planfile
  artifacts:
    paths: [planfile]
    expire_in: 1 day

apply:
  stage: apply
  script:
    - terraform apply -input=false planfile
  dependencies: [plan] # récupère l'artefact planfile
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual # bouton ► dans l'interface
  environment: staging

destroy:
  stage: destroy
  script:
    - terraform destroy -input=false -auto-approve
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual
  environment:
    name: staging
    action: stop

```

🧠 LES MÉCANISMES CLÉS

- **artifacts** : le fichier `planfile` produit par le job `plan` est conservé et transmis au job `apply` (`dependencies`). On applique exactement le plan affiché.
- **when: manual** : `apply` et `destroy` attendent un clic humain, après lecture du plan dans les logs du job `plan`.
- **cache** : le dossier `.terraform` (les providers, plusieurs centaines de Mo) est réutilisé entre jobs et pipelines.
- **before_script** : exécuté avant chaque job (ici, l' `init` avec le backend partiel).
- **environment** : GitLab garde l'historique des déploiements de l'environnement *staging* (*Operate* → *Environments*).

Étape 5 — Exécuter

Pousse le fichier → *Build* → *Pipelines* : `validate` et `plan` s'exécutent seuls. Ouvre les logs de `plan`, lis-le, puis clique ► sur `apply`.

✅ VÉRIFICATION

- L'instance `my_server` apparaît dans EC2 ; tu peux t'y connecter en SSH si tu as ajouté IGW, route et `key_name`.
- Le fichier d'état est dans `s3://io-tfstate-<prenom>/terraform/staging/terraform.tfstate`.
- Lance `terraform plan` en local (avec `init -backend-config=tfstate.config`) : « No changes » — ton poste et le pipeline partagent **le même état**.
- Clique ► sur `destroy` : tout disparaît.

Étape 6 (recommandée) — Sans clé d'accès : OIDC GitLab → AWS

Comme GitHub, GitLab peut fournir un jeton OIDC à chaque job :

1. IAM → *Identity providers* → *Add* → OpenID Connect → URL `https://gitlab.com`, audience `https://gitlab.com`.
2. Rôle IAM `gitlab-terraform` avec une politique de confiance limitée à ton projet et à `main` :

```

"Condition": {
  "StringEquals": { "gitlab.com:aud": "https://gitlab.com" },
  "StringLike": { "gitlab.com:sub": "project_path:<toi>/aws-terraform-gitlab:ref_type:branch:ref:main" }
}

```

3. Dans `.gitlab-ci.yml`, supprime `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` et ajoute :

```

default:
  id_tokens:
    GITLAB_OIDC_TOKEN:
      aud: https://gitlab.com

variables:
  AWS_ROLE_ARN: arn:aws:iam::<compte>:role/gitlab-terraform
  AWS_WEB_IDENTITY_TOKEN_FILE: /tmp/web_identity_token

before_script:

```

```
- echo "$GITLAB_OIDC_TOKEN" > /tmp/web_identity_token # le SDK AWS (utilisé par Terraform) lit ce fichier et endosse le rôle
- terraform init -input=false -backend-config=tfstate.config
```

Supprime ensuite les variables `MY_AWS_KEY` / `MY_AWS_ACCESS_KEY` et **désactive les clés** de l'utilisateur IAM.

Dépannage

🔧 DÉPANNAGE

- `Unsupported argument "aws_dynamodb_table"` : étape 1, point 1.
- `No valid credential sources found` : variables protégées sur une branche non protégée, ou noms de variables différents.
- `terraform: not found /le job se termine immédiatement` : `entrypoint: [""]` manquant avec l'image `hashicorp/terraform`.
- `Saved plan is stale` au job `apply` : un autre pipeline a modifié l'état entre-temps ; relance le pipeline.
- **L'instance est recrée à chaque `apply`** : `security_groups` au lieu de `vpc_security_group_ids`.
- `Error acquiring the state lock` : un job annulé a laissé le verrou ; `terraform force-unlock <ID>` en local après vérification.

Nettoyage

👉 NETTOYAGE OBLIGATOIRE

Job `destroy` (ou `terraform destroy` en local). Supprime les variables CI/CD, l'utilisateur IAM et ses clés (ou le rôle OIDC), et éventuellement le projet GitLab.

🔧 QUESTIONS D'ENTRETIEN

- Comment transmetts-tu un plan Terraform d'un job à l'autre dans GitLab CI ?
- Pourquoi un `apply` manuel ? Comment l'automatiserais-tu en sécurité ?
- Qu'est-ce qu'une variable CI/CD protégée et masquée ?
- Comment configures-tu un backend Terraform différent par environnement sans dupliquer le code ?

PARTIE 5

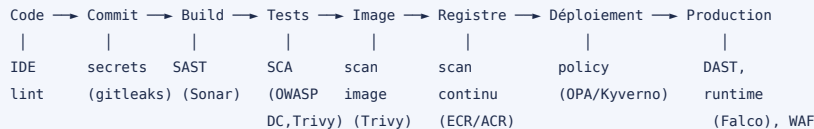
DevSecOps & Observabilité

Intégrer la sécurité à chaque étape du pipeline, et savoir en permanence ce qui se passe en production. Neuf projets : Netflix, Zomato, Swiggy en blue/green, .NET, Petshop, Reddit avec Argo CD, Chatbot, Voting App microservices et Blog App avec Nexus.

Leçon 14 — DevSecOps : la sécurité dans le pipeline

Le principe du « shift left »

Autrefois, la sécurité était vérifiée **à la fin**, juste avant la mise en production, par une équipe séparée. Découvrir une faille à ce moment-là coûte cher (il faut tout refaire) et bloque les livraisons. Le **DevSecOps** déplace les contrôles **vers la gauche** de la boucle DevOps : dans l'éditeur du développeur, dans la pull request, dans chaque exécution du pipeline. **Chaque commit est vérifié automatiquement.**



Les familles de contrôles

Sigle	Nom	Ce que ça vérifie	Outils du parcours
SAST	<i>Static Application Security Testing</i>	le code source : injections, mauvaises pratiques, secrets codés en dur	SonarQube , Semgrep, CodeQL
SCA	<i>Software Composition Analysis</i>	les dépendances (bibliothèques open source) : vulnérabilités connues (CVE), licences	OWASP Dependency-Check , Trivy fs , Snyk, Dependabot
Scan d'image	—	les paquets de l' image Docker (OS + runtime)	Trivy image , scan ECR/ACR, Grype
IaC scanning	—	les fichiers d'infrastructure : bucket public, port 22 ouvert, chiffrement absent	Trivy config , Checkov, tfsec, KICS
Secret scanning	—	les secrets commités par erreur	gitleaks, trufflehog, GitHub secret scanning
DAST	<i>Dynamic Application Security Testing</i>	l' application en fonctionnement , attaquée de l'extérieur	OWASP ZAP
Runtime	—	les comportements suspects en production	Falco, GuardDuty, WAF

Les vulnérabilités : CVE et CVSS

Une **CVE** (*Common Vulnerabilities and Exposures*) est l'identifiant public d'une faille connue, par exemple [CVE-2021-44228](#) (Log4Shell). Le score **CVSS** (de 0 à 10) mesure sa gravité : **LOW**, **MEDIUM**, **HIGH**, **CRITICAL**. Les scanners comparent les versions de tes paquets à des bases de CVE (la **NVD** américaine, les bases des distributions, GitHub Advisories).

POURQUOI LA PLUPART DES FAILLES VIENNENT DES DÉPENDANCES

Une application moderne contient souvent **80 à 90 % de code open source** (bibliothèques npm, Maven, PyPI, paquets de l'image de base). Ton code est peut-être parfait, mais une vieille version de bibliothèque peut ouvrir la porte. D'où l'importance du SCA et du scan d'image, et de **mettre à jour régulièrement** (Dependabot, Renovate).

SonarQube

SonarQube analyse le code et classe les problèmes en **bugs**, **vulnérabilités**, **security hotspots** (code sensible à revoir à la main), **code smells** (maintenabilité), et mesure la **couverture de tests** et la **duplication**. La **Quality Gate** définit les conditions d'acceptation, surtout pour le **nouveau code** (« pas de nouvelle vulnérabilité, couverture ≥ 80 % sur le code ajouté »). Dans Jenkins :

```
stage('Analyse SonarQube') {
  steps {
    withSonarQubeEnv('sonar-server') { // nom du serveur déclaré dans Manage Jenkins > System
      sh "$SCANNER_HOME/bin/sonar-scanner -Dsonar.projectKey=monapp -Dsonar.projectName=monapp"
    }
  }
}
stage('Quality Gate') {
  steps {
    timeout(time: 5, unit: 'MINUTES') {
      waitForQualityGate abortPipeline: true // true = le pipeline ÉCHoue si la porte est rouge
    }
  }
}
```

⚠️ `ABORTPIPELINE: FALSE`

Beaucoup de projets du dépôt d'origine utilisent `waitForQualityGate abortPipeline: false` : le résultat est affiché, mais **le pipeline continue même si la Quality Gate échoue**. C'est utile pour démarrer sans tout bloquer, mais ce n'est **pas** du DevSecOps : un contrôle qui ne bloque jamais finit par être ignoré. Une fois le projet stabilisé, passe à `true`.

Trivy

Trivy (Aqua Security) est le couteau suisse du scan, open source et rapide :

```
trivy fs . # dépendances et secrets du dossier (SCA)
trivy image monapp:1.0 # paquets de l'image
trivy image --severity HIGH,CRITICAL --exit-code 1 monapp:1.0 # code retour 1 s'il y a du HIGH/CRITICAL → fait échouer le pipeline
trivy config ./terraform # erreurs de configuration IaC
trivy k8s --report summary cluster # le cluster Kubernetes entier
trivy image --format template --template "@contrib/html.tpl" -o rapport.html monapp:1.0 # rapport HTML
```

Le dépôt d'origine redirige souvent la sortie dans un fichier (`trivy fs . > trivyfs.txt`) sans jamais faire échouer le build : là encore, c'est informatif. L'option `--exit-code 1` en fait un vrai contrôle.

OWASP Dependency-Check

Outil de SCA de l'OWASP, très utilisé avec Jenkins (plugin *OWASP Dependency-Check*). Il télécharge la base **NVD** puis compare les dépendances du projet.

⚠️ LA CLÉ D'API NVD

Depuis 2023, la NVD limite fortement les téléchargements anonymes : sans clé, la première exécution peut prendre **des heures** ou échouer (erreurs 403/429). Demande une clé gratuite (<https://nvd.nist.gov/developers/request-an-api-key>), range-la dans les credentials Jenkins (*Secret text*, ID `nvd-api-key`) et passe-la à l'outil :

```
withCredentials([string(credentialsId: 'nvd-api-key', variable: 'NVD_KEY')]) {
    dependencyCheck additionalArguments: "--scan ./ --format XML --nvdApiKey ${NVD_KEY} --disableYarnAudit --disableNodeAudit",
                    odcInstallation: 'DP-Check'
}
dependencyCheckPublisher pattern: '**/dependency-check-report.xml'
```

La base est mise en cache sur le serveur Jenkins : seules les exécutions suivantes sont rapides.

Les bonnes pratiques de sécurité à appliquer partout

- **Secrets** : jamais dans Git ni dans une image ; un coffre (Credentials, Secrets Manager, Key Vault) ; rotation ; détection automatique (gitleaks en pre-commit et en CI).
- **Images** : image de base minimale et à jour, utilisateur non-root, multi-stage, tag immuable, scan, signature (cosign).
- **Moindre privilège** partout : IAM, RBAC Kubernetes, Security Groups, jetons à portée limitée.
- **Pas de `chmod 777`**, pas de ports ouverts à `0.0.0.0/0` sans raison, pas de mots de passe par défaut (`admin/admin` de SonarQube, Grafana, Nexus...).
- **Mettre à jour** : OS, dépendances, images, outils (Jenkins et ses plugins sont des cibles fréquentes).

Leçon 15 — Observabilité : monitoring, logs, alertes

Les trois piliers

Pilier	Question	Exemple	Outils
Métriques	« Combien ? À quelle vitesse ? »	CPU 85 %, 230 requêtes/s, 2 % d'erreurs, latence p95 = 300 ms	Prometheus, CloudWatch, Azure Monitor
Logs	« Que s'est-il passé exactement ? »	ERROR Payment failed: card declined (order 4521)	Loki, ELK/OpenSearch, CloudWatch Logs
Traces	« Où le temps a-t-il été passé dans cette requête ? »	front 20 ms → API 40 ms → base 350 ms	OpenTelemetry, Jaeger, Tempo, X-Ray

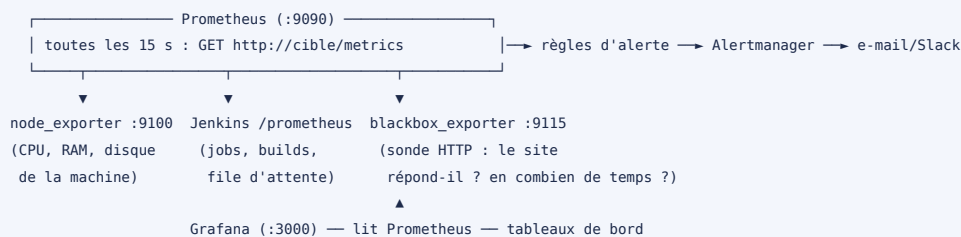
Et par-dessus : **tableaux de bord** (Grafana) et **alertes** (Alertmanager, CloudWatch Alarms) qui préviennent un humain quand quelque chose ne va pas.

🔴 QUOI SURVEILLER ? LES « GOLDEN SIGNALS »

Google propose 4 signaux d'or pour tout service : **latence** (temps de réponse), **trafic** (requêtes par seconde), **erreurs** (taux de réponses en échec), **saturation** (à quel point la ressource est pleine : CPU, mémoire, disque, connexions). Pour l'infrastructure, la méthode **USE** : *Utilization, Saturation, Errors*. Commence toujours par là avant d'ajouter 200 graphiques.

Prometheus

Prometheus collecte des métriques en **interrogeant** (*scrape*) à intervalle régulier des points d'accès HTTP `/metrics` exposés par les applications ou par des **exporters**, et les stocke dans une base de séries temporelles.



```
# /etc/prometheus/prometheus.yml
global:
  scrape_interval: 15s
scrape_configs:
  - job_name: prometheus
    static_configs: [{ targets: ['localhost:9090'] }]
  - job_name: node_exporter
    static_configs: [{ targets: ['localhost:9100', '10.0.1.25:9100'] }]
  - job_name: jenkins
    metrics_path: /prometheus
    static_configs: [{ targets: ['10.0.1.10:8080'] }]
```

Le langage de requête **PromQL** :

```
up # 1 = cible joignable, 0 = en panne
100 - (avg by (instance)(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) # % CPU utilisé
node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes * 100 # % mémoire libre
probe_success{job="blackbox"} # le site répond-il ?
```

L'installation complète (Prometheus, node_exporter, Grafana, blackbox_exporter en services systemd) est en **annexe A.7** ; sur Kubernetes, on utilise le chart **kube-prometheus-stack** (annexe A.10).

Grafana

Grafana se connecte à des sources de données (Prometheus, Loki, CloudWatch...) et affiche des tableaux de bord. On importe des tableaux prêts à l'emploi depuis grafana.com/grafana/dashboards par leur **ID** :

ID	Tableau
1860	Node Exporter Full (machine Linux)

ID	Tableau
9964	Jenkins: Performance and Health Overview
7587	Prometheus Blackbox Exporter
15757 à 15762	Kubernetes (Views / Global, Namespaces, Pods...)

Les alertes

Une bonne alerte est **actionnable** (quelqu'un doit faire quelque chose), **rare** (sinon on finit par l'ignorer : *alert fatigue*) et décrit le **symptôme** vu par l'utilisateur (« le site répond en plus de 2 s ») plutôt qu'une cause interne (« le CPU est à 80 % »).

```
# règle Prometheus
groups:
- name: site
  rules:
  - alert: SiteDown
    expr: probe_success{job="blackbox"} == 0
    for: 2m # doit durer 2 minutes avant de déclencher
    labels: { severity: critical }
    annotations:
      summary: "Le site {{ $labels.instance }} ne répond plus"
```

YAML

🔑 QUESTIONS D'ENTRETIEN

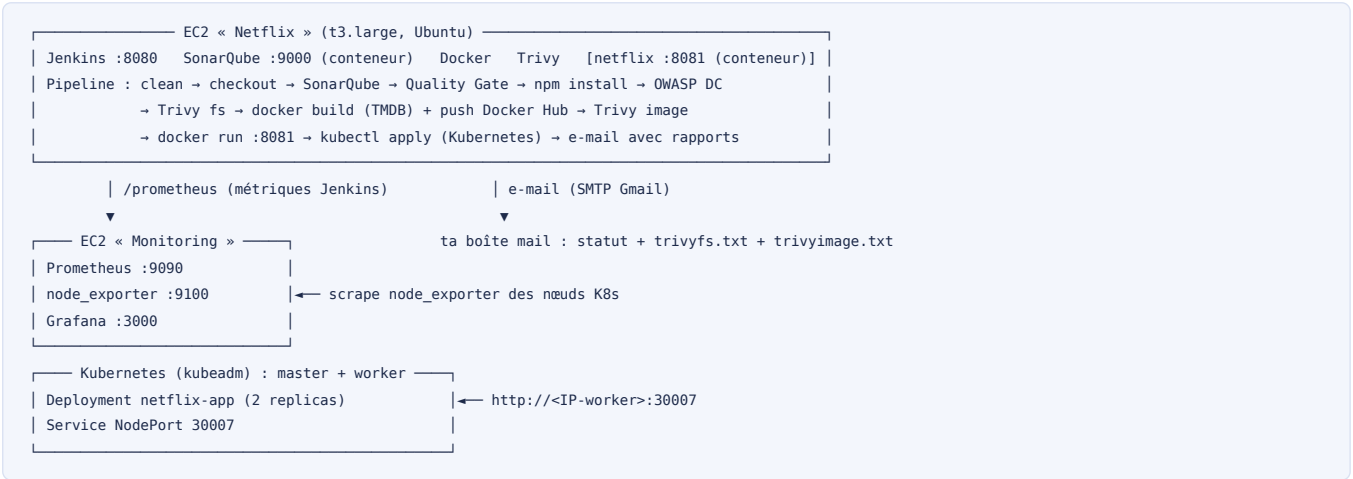
- Différence entre SAST, SCA, DAST ? Donne un outil pour chacun.
- Que fais-tu si Trivy trouve une CVE CRITICAL dans ton image ? (*Mettre à jour l'image de base ou la dépendance, reconstruire ; si aucun correctif n'existe : évaluer l'exploitabilité, documenter une exception temporaire, compenser.*)
- Quels sont les trois piliers de l'observabilité ? Les 4 golden signals ?
- Prometheus fonctionne-t-il en push ou en pull ? Quels avantages ?
- Qu'est-ce qu'une bonne alerte ?

Projet 09 — DevSecOps : clone de Netflix, CI/CD, monitoring et e-mails

 FICHE PROJET

Niveau	★★★★☆ Avancé — le projet de référence de la partie 5
Durée	10 à 15 heures
Coût	~0,15 \$/h : t3.large (Jenkins + SonarQube + Docker), t3.medium (Prometheus/Grafana), 2 t3.medium (Kubernetes)
Compétences	Jenkins (outils JDK/Node, plugins), SonarQube + Quality Gate, OWASP Dependency-Check, Trivy (fs et image), build multi-stage avec argument (clé TMDB), Docker Hub, déploiement Docker puis Kubernetes (kubeadm), Prometheus, node_exporter, Grafana, plugin Prometheus de Jenkins, notifications e-mail
Prérequis	Leçons 14 et 15, projets 05 et 16
Dépôt d'origine	DevOps-Project-09/README.md + jenkinsfile ; application github.com/Aj7Ay/Netflix-clone (React/Vite, Dockerfile , dossier Kubernetes/)

Ce qu'on construit



Étape 1 — Le serveur principal

Instance **Ubuntu 24.04**, [t3.large](#) (8 Go : Jenkins + SonarQube + builds Node), **30 Go** de disque, key pair [io-key](#) .
Security Group : **22** (ton IP), **8080** Jenkins, **9000** SonarQube, **8081** application Docker (ton IP ; 8080 aussi depuis GitHub si webhook).

 « OPEN ALL PORTS »

Le tutoriel d'origine dit d'ouvrir **tous** les ports dans le Security Group. Ne le fais pas : ouvre uniquement les ports listés, et seulement depuis ton IP. Chaque port ouvert inutilement à Internet est une porte d'entrée (Jenkins et SonarQube avec des mots de passe faibles sont attaqués en quelques heures).

Étape 2 — Jenkins, Docker, SonarQube, Trivy

```
ssh -i ~/.ssh/io-key.pem ubuntu@<IP>
sudo hostnamectl set-hostname netfliX && exec bash
```

1. **Jenkins** : recette de l'annexe A.3 (Java 21 + dépôt Jenkins). (Le tutoriel installe Temurin 17 depuis le dépôt Adoptium : c'est aussi valable.)
2. **Docker** :

```
sudo apt install -y docker.io
sudo usermod -aG docker ubuntu
sudo usermod -aG docker jenkins # Jenkins doit pouvoir lancer docker
sudo systemctl restart jenkins
newgrp docker # appliquer le groupe dans ta session actuelle
```

● PAS DE `CHMOD 777 /VAR/RUN/DOCKER.SOCK`

Le tutoriel fait `sudo chmod 777 /var/run/docker.sock` : n'importe quel utilisateur ou processus de la machine devient alors root via Docker, et le réglage disparaît au redémarrage de Docker. L'ajout de `jenkins` au groupe `docker` + redémarrage de Jenkins est la bonne méthode.

3. SonarQube en conteneur :

```
echo "vm.max_map_count=524288" | sudo tee /etc/sysctl.d/99-sonar.conf && sudo sysctl --system
docker run -d --name sonar --restart unless-stopped -p 9000:9000 \
-v sonar_data:/opt/sonarqube/data -v sonar_ext:/opt/sonarqube/extensions sonarqube:community
```

BASH

(Le tag `sonarqube:lts-community` du tutoriel est l'ancien nom ; `community` est la version Community Build actuelle. Le volume garde les données si le conteneur est recréé.)

4. Trivy : recette de l'annexe A.5.

```
trivy --version
```

BASH

Étape 3 — La clé d'API TMDb

L'application affiche des films grâce à **The Movie Database** : crée un compte sur themoviedb.org → profil → *Settings* → *API* → *Create* → *Developer* → accepte les conditions → remplis le formulaire → copie la **clé API (v3)**.

🧠 POURQUOI PASSER LA CLÉ AU MOMENT DU BUILD ?

C'est une application **front-end** (React compilé par Vite) : le code s'exécute dans le navigateur de l'utilisateur. La clé est injectée à la **compilation** (`ARG TMDb_V3_API_KEY` → `ENV VITE_APP_TMDb_V3_API_KEY` dans le Dockerfile) et se retrouve **dans le JavaScript envoyé aux navigateurs**. Elle n'est donc **pas secrète** pour les visiteurs : c'est acceptable pour une clé TMDb gratuite, jamais pour une vraie clé sensible (qui doit rester côté serveur, derrière une API).

Étape 4 — Le serveur de monitoring

Deuxième instance Ubuntu 24.04 [t3.medium](#), 20 Go, ports **9090** (Prometheus), **9100** (node_exporter), **3000** (Grafana) depuis ton IP.

Installe **Prometheus**, **node_exporter** et **Grafana** en suivant l'**annexe A.7** (utilisateurs système dédiés, services systemd, fichier `prometheus.yml`). Le tutoriel d'origine décrit exactement cette procédure, avec des versions de 2023 ; l'annexe utilise une variable `VERSION` à remplacer par la dernière version publiée.

✓ VÉRIFICATION

- `http://<IP_MONITORING>:9090/targets` : les cibles `prometheus` et `node_exporter` sont **UP**.
- `http://<IP_MONITORING>:3000` : connexion `admin / admin` (change le mot de passe), source de données Prometheus `http://localhost:9090` → *Save & test* OK, import du tableau **1860** (Node Exporter Full).

Étape 5 — Surveiller Jenkins avec Prometheus

- Jenkins → *Plugins* → installe **Prometheus metrics**. Il expose les métriques sur `http://<IP_JENKINS>:8080/prometheus` (*Manage Jenkins* → *System* → *Prometheus* pour les options).
- Sur le serveur de monitoring, ajoute la cible dans `/etc/prometheus/prometheus.yml` :

```
- job_name: jenkins
  metrics_path: /prometheus
  static_configs:
    - targets: ['<IP_PRIVÉE_JENKINS>:8080']
```

YAML

```
promtool check config /etc/prometheus/prometheus.yml # toujours valider avant de recharger
curl -X POST http://localhost:9090/-/reload           # rechargement à chaud (--web.enable-lifecycle)
```

BASH

3. Le Security Group du serveur Jenkins doit autoriser **8080** depuis le serveur de monitoring.

4. Grafana → import du tableau **9964** (Jenkins performance and health).

Étape 6 — Les notifications e-mail

- Compte Gmail dédié → **validation en deux étapes** activée → *Gérer votre compte* → *Sécurité* → *Mots de passe des applications* → crée un mot de passe d'application « Jenkins » (16 caractères).
- Jenkins → plugin **Email Extension** (souvent déjà installé).
- Manage Jenkins* → *Credentials* → *Username with password* : `ton.compte@gmail.com` + mot de passe d'application, ID `mail-cred`.
- Manage Jenkins* → *System* :
 - Extended E-mail Notification** : SMTP server `smtp.gmail.com`, port **465**, *Advanced* → credentials `mail-cred`, coche **Use SSL** ; *Default Content Type* : HTML.

- **E-mail Notification** (le plugin de base, pour tester) : même serveur, *Use SMTP Authentication, Use SSL*, port 465 → *Test configuration by sending test e-mail*.

Étape 7 — Les plugins et outils du pipeline

Plugins : Eclipse Temurin installer, SonarQube Scanner, NodeJS, OWASP Dependency-Check, Docker, Docker Commons, Docker Pipeline, Docker API, docker-build-step, Kubernetes CLI (pour [withKubeConfig](#)), Kubernetes Credentials.

Tools (Manage Jenkins → Tools) :

Type	Nom (utilisé dans le Jenkinsfile)	Réglage
JDK	jdk17	Install automatically → adoptium.net → 17
NodeJS	node16	(mieux : une version LTS actuelle, 20 ou 22, avec le nom node20 dans le Jenkinsfile)
SonarQube Scanner	sonar-scanner	Install automatically
Dependency-Check	DP-Check	Install automatically from github.com
Docker	docker	Install automatically from docker.com

SonarQube : SonarQube → My Account → Security → token → Jenkins Credentials → Secret text ID [Sonar-token](#) → Manage Jenkins → System → SonarQube servers : nom [sonar-server](#), URL http://<IP_PRIVÉE>:9000, token [Sonar-token](#). Dans SonarQube : Administration → Webhooks → http://<IP_PRIVÉE>:8080/sonarqube-webhook/.

Docker Hub : credentials Username with password, ID [docker](#) (pseudo + access token Docker Hub).

NVD : Secret text ID [nvd-api-key](#) (voir leçon 14).

Étape 8 — Le pipeline

Fork github.com/Aj7Ay/Netflix-clone sur ton GitHub. Jenkins → New Item [netfliX](#) → Pipeline → script :

```
pipeline {
    agent any
    tools {
        jdk 'jdk17'
        nodejs 'node16'
    }
    environment {
        SCANNER_HOME = tool 'sonar-scanner'
        IMAGE = '<toi-dockerhub>/netfliX'
    }
    stages {
        stage('Clean workspace') { steps { cleanWs() } }

        stage('Checkout') {
            steps { git branch: 'main', url: 'https://github.com/<toi>/Netflix-clone.git' }
        }

        stage('Analyse SonarQube') {
            steps {
                withSonarQubeEnv('sonar-server') {
                    sh '$SCANNER_HOME/bin/sonar-scanner -Dsonar.projectName=Netflix -Dsonar.projectKey=Netflix'
                }
            }
        }

        stage('Quality Gate') {
            steps {
                script { waitForQualityGate abortPipeline: false, credentialsId: 'Sonar-token' }
            }
        }

        stage('Dépendances') { steps { sh 'npm install' } }

        stage('OWASP Dependency-Check') {
            steps {
                withCredentials([string(credentialsId: 'nvd-api-key', variable: 'NVD_KEY')]) {
                    dependencyCheck additionalArguments: "--scan ./ --disableYarnAudit --disableNodeAudit --nvdApiKey ${NVD_KEY}",
                                odcInstallation: 'DP-Check'
                }
                dependencyCheckPublisher pattern: '**/dependency-check-report.xml'
            }
        }

        stage('Trivy FS') { steps { sh 'trivy fs . > trivyfs.txt' } }
    }
}
```

```

stage('Build & push Docker') {
    steps {
        withCredentials([string(credentialsId: 'tmdb-api-key', variable: 'TMDB')]) {
            script {
                withDockerRegistry(credentialsId: 'docker', toolName: 'docker') {
                    sh '''
                        docker build --build-arg TMDB_V3_API_KEY=$TMDB -t $IMAGE:$BUILD_NUMBER -t $IMAGE:latest .
                        docker push $IMAGE:$BUILD_NUMBER
                        docker push $IMAGE:latest
                    '''
                }
            }
        }
    }
}

stage('Trivy image') { steps { sh 'trivy image $IMAGE:$BUILD_NUMBER > trivyimage.txt' } }

stage('Déployer en conteneur') {
    steps {
        sh '''
            docker rm -f netflix || true
            docker run -d --name netflix -p 8081:80 $IMAGE:$BUILD_NUMBER
        '''
    }
}

// Étape 9 : décommenter une fois le cluster Kubernetes prêt
// stage('Déployer sur Kubernetes') {
//     steps {
//         dir('Kubernetes') {
//             withKubeConfig(credentialsId: 'k8s') {
//                 sh 'kubectl apply -f deployment.yml -f service.yml'
//             }
//         }
//     }
// }

}

post {
    always {
        emailx attachLog: true,
            subject: "${currentBuild.result} : ${env.JOB_NAME} #${env.BUILD_NUMBER}",
            body: "Projet : ${env.JOB_NAME}<br/>Build : ${env.BUILD_NUMBER}<br/>URL : ${env.BUILD_URL}",
            to: 'ton.compte@gmail.com',
            attachmentsPattern: 'trivyfs.txt,trivyimage.txt'
    }
}
}

```

Ajoute aussi la clé TMDB en credential *Secret text* ID `tmdb-api-key` .

⚠ LES ERREURS DU JENKINSFILE D'ORIGINE

Le fichier `jenkinsfile` du dépôt contient : la clé TMDB **en clair** dans le pipeline (`--build-arg TMDB_V3_API_KEY=AJ7AYe14...`), un nom d'image différent entre le push (`rutik/netflix`) et le scan (`sevenajay/netflix`) — on scanne donc **une autre image** que celle construite —, et un `docker run --name netflix` qui échoue au deuxième build (conteneur déjà existant). La version ci-dessus corrige les trois.

✅ VÉRIFICATION

- Pipeline vert, *Stage View* complète.
- SonarQube : projet *Netflix* avec ses résultats ; Jenkins : graphique *Dependency-Check* sur la page du job.
- `http://<IP>:8081` : le clone de Netflix avec les affiches de films.
- Un e-mail avec le statut, le log et les deux rapports Trivy en pièces jointes.
- Grafana (tableau 9964) : le nombre de builds et leur durée.

Étape 9 — Kubernetes (master + worker) et déploiement

1. Deux instances Ubuntu 24.04 `t3.medium` (`k8s-master` , `k8s-worker`), SG autorisant tout le trafic **entre elles**, 22 depuis ton IP, **30007** depuis ton IP (NodePort de l'application).
2. Installe le cluster avec **kubeadm** : **annexe A.6** (containerd, kubelet/kubeadm/kubectl, `kubeadm init` sur le master, réseau de pods, `kubeadm join` sur le worker). (Le *tutoriel d'origine* installe `docker.io` comme moteur de conteneurs et une ancienne version de Kubernetes depuis `apt.kubernetes.io` , dépôt **supprimé** en 2024 : l'annexe utilise le dépôt actuel `pkgs.k8s.io` .)

3. Sur le master : `kubectl get nodes` → 2 nœuds `Ready` . Copie le contenu de `~/.kube/config` .
4. Jenkins → *Credentials* → **Secret file**, ID `k8s` , fichier = ce kubeconfig. (L'adresse du serveur dans ce fichier doit être joignable depuis Jenkins : l'IP **privée** du master si tout est dans le même VPC, et le port 6443 ouvert depuis Jenkins.)
5. Dans ton fork, `Kubernetes/deployment.yml` : remplace l'image par `<toi-dockerhub>/netflix:latest` . Décommente le stage Kubernetes et relance.

```
kubectl get all # sur le master
```

BASH

Ouvre `http://<IP_PUBLIQUE_WORKER>:30007` .

Monitoring des nœuds Kubernetes : installe `node_exporter` (annexe A.7) sur le master et le worker, et ajoute-les à Prometheus :

```
- job_name: k8s-nodes
  static_configs:
    - targets: ['<IP_PRIVATE_MASTER>:9100', '<IP_PRIVATE_WORKER>:9100']
```

YAML

Dépannage

🔧 DÉPANNAGE

- **docker: permission denied dans Jenkins** : `jenkins` pas dans le groupe `docker`, ou Jenkins pas redémarré.
- **SonarQube s'arrête au démarrage** : `docker logs sonar` → `vm.max_map_count` (étape 2) ou mémoire insuffisante.
- **Dependency-Check très long ou en erreur 403** : clé NVD absente.
- **npm install échoue** (versions de Node) : le projet date de 2023 ; Node 16 fonctionne, Node 20 aussi la plupart du temps. En cas d'erreur, garde `node16` comme dans l'original.
- **Pas d'affiches de films** : clé TMDB absente ou invalide au moment du build.
- **E-mail non reçu** : mot de passe d'application (pas ton mot de passe Gmail), port 465 + SSL, ou dossier spam.
- **withKubeConfig : Unable to connect to the server** : adresse du master injoignable depuis Jenkins, ou port 6443 fermé.
- **Pods ImagePullBackOff** : nom d'image du `deployment.yml` non modifié.

Nettoyage

🧹 NETTOYAGE OBLIGATOIRE

Terminate les 4 instances (Netflix, Monitoring, master, worker), supprime les Security Groups et les volumes restants. Révoque le mot de passe d'application Gmail et le token Docker Hub si tu ne t'en sers plus.

🔑 QUESTIONS D'ENTRETIEN

- Décris ton pipeline DevSecOps et ce que contrôle chaque étape de sécurité.
- Pourquoi la clé TMDB n'est-elle pas vraiment secrète dans ce projet ? Comment protéger une vraie clé dans une application front-end ?
- Comment Prometheus récupère-t-il les métriques de Jenkins ?
- Que changerais-tu pour que les failles critiques bloquent réellement le déploiement ? (Quality Gate `abortPipeline: true`, `trivy --exit-code 1 --severity CRITICAL`, seuil sur *Dependency-Check*.)

Projet 13 — Clone de Zomato : déploiement sécurisé avec une CI/CD DevSecOps

FICHE PROJET

Niveau	★★★☆☆ Intermédiaire (après le projet 09)
Durée	4 à 6 heures
Coût	~0,10 \$/h (t3.large + éventuellement un serveur de monitoring)
Compétences	rejouer seul un pipeline DevSecOps complet sur une nouvelle application ; durcir le Dockerfile ; rendre les contrôles bloquants
Prérequis	Projet 09 terminé
Dépôt d'origine	DevOps-Project-13/README.md (lien vers le blog et vers le code) ; application github.com/Aj7Ay/Zomato-Clone (React, Create React App, port 3000)

Ce qu'on construit

Le **même** pipeline que le projet 09, appliqué à une autre application : un clone de Zomato (le « Deliveroo » indien). L'objectif pédagogique est différent : **tu le fais seul**, à partir de tes notes du projet 09, puis tu **l'améliores** sur trois points que le tutoriel d'origine laisse de côté.

```
GitHub (fork Zomato-Clone) → Jenkins
clean → checkout → SonarQube → Quality Gate (BLOQUANTE) → npm install
→ OWASP Dependency-Check → Trivy fs (BLOQUANT sur CRITICAL)
→ docker build (image durcie, multi-stage) → push Docker Hub → Trivy image
→ docker run -p 3000:... → e-mail
```

LA RÈGLE DU JEU

Ouvre ton [NOTES.md](#) du projet 09 et refais l'installation **sans** relire le projet 09 (sauf en cas de blocage). Note le temps passé : la deuxième fois doit être deux fois plus rapide. C'est exactement ce qu'on attend d'un ingénieur : capitaliser.

Étape 1 — L'infrastructure et les outils

Reprends les étapes 1, 2 et 7 du projet 09 : instance [t3.large](#), Jenkins, Docker (groupe [docker](#) pour [jenkins](#)), SonarQube en conteneur, Trivy, plugins, outils ([jdk17](#), [node16](#) ou [node20](#), [sonar-scanner](#), [DP-Check](#), [docker](#)), credentials ([Sonar-token](#), [docker](#), [nvd-api-key](#), e-mail).

Le Security Group doit ouvrir le port de l'application (**3000** dans le tutoriel, ou 8081 avec le Dockerfile durci ci-dessous).

Étape 2 — Amélioration n°1 : un Dockerfile de production

Le Dockerfile d'origine utilise [node:16-slim](#), installe toutes les dépendances, construit l'application... puis la lance avec [npm start](#), c'est-à-dire le **serveur de développement** de Create React App. Résultat : une image lourde (~1 Go), un serveur non prévu pour la production, et toutes les dépendances de développement (avec leurs failles) embarquées.

Remplace-le par un build **multi-stage** servi par Nginx, avec un utilisateur non-root :

```
# Étape 1 : construire les fichiers statiques
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build                # produit /app/build

# Étape 2 : les servir avec Nginx, sans Node ni sources
FROM nginxinc/nginx-unprivileged:stable-alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 8080                      # l'image "unprivileged" écoute sur 8080, utilisateur non-root
```

DOCKERFILE

✓ COMPARE

```
docker images | grep zomato
trivy image <ancienne-image> | grep Total
trivy image <nouvelle-image> | grep Total
```

BASH

Note dans ton `NOTES.md` la taille et le nombre de vulnérabilités **avant/après**. La différence est en général spectaculaire (taille divisée par 20, vulnérabilités divisées par 10 ou plus). C'est un excellent argument d'entretien.

⚠ 'NPM CI' ET NODE 20 SUR UN VIEUX PROJET

`npm ci` exige un `package-lock.json` cohérent avec `package.json`. Si le build échoue avec des erreurs OpenSSL (`ERR_OSSL_EVP_UNSUPPORTED`) typiques des vieux projets Create React App sur Node ≥ 17 , ajoute `ENV NODE_OPTIONS=--openssl-legacy-provider` dans l'étape de build, ou reste sur `node:16-alpine` pour cette étape.

Étape 3 — Amélioration n°2 : des contrôles qui bloquent

```
stage('Quality Gate') {
  steps {
    timeout(time: 5, unit: 'MINUTES') {
      waitForQualityGate abortPipeline: true, credentialsId: 'Sonar-token'
    }
  }
}
stage('Trivy FS') {
  steps {
    sh 'trivy fs --format table -o trivyfs.txt .'
    sh 'trivy fs --severity CRITICAL --exit-code 1 --ignore-unfixed .' // échoue s'il reste du CRITICAL corrigéable
  }
}
stage('Trivy image') {
  steps {
    sh 'trivy image --format table -o trivyimage.txt $IMAGE:$BUILD_NUMBER'
    sh 'trivy image --severity CRITICAL --exit-code 1 --ignore-unfixed $IMAGE:$BUILD_NUMBER'
  }
}
```

GROOVY

🧠 '--IGNORE-UNFIXED'

Certaines CVE n'ont **pas encore de correctif**. Bloquer le pipeline pour elles empêcherait toute livraison sans que tu puisses rien y faire. `--ignore-unfixed` ne retient que les failles **corrigéables** : tu peux agir (mettre à jour la dépendance ou l'image de base). Pour les autres, on documente une exception dans un fichier `.trivyignore`, avec une date de revue.

Quand le pipeline échoue sur une faille, **corrige-la** (mise à jour de la dépendance avec `npm audit fix` ou `npm install <paquet>@<version>`, ou de l'image de base), commit, relance. C'est le vrai travail DevSecOps.

Étape 4 — Amélioration n°3 : la détection de secrets

Ajoute un stage `gitleaks` juste après le checkout :

```
stage('Secrets (gitleaks)') {
  steps {
    sh 'docker run --rm -v "$PWD:/repo" zricethezav/gitleaks:latest detect --source /repo --no-git -v'
  }
}
```

GROOVY

Teste-le : ajoute dans un fichier une fausse clé AWS (`AKIAIOSFODNN7EXAMPLE` + une chaîne de 40 caractères) sur une branche, pousse, observe l'échec, puis supprime-la.

Étape 5 — Déploiement et vérification

```
stage('Déployer') {
  steps {
    sh '''
      docker rm -f zomato || true
      docker run -d --name zomato -p 3000:8080 $IMAGE:$BUILD_NUMBER
    '''
  }
}
```

GROOVY

✓ VÉRIFICATION

- `http://<IP>:3000` affiche le clone de Zomato.
- Un pipeline volontairement « cassé » (fausse clé, dépendance vulnérable, code qui fait échouer la Quality Gate) **s'arrête** avant le déploiement.
- Ton `NOTES.md` contient le comparatif avant/après du Dockerfile.

Pour aller plus loin

- Ajoute le monitoring du projet 09 (Prometheus + Grafana) et un **blackbox_exporter** qui sonde `http://<IP>:3000` (annexe A.7).
- Déploie sur Kubernetes (kubeadm ou EKS) avec un `Deployment` dont le conteneur tourne en non-root (`securityContext.runAsNonRoot: true`).

🔧 NETTOYAGE OBLIGATOIRE

Terminate l'instance, supprime le SG, garde ton fork.

🔑 QUESTIONS D'ENTRETIEN

- Pourquoi ne faut-il jamais lancer `npm start` (serveur de développement) en production ?
- Comment fais-tu pour qu'une faille critique bloque une livraison sans paralyser l'équipe ?
- Qu'est-ce qu'un build multi-stage et qu'apporte-t-il à la sécurité ?

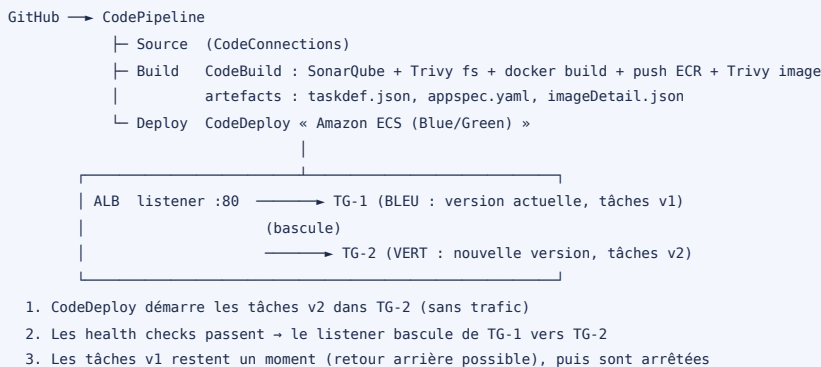
Projet 23 — DevSecOps : déploiement blue/green du clone de Swiggy sur ECS avec AWS CodePipeline

FICHE PROJET

Niveau	★★★★☆ Avancé
Durée	6 à 10 heures
Coût	~0,15 \$/h (SonarQube <code>t3.medium</code> , cluster ECS, ALB)
Compétences	CodeBuild avec scans de sécurité, ECR, ECS (task definition, service), ALB avec deux target groups , déploiement blue/green avec CodeDeploy , CodePipeline, <code>taskdef.json</code> / <code>appspec.yaml</code> / <code>imageDetail.json</code> , notification SES ou SNS
Prérequis	Projets 04 (ECS) et 21 (CodePipeline), leçon 14
Dépôt d'origine	<code>DevOps-Project-23/README.md</code> ; application <code>DevOps-Project-23/Swiggy_clone</code> (React, port 3000, <code>buildspec.yaml</code> , <code>appspec.yaml</code>)

Ce qu'on construit

Le même type de pipeline AWS que le projet 21, mais avec une stratégie de déploiement **sans interruption** et avec **retour arrière instantané** : le **blue/green**.



Les concepts

LE DÉPLOIEMENT BLUE/GREEN

Deux environnements identiques : **bleu** (la version en production) et **vert** (la nouvelle version). On déploie la nouvelle version à **côté** de l'ancienne, on la teste, puis on **bascule le trafic** d'un coup (au niveau du load balancer). Si un problème apparaît, on rebascule vers le bleu en quelques secondes, sans redéployer.

Comparé au **rolling update** (projets 04, 08) où les anciennes instances sont remplacées progressivement : le blue/green coûte temporairement le double de ressources, mais offre un **retour arrière instantané** et aucun mélange de versions. Il existe aussi le **canary** : envoyer d'abord 10 % du trafic à la nouvelle version, puis augmenter si tout va bien (CodeDeploy le propose : `CodeDeployDefault::ECSCanary10Percent5Minutes`).

LES TROIS FICHIERS D'UN DÉPLOIEMENT ECS BLUE/GREEN

L'action *Amazon ECS (Blue/Green)* de CodePipeline attend dans l'artefact :

- `taskdef.json` : la task definition, avec un **marqueur** `<IMAGE1_NAME>` à la place de l'image ;
- `appspec.yaml` : indique à CodeDeploy quel conteneur et quel port enregistrer dans le load balancer, avec un marqueur `<TASK_DEFINITION>` ;
- `imageDetail.json` : produit par le build, contient l'URI de l'image qui vient d'être poussée (`{ "ImageURI": "...:42" }`).

CodePipeline remplace les marqueurs, enregistre une **nouvelle révision** de la task definition et la passe à CodeDeploy. Le `appspec.yaml` du dépôt d'origine contient un ARN de task definition **codé en dur** (`...task-definition/swiggy:1`) : chaque déploiement redéploierait la révision 1, jamais la nouvelle image. On corrige ça.

Étape 1 — Le serveur SonarQube

EC2 Ubuntu `t3.medium`, port 9000 ouvert **depuis Internet** (CodeBuild tourne hors de ton VPC par défaut et doit le joindre — à restreindre en production en plaçant CodeBuild dans ton VPC). Installe Docker et lance SonarQube (projet 09, étape 2). Crée un projet `swiggy` et un `token`.

Étape 2 — Le code et les paramètres

1. Dépôt GitHub `swiggy-clone` avec le contenu de `DevOps-Project-23/Swiggy-clone`.
2. Dépôt ECR : `aws ecr create-repository --repository-name swiggy --image-scanning-configuration scanOnPush=true`.
3. Paramètres SSM :

```
aws ssm put-parameter --name /cicd/sonar/sonar-token --type SecureString --value '<token-sonar>'
aws ssm put-parameter --name /cicd/sonar/url --type String --value 'http://<IP_SONAR>:9000'
```

BASH

4. **Dockerfile** : celui d'origine lance `npm start` (serveur de développement, port 3000). Garde-le pour coller au projet (le port 3000 est utilisé partout), ou applique le Dockerfile durci du projet 13 (et adapte le port à 8080 dans la task definition et l' `appspec.yaml`).

Étape 3 — Le `buildspec.yaml`, réécrit

Le `buildspec.yaml` d'origine est **presque entièrement commenté** (rien n'est construit) et son bloc `post_build` est mal indenté (hors de `phases`). Voici une version fonctionnelle :

```
version: 0.2

env:
  parameter-store:
    SONAR_TOKEN: /cicd/sonar/sonar-token
    SONAR_URL: /cicd/sonar/url
  variables:
    REPO_NAME: swiggy

phases:
  install:
    runtime-versions:
      nodejs: 20
      java: corretto17
    commands:
      - curl -sfl https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh | sh -s -- -b /usr/local/bin
      - npm install -g sonarqube-scanner
  pre_build:
    commands:
      - ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
      - REGISTRY=$ACCOUNT_ID.dkr.ecr.$AWS_DEFAULT_REGION.amazonaws.com
      - IMAGE_URI=$REGISTRY/$REPO_NAME:$CODEBUILD_RESOLVED_SOURCE_VERSION
      - aws ecr get-login-password | docker login --username AWS --password-stdin $REGISTRY
      - echo "Analyse SonarQube"
      - sonar-scanner -Dsonar.projectKey=swiggy -Dsonar.sources=src -Dsonar.host.url=$SONAR_URL -Dsonar.token=$SONAR_TOKEN
      - echo "Scan Trivy du code"
      - trivy fs --format table -o trivyfs.txt .
  build:
    commands:
      - docker build -t $IMAGE_URI .
      - trivy image --format table -o trivyimage.txt $IMAGE_URI
      - trivy image --severity CRITICAL --exit-code 1 --ignore-unfixed $IMAGE_URI # bloque si CRITICAL corrigé
  post_build:
    commands:
      - docker push $IMAGE_URI
      - printf '{"ImageURI":"%s"}' $IMAGE_URI > imageDetail.json

artifacts:
  files:
    - appspec.yaml
    - taskdef.json
    - imageDetail.json

reports:
  security:
    files: [trivyfs.txt, trivyimage.txt]
    file-format: CUCUMBERJSON # (format indicatif ; sinon publier les fichiers dans S3)
```

YAML

`'$CODEBUILD_RESOLVED_SOURCE_VERSION'`

C'est le **hash du commit** construit. Utilisé comme tag d'image, il relie chaque image à un commit précis : on sait toujours exactement quel code tourne en production. C'est bien plus traçable que `latest`.

(La section `reports` attend des formats précis (JUnit, Cucumber...). Si elle pose problème, supprime-la et copie plutôt les rapports dans un bucket S3 en `post_build`.)

Le rôle de service CodeBuild a besoin de : lire les paramètres SSM (`ssm:GetParameters` sur `/cicd/*`), pousser dans ECR (politique `AmazonEC2ContainerRegistryPowerUser`), et le **mode privilégié** activé (pour Docker). Le tutoriel donne `AmazonSSMFullAccess` et `AmazonS3FullAccess` : beaucoup trop large, limite aux ressources nécessaires.

Étape 4 — `taskdef.json` et `appspec.yaml`

À la racine du dépôt, `taskdef.json` :

```
{
  "family": "swiggy",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "512",
  "memory": "1024",
  "executionRoleArn": "arn:aws:iam::<compte>:role/ecsTaskExecutionRole",
  "containerDefinitions": [
    {
      "name": "swiggy",
      "image": "<IMAGE1_NAME>",
      "essential": true,
      "portMappings": [{ "containerPort": 3000, "protocol": "tcp" }],
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/swiggy",
          "awslogs-region": "eu-west-3",
          "awslogs-stream-prefix": "swiggy",
          "awslogs-create-group": "true"
        }
      }
    }
  ]
}
```

Et `appspec.yaml` (remplace celui d'origine) :

```
version: 0.0
Resources:
  - TargetService:
      Type: AWS::ECS::Service
      Properties:
        TaskDefinition: <TASK_DEFINITION>
        LoadBalancerInfo:
          ContainerName: "swiggy"
          ContainerPort: 3000
```

(Le tutoriel utilise un cluster ECS en mode **EC2** avec des `t2.medium`. **Fargate** fonctionne de la même façon et évite de gérer des instances ; c'est ce que décrit le `taskdef.json` ci-dessus. Si tu préfères EC2, mets `"requiresCompatibilities": ["EC2"]` et `"networkMode": "awsvpc"` ou `"bridge"` selon ton cluster.)

Étape 5 — ALB, deux target groups, cluster et service

1. Deux **target groups** de type **IP** (Fargate), port 3000, VPC par défaut, health check `/` : `swiggy-tg-1` et `swiggy-tg-2`.
2. **ALB** `swiggy-alb` (Internet-facing, subnets publics, SG HTTP 80), listener **HTTP 80** → `swiggy-tg-1`. (Optionnel : un listener de test sur 8080 → `swiggy-tg-2`, pour tester la version verte avant bascule.)
3. **Rôle CodeDeploy pour ECS** : IAM → *Create role* → *CodeDeploy* → cas d'usage **CodeDeploy - ECS** (politique `AWSCodeDeployRoleForECS`) → `io-23-codedeploy-ecs`.
4. **Cluster ECS** `swiggy-cluster` (Fargate).
5. **Enregistre une première task definition** à la main (remplace `<IMAGE1_NAME>` par une image existante, par exemple celle du premier build CodeBuild lancé à la main) : `aws ecs register-task-definition --cli-input-json file://taskdef-initial.json`.
6. **Service** : cluster → *Create service* → Fargate, task definition `swiggy`, 1 tâche → *Deployment options* : **Blue/green deployment (powered by AWS CodeDeploy)** → rôle `io-23-codedeploy-ecs` → *Load balancing* : ALB `swiggy-alb`, listener 80, **target group 1** : `swiggy-tg-1`, **target group 2** : `swiggy-tg-2` → SG des tâches : 3000 depuis le SG de l'ALB.

La création du service crée automatiquement une **application** et un **deployment group** dans CodeDeploy (`AppECS-swiggy-cluster-<service>`).

✓ VÉRIFICATION

`http://<DNS-de-l'ALB>` affiche l'application ; l'onglet du navigateur s'appelle « Swiggy Application ». Dans EC2 → *Target groups*, `swiggy-tg-1` a une cible *healthy*, `swiggy-tg-2` aucune.

Étape 6 — Le pipeline

CodePipeline → Create pipeline :

- **Source** : GitHub via CodeConnections, branche `main` .
- **Build** : CodeBuild (le projet de l'étape 3).
- **Deploy** : fournisseur **Amazon ECS (Blue/Green)** → application et deployment group CodeDeploy créés à l'étape 5 → *Amazon ECS task definition* : `BuildArtifact`, fichier `taskdef.json` → *AWS CodeDeploy AppSpec file* : `BuildArtifact`, `appspec.yaml` → *Dynamically update task definition image* : `BuildArtifact`, marqueur `IMAGE1_NAME` .

Étape 7 — Observer une bascule blue/green

Modifie un texte visible de l'application (par exemple le titre de `public/index.html`), commit, push. Suis le déploiement dans **CodeDeploy** → **Deployments** :

1. *Deploying replacement task set* : les tâches de la nouvelle version démarrent dans `swiggy-tg-2` .
2. *Rerouting production traffic* : le listener 80 pointe maintenant vers `swiggy-tg-2` .
3. *Wait* (durée configurable dans le deployment group, par défaut une heure → réduis-la à 5 minutes) : l'ancienne version reste disponible.
4. *Terminate original task set*.

Pendant l'attente, le bouton **Stop and roll back deployment** ramène instantanément le trafic vers l'ancienne version. Essaie-le une fois !

✓ VÉRIFICATION

- Recharge `http://<DNS-ALB>` pendant le déploiement : aucune erreur, puis la nouvelle version apparaît d'un coup.
- Le listener 80 de l'ALB alterne entre TG-1 et TG-2 à chaque déploiement.
- ECR contient une image par commit ; les rapports Trivy et l'analyse SonarQube sont disponibles.

Étape 8 (optionnelle) — Notification

Le tutoriel envoie un e-mail via **SES** (`aws ses send-email`), ce qui exige de vérifier les adresses dans SES (et de sortir du *sandbox* pour écrire à d'autres adresses). Plus simple : une **règle de notification** sur le pipeline (*Settings* → *Notifications* → *Create notification rule*) vers un topic **SNS** auquel tu es abonné par e-mail, sur les événements *Failed* et *Succeeded*.

Dépannage

🔧 DÉPANNAGE

- **Le déploiement échoue avec « The ECS service cannot be updated due to an unexpected error »** : le service n'a pas été créé avec le contrôleur de déploiement `CODE_DEPLOY` (il faut le recréer, ce réglage ne se modifie pas).
- **Invalid action configuration: ... IMAGE1_NAME** : le marqueur dans `taskdef.json` doit être exactement `<IMAGE1_NAME>` et le nom dans l'action `IMAGE1_NAME` .
- **Nouvelles tâches jamais saines** : SG des tâches (3000 depuis l'ALB), health check, ou l'application met longtemps à démarrer (`npm start` compile au démarrage : augmente le *health check grace period*).
- **CodeBuild** : `sonar-scanner: Connection refused` : SonarQube injoignable depuis CodeBuild (port 9000, IP publique).
- **CodeBuild** : `exit status 1` au **Trivy image** : c'est voulu, une faille CRITICAL corrigable a été trouvée. Lis `trivyimage.txt` .

Nettoyage

🔪 NETTOYAGE OBLIGATOIRE

Supprime : le pipeline, le projet CodeBuild, l'application CodeDeploy, le **service** ECS (passe-le à 0 tâche puis supprime-le), le cluster, l'ALB et les deux target groups, le dépôt ECR (`--force`), l'instance SonarQube, les paramètres SSM, les rôles IAM, le log group `/ecs/swiggy` .

🔑 QUESTIONS D'ENTRETIEN

- Explique la différence entre rolling update, blue/green et canary. Avantages et coûts de chacun ?
- Comment fonctionne un déploiement blue/green sur ECS avec CodeDeploy ? Que contiennent `taskdef.json`, `appspec.yaml` et `imageDetail.json` ?
- Comment revenir en arrière en blue/green ?
- Pourquoi taguer les images avec le hash du commit ?

Projet 24 — Pipeline DevSecOps temps réel pour une application web .NET

FICHE PROJET

Niveau	★★☆☆ Intermédiaire
Durée	5 à 8 heures
Coût	~0,10 \$/h (t3.large 30 Go) + 2 t3.medium pour Kubernetes
Compétences	pipeline DevSecOps pour une application .NET, Makefile comme interface de build, conventions de nommage Docker, déploiement Docker puis Kubernetes, application de monitoring système
Prérequis	Projet 09
Dépôt d'origine	DevOps-Project-24/README.md ; application DevOps-Project-24/DotNet-monitoring (ASP.NET Core, makefile, build/Dockerfile, K8S/deployment.yaml)

Ce qu'on construit

Une application **ASP.NET Core** qui affiche en temps réel les métriques de la machine où elle tourne (CPU, mémoire, informations système), livrée par le même type de pipeline DevSecOps que le projet 09 :

```
GitHub → Jenkins : clean → checkout → SonarQube → Quality Gate → Trivy fs → OWASP DC
                → make image (docker build) → Trivy image → make push (Docker Hub)
                → docker run :5000 → (Kubernetes : kubectl apply K8S/deployment.yaml, NodePort)
```

La nouveauté : le pipeline appelle un **Makefile** (`make image`, `make push`) au lieu de commandes `docker` directes.

Les concepts

LE MAKEFILE COMME INTERFACE COMMUNE

Ouvre `DotNet-monitoring/makefile` : il définit des **cibles** (`lint`, `image`, `push`, `run`, `test`, `deploy` ...) avec des variables surchargeables :

```
IMAGE_REG ?= docker.io
IMAGE_REPO ?= writetoritika/dotnet-monitoring
IMAGE_TAG ?= latest

image: ## 🐳 Build container image from Dockerfile
    docker build . --file build/Dockerfile --tag $(IMAGE_REG)/$(IMAGE_REPO):$(IMAGE_TAG)
```

Le même `make image` fonctionne sur le poste du développeur **et** dans le pipeline : la logique de build est écrite **une seule fois**, dans le dépôt, et le pipeline reste simple. `?=` signifie « valeur par défaut, sauf si on en fournit une » : `make image IMAGE_REPO=moi/dotnet-monitoring IMAGE_TAG=42`. Tape `make help` pour voir les cibles documentées.

Étape 1 — Préparer le dépôt

⚠ L'URL GIT DU TUTORIEL NE FONCTIONNE PAS

Le Jenkinsfile d'origine fait `git url: 'https://github.com/devcloudninjas/DevOps-Projects/tree/master/DevOps-Project-24/DotNet-monitoring'`. **On ne peut pas cloner un sous-dossier** d'un dépôt GitHub avec cette URL (`/tree/...` est une page web, pas un dépôt Git). Il faut un dépôt dont l'application est à la racine.

```
mkdir dotnet-monitoring && cp -r /tmp/dp/DevOps-Project-24/DotNet-monitoring/. dotnet-monitoring/
cd dotnet-monitoring && git init && git add . && git commit -m "Application DotNet-monitoring"
git branch -M main
git remote add origin git@github.com:<toi>/dotnet-monitoring.git && git push -u origin main
```

Dans le `makefile`, remplace `IMAGE_REPO ?= writetoritika/dotnet-monitoring` par **ton** pseudo Docker Hub. Dans `K8S/deployment.yaml`, l'image aussi.

Étape 2 — Le serveur

Reprends les étapes 1, 2 et 7 du projet 09 (instance [t3.large](#) 30 Go, Jenkins, Docker, SonarQube, Trivy, plugins, outils et credentials). Ajoute :

```
sudo apt install -y make
```

BASH

Port de l'application : **5000** dans le Security Group.

Étape 3 — Le pipeline

```
pipeline {
    agent any
    tools { jdk 'jdk17' }
    environment {
        SCANNER_HOME = tool 'sonar-scanner'
        IMAGE_REPO    = '<toi>-dockerhub>/dotnet-monitoring'
    }
    stages {
        stage('Clean workspace') { steps { cleanWs() } }
        stage('Checkout') {
            steps { git branch: 'main', url: 'https://github.com/<toi>/dotnet-monitoring.git' }
        }
        stage('Analyse SonarQube') {
            steps {
                withSonarQubeEnv('sonar-server') {
                    sh '$SCANNER_HOME/bin/sonar-scanner -Dsonar.projectName=Dotnet-Webapp -Dsonar.projectKey=Dotnet-Webapp'
                }
            }
        }
        stage('Quality Gate') {
            steps { script { waitForQualityGate abortPipeline: false, credentialsId: 'Sonar-token' } }
        }
        stage('Trivy FS') { steps { sh 'trivy fs . > trivy-fs_report.txt' } }
        stage('OWASP Dependency-Check') {
            steps {
                withCredentials([string(credentialsId: 'nvd-api-key', variable: 'NVD_KEY')]) {
                    dependencyCheck additionalArguments: "--scan ./ --format XML --nvdApiKey ${NVD_KEY}", odcInstallation: 'DP-Check'
                }
                dependencyCheckPublisher pattern: '**/dependency-check-report.xml'
            }
        }
        stage('Image Docker') {
            steps {
                script {
                    withDockerRegistry(credentialsId: 'docker', toolName: 'docker') {
                        sh 'make image IMAGE_REPO=$IMAGE_REPO IMAGE_TAG=$BUILD_NUMBER'
                    }
                }
            }
        }
        stage('Trivy image') {
            steps { sh 'trivy image docker.io/$IMAGE_REPO:$BUILD_NUMBER > trivy.txt' }
        }
        stage('Push') {
            steps {
                script {
                    withDockerRegistry(credentialsId: 'docker', toolName: 'docker') {
                        sh 'make push IMAGE_REPO=$IMAGE_REPO IMAGE_TAG=$BUILD_NUMBER'
                    }
                }
            }
        }
        stage('Déployer en conteneur') {
            steps {
                sh '''
                    docker rm -f dotnet || true
                    docker run -d --name dotnet -p 5000:5000 docker.io/$IMAGE_REPO:$BUILD_NUMBER
                '''
            }
        }
    }
}
```

GROOVY

⚠ LES NOMS D'IMAGES DOCKER SONT EN MINUSCULES

Le tutoriel utilise `NotHarshaa/dotnet-monitoring` : Docker **refuse** les majuscules dans un nom de dépôt (`invalid reference format: repository name must be lowercase`). Ton pseudo Docker Hub est toujours en minuscules dans les noms d'images.

✓ VÉRIFICATION

- `http://<IP>:5000` : l'application affiche les informations et la charge CPU/mémoire **du conteneur**. Lance une charge (`docker exec dotnet sh -c "yes > /dev/null &"`) et observe la courbe monter.
- SonarQube : projet *Dotnet-Webapp* ; Jenkins : graphique Dependency-Check.
- Docker Hub : `dotnet-monitoring:<numéro de build>`.

Étape 4 — Kubernetes

1. Cluster kubernetes master + worker (**annexe A.6**), comme au projet 09.
2. `k8s/deployment.yaml` contient un Deployment (port 5000) et un Service **NodePort** (port 80 → 5000, NodePort attribué automatiquement).
3. Ajoute le credential kubeconfig (*Secret file* `k8s`) et le stage :

```
stage('Déployer sur Kubernetes') {  
    steps {  
        withKubeConfig(credentialsId: 'k8s') {  
            sh 'kubectl apply -f k8s/deployment.yaml'  
            sh 'kubectl set image deployment/dotnet-app-deployment dotnet-app=docker.io/$IMAGE_REPO:$BUILD_NUMBER'  
            sh 'kubectl rollout status deployment/dotnet-app-deployment --timeout=120s'  
        }  
    }  
}
```

GROOVY

```
kubectl get svc dotnet-app-service      # repère le NodePort (30000-32767)
```

BASH

Ouvre `http://<IP_WORKER>:<NodePort>` (pense au Security Group).

🧠 'KUBECTL SET IMAGE' APRÈS 'APPLY'

Le fichier YAML contient l'image `:latest`. Avec `kubectl apply` seul, Kubernetes ne voit **aucun changement** d'un build à l'autre (le texte est identique) et ne redéploie pas, même si l'image `latest` a changé sur Docker Hub. `kubectl set image` avec le numéro de build modifie réellement le Deployment et déclenche un rolling update traçable. C'est une raison de plus pour ne jamais déployer `latest`.

Dépannage

🔧 DÉPANNAGE

- `make: command not found` dans Jenkins : `sudo apt install make`.
- `invalid reference format` : majuscules dans le nom d'image.
- **SonarQube analyse peu de choses** : pour une analyse C# complète, SonarQube recommande le *SonarScanner for .NET* (`dotnet sonarscanner begin/end` autour d'un `dotnet build`). Le scanner générique utilisé ici analyse surtout les fichiers non compilés. Bonne piste d'amélioration.
- **Port 5000 injoignable** : Security Group, ou l'application écoute sur un autre port (vérifie `EXPOSE` dans `build/Dockerfile` et `docker logs dotnet`).

🧹 NETTOYAGE OBLIGATOIRE

Terminate les instances (Jenkins, master, worker), SG, volumes.

🔑 QUESTIONS D'ENTRETIEN

- Pourquoi utiliser un Makefile entre le pipeline et les commandes Docker ?
- Pourquoi `kubectl apply` d'un manifest qui référence `:latest` ne redéploie-t-il pas la nouvelle image ?
- Comment analyserais-tu correctement du code C# avec SonarQube ?

Jenkins : plugin **Ansible** → *Tools* → *Ansible installations* : nom `ansible` , chemin `/usr/bin` .

L'inventaire `/etc/ansible/hosts` :

```
[local]
localhost ansible_connection=local

[k8s]
k8s-master ansible_host=<IP_PRIVÉE_MASTER> ansible_user=ubuntu
```

La clé SSH pour joindre le master : Jenkins → *Credentials* → *SSH Username with private key*, ID `ssh` , user `ubuntu` , clé privée `io-key.pem` (celle des instances). Le plugin la fournira à Ansible.

Étape 4 — Kubernetes

Cluster kubeadm sur `k8s-master` + `k8s-worker` (annexe A.6). Vérifie `kubectl get nodes` en tant qu'utilisateur `ubuntu` sur le master (le playbook `kube.yaml` exécutera `kubectl` sous cet utilisateur).

Étape 5 — Adapter les playbooks

Fork `jpetstore-6` et ouvre `Ansible/docker.yaml` :

```
- name: docker build and push
  hosts: local
  become: yes
  tasks:
    - name: Build Docker Image
      command: docker build -t petstore .
      args:
        chdir: /var/lib/jenkins/workspace/petstore      # le workspace du job : le job doit s'appeler "petstore"
    - name: tag image
      command: docker tag petstore:latest harshitgupta21/petstore:latest
    - name: Log in to Docker Hub
      community.docker.docker_login:
        registry_url: https://index.docker.io/v1/
        username: harshitgupta21
        password: <password>                          # ~ mot de passe EN CLAIR dans le dépôt !
    - name: Push image
      command: docker push harshitgupta21/petstore:latest
    - name: Run container
      command: docker run -d --name pet1 -p 8081:8080 harshitgupta21/petstore:latest
```

À corriger :

1. Le pseudo Docker Hub → le tien.
2. **Le mot de passe en clair** : passe-le en variable. Dans le pipeline, `ansiblePlaybook` accepte `extraVars` ; on y injecte un credential Jenkins (étape 6). Dans le playbook : `password: "{{ docker_password }}"` et `username: "{{ docker_user }}"` . (Encore mieux : **Ansible Vault** pour chiffrer les variables sensibles dans le dépôt.)
3. Le `docker run` échoue au deuxième passage (conteneur `pet1` existant). Ajoute avant une tâche `command: docker rm -f pet1` avec `ignore_errors: true` , ou utilise le module `community.docker.docker_container` avec `state: started` et `recreate: true` (idempotent).
4. Le `chdir` dépend du **nom du job** : nomme ton job `petstore` , ou utilise `{{ workspace }}` passé en `extraVars` avec la valeur `${env.WORKSPACE}` .

`Ansible/kube.yaml` supprime l'ancien déploiement, copie `deployment.yaml` sur le master puis l'applique. Remplace l'image dans `deployment.yaml` par la tienne, et préfère `kubectl apply` seul (le `delete` provoque une coupure à chaque déploiement ; `apply` + changement de tag fait un rolling update sans interruption).

i LE DOCKERFILE DE JPETSTORE

```
FROM openjdk:17.0.2
COPY . /usr/src/myapp
WORKDIR /usr/src/myapp
RUN ./mvnw clean package
CMD ./mvnw cargo:run -P tomcat90
```

DOCKERFILE

L'image recompile tout le projet et lance Tomcat via le plugin Maven **Cargo** : simple mais lourd (JDK complet, sources, dépôt Maven dans l'image). L'image `openjdk` est en outre **dépréciée**. Exercice : réécris-le en multi-stage, avec `maven:3.9-eclipse-temurin-17` pour builder et `tomcat:9.0-jre17` pour exécuter (`COPY --from=build /app/target/jpetstore.war /usr/local/tomcat/webapps/`).

Étape 6 — Le pipeline

Credential *Username with password* ID `dockerhub` . Job **Pipeline** nommé `petstore` :

```

pipeline {
    agent any
    tools {
        jdk 'jdk17'
        maven 'maven3'
    }
    environment {
        SCANNER_HOME = tool 'sonar-scanner'
    }
    stages {
        stage('Clean workspace') { steps { cleanWs() } }
        stage('Checkout') {
            steps { git branch: 'master', url: 'https://github.com/<toi>/jpetstore-6.git' }
        }
        stage('Compilation') { steps { sh 'mvn -B clean compile' } }
        stage('Tests') { steps { sh 'mvn -B test' } }
        stage('Analyse SonarQube') {
            steps {
                withSonarQubeEnv('sonar-server') {
                    sh '$SCANNER_HOME/bin/sonar-scanner -Dsonar.projectName=Petstore -Dsonar.projectKey=Petstore -
Dsonar.java.binaries=target/classes'
                }
            }
        }
        stage('Quality Gate') {
            steps { script { waitForQualityGate abortPipeline: false, credentialsId: 'Sonar-token' } }
        }
        stage('Build du war') { steps { sh 'mvn -B clean install -DskipTests=true' } }
        stage('OWASP Dependency-Check') {
            steps {
                withCredentials([string(credentialsId: 'nvd-api-key', variable: 'NVD_KEY')]) {
                    dependencyCheck additionalArguments: "--scan ./ --format XML --nvdApiKey ${NVD_KEY}", odcInstallation: 'DP-Check'
                }
                dependencyCheckPublisher pattern: '**/dependency-check-report.xml'
            }
        }
        stage('Docker via Ansible') {
            steps {
                withCredentials([usernamePassword(credentialsId: 'dockerhub', usernameVariable: 'DH_USER', passwordVariable: 'DH_PASS')]) {
                    dir('Ansible') {
                        ansiblePlaybook credentialsId: 'ssh', disableHostKeyChecking: true, installation: 'ansible',
                        inventory: '/etc/ansible/hosts', playbook: 'docker.yaml',
                        extraVars: [docker_user: env.DH_USER, docker_password: [value: env.DH_PASS, hidden: true]]
                    }
                }
            }
        }
        stage('Kubernetes via Ansible') {
            steps {
                dir('Ansible') {
                    ansiblePlaybook credentialsId: 'ssh', disableHostKeyChecking: true, installation: 'ansible',
                    inventory: '/etc/ansible/hosts', playbook: 'kube.yaml'
                }
            }
        }
    }
}

```

'SONAR.JAVA.BINARIES'

Le scanner Sonar a besoin des fichiers **compilés** pour analyser du Java. Le tutoriel met `-Dsonar.java.binaries=.` (tout le dossier) : ça fonctionne, mais `target/classes` est plus précis et plus rapide.

VÉRIFICATION

- http://<IP_JENKINS>:8081/jpetstore : l'animalerie (via le conteneur lancé par `docker.yaml`).
- Sur le master : `kubectl get all` → le pod `petshop` *Running* et le Service ; http://<IP_WORKER>:<NodePort>/jpetstore.
- La console Jenkins affiche la sortie d'Ansible (`ok` , `changed`) pour chaque tâche ; aucun mot de passe en clair.

Dépannage

DÉPANNAGE

- `ERROR! couldn't resolve module/action 'community.docker.docker_login'` : la collection n'est pas installée **pour l'utilisateur jenkins** (`sudo -u jenkins ansible-galaxy collection install community.docker`).
- `Permission denied (publickey) vers le master` : credential `ssh` incorrect, ou mauvais `ansible_user`.
- `kubectrl: connection refused dans kube.yaml` : sur le master, le kubeconfig n'est configuré que pour root, ou la tâche s'exécute avec `become`. Lance `kubectrl` en tant qu'`ubuntu`.
- `docker build : chdir introuvable` : le job ne s'appelle pas `petstore`.
- La page JPetStore renvoie `404` : l'URL inclut le contexte `/jpetstore`.

NETTOYAGE OBLIGATOIRE

Terminate les trois instances (ou `terraform destroy`), révoque le token Docker Hub si besoin.

QUESTIONS D'ENTRETIEN

- Comment Jenkins exécute-t-il un playbook Ansible ? Comment lui passes-tu des secrets ?
- Qu'est-ce qu'Ansible Vault ?
- Pourquoi `kubectrl delete` puis `apply` provoque-t-il une coupure, et comment l'éviter ?
- Comment rendre une tâche Ansible idempotente quand elle utilise `command` ?

Projet 27 — DevSecOps : l'application Reddit sur EKS avec Argo CD, et son monitoring

FICHE PROJET

Niveau	★★★★★ Expert
Durée	10 à 15 heures
Coût	~0,40 \$/h (Jenkins <code>t3.large</code> , EKS + 2 nœuds, ALB, LB Argo CD/Grafana si exposés)
Compétences	serveur Jenkins en Terraform, EKS créé par un pipeline Jenkins, pipeline DevSecOps qui met à jour le manifest <code>GitOps</code> , Argo CD, Ingress ALB, Prometheus/Grafana avec Helm, analyse des métriques
Prérequis	Projets 15, 16, 19 et 09 — c'est la synthèse de tout le parcours
Dépôt d'origine	<code>DevOps-Project-27/README.md</code> ; code github.com/uniqueSreedhar/Reddit-Project (<code>Jenkins-server-TF/</code> , <code>EKS-TF/</code> , <code>Jenkins-Pipeline-Code/</code> , <code>K8s/</code> , <code>deployment.yml</code> , <code>service.yml</code> , <code>ingress.yml</code>)

Ce qu'on construit

1. Terraform (poste) → EC2 Jenkins (Jenkins, Docker, SonarQube, Trivy, Terraform, kubectl, AWS CLI)
2. Job Jenkins « eks-terraform » (apply/destroy) → cluster EKS
3. Job Jenkins « reddit » :
 - checkout → npm install → SonarQube → Quality Gate → OWASP DC → Trivy fs
 - docker build + push Docker Hub (tag = BUILD_NUMBER) → Trivy image
 - sed : `K8s/deployment.yml` → image reddit:<BUILD_NUMBER> → git commit + push
4. Argo CD (dans EKS) voit le commit → synchronise → Deployment reddit
 - └─ Ingress → ALB → navigateur
5. kube-prometheus-stack (Helm) → Grafana : CPU, mémoire, réseau des pods, état des nœuds

L'application est un clone de **Reddit** écrit en **Next.js**.

Étape 1 — Le serveur Jenkins avec Terraform

```
git clone https://github.com/uniqueSreedhar/Reddit-Project.git && cd Reddit-Project
ls Jenkins-server-TF EKS-TF Jenkins-Pipeline-Code K8s
```

BASH

LE DOSSIER 'JENKINS-SERVER-TF' EST VIDE

Dans la version actuelle du dépôt, `Jenkins-server-TF/` ne contient qu'un fichier texte. Utilise le Terraform du serveur Jenkins du **projet 19** (`jenkins_server/tf-aws-ec2`, avec les corrections d'AMI et de rôle IAM), et le script d'installation `Installation-script.sh` du dépôt Reddit en user data (relis-le : il installe Jenkins, Docker, SonarQube en conteneur, Trivy, Terraform, kubectl, AWS CLI).

Après `terraform apply` : `http://<IP>:8080` (mot de passe initial dans `/var/lib/jenkins/secrets/initialAdminPassword`), crée ton utilisateur, installe les plugins (*Eclipse Temurin installer*, *SonarQube Scanner*, *NodeJS*, *OWASP Dependency-Check*, *Docker* et *Docker Pipeline*, *Pipeline: AWS Steps*, *Pipeline: Stage View*), puis SonarQube sur `http://<IP>:9000`.

Outils : `jdk` (17), `nodejs` (16 ou 20), `sonar-scanner`, `DP-Check`, `docker`. **Credentials** : `sonar-token`, `docker` (Docker Hub), `github` (token avec écriture sur ton fork), `nvd-api-key`. Avec un **rôle IAM** sur l'instance, le credential `aws-key` (access keys) du tutoriel est inutile.

Étape 2 — Le cluster EKS par un pipeline

Fork `Reddit-Project`. Dans `EKS-TF/` : adapte `backend.tf` (ton bucket, ta région), `variables.tfvars` (région, nom du cluster, type d'instance `t3.medium`), et vérifie que l'identité qui crée le cluster en devient administratrice (projet 12, `access_config`).

Job `eks-terraform` : *Pipeline from SCM* → ton fork → `Jenkins-Pipeline-Code/Jenkinsfile-EKS-Terraform`. Il est paramétré (`File-Name`, `Terraform-Action`) comme au projet 28 ; remplace l'URL Git par ton fork, supprime ou adapte `withAWS(credentials: 'aws-key', region: 'us-east-1')`, et ajoute un **input de validation** avant l'apply/destroy.

```
# sur le serveur Jenkins, une fois le cluster créé
aws eks update-kubeconfig --name <cluster> --region eu-west-3
kubectl get nodes
```

BASH

Étape 3 — Le pipeline applicatif

Job `reddit` : *Script Path* `Jenkins-Pipeline-Code/Jenkinsfile-Reddit`. Après les étapes de scan (identiques au projet 09), la fin du pipeline doit **construire, pousser, scanner, puis mettre à jour le manifest** :

```
environment {
    SCANNER_HOME = tool 'sonar-scanner'
    IMAGE         = '<toi-dockerhub>/reddit'
    GIT_REPO      = 'github.com/<toi>/Reddit-Project.git'
}
// ...
stage('Build & push Docker') {
    steps {
        script {
            withDockerRegistry(credentialsId: 'docker', toolName: 'docker') {
                sh 'docker build -t $IMAGE:$BUILD_NUMBER . && docker push $IMAGE:$BUILD_NUMBER'
            }
        }
    }
}
stage('Trivy image') {
    steps { sh 'trivy image $IMAGE:$BUILD_NUMBER > trivyimage.txt' }
}
stage('Mise à jour du manifest GitOps') {
    steps {
        withCredentials([string(credentialsId: 'github', variable: 'GITHUB_TOKEN')]) {
            sh '''
                git config user.email "jenkins@exemple.com"
                git config user.name "Jenkins"
                sed -i -E "s#image: .*/reddit:.*#image: ${IMAGE}:${BUILD_NUMBER}#" K8s/deployment.yml
                git add K8s/deployment.yml
                git commit -m "ci: reddit:${BUILD_NUMBER}"
                git push https://${GITHUB_TOKEN}@${GIT_REPO} HEAD:main
            '''
        }
    }
}
```

GROOVY

⚠ LE PIPELINE MODIFIE LE DÉPÔT QUI LE DÉCLENCHE

Comme au projet 18 : si ce job est déclenché par webhook sur **tout** le dépôt, le commit du manifest relance le pipeline indéfiniment. Excluez le dossier `K8s/` du déclenchement (*Polling ignores commits in certain paths*), ou sépare les manifests dans un **second dépôt** (méthode du projet 16, plus propre).

Vérifiez que `K8s/deployment.yml` référence bien `<toi-dockerhub>/reddit:...` au départ (sinon le `sed` ne trouvera rien).

Étape 4 — Argo CD

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
# (le tutoriel épingle la v2.4.7, très ancienne : "stable" pointe vers la dernière version stable)
kubectl -n argocd rollout status deploy/argocd-server
kubectl -n argocd patch svc argocd-server -p '{"spec": {"type": "LoadBalancer"}}' # ou port-forward (gratuit)

export ARGOCD_SERVER=$(kubectl get svc argocd-server -n argocd -o json | jq -r '.status.loadBalancer.ingress[0].hostname')
export ARGO_PWD=$(kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d)
echo "https://$ARGOCD_SERVER admin / $ARGO_PWD"
```

BASH

Crée l'Application (fichier versionné, plutôt que des clics) :

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: { name: reddit, namespace: argocd }
spec:
  project: default
  source:
    repoURL: https://github.com/<toi>/Reddit-Project.git
    targetRevision: main
    path: K8s
  destination: { server: https://kubernetes.default.svc, namespace: default }
  syncPolicy:
    automated: { prune: true, selfHeal: true }
```

YAML

L'**Ingress** (`K8s/ingress.yml`) nécessite un contrôleur d'Ingress : installe l'**AWS Load Balancer Controller** (projet 15, étapes 2 à 4) et vérifie `ingressClassName: alb` et les annotations `alb.ingress.kubernetes.io/scheme: internet-facing` et `target-type: ip`. (Sans contrôleur, l'Ingress reste sans adresse ; à défaut, expose le Service en `LoadBalancer`.)

✓ VÉRIFICATION

- Argo CD : application `reddit` *Synced* et *Healthy* ; l'arbre des ressources (Deployment → ReplicaSet → Pods, Service, Ingress).
- `kubectl get ingress` → ADDRESS (ALB) → l'application Reddit dans le navigateur.
- Modifie un texte de l'application, push : Jenkins construit `reddit:<n+1>`, commit le manifest, Argo CD déploie. Le numéro de build est visible dans `kubectl describe deploy reddit | grep Image`.

Étape 5 — Monitoring avec Prometheus et Grafana

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts && helm repo update
helm install monitoring prometheus-community/kube-prometheus-stack -n monitoring --create-namespace
kubectl -n monitoring get pods
kubectl -n monitoring port-forward svc/monitoring-grafana 3000:80
kubectl -n monitoring get secret monitoring-grafana -o jsonpath="{.data.admin-password}" | base64 -d; echo
```

BASH

(Le tutoriel installe Prometheus et Grafana séparément et expose leurs Services en `LoadBalancer` ; la pile `kube-prometheus-stack` installe tout, déjà relié, avec des dizaines de tableaux de bord Kubernetes. Détails en annexe A.10.)

Étape 6 — Analyser et interpréter les métriques

Ouvre Grafana (`http://localhost:3000`) et explore :

Tableau (dossier <i>Kubernetes</i>)	Ce qu'il montre	Question à se poser
<i>Compute Resources / Cluster</i>	CPU et mémoire par namespace	Le cluster est-il trop grand (gaspillage) ou trop petit ?
<i>Compute Resources / Namespace (Pods)</i>	conso de chaque pod de <code>default</code>	Les <code>requests</code> / <code>limits</code> du Deployment sont-ils réalistes ?
<i>Compute Resources / Pod</i>	un pod précis, avec ses limites	Le pod frôle-t-il sa limite mémoire (risque d'OOMKilled) ?
<i>Networking / Pod</i>	trafic réseau entrant/sortant	Le trafic suit-il la charge attendue ?
<i>Node Exporter / Nodes</i>	santé des nœuds (CPU, disque, réseau)	Un nœud sature-t-il ?

🔧 PROVOQUER ET OBSERVER

- Génère de la charge : `kubectl run load --rm -it --image=busybox -- sh -c "while true; do wget -q -O- http://<service-reddit>; done"`.
- Observe la montée du CPU dans *Compute Resources / Pod*.
- Ajoute un **HPA** (`kubectl autoscale deployment reddit --cpu-percent=50 --min=2 --max=6`) et regarde le nombre de pods augmenter, puis redescendre après la charge.
- Note dans ton `NOTES.md` les valeurs observées et ce que tu en conclus sur le dimensionnement.

Dépannage

🔧 DÉPANNAGE

- Argo CD *OutOfSync* permanent** : un champ modifié dans le cluster par un autre acteur (par exemple `replicas` par le HPA). Solution : retirer `replicas` du manifest quand un HPA gère la taille, ou configurer `ignoreDifferences` dans l'Application.
- Ingress sans ADDRESS** : contrôleur ALB absent ou mal configuré (projet 15, dépannage).
- Le `sed` ne modifie rien** : l'image du manifest ne correspond pas au motif `.../reddit:...`.
- Pods en *Pending*** : nœuds trop petits pour l'application + Argo CD + la pile de monitoring : passe à 3 nœuds ou à `t3.large`.

Nettoyage

🧹 NETTOYAGE OBLIGATOIRE

- Supprime l'Application Argo CD (cascade), `helm uninstall monitoring -n monitoring`, les Services LoadBalancer d'Argo CD/Grafana s'il y en a, l'Ingress, puis le contrôleur ALB.
- Job `eks-terraform` avec `destroy`.
- `terraform destroy` du serveur Jenkins.
- Vérifie load balancers, volumes EBS (Prometheus en crée parfois), NAT Gateways.

QUESTIONS D'ENTRETIEN

- Raconte ce projet de bout en bout en 3 minutes, comme en entretien. *(C'est l'exercice le plus important de toute la bible.)*
- Comment Argo CD et un HPA peuvent-ils entrer en conflit, et comment le résoudre ?
- Quelles métriques regardes-tu en premier quand une application Kubernetes est lente ?
- Quelles améliorations de sécurité apporterais-tu à ce pipeline ?

Projet 28 — DevSecOps : déployer une interface de chatbot OpenAI sur EKS avec Jenkins et Terraform

FICHE PROJET

Niveau	★★★★☆ Avancé
Durée	6 à 10 heures
Coût	Jenkins t3.large + EKS + nœuds + load balancer : ~0,30 \$/h. Utilisation de l'API OpenAI facturée à part (quelques centimes pour des tests)
Compétences	serveur Jenkins créé par Terraform, pipeline Terraform paramétré (apply/destroy) pour EKS, pipeline DevSecOps pour une application Next.js, déploiement sur EKS, gestion d'une clé d'API d'IA
Prérequis	Projets 09 et 19 (c'est la combinaison des deux)
Dépôt d'origine	DevOps-Project-28/Chatbot-UI : application Next.js (<i>Chatbot UI</i>), EKS-TF , JenkinsFile/EKS-Jenkinsfile , JenkinsFile/Chatbot-Jenkinsfile , k8s/chatbot-ui.yaml

Ce qu'on construit

Une interface web de type ChatGPT (projet open source *Chatbot UI*) qui envoie les conversations à l'**API d'OpenAI**, livrée par une chaîne DevSecOps complète :

```
1. Terraform (poste) → EC2 Jenkins + SonarQube + Trivy + Terraform + kubectl
2. Job "eks-terraform" (paramètres : File-Name, Terraform-Action) → EKS (VPC, rôles, cluster, node group)
3. Job "chatbot" : checkout → npm install → SonarQube → Quality Gate → OWASP DC → Trivy fs
                  → docker build/push (Docker Hub) → Trivy image → docker run :3000
                  → kubectl apply k8s/chatbot-ui.yaml → EKS : Deployment + Service LoadBalancer :80
                  ▼
                  navigateur → Chatbot UI → API OpenAI
```

Ce qui est nouveau par rapport aux projets 09 et 19

1. Deux pipelines dans le même dépôt : un pour l'infrastructure ([EKS-Jenkinsfile](#)), un pour l'application ([Chatbot-Jenkinsfile](#)).
2. Le pipeline Terraform est **paramétré** avec la syntaxe `properties(parameters[...])` (équivalent du bloc `parameters {}` déclaratif).
3. L'application a besoin d'une **clé d'API d'IA**, à gérer comme un vrai secret.

Étape 1 — Le dépôt

⚠ MÊME PIÈGE QU'AU PROJET 24

Les Jenkinsfiles font `git url: 'https://github.com/devcloudninjas/DevOps-Projects/DevOps-Project-28/Chatbot-UI.git'` : cette URL n'existe pas (on ne clone pas un sous-dossier). Copie le dossier [Chatbot-UI](#) à la racine d'un dépôt à toi :

```
mkdir chatbot-ui && cp -r /tmp/dp/DevOps-Project-28/Chatbot-UI/. chatbot-ui/ && cd chatbot-ui
git init && git add . && git commit -m "Chatbot UI" && git branch -M main
git remote add origin git@github.com:<toi>/chatbot-ui.git && git push -u origin main
```

BASH

Dans [k8s/chatbot-ui.yaml](#), remplace l'image [sreedhar8897/chatbot:latest](#) par [<toi-dockerhub>/chatbot:latest](#).

Étape 2 — Le serveur Jenkins

Comme au projet 19 (Terraform du serveur, ou instance à la main) : Ubuntu [t3.large](#) 30 Go, avec Jenkins, Docker, SonarQube (conteneur), Trivy, **Terraform**, **AWS CLI**, **kubectl**. Donne à l'instance un **rôle IAM** (plutôt que le credential `aws-key` d'access keys du tutoriel) : le plugin *Pipeline: AWS Steps* et les commandes `aws` / `terraform` utiliseront automatiquement le rôle.

Outils Jenkins : `jdk17`, `node19` dans le Jenkinsfile → déclare plutôt une LTS (`node20`) et adapte le nom ; `sonar-scanner`, `DP-Check`, `docker`. Credentials : `Sonar-token`, `docker`, `nvd-api-key`.

Étape 3 — Le pipeline EKS

Lis `EKS-TF/` : `vpc.tf` (VPC, subnets publics, IGW), `iam-role.tf` et `iam-policy.tf` (rôles cluster et nœuds), `eks-cluster.tf`, `eks-node-group.tf`, `backend.tf`, `variables.tfvars`. Adapte le **backend** (ton bucket, ta région), la **région** et le **type d'instance** des nœuds (`t3.medium`), et assure-toi que l'identité de Jenkins sera administratrice du cluster (bloc `access_config` avec `bootstrap_cluster_creator_admin_permissions = true`, voir projet 12).

Job `eks-terraform` (Pipeline from SCM, *Script Path* `JenkinsFile/EKS-Jenkinsfile`). Avec le rôle IAM, supprime les blocs `withAWS(credentials: 'aws-key', ...)` ou remplace-les par `withAWS(region: 'eu-west-3')`. Corrige l'URL Git.

```
properties([
  parameters([
    string(defaultValue: 'variables.tfvars', description: 'Fichier de variables', name: 'File-Name'),
    choice(choices: ['apply', 'destroy'], description: 'Action Terraform', name: 'Terraform-Action')
  ])
])
// ... stages : init → validate → plan → (apply | destroy) selon params.'Terraform-Action'
```

GROOVY

💡 AJOUTE UNE VALIDATION HUMAINE

Le pipeline d'origine fait `terraform apply -auto-approve` ou `destroy -auto-approve` sans relecture. Ajoute un `input(message: "Confirmer ${params.'Terraform-Action'} ?")` après le plan, comme au projet 19. Un `destroy` lancé par erreur sur un cluster de production est une catastrophe classique.

Build with Parameters → `apply`. 15 minutes plus tard :

```
aws eks update-kubeconfig --name <nom-du-cluster> --region eu-west-3
kubectl get nodes
```

BASH

Étape 4 — La clé d'API OpenAI

Crée une clé sur la plateforme OpenAI (*API keys*), avec une **limite de dépense** basse (*Billing → Limits*). L'application Chatbot UI lit la variable d'environnement `OPENAI_API_KEY` côté serveur (sinon, chaque utilisateur doit saisir sa propre clé dans l'interface).

🔴 UNE CLÉ D'IA EST UN SECRET QUI COÛTE DE L'ARGENT

Une clé OpenAI fuitée peut être utilisée par d'autres à **tes frais**, très vite. Ne la mets jamais dans le dépôt, dans le Dockerfile, ni dans un manifest Kubernetes en clair. Stocke-la dans un **Secret Kubernetes** créé à la main (ou mieux, via AWS Secrets Manager + External Secrets Operator), et référence-le dans le Deployment.

```
kubectl create secret generic openai --from-literal=OPENAI_API_KEY='sk-...'
```

BASH

Dans `k8s/chatbot-ui.yaml`, ajoute au conteneur :

```
env:
- name: OPENAI_API_KEY
  valueFrom:
    secretKeyRef: { name: openai, key: OPENAI_API_KEY }
```

YAML

Étape 5 — Le pipeline de l'application

Job `chatbot` (*Script Path* `JenkinsFile/Chatbot-Jenkinsfile`). Corrections à apporter au fichier d'origine :

Ligne d'origine	Problème	Correction
<code>git ... url: '.../DevOps-Project-28/Chatbot-UI.git'</code>	URL inexistante	ton dépôt, branche <code>main</code>
<code>docker tag chatbot NotHarshhaa/chatbot:latest</code>	majuscules interdites dans un nom d'image	<code><toi-dockerhub>/chatbot:\${BUILD_NUMBER}</code>
<code>sh "docker stop chatbot true"</code>	<code> </code> est un pipe , pas un « ou » : le résultat dépend du hasard	<code>docker rm -f chatbot true</code>
<code>dependencyCheck ... sans clé NVD</code>	très lent / échec	<code>--nvdApiKey</code> (leçon 14)
<code>withAWS(credentials: 'aws-key', ...)</code>	clés d'accès stockées	rôle IAM de l'instance
<code>kubectl apply -f k8s/chatbot-ui.yaml avec :latest</code>	pas de redéploiement	<code>kubectl set image deployment/chatbot chatbot=<image>:\${BUILD_NUMBER}</code> après l'apply

Pour le déploiement Kubernetes, avec le rôle IAM et `aws eks update-kubeconfig` exécuté une fois sur le serveur (dans le `HOME` de l'utilisateur `jenkins` : `sudo -u jenkins aws eks update-kubeconfig ...`), le stage devient simplement :

```
stage('Déployer sur EKS') {
  steps {
    sh '''
```

GROOVY

```

    kubectl apply -f k8s/chatbot-ui.yaml
    kubectl set image deployment/chatbot chatbot=<toi-dockerhub>/chatbot:$BUILD_NUMBER
    kubectl rollout status deployment/chatbot --timeout=180s
    ...
  }
}

```

✓ VÉRIFICATION

- `http://<IP_JENKINS>:3000` : l'interface tourne en conteneur sur le serveur Jenkins.
- `kubectl get svc chatbot-service` → EXTERNAL-IP → `http://<EXTERNAL-IP>` : la même interface servie par EKS. Pose une question : la réponse vient de l'API OpenAI.
- SonarQube, Dependency-Check, rapports Trivy disponibles.

Dépannage

🔧 DÉPANNAGE

- `npm install` très long ou en échec mémoire : Next.js est gourmand ; `t3.large` minimum, ou ajoute du swap.
- L'interface s'affiche mais les réponses échouent : clé OpenAI absente/invalid, quota épuisé, ou le pod n'a pas la variable (`kubectl exec deploy/chatbot -- printenv | grep OPENAI`).
- `error: You must be logged in to the server` : l'utilisateur `jenkins` n'a pas de kubeconfig, ou son identité n'est pas autorisée dans le cluster.
- `terraform destroy` bloqué sur le VPC : supprime d'abord le Service LoadBalancer (`kubectl delete -f k8s/chatbot-ui.yaml`).

Nettoyage

🔧 NETTOYAGE OBLIGATOIRE

1. `kubectl delete -f k8s/chatbot-ui.yaml` (supprime le load balancer) et `kubectl delete secret openai`.
2. Job `eks-terraform` avec `destroy`.
3. Destruction du serveur Jenkins (Terraform ou console).
4. Révoque la clé OpenAI sur la plateforme OpenAI.

🔧 QUESTIONS D'ENTRETIEN

- Comment gères-tu une clé d'API tierce (IA, paiement) dans une application déployée sur Kubernetes ?
- Pourquoi séparer le pipeline d'infrastructure et le pipeline applicatif ?
- `cmd | true` et `cmd || true` : quelle différence ?

Projet 29 — CI/CD d'une application de vote en microservices avec Azure DevOps et Argo CD

FICHE PROJET

Niveau	★★★★☆ Avancé
Durée	8 à 12 heures
Coût	VM Azure B2s (agent) + AKS 1 à 2 nœuds + ACR Basic : ~0,15 \$/h sur le crédit Azure
Compétences	microservices, Docker Compose, Azure Repos, agent Azure DevOps auto-hébergé , un pipeline par microservice avec filtres de chemins, ACR, AKS, Argo CD, script de mise à jour des manifests (GitOps) depuis le pipeline, imagePullSecrets
Prérequis	Projets 07, 16 et 17
Dépôt d'origine	DevOps-Project-29/README.md ; application github.com/dockeramples/example-voting-app (dossiers vote/ , result/ , worker/ , k8s-specifications/ , docker-compose.yml)

Ce qu'on construit

L'**Exemple Voting App** de Docker est l'application microservices de référence : 5 composants dans 3 langages.



Et la chaîne CI/CD :

```
commit dans vote/ → pipeline « vote » (agent auto-hébergé sur une VM Azure)
    Build : docker build vote/ → Push : ACR votingapp/vote:<BuildId>
    Update : script → k8s-specifications/vote-deployment.yaml → image ...:<BuildId> → git push (Azure Repos)
    |
Argo CD (dans AKS) surveille k8s-specifications/ ← synchronise → nouveau pod vote
(même chose pour result/ et worker/, chacun avec son pipeline)
```

Étape 1 — Tester l'application en local avec Docker Compose

Sur une VM Ubuntu Azure ([Standard_B2s](#) , port 22, et 5000/5001 depuis ton IP), qui servira aussi d'agent à l'étape 4 :

```
sudo apt update && sudo apt install -y docker.io docker-compose-v2 git
sudo usermod -aG docker $USER && newgrp docker
git clone https://github.com/dockeramples/example-voting-app.git && cd example-voting-app
docker compose up -d          # (le tutoriel installe l'ancien binaire "docker-compose" : le plugin "docker compose" le remplace)
docker compose ps
```

Ouvre http://<IP_VM>:5000 (vote) et http://<IP_VM>:5001 (résultats). Vote : le résultat change en temps réel. Lis [docker-compose.yml](#) : il décrit les 5 services, leurs réseaux, leurs volumes et leurs [healthcheck](#). `docker compose down` quand tu as fini.

Étape 2 — Projet Azure DevOps et import du dépôt

[dev.azure.com](#) → New project `votingApp` (privé) → Repos → Import a repository → <https://github.com/dockeramples/example-voting-app.git>.

Étape 3 — Azure Container Registry

```
az group create -n voting-rg -l francecentral
az acr create -g voting-rg -n votingacr$RANDOM --sku Basic
```

Puis une **service connection** Docker Registry → Azure Container Registry → ton ACR (nom : par exemple `votingACR`).

Étape 4 — L'agent auto-hébergé

🧠 POURQUOI UN AGENT AUTO-HÉBERGÉ ?

1. Les nouvelles organisations n'ont pas de parallélisme gratuit sur les agents Microsoft (voir projet 07) : un agent à toi fonctionne tout de suite.
2. Les builds Docker profitent du **cache** local de l'agent (les couches déjà construites) : beaucoup plus rapides.
3. En entreprise, un agent dans ton réseau peut joindre des ressources privées.

Contrepartie : tu dois maintenir la machine (mises à jour, espace disque, sécurité).

1. *Project settings* → *Agent pools* → *Add pool* → *Self-hosted* → nom `votingApp-Agent` → *Grant access permission to all pipelines*.
2. Le pool → *New agent* → *Linux* → copie l'URL de téléchargement affichée.
3. Sur la VM :

```
mkdir ~/myagent && cd ~/myagent
wget <URL-copiée-depuis-Azure-DevOps> # vsts-agent-linux-x64-<version>.tar.gz
tar xzvf vsts-agent-linux-x64-*.tar.gz
./config.sh # URL : https://dev.azure.com/<org> ; authentication : PAT (droits Agent Pools: Read & manage) ; pool : votingApp-Agent
sudo ./svc.sh install && sudo ./svc.sh start # en service : l'agent survit à la déconnexion SSH et au redémarrage
```

BASH

(Le tutoriel lance `./run.sh` : l'agent s'arrête dès que tu fermes le terminal. `svc.sh` l'installe comme service systemd.)

L'agent apparaît **Online** dans le pool. L'utilisateur qui exécute l'agent doit être dans le groupe `docker`.

Étape 5 — Un pipeline par microservice

Crée `vote/azure-pipelines-vote.yml` (dans Azure Repos) :

```
trigger:
  branches:
    include: [ main ]
  paths:
    include: [ vote/* ] # ne se déclenche QUE si quelque chose change dans vote/

resources:
- repo: self

variables:
  dockerRegistryServiceConnection: 'votingACR'
  imageRepository: 'votingapp/vote'
  containerRegistry: '<ton-acr>.azurecr.io'
  dockerfilePath: '$(Build.SourcesDirectory)/vote/Dockerfile'
  tag: '$(Build.BuildId)'

pool:
  name: 'votingApp-Agent' # l'agent auto-hébergé

stages:
- stage: Build
  jobs:
  - job: Build
    steps:
    - task: Docker@2
      inputs:
        containerRegistry: $(dockerRegistryServiceConnection)
        repository: $(imageRepository)
        command: build
        Dockerfile: vote/Dockerfile
        tags: $(tag)

- stage: Push
  jobs:
  - job: Push
    steps:
    - task: Docker@2
      inputs:
        containerRegistry: $(dockerRegistryServiceConnection)
        repository: $(imageRepository)
        command: push
        tags: $(tag)

- stage: Update_K8s
  jobs:
  - job: Update
```

YAML

```
steps:
- task: ShellScript@2
  inputs:
    scriptPath: 'scripts/updateK8sManifests.sh'
    args: 'vote $(imageRepository) $(tag)'
  env:
    AZDO_PAT: $(AZDO_PAT) # variable secrète du pipeline
```

Pipelines → New pipeline → Azure Repos Git → Existing YAML → `vote/azure-pipelines-vote.yml`. Duplique pour `result` et `worker` (chemins, Dockerfile, `imageRepository`).

🔧 SÉPARER BUILD ET PUSH EN DEUX STAGES... SUR UN AGENT AUTO-HÉBERGÉ

Les stages peuvent s'exécuter sur des machines différentes : l'image construite dans *Build* n'existerait pas pour *Push*. Ça fonctionne ici **parce que** le même agent auto-hébergé exécute les deux stages et garde l'image en local. Sur des agents hébergés par Microsoft, on ferait `buildAndPush` dans une seule étape.

Étape 6 — Le script de mise à jour des manifests

Crée `scripts/updateK8sManifests.sh` :

```
#!/bin/bash
# Usage : updateK8sManifests.sh <service> <imageRepository> <tag>
set -euo pipefail

SERVICE=$1 # vote | result | worker
IMAGE_REPO=$2 # votingapp/vote
TAG=$3 # numéro de build
ACR="<ton-acr>.azurecr.io"
REPO_URL="https://$${AZDO_PAT}@dev.azure.com/<org>/votingApp/_git/votingApp"

rm -rf /tmp/temp_repo
git clone "$REPO_URL" /tmp/temp_repo
cd /tmp/temp_repo

sed -i "s|image:.*|image: ${ACR}/${IMAGE_REPO}:${TAG}|g" "k8s-specifications/${SERVICE}-deployment.yaml"

git config user.email "pipeline@exemple.com"
git config user.name "Azure Pipeline"
git add "k8s-specifications/${SERVICE}-deployment.yaml"
git commit -m "ci: ${SERVICE} → ${TAG} [skip ci]"
git push

rm -rf /tmp/temp_repo
```

🔴 LE JETON DANS LE SCRIPT

Le tutoriel écrit le **PAT directement dans le script** (`https://<ACCESS-TOKEN>@dev.azure.com/...`), donc dans le dépôt. Utilise une **variable secrète** de pipeline (`AZDO_PAT`, cadenas coché) passée en `env` (les secrets ne sont pas exposés automatiquement aux scripts, voir projet 20). Donne au PAT uniquement *Code: Read & write*. `[skip ci]` dans le message empêche Azure Pipelines de se redéclencher sur ce commit (en plus, le filtre `paths` exclut déjà `k8s-specifications/`).

Étape 7 — AKS et Argo CD

```
az aks create -g voting-rg -n voting-aks --node-count 1 --node-vm-size Standard_B2s --generate-ssh-keys
az aks get-credentials -g voting-rg -n voting-aks
kubectl get nodes
```

Installe Argo CD (le script du tutoriel est correct ; version courte) :

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl wait --for=condition=Ready pods --all -n argocd --timeout=600s
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d; echo
kubectl -n argocd patch svc argocd-server -p '{"spec": {"type": "NodePort"}}'
kubectl -n argocd get svc argocd-server # port NodePort (ex. 31xxx)
kubectl get nodes -o wide # EXTERNAL-IP d'un nœud
```

⚠️ NODEPORT SUR AKS

Pour joindre un NodePort depuis Internet, le nœud doit avoir une IP publique (option `--enable-node-public-ip` à la création) et le groupe de sécurité réseau (NSG) des nœuds doit autoriser le port. Plus simple : `kubectl -n argocd port-forward svc/argocd-server 8443:443` depuis ton poste.

Configurer Argo CD : *Settings* → *Repositories* → *Connect repo* (HTTPS) → URL du dépôt Azure Repos, utilisateur quelconque, mot de passe = un PAT *Code*: Read. Puis *New App* : nom `voting-app`, chemin `k8s-specifications`, cluster `in-cluster`, namespace `default`, synchro automatique.

Étape 8 — Autoriser AKS à tirer les images

Deux options :

- **Recommandée** : `az aks update -g voting-rg -n voting-aks --attach-acr <ton-acr>` (rôle AcrPull pour les nœuds, aucun secret).
- **Celle du tutorial** : un `imagePullSecret` :

```
kubectl create secret docker-registry acr-secret \
  --docker-server=<ton-acr>.azurecr.io \
  --docker-username=<nom-acr> --docker-password=<mot-de-passe-acr> # nécessite l'utilisateur admin de l'ACR activé
```

puis ajouter dans chaque `*-deployment.yaml` : `imagePullSecrets: [{ name: acr-secret }]` (sous `spec.template.spec`).

Étape 9 — Vérifier le CI/CD

Modifie `vote/app.py` : remplace les options `Cats / Dogs` (variables `option_a / option_b`) par `Summer / Winter`. Commit sur `main`.

✓ VÉRIFICATION

1. Seul le pipeline **vote** démarre (filtre de chemins) : Build → Push → Update_K8s.
2. ACR : `votingapp/vote:<BuildId>`.
3. Azure Repos : un commit « ci: vote → » dans `k8s-specifications/vote-deployment.yaml`.
4. Argo CD : `OutOfSync` → `Synced` (synchro toutes les 3 minutes, ou *Refresh*).
5. `kubectl get pods` : un nouveau pod `vote` ; la page de vote propose « Summer / Winter ».

Pour accéder aux pages : `kubectl get svc vote result` (NodePort 31000/31001 dans les manifests d'exemple), ou `kubectl port-forward svc/vote 5000:5000`.

Dépannage

🔧 DÉPANNAGE

- **Le pipeline reste en file d'attente** : l'agent est *Offline* (service arrêté), ou le pool n'est pas autorisé pour ce pipeline.
- **permission denied ... docker.sock sur l'agent** : l'utilisateur de l'agent n'est pas dans le groupe docker (redémarre le service de l'agent après l'ajout).
- **Push** : `unauthorized` : service connection ACR incorrecte.
- **Le script échoue au git push** : PAT absent (secret non passé en `env` :) ou sans droit d'écriture.
- **Pods ImagePullBackOff** : rôle AcrPull ou `imagePullSecret` manquant (étape 8).
- **Les trois pipelines se déclenchent à chaque commit** : filtre `paths` absent ou incorrect.

Nettoyage

🔧 NETTOYAGE OBLIGATOIRE

```
az group delete -n voting-rg --yes --no-wait # AKS, ACR, VM (si elle est dans ce groupe)
```

Désinstalle l'agent de la VM si tu la gardes (`./config.sh remove`), supprime le pool, révoque les PAT.

🔧 QUESTIONS D'ENTRETIEN

- Comment organises-tu la CI de plusieurs microservices dans un même dépôt (*monorepo*) ? (Un pipeline par service avec filtres de chemins, ou un pipeline unique qui détecte les changements.)
- Avantages et inconvénients d'un agent auto-hébergé ?
- Explique la chaîne : commit → image → manifest → Argo CD → pod.
- `imagePullSecret` ou identité managée pour tirer les images d'ACR ?

Projet 30 — Le projet final : Blog App sur EKS avec Nexus, SonarQube, Trivy et monitoring

FICHE PROJET

Niveau	★★★★★ Expert — le projet de fin de parcours
Durée	12 à 20 heures
Coût	Jenkins <code>t3.large</code> , SonarQube <code>t3.medium</code> , Nexus <code>t3.medium</code> , monitoring <code>t3.medium</code> , EKS + 2 nœuds, load balancer : ~0,55 \$/h quand tout tourne
Compétences	toute la chaîne : Git + token, Jenkins, SonarQube, Nexus (dépôt Maven), Trivy, Docker Hub privé, EKS en Terraform, RBAC Kubernetes et ServiceAccount pour Jenkins, <code>imagePullSecret</code> , domaine personnalisé, Blackbox Exporter + Prometheus + Grafana, e-mails
Prérequis	Tout le parcours
Dépôt d'origine	<code>DevOps-Project-30/README.md</code> ; application <code>full-stack-blogging-app</code> (Spring Boot, <code>pom.xml</code> , <code>Dockerfile</code> , <code>deployment-service.yml</code>) — copie dans <code>DevOps-Projects/full-stack-blogging-app</code>

Ce qu'on construit

Le scénario d'une vraie mission : « Mets en place la CI/CD et le monitoring de notre application de blog. »



C'est le **projet de démonstration** à mettre en avant sur ton CV : fais-le proprement, documente-le dans `projects/30-blog-app/NOTES.md` avec des captures, et prépare un schéma d'architecture.

Étape 1 — Le dépôt et le token GitHub

- Copie `full-stack-blogging-app` dans un dépôt GitHub **privé** à toi (`blogging-app`).
- Crée un **fine-grained token** (lecture du contenu de ce dépôt) → Jenkins *Credentials* → *Username with password*, ID `git-cred` .

Étape 2 — Les serveurs

Quatre instances Ubuntu 24.04 dans le même VPC et le même Security Group (trafic interne autorisé, ports d'administration depuis ton IP uniquement) :

Serveur	Type	Ports	Contenu
<code>jenkins</code>	<code>t3.large</code> , 30 Go	8080	Jenkins, Docker, Trivy, kubectl, Maven (via Jenkins)
<code>sonarqube</code>	<code>t3.medium</code> , 20 Go	9000	SonarQube (conteneur)
<code>nexus</code>	<code>t3.medium</code> , 20 Go	8081	Nexus (conteneur)
<code>monitoring</code>	<code>t3.medium</code> , 25 Go	9090, 9115, 3000	Prometheus, Blackbox Exporter, Grafana

(Le tutoriel utilise des `t2.large` partout : `t3.medium` suffit pour SonarQube, Nexus et le monitoring dans un lab. Bonus : décris ces 4 instances en **Terraform**, comme au projet 06.)

Étape 3 — Jenkins, SonarQube et Nexus

Jenkins : annexe A.3 + Docker (groupe `docker` pour `jenkins`) + Trivy (annexe A.5) + kubectl.

SonarQube : `docker run -d --name sonar -p 9000:9000 --restart unless-stopped sonarqube:community` (après le réglage `vm.max_map_count` , annexe A.4).

Nexus (annexe A.9) :

```
sudo apt install -y docker.io
sudo docker run -d --name nexus -p 8081:8081 --restart unless-stopped -v nexus-data:/nexus-data sonatype/nexus3
sudo docker exec nexus cat /nexus-data/admin.password ; echo # mot de passe initial (attendre 2 à 3 minutes)
```

BASH

Ouvre http://<IP_NEXUS>:8081 → Sign in [admin](#) + mot de passe initial → assistant : nouveau mot de passe, **désactive l'accès anonyme**.

NEXUS, C'EST QUOI ?

Sonatype Nexus Repository est un dépôt d'artefacts, comme JFrog Artifactory (projets 01 et 06), très répandu et utilisable gratuitement en auto-hébergé. Il héberge par défaut [maven-releases](#) (versions finales, immuables), [maven-snapshots](#) (versions de développement, suffixe `-SNAPSHOT`, remplaçables) et [maven-central](#) (cache de Maven Central). Il peut aussi servir de registre Docker, npm, PyPI...

Étape 4 — Plugins, outils et configurations

Plugins : SonarQube Scanner, Eclipse Temurin installer, Config File Provider, Maven Integration, Pipeline Maven Integration, Kubernetes, Kubernetes Credentials, Kubernetes CLI, Kubernetes Client API, Docker, Docker Pipeline, Email Extension Template, Pipeline: Stage View.

Tools : JDK [jdk17](#) (adoptium, 17), Maven [maven3](#), SonarQube Scanner [sonar-scanner](#), Docker [docker](#).

SonarQube : token (Administration → Security → Users → Tokens) → credential *Secret text* [sonar-token](#) → System → SonarQube servers : [sonar-server](#), http://<IP_PRIVÉE_SONAR>:9000. Webhook SonarQube → http://<IP_PRIVÉE_JENKINS>:8080/sonarqube-webhook/.

Nexus :

1. Dans le `pom.xml` de l'application, la section `distributionManagement` doit pointer vers **ton** Nexus :

```
<distributionManagement>
  <repository>
    <id>maven-releases</id>
    <url>http://<IP_PRIVÉE_NEXUS>:8081/repository/maven-releases/</url>
  </repository>
  <snapshotRepository>
    <id>maven-snapshots</id>
    <url>http://<IP_PRIVÉE_NEXUS>:8081/repository/maven-snapshots/</url>
  </snapshotRepository>
</distributionManagement>
```

XML

2. Manage Jenkins → Managed files → Add a new Config → Global Maven settings.xml, ID [maven-settings](#). Dans le contenu, la section `<servers>` — **sans mot de passe en clair** : le plugin *Config File Provider* permet d'associer un credential Jenkins à chaque `serverId` (*Server Credentials* → Add → ServerId [maven-releases](#) → credential [nexus-cred](#), idem pour [maven-snapshots](#)). Le tutoriel écrit `admin / admin` en clair dans le fichier : évite-le.

3. Dans Nexus : [maven-releases](#) → Deployment policy : **Allow redeploy** (sinon le second `mvn deploy` d'une même version échoue avec une erreur 400). (*En production, on garde « Disable redeploy » et on incrémente la version : une release est immuable.*)

Docker Hub : crée un dépôt **privé** [blogging-app](#) ; credential *Username with password* [dockerhub-cred](#).

Étape 5 — Le pipeline CI

```
pipeline {
  agent any
  tools {
    jdk 'jdk17'
    maven 'maven3'
  }
  environment {
    SCANNER_HOME = tool 'sonar-scanner'
    IMAGE = 'toi-dockerhub/blogging-app'
  }
  stages {
    stage('Checkout') {
      steps { git branch: 'main', credentialsId: 'git-cred', url: 'https://github.com/<toi>/blogging-app.git' }
    }
    stage('Compilation') { steps { sh 'mvn -B compile' } }
    stage('Tests') { steps { sh 'mvn -B test' } }
    stage('Trivy FS') { steps { sh 'trivy fs --format table -o fs.html .' } }
    stage('Analyse SonarQube') {
      steps {
        withSonarQubeEnv('sonar-server') {
          sh '$SCANNER_HOME/bin/sonar-scanner -Dsonar.projectName=Blogging-app -Dsonar.projectKey=Blogging-app -Dsonar.java.binaries=target'
        }
      }
    }
    stage('Quality Gate') {
      steps { timeout(time: 5, unit: 'MINUTES') { waitForQualityGate abortPipeline: true, credentialsId: 'sonar-token' } }
    }
  }
}
```

GROOVY

```

    }
    stage('Build') { steps { sh 'mvn -B package -DskipTests' } }
    stage('Publication Nexus') {
        steps {
            withMaven(globalMavenSettingsConfig: 'maven-settings', jdk: 'jdk17', maven: 'maven3', traceability: true) {
                sh 'mvn -B deploy -DskipTests'
            }
        }
    }
}
stage('Build image') {
    steps {
        script {
            withDockerRegistry(credentialsId: 'dockerhub-cred', url: 'https://index.docker.io/v1/') {
                sh 'docker build -t $IMAGE:$BUILD_NUMBER .'
            }
        }
    }
}
stage('Trivy image') { steps { sh 'trivy image --format table -o image.html $IMAGE:$BUILD_NUMBER' } }
stage('Push image') {
    steps {
        script {
            withDockerRegistry(credentialsId: 'dockerhub-cred', url: 'https://index.docker.io/v1/') {
                sh 'docker push $IMAGE:$BUILD_NUMBER'
            }
        }
    }
}
// stage('Déploiement EKS') : ajouté à l'étape 6
}
}

```

✓ VÉRIFICATION

Pipeline vert jusqu'au push ; Nexus → *Browse* → [maven-releases](#) contient le `.jar` (la version du `pom.xml`, `0.0.3`, n'a pas le suffixe `-SNAPSHOT` : c'est une release) ; Docker Hub contient l'image (privée) ; SonarQube affiche le projet.

Étape 6 — EKS, RBAC et déploiement

6.1 — Le cluster. Depuis une machine d'administration (ou ton poste) avec AWS CLI, Terraform et kubectl : crée le cluster avec le Terraform du dépôt ([main.tf](#), [variables.tf](#), [output.tf](#), adapte région et AZ) **ou** avec le module EKS du projet 19. Puis :

```

aws eks update-kubeconfig --region eu-west-3 --name <cluster>
kubectl get nodes
kubectl create namespace webapps

```

BASH

6.2 — Une identité dédiée à Jenkins, limitée au namespace. Au lieu de donner à Jenkins le kubeconfig administrateur, on crée un **ServiceAccount** avec des droits **seulement** sur [webapps](#) :

```

# rbac-jenkins.yaml
apiVersion: v1
kind: ServiceAccount
metadata: { name: jenkins, namespace: webapps }
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { name: app-role, namespace: webapps }
rules:
- apiGroups: ["", "apps", "autoscaling", "networking.k8s.io"]
  resources: ["pods", "services", "configmaps", "secrets", "deployments", "replicasets",
    "horizontalpodautoscalers", "ingresses", "persistentvolumeclaims"]
  verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: app-rolebinding, namespace: webapps }
roleRef: { apiGroup: rbac.authorization.k8s.io, kind: Role, name: app-role }
subjects:
- kind: ServiceAccount
  name: jenkins
  namespace: webapps
---
apiVersion: v1
kind: Secret

```

YAML

```

metadata:
  name: jenkins-token
  namespace: webapps
  annotations: { kubernetes.io/service-account.name: jenkins }
type: kubernetes.io/service-account-token

```

```

kubectl apply -f rbac-jenkins.yaml
kubectl -n webapps get secret jenkins-token -o jsonpath='{.data.token}' | base64 -d; echo # le jeton
kubectl config view --minify -o jsonpath='{.clusters[0].cluster.server}'; echo # l'URL de l'API

```

BASH

🔑 RBAC : ROLE, ROLEBINDING, SERVICEACCOUNT

- Un **ServiceAccount** est une identité pour un programme (ici Jenkins).
- Un **Role** liste des permissions (*verbes* sur des *ressources*) **dans un namespace**. (Un **ClusterRole** vaut pour tout le cluster.)
- Un **RoleBinding** attribue le Role au ServiceAccount.

Si le jeton de Jenkins fuit, l'attaquant ne peut agir que dans `webapps`, pas sur tout le cluster : c'est encore le **moindre privilège**. Vérifie :

```
kubectl auth can-i delete nodes --as=system:serviceaccount:webapps:jenkins → no.
```

Jenkins → *Credentials* → *Secret text*, ID `k8-cred`, valeur = le jeton.

6.3 — Le secret pour tirer l'image privée :

```

kubectl -n webapps create secret docker-registry regcred \
  --docker-server=https://index.docker.io/v1/ \
  --docker-username=<toi-dockerhub> --docker-password=<token-dockerhub>

```

BASH

Dans `deployment-service.yml`, l'image doit être `<toi-dockerhub>/blogging-app:IMAGE_TAG` et le pod doit référencer `imagePullSecrets: [{ name: regcred }]`.

6.4 — Le stage de déploiement :

```

stage('Déploiement EKS') {
  steps {
    withKubeConfig(credentialsId: 'k8-cred', serverUrl: '<URL-API-EKS>', namespace: 'webapps',
      clusterName: '<cluster>', caCertificate: '', restrictKubeConfigAccess: false) {
      sh '''
        sed -i "s#IMAGE_TAG#${BUILD_NUMBER}#" deployment-service.yml
        kubectl apply -f deployment-service.yml
        kubectl rollout status deployment/bloggingapp-deployment --timeout=180s
      '''
    }
  }
}

stage('Vérification') {
  steps {
    withKubeConfig(credentialsId: 'k8-cred', serverUrl: '<URL-API-EKS>', namespace: 'webapps', clusterName: '<cluster>') {
      sh 'kubectl get pods,svc -n webapps'
    }
  }
}

```

GROOVY

(`bloggingapp-deployment` est le nom du Deployment dans `deployment-service.yml`. `caCertificate: ''` désactive la vérification du certificat du serveur : acceptable pour un lab ; en production, fournis le certificat du cluster.)

6.5 — L'e-mail : configure *Extended E-mail Notification* (projet 09, étape 6) et ajoute un bloc `post { always { emailtext ... } }` avec un corps HTML (statut coloré, numéro de build, lien), et en pièces jointes `fs.html` et `image.html`.

✅ VÉRIFICATION

`kubectl get svc -n webapps` → EXTERNAL-IP → le blog s'affiche. Inscris-toi, écris un article.

Étape 7 — Un domaine personnalisé (optionnel)

Chez ton registrar (Route 53, OVH, Gandi, GoDaddy...), crée un enregistrement **CNAME** `blog.<ton-domaine>` → le nom DNS du load balancer. Vérifie : `nslookup blog.<ton-domaine>`. (Pour HTTPS : Ingress ALB + certificat ACM, projet 15.)

Étape 8 — Le monitoring de l'application

Sur `monitoring` : installe **Prometheus**, **Grafana** et **Blackbox Exporter** (annexe A.7).

🐛 LE BLACKBOX EXPORTER

node_exporter mesure **la machine de l'intérieur**. Le **Blackbox Exporter** teste l'application **de l'extérieur**, comme un utilisateur : il envoie une requête HTTP à l'URL et mesure si elle répond (`probe_success`), en combien de temps (`probe_duration_seconds`), avec quel code HTTP, et quand expire le certificat TLS. C'est la base du **monitoring de disponibilité** (*uptime*).

Ajoute dans `prometheus.yml` (configuration standard du Blackbox Exporter, avec réécriture des labels) :

```
- job_name: blackbox
  metrics_path: /probe
  params:
    module: [http_2xx]          # module défini dans blackbox.yml : attend un code 2xx
  static_configs:
    - targets:
      - http://blog.<ton-domaine>    # ou http://<DNS-du-load-balancer>
      - https://www.google.com      # une cible témoin
  relabel_configs:
    - source_labels: [__address__]
      target_label: __param_target  # l'URL à sonder devient le paramètre ?target=
    - source_labels: [__param_target]
      target_label: instance        # et le label "instance" affiché dans Grafana
    - target_label: __address__
      replacement: localhost:9115  # Prometheus interroge en réalité le Blackbox Exporter
```

YAML

```
promtool check config /etc/prometheus/prometheus.yml && curl -X POST http://localhost:9090/-/reload
```

BASH

Grafana → source Prometheus → import du tableau **7587** (*Prometheus Blackbox Exporter*). Tu vois : statut UP/DOWN, temps de réponse, phases de la requête (DNS, connexion, TLS, traitement), code HTTP, expiration du certificat.

🚨 LE TEST DE PANNE

1. `kubectl scale deploy -n webapps --all --replicas=0` : en moins d'une minute, Grafana passe au rouge.
2. Ajoute une **règle d'alerte** `probe_success = 0` pendant 1 minute (leçon 15) et branche Alertmanager ou une alerte Grafana sur ton e-mail.
3. Remets les replicas : retour au vert. Note le temps de détection dans tes notes : c'est ton **MTTD** (*Mean Time To Detect*).

Dépannage

🔧 DÉPANNAGE

- `mvn deploy` : **401** : credentials Nexus non associés aux `serverId` du `settings.xml`, ou `id` du `pom.xml` différent (`maven-releases` / `maven-snapshots`).
- `mvn deploy` : **400 Repository does not allow updating assets** : `Allow redeploy` non activé, ou tu déploies une version release vers `maven-snapshots` (ou l'inverse).
- `kubectl` dans Jenkins : **Forbidden** : le Role ne couvre pas la ressource demandée (lis le message : il nomme la ressource et le verbe), ou mauvais namespace.
- **Pods ImagePullBackOff** : `regcred` absent du namespace `webapps` ou non référencé dans le manifest ; image privée.
- `probe_success` à 0 **alors que le site marche** : le monitoring n'atteint pas l'URL (SG, DNS), ou le module attend du 2xx et le site redirige (301) : utilise un module qui suit les redirections.

Nettoyage

🧹 NETTOYAGE OBLIGATOIRE

1. `kubectl delete -f deployment-service.yml -n webapps` (supprime le load balancer), puis destruction du cluster (Terraform ou `eksctl`).
2. *Terminate* les 4 instances, supprime le SG et les volumes.
3. Supprime l'enregistrement DNS, révoque les tokens (GitHub, Docker Hub), supprime le dépôt Docker Hub privé si besoin.

Et maintenant ?

Félicitations : si tu es arrivé ici en ayant fait les projets, tu as manipulé l'essentiel de ce qu'un ingénieur DevOps/Cloud utilise au quotidien. Pour transformer ce parcours en emploi :

1. **Portfolio** : ton dépôt IO, avec un `NOTES.md` soigné par projet, des schémas et des captures. Épingles-le sur ton profil GitHub.
2. **Trois projets « vitrine »** à présenter en entretien : par exemple 01 (architecture AWS), 27 ou 30 (DevSecOps + GitOps + monitoring), et un projet Terraform (11 ou 19).
3. **Certifications** utiles : *AWS Certified Cloud Practitioner* puis *Solutions Architect – Associate*, *HashiCorp Terraform Associate*, *CKA* (Kubernetes), *AZ-900 / AZ-104* pour Azure.
4. **Continue** : Packer, Karpenter, OpenTelemetry, Loki, Kyverno, Crossplane, platform engineering (Backstage), FinOps.

LES QUESTIONS DE SYNTHÈSE

- Présente ton projet le plus complet en 3 minutes : contexte, architecture, outils, difficultés, ce que tu as appris.
- Si tu devais réduire de 50 % le coût de cette architecture, que ferais-tu ?
- Comment sécuriserais-tu ce pipeline pour une banque ?
- Un développeur dit « mon déploiement a cassé la production ». Comment réagis-tu, et que mets-tu en place pour que ça ne se reproduise pas ?

ANNEXES

Recettes, aide-mémoire et glossaire

Les procédures d'installation réutilisées dans plusieurs projets, les commandes à avoir sous la main, le vocabulaire et un planning type pour suivre le parcours.

Annexe A — Les recettes d'installation

Ces recettes sont écrites pour **Ubuntu 24.04** (serveur). Elles sont référencées dans les projets (« voir annexe A.x »). Quand une version apparaît (`VERSION=...`), va chercher la dernière sur la page indiquée : la procédure ne change pas.

A.1 — Préparer un serveur Ubuntu

```
sudo apt update && sudo apt upgrade -y
sudo hostnamectl set-hostname <nom> && exec bash
sudo timedatectl set-timezone Europe/Paris
sudo apt install -y curl wget unzip git jq ca-certificates gnupg

# Optionnel mais très utile sur une petite instance : 2 Go de swap
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

BASH

⚠ PAS DE SWAP SUR UN NŒUD KUBERNETES

Kubernetes (kubeadm) exige par défaut que le swap soit **désactivé**. Ne fais pas cette étape sur les machines de l'annexe A.6.

A.2 — Docker (dépôt officiel)

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" \
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker

sudo usermod -aG docker $USER          # puis se déconnecter / reconnecter
# Sur un serveur Jenkins :
sudo usermod -aG docker jenkins && sudo systemctl restart jenkins

docker run --rm hello-world
docker compose version
```

BASH

(Version rapide, suffisante pour les exercices : `sudo apt install -y docker.io docker-compose-v2`.)

A.3 — Jenkins

```
# Java (Jenkins exige Java 17 ou 21)
sudo apt update && sudo apt install -y fontconfig openjdk-21-jre
java -version

# Dépôt officiel Jenkins (version LTS "stable")
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/" \
| sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt update && sudo apt install -y jenkins
sudo systemctl enable --now jenkins
sudo systemctl status jenkins --no-pager

# Mot de passe du premier démarrage
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

BASH

Puis `http://<IP>:8080` → coller le mot de passe → *Install suggested plugins* → créer l'administrateur.

💡 SI LA CLÉ DU DÉPÔT A CHANGÉ

Le nom du fichier de clé (`jenkins.io-2023.key`) est indiqué sur <https://pkg.jenkins.io/debian-stable/> . Si `apt update` affiche `NO_PUBKEY` , récupère le nom actuel sur cette page.

Réglages de base à faire tout de suite :

- *Manage Jenkins* → *Security* : vérifie que l'accès anonyme est désactivé.
- *Manage Jenkins* → *Nodes* → *Built-In Node* : 0 exécuteur si tu as des agents.
- *Manage Jenkins* → *Plugins* → *Updates* : mets à jour les plugins.
- **Sauvegarde** : tout Jenkins est dans `/var/lib/jenkins` (jobs, credentials chiffrés, plugins).

A.4 — SonarQube

Prérequis noyau (Elasticsearch embarqué) :

```
echo "vm.max_map_count=524288" | sudo tee /etc/sysctl.d/99-sonarqube.conf
echo "fs.file-max=131072" | sudo tee -a /etc/sysctl.d/99-sonarqube.conf
sudo sysctl --system
```

Version rapide (base de données embarquée, pour un lab) :

```
docker run -d --name sonar --restart unless-stopped -p 9000:9000 \
-v sonar_data:/opt/sonarqube/data -v sonar_ext:/opt/sonarqube/extensions -v sonar_logs:/opt/sonarqube/logs \
sonarqube:community
docker logs -f sonar # attendre "SonarQube is operational"
```

Version solide (avec PostgreSQL) : le fichier `compose.yaml` du **projet 16, étape 3**.

Premier accès : <http://<IP>:9000>, `admin` / `admin` → changer le mot de passe → *My Account* → *Security* → générer un token. Machine : 4 Go de RAM minimum (`t3.medium`).

A.5 — Trivy

```
# Script d'installation officiel (installe le binaire dans /usr/local/bin)
curl -sL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh | sudo sh -s -- -b /usr/local/bin
trivy --version

# Premier lancement : télécharge la base de vulnérabilités
trivy image --download-db-only
```

(Alternative : le dépôt APT officiel, décrit dans la documentation de Trivy, rubrique « Installation ».) Commandes : voir leçon 14.

A.6 — Un cluster Kubernetes avec kubernetes (1 master + 1 ou plusieurs workers)

Machines : Ubuntu 24.04, `t3.medium` minimum (2 vCPU exigés pour le master), même VPC, Security Group autorisant **tout le trafic entre les nœuds** (et 22 depuis ton IP).

Sur TOUS les nœuds :

```
# 1. Pas de swap
sudo swapoff -a && sudo sed -i 's/ swap / s/^#/' /etc/fstab

# 2. Modules noyau et réseau
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
sudo modprobe overlay && sudo modprobe br_netfilter
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF
sudo sysctl --system

# 3. Le moteur de conteneurs : containerd, avec le pilote de cgroups systemd
sudo apt update && sudo apt install -y containerd
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml > /dev/null
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
sudo systemctl restart containerd && sudo systemctl enable containerd

# 4. kubernetes, kubelet, kubectl depuis le dépôt officiel pkgs.k8s.io
K8S=v1.34 # ← mets la version mineure stable actuelle (voir https://kubernetes.io/releases/)
sudo apt install -y apt-transport-https ca-certificates curl gpg
curl -fsSL https://pkgs.k8s.io/core:stable:${K8S}/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:stable:${K8S}/deb/ /" \
| sudo tee /etc/apt/sources.list.d/kubernetes.list
```

```
sudo apt update && sudo apt install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl      # empêche les mises à jour accidentelles
sudo systemctl enable --now kubelet
```

⚠ L'ANCIEN DÉPÔT `APT.KUBERNETES.IO` N'EXISTE PLUS

Beaucoup de tutoriels (dont ceux du dépôt d'origine) utilisent apt.kubernetes.io ou packages.cloud.google.com, gelés puis supprimés en 2024. Seul pkgs.k8s.io fonctionne, avec un dépôt par version mineure.

Sur le MASTER uniquement :

```
sudo kubeadm init --pod-network-cidr=10.244.0.0/16
# À la fin, kubeadm affiche une commande "kubeadm join ..." : COPIE-LA.

mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

# Le réseau des pods (CNI) : Flannel (simple ; utilise 10.244.0.0/16 par défaut)
kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
kubectl get pods -A      # attendre que tout soit Running
```

BASH

Sur chaque WORKER :

```
sudo kubeadm join <IP_MASTER>:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash>
# Token perdu ou expiré ? Sur le master : kubeadm token create --print-join-command
```

BASH

Vérification (sur le master) :

```
kubectl get nodes -o wide      # tous les nœuds "Ready" (1 à 2 minutes)
kubectl run test --image=nginx --restart=Never && kubectl get pod test -o wide && kubectl delete pod test
```

BASH

🐛 CE QUE FAIT `KUBEADM INIT`

Il génère les certificats du cluster, démarre les composants du plan de contrôle (apiserver, etcd, scheduler, controller-manager) sous forme de pods statiques, crée la configuration d'administration (`admin.conf`) et un jeton pour que les workers puissent rejoindre le cluster. Sans plugin réseau (CNI), les nœuds restent `NotReady` et CoreDNS en `Pending` : c'est normal tant que Flannel n'est pas installé.

A.7 — Prometheus, node_exporter, Blackbox Exporter et Grafana (serveur de monitoring)

Prometheus (dernière version : <https://github.com/prometheus/prometheus/releases>) :

```
sudo useradd --system --no-create-home --shell /bin/false prometheus
VERSION=3.x.y      # ← à remplacer
cd /tmp && wget https://github.com/prometheus/prometheus/releases/download/v$VERSION/prometheus-$VERSION.linux-amd64.tar.gz
tar xzf prometheus-$VERSION.linux-amd64.tar.gz && cd prometheus-$VERSION.linux-amd64
sudo mv prometheus promtool /usr/local/bin/
sudo mkdir -p /etc/prometheus /data
sudo mv prometheus.yml /etc/prometheus/prometheus.yml
[ -d consoles ] && sudo mv consoles console_libraries /etc/prometheus/      # (absents des versions 3.x)
sudo chown -R prometheus:prometheus /etc/prometheus /data

sudo tee /etc/systemd/system/prometheus.service > /dev/null <<'EOF'
[Unit]
Description=Prometheus
Wants=network-online.target
After=network-online.target

[Service]
User=prometheus
Group=prometheus
Type=simple
Restart=on-failure
RestartSec=5s
ExecStart=/usr/local/bin/prometheus \
  --config.file=/etc/prometheus/prometheus.yml \
  --storage.tsdb.path=/data \
  --web.listen-address=0.0.0.0:9090 \
  --web.enable-lifecycle

[Install]
WantedBy=multi-user.target
EOF
```

BASH

```
sudo systemctl daemon-reload && sudo systemctl enable --now prometheus
sudo journalctl -u prometheus -f --no-pager      # en cas de problème
```

`--web.enable-lifecycle` permet de recharger la configuration sans redémarrer : `curl -X POST http://localhost:9090/-/reload` (après `promtool check config /etc/prometheus/prometheus.yml`).

node_exporter (sur chaque machine à surveiller ; https://github.com/prometheus/node_exporter/releases) :

```
sudo useradd --system --no-create-home --shell /bin/false node_exporter
VERSION=1.x.y
cd /tmp && wget https://github.com/prometheus/node_exporter/releases/download/v$VERSION/node_exporter-$VERSION.linux-amd64.tar.gz
tar xzf node_exporter-$VERSION.linux-amd64.tar.gz
sudo mv node_exporter-$VERSION.linux-amd64/node_exporter /usr/local/bin/

sudo tee /etc/systemd/system/node_exporter.service > /dev/null <<'EOF'
[Unit]
Description=Node Exporter
After=network-online.target

[Service]
User=node_exporter
Group=node_exporter
Restart=on-failure
ExecStart=/usr/local/bin/node_exporter --collector.logind

[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload && sudo systemctl enable --now node_exporter
curl -s localhost:9100/metrics | head
```

Blackbox Exporter (https://github.com/prometheus/blackbox_exporter/releases) :

```
sudo useradd --system --no-create-home --shell /bin/false blackbox
VERSION=0.x.y
cd /tmp && wget https://github.com/prometheus/blackbox_exporter/releases/download/v$VERSION/blackbox_exporter-$VERSION.linux-amd64.tar.gz
tar xzf blackbox_exporter-$VERSION.linux-amd64.tar.gz && cd blackbox_exporter-$VERSION.linux-amd64
sudo mv blackbox_exporter /usr/local/bin/
sudo mkdir -p /etc/blackbox && sudo mv blackbox.yml /etc/blackbox/      # contient le module http_2xx par défaut

sudo tee /etc/systemd/system/blackbox.service > /dev/null <<'EOF'
[Unit]
Description=Blackbox Exporter
After=network-online.target

[Service]
User=blackbox
Restart=on-failure
ExecStart=/usr/local/bin/blackbox_exporter --config.file=/etc/blackbox/blackbox.yml --web.listen-address=:9115

[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload && sudo systemctl enable --now blackbox
curl "http://localhost:9115/probe?target=https://www.google.com&module=http_2xx" | grep probe_success
```

La configuration Prometheus type :

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: prometheus
    static_configs: [{ targets: ['localhost:9090'] }]

  - job_name: node_exporter
    static_configs: [{ targets: ['localhost:9100', '<IP_AUTRE_MACHINE>:9100'] }]

  - job_name: jenkins
    metrics_path: /prometheus
    static_configs: [{ targets: ['<IP_JENKINS>:8080'] }]

  - job_name: blackbox
    metrics_path: /probe
    params: { module: [http_2xx] }
```

```
static_configs: [{ targets: ['http://<URL_DE_L_APPLICATION>'] }]
relabel_configs:
- { source_labels: [__address__], target_label: __param_target }
- { source_labels: [__param_target], target_label: instance }
- { target_label: __address__, replacement: 'localhost:9115' }
```

Grafana (dépôt officiel) :

```
sudo apt install -y apt-transport-https software-properties-common wget
sudo mkdir -p /etc/apt/keyrings
wget -q -O - https://apt.grafana.com/gpg.key | gpg --dearmor | sudo tee /etc/apt/keyrings/grafana.gpg > /dev/null
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable main" \
| sudo tee /etc/apt/sources.list.d/grafana.list
sudo apt update && sudo apt install -y grafana
sudo systemctl enable --now grafana-server
```

BASH

<http://<IP>:3000> → admin / admin → nouveau mot de passe → Connections → Data sources → Add → Prometheus → URL <http://localhost:9090> → Save & test → Dashboards → New → Import → ID (1860 Node Exporter Full, 9964 Jenkins, 7587 Blackbox).

⚠ 'APT-KEY' EST OBSOLETE

Le tutoriel d'origine utilise `wget ... | sudo apt-key add -` et le dépôt `packages.grafana.com`. `apt-key` est déprécié (et absent des Ubuntu récents) ; la méthode moderne place la clé dans `/etc/apt/keyrings/` et la référence avec `signed-by=`, comme ci-dessus.

A.8 — Un cluster EKS avec eksctl (la méthode rapide)

```
eksctl create cluster \
--name io-lab --region eu-west-3 \
--nodegroup-name workers --node-type t3.medium \
--nodes 2 --nodes-min 1 --nodes-max 3 --managed
kubectl get nodes

# Au besoin : fournisseur OIDC (pour IRSA, projet 15)
eksctl utils associate-iam-oidc-provider --cluster io-lab --region eu-west-3 --approve

# Changer la taille du node group
eksctl scale nodegroup --cluster io-lab --region eu-west-3 --name workers --nodes 3

# SUPPRIMER (après avoir supprimé les Services LoadBalancer et les Ingress !)
kubectl get svc -A | grep LoadBalancer
eksctl delete cluster --name io-lab --region eu-west-3
```

BASH

Version « fichier de configuration », à versionner dans Git :

```
# cluster.yaml → eksctl create cluster -f cluster.yaml
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig
metadata:
  name: io-lab
  region: eu-west-3
iam:
  withOIDC: true
managedNodeGroups:
- name: workers
  instanceType: t3.medium
  desiredCapacity: 2
  minSize: 1
  maxSize: 3
  volumeSize: 20
```

YAML

A.9 — Nexus Repository

```
docker run -d --name nexus --restart unless-stopped -p 8081:8081 -v nexus-data:/nexus-data sonatype/nexus3
sleep 120
docker exec nexus cat /nexus-data/admin.password; echo
```

BASH

<http://<IP>:8081> → Sign in (admin + mot de passe initial) → assistant (nouveau mot de passe, désactiver l'accès anonyme). Dépôts fournis par défaut : maven-releases, maven-snapshots, maven-central (proxy), maven-public (groupe). Machine : 4 Go de RAM minimum.

A.10 — La pile de monitoring sur Kubernetes (kube-prometheus-stack)

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm install monitoring prometheus-community/kube-prometheus-stack -n monitoring --create-namespace
kubectl -n monitoring get pods

# Grafana
kubectl -n monitoring port-forward svc/monitoring-grafana 3000:80
kubectl -n monitoring get secret monitoring-grafana -o jsonpath="{.data.admin-password}" | base64 -d; echo

# Prometheus
kubectl -n monitoring port-forward svc/monitoring-kube-prometheus-prometheus 9090:9090

# Désinstaller
helm uninstall monitoring -n monitoring
kubectl delete namespace monitoring
```

BASH

La pile installe : Prometheus (via le *Prometheus Operator*), Alertmanager, Grafana avec des dizaines de tableaux de bord Kubernetes prêts, node-exporter (DaemonSet), kube-state-metrics, et des règles d'alerte par défaut. Pour surveiller tes propres applications, on déclare des objets **ServiceMonitor**.

A.11 — Argo CD (rappel)

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl -n argocd rollout status deploy/argocd-server
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d; echo
kubectl -n argocd port-forward svc/argocd-server 8443:443 # https://localhost:8443 (admin)

# CLI
curl -sSL -o argocd https://github.com/argoproj/argo-cd/releases/latest/download/argocd-linux-amd64
sudo install -m 555 argocd /usr/local/bin/argocd && rm argocd
argocd login localhost:8443 --username admin --insecure
```

BASH

Annexe B — Aide-mémoire

Linux

Besoin	Commande
Où suis-je / lister / aller	<code>pwd</code> · <code>ls -la</code> · <code>cd /chemin</code>
Créer / copier / déplacer / supprimer	<code>mkdir -p</code> · <code>cp -r</code> · <code>mv</code> · <code>rm -r</code>
Lire un fichier	<code>cat</code> · <code>less</code> · <code>head -n 20</code> · <code>tail -f</code>
Chercher	<code>grep -ri "texte" dossier/</code> · <code>find / -name "x" 2>/dev/null</code>
Remplacer dans un fichier	<code>sed -i 's/ancien/nouveau/g' fichier</code>
Écrire en root	<code>echo "x" sudo tee -a /etc/fichier</code>
Droits / propriétaire	<code>chmod 640 f</code> · <code>chmod +x f</code> · <code>chown user:groupe f</code>
Utilisateurs / groupes	<code>useradd -m -s /bin/bash u</code> · <code>usermod -aG g u</code> · <code>id u</code>
Services	<code>systemctl status/start/stop/restart/enable X</code> · <code>journalctl -u X -f</code>
Processus / ressources	<code>ps aux</code> · <code>top</code> · <code>free -h</code> · <code>df -h</code> · <code>du -sh</code>
Réseau	<code>ip a</code> · <code>ss -tlnp</code> · <code>curl -I url</code> · <code>ping</code> · <code>dig nom</code>
SSH / copie	<code>ssh -i cle.pem user@ip</code> · <code>scp -i cle.pem f user@ip:/tmp/</code> · <code>ssh -J bastion cible</code>
Paquets	<code>apt update && apt install x</code> (Ubuntu) · <code>dnf install x</code> (Amazon Linux 2023)

Git

Besoin	Commande
État / différences	<code>git status</code> · <code>git diff</code> · <code>git diff --staged</code>
Enregistrer	<code>git add -p</code> · <code>git commit -m "feat: ..."</code>
Branches	<code>git switch -c feature/x</code> · <code>git switch main</code> · <code>git merge feature/x</code>
Synchroniser	<code>git pull</code> · <code>git push -u origin <branche></code>
Historique	<code>git log --oneline --graph --all</code>
Annuler	<code>git restore f</code> · <code>git revert <hash></code> · <code>git stash</code> / <code>git stash pop</code>

Docker

Besoin	Commande
Construire / lister	<code>docker build -t nom:tag .</code> · <code>docker images</code>
Lancer / lister	<code>docker run -d --name n -p 8080:80 image</code> · <code>docker ps -a</code>
Logs / shell	<code>docker logs -f n</code> · <code>docker exec -it n sh</code>
Arrêter / supprimer	<code>docker rm -f n</code> · <code>docker rmi image</code> · <code>docker system prune -a</code>
Registre	<code>docker login</code> · <code>docker tag a b</code> · <code>docker push b</code>
Compose	<code>docker compose up -d</code> · <code>docker compose logs -f</code> · <code>docker compose down</code>

Kubernetes

Besoin	Commande
Voir	<code>kubectl get pods,svc,deploy -n ns -o wide</code> · <code>kubectl get all -A</code>
Comprendre un problème	<code>kubectl describe pod p</code> · <code>kubectl logs p [--previous]</code> · <code>kubectl get events --sort-by=.lastTimestamp</code>
Appliquer / supprimer	<code>kubectl apply -f f.yaml</code> · <code>kubectl delete -f f.yaml</code>
Accéder	<code>kubectl port-forward svc/s 8080:80</code> · <code>kubectl exec -it p -- sh</code>
Mettre à jour	<code>kubectl set image deploy/d c=img:tag</code> · <code>kubectl rollout status/undo deploy/d</code>
Taille	<code>kubectl scale deploy/d --replicas=3</code> · <code>kubectl autoscale deploy/d --cpu-percent=50 --min=2 --max=6</code>

Besoin	Commande
Contexte	<code>kubectll config get-contexts</code> · <code>kubectll config use-context c</code>
EKS / AKS	<code>aws eks update-kubeconfig --name c --region r</code> · <code>az aks get-credentials -g rg -n c</code>

Terraform

Besoin	Commande
Démarrer	<code>terraform init [-backend-config=f]</code>
Vérifier	<code>terraform fmt -recursive</code> · <code>terraform validate</code>
Prévoir / appliquer	<code>terraform plan -out=tfplan</code> · <code>terraform apply tfplan</code>
Détruire	<code>terraform destroy</code>
État	<code>terraform state list</code> · <code>terraform state show <res></code> · <code>terraform output</code>
Débloquer	<code>terraform force-unlock <ID></code> (après vérification !)

Helm

`helm repo add n url` · `helm repo update` · `helm upgrade --install r chart -n ns --create-namespace -f values.yaml` · `helm list -A` · `helm history r` · `helm rollback r 1` · `helm template chart` · `helm uninstall r -n ns`

AWS CLI

Besoin	Commande
Qui suis-je ?	<code>aws sts get-caller-identity</code>
Instances	<code>aws ec2 describe-instances --query "Reservations[].Instances[].[InstanceId,State.Name,PublicIpAddress]" --output table</code>
S3	<code>aws s3 ls</code> · <code>aws s3 cp f s3://b/</code> · <code>aws s3 rb s3://b --force</code>
ECR	<code>aws ecr get-login-password docker login --username AWS --password-stdin <compte>.dkr.ecr.<r>.amazonaws.com</code>
Paramètres	<code>aws ssm put-parameter --name /x --type SecureString --value v</code> · <code>aws ssm get-parameter --name /x --with-decryption</code>
Session sans SSH	<code>aws ssm start-session --target i-...</code>

La check-list de fin de session (anti-facture)

🔧 AVANT DE FERMER TON ORDINATEUR

- ☐ Services Kubernetes `LoadBalancer` et Ingress supprimés (`kubectll get svc -A | grep LoadBalancer`)
- ☐ Clusters EKS / AKS supprimés (`eksctl get cluster --region ...` , `az aks list -o table`)
- ☐ `terraform destroy` exécuté dans chaque dossier utilisé
- ☐ Instances EC2 terminées ou arrêtées, **dans toutes les régions utilisées** (EC2 Global View)
- ☐ NAT Gateways, Transit Gateways, Elastic IPs non attachées supprimées
- ☐ Load balancers, target groups, volumes EBS « available », snapshots inutiles supprimés
- ☐ Bases RDS / Aurora supprimées
- ☐ Groupes de ressources Azure supprimés
- ☐ Coup d'œil à *Billing* → *Bills* / *Cost Explorer* (et le lendemain matin)

Annexe C — Glossaire

Terme	Définition
ACR / ECR	registres d'images de conteneurs d'Azure / d'AWS
Agent (CI)	machine qui exécute les jobs d'un pipeline (agent Jenkins, runner GitHub/GitLab, agent Azure DevOps)
AKS / EKS / GKE	Kubernetes managé par Azure / AWS / Google
ALB / NLB	load balancers AWS de couche 7 (HTTP) / couche 4 (TCP)
AMI	image de disque à partir de laquelle on lance une instance EC2
Ansible	outil de configuration de serveurs, sans agent, par SSH et playbooks YAML
Argo CD	outil de déploiement continu GitOps pour Kubernetes
Artefact	produit d'un build (jar, war, image, zip), versionné et stocké
ARN	identifiant unique d'une ressource AWS
ASG	Auto Scaling Group : groupe d'instances EC2 dont le nombre s'adapte
AZ	zone de disponibilité : datacenter(s) isolé(s) d'une région
Backend (Terraform)	lieu de stockage de l'état Terraform (S3, Azure Storage...)
Bastion	machine exposée qui sert de point d'entrée SSH vers des machines privées
Blue/green	déploiement de la nouvelle version à côté de l'ancienne, puis bascule du trafic
Canary	déploiement progressif de la nouvelle version à une petite partie du trafic
CDN	réseau de serveurs qui met en cache les contenus près des utilisateurs (CloudFront)
CI / CD	intégration continue / livraison ou déploiement continu
CIDR	notation d'une plage d'adresses IP (10.0.0.0/16)
CNI	plugin réseau de Kubernetes (VPC CNI, Flannel, Calico)
ConfigMap / Secret	configuration non sensible / sensible injectée dans les pods
Conteneur	processus isolé qui embarque une application et ses dépendances
CRD	type d'objet Kubernetes personnalisé (Application Argo CD, ServiceMonitor...)
CVE / CVSS	identifiant public d'une vulnérabilité / score de gravité (0 à 10)
DaemonSet	objet Kubernetes qui place un pod sur chaque nœud
DAST / SAST / SCA	tests de sécurité dynamiques / statiques du code / des dépendances
Deployment	objet Kubernetes qui gère des pods identiques et leurs mises à jour
Dockerfile	recette de construction d'une image Docker
Drift	écart entre l'état décrit (Terraform, Git) et la réalité
EBS	disque réseau attaché à une instance EC2
Elastic IP	IP publique fixe d'AWS
Exporter	programme qui expose des métriques au format Prometheus
Fargate	exécution de conteneurs sans gérer de serveurs (ECS, EKS)
GitOps	Git comme source de vérité de l'état d'un système, appliqué par un agent
Golden AMI	AMI préconfigurée pour un rôle précis
Grafana	outil de tableaux de bord et de visualisation
Health check	vérification régulière qu'un service répond correctement
Helm / chart	gestionnaire de paquets Kubernetes / paquet Helm
HPA	autoscaling horizontal des pods selon des métriques
IaC	Infrastructure as Code : infrastructure décrite dans des fichiers versionnés
IAM	gestion des identités et des droits (AWS)
Idempotence	propriété d'une opération rejouable sans effet supplémentaire
IGW	Internet Gateway : porte entre un VPC et Internet
Image (Docker)	modèle en lecture seule à partir duquel on crée des conteneurs
IMDS	service de métadonnées d'une instance EC2 (169.254.169.254)

Terme	Définition
Ingress	point d'entrée HTTP d'un cluster Kubernetes, avec règles de routage
IRSA / Pod Identity	mécanismes donnant un rôle IAM à un pod EKS
Jenkinsfile	pipeline Jenkins décrit en code (Groovy)
kubeconfig	fichier de connexion de kubectl à un ou plusieurs clusters
kubelet	agent Kubernetes présent sur chaque nœud
Lambda	fonction exécutée à la demande, sans serveur à gérer
Launch Template	modèle de lancement d'instances EC2 (AMI, type, SG, user data...)
Manifest	fichier YAML décrivant des objets Kubernetes
Moindre privilège	n'accorder que les droits strictement nécessaires
MTTD / MTTR	temps moyen de détection / de rétablissement d'un incident
Multi-stage build	Dockerfile en plusieurs étapes, l'image finale ne gardant que le nécessaire
Namespace	découpage logique d'un cluster Kubernetes
NAT Gateway	permet aux ressources privées de sortir sur Internet
Nexus / Artifactory	dépôts d'artefacts
Node group	groupe de nœuds (EC2) d'un cluster EKS
NodePort	Service Kubernetes accessible sur un port (30000-32767) de chaque nœud
OIDC	protocole d'identité utilisé pour des connexions sans secret (GitHub/GitLab → AWS, IRSA)
Pipeline	suite automatisée d'étapes de build, test et déploiement
Pod	plus petite unité de Kubernetes : un ou plusieurs conteneurs
Probe	test de vie (<i>liveness</i>) ou de disponibilité (<i>readiness</i>) d'un conteneur
Prometheus	système de collecte et de stockage de métriques
PromQL	langage de requête de Prometheus
PVC / PV / StorageClass	demande de stockage / volume / classe de stockage dans Kubernetes
Quality Gate	ensemble de conditions de qualité qu'un code doit respecter (SonarQube)
RBAC	contrôle d'accès basé sur des rôles (Kubernetes, Azure)
Région	zone géographique d'un fournisseur cloud (eu-west-3 = Paris)
Registre	serveur qui stocke des images de conteneurs
Replica / ReplicaSet	copie d'un pod / objet qui maintient N copies
Reverse proxy	serveur intermédiaire qui transmet les requêtes à une application
Rolling update	mise à jour progressive, instance par instance
Route 53	DNS d'AWS
Runner	agent d'exécution de GitHub Actions ou GitLab CI
S3	stockage d'objets d'AWS
Security Group	pare-feu stateful attaché à une ressource AWS
Service (K8s)	adresse stable devant un groupe de pods
Service connection	connexion d'Azure DevOps vers un service externe (Azure, registre, GitHub)
ServiceAccount	identité d'un programme dans Kubernetes
Shift left	déplacer les contrôles (tests, sécurité) plus tôt dans le cycle
SonarQube	outil d'analyse statique de qualité et de sécurité du code
SSM	AWS Systems Manager (Session Manager, Parameter Store...)
State (Terraform)	fichier qui associe le code Terraform aux ressources réelles
StatefulSet	objet Kubernetes pour les applications avec état
Subnet	sous-réseau d'un VPC, dans une seule AZ
Target Group	groupe de cibles vers lesquelles un load balancer envoie le trafic
Terraform	outil d'Infrastructure as Code multi-cloud
Transit Gateway	routeur central reliant plusieurs VPC
Trivy	scanner de vulnérabilités (images, fichiers, IaC, clusters)
User data	script exécuté au premier démarrage d'une instance
VPC / VNet	réseau privé virtuel dans AWS / Azure

Terme	Définition
VPC endpoint	accès privé à un service AWS depuis un VPC
WAF	pare-feu applicatif web (filtre les requêtes HTTP)
Webhook	appel HTTP envoyé automatiquement lors d'un événement (push Git...)
Workload Identity Federation	authentification d'un pipeline Azure DevOps sans secret

Annexe D — Planning type et suivi

Un planning sur 7 mois (8 à 10 h par semaine)

Mois	Contenu	Jalons
1	Partie 0 + leçons 1 à 3, projets 03 et 02	compte AWS sécurisé, budget, premières instances
2	Projet 01, leçons 4 à 8, projets 04 et 05	première appli 3-tiers, premier pipeline Jenkins
3	Projets 06, 14, 07, 10	Ansible, GitHub Actions, Azure DevOps
4	Leçons 9 à 12, projets 08, 11, 12, 17	Kubernetes, Terraform, Helm
5	Projets 15, 16, 18, 19, 20	GitOps, EKS en Terraform
6	Leçon 13, projets 21, 22, 26, leçons 14-15, projet 09	AWS natif, serverless, DevSecOps
7	Projets 13, 23, 24, 25, 27, 28, 29, 30	portfolio, présentation des projets vitrines

Si tu as moins de temps, le **parcours essentiel** (≈ 3 mois) : 03 → 02 → 01 → 04 → 05 → 14 → 08 → 11 → 19 → 16 → 09 → 30.

Le tableau de suivi

#	Projet	Commencé	Terminé	Nettoyé	NOTES.md	Présenté à voix haute
03	Linux	☐	☐	☐	☐	☐
02	VPC scalable	☐	☐	☐	☐	☐
01	Java 3-tiers	☐	☐	☐	☐	☐
04	Django ECS/ECR	☐	☐	☐	☐	☐
05	Jenkins + Docker	☐	☐	☐	☐	☐
06	Pipeline avancé	☐	☐	☐	☐	☐
07	Azure DevOps	☐	☐	☐	☐	☐
10	.NET sur Azure	☐	☐	☐	☐	☐
14	GitHub Actions Android	☐	☐	☐	☐	☐
08	EKS 2048	☐	☐	☐	☐	☐
11	Terraform 2-tiers	☐	☐	☐	☐	☐
12	Super Mario	☐	☐	☐	☐	☐
15	Robot Shop + Helm	☐	☐	☐	☐	☐
16	Jenkins + Argo CD	☐	☐	☐	☐	☐
17	AKS + Azure DevOps	☐	☐	☐	☐	☐
18	GitOps Java	☐	☐	☐	☐	☐
19	EKS Terraform + Jenkins	☐	☐	☐	☐	☐
20	Azure DevOps + Terraform	☐	☐	☐	☐	☐
21	AWS CodePipeline	☐	☐	☐	☐	☐
22	Serverless	☐	☐	☐	☐	☐
26	GitLab CI + Terraform	☐	☐	☐	☐	☐
09	Netflix DevSecOps	☐	☐	☐	☐	☐
13	Zomato	☐	☐	☐	☐	☐
23	Swiggy blue/green	☐	☐	☐	☐	☐
24	.NET DevSecOps	☐	☐	☐	☐	☐
25	Petshop + Ansible	☐	☐	☐	☐	☐
27	Reddit + Argo CD	☐	☐	☐	☐	☐
28	Chatbot EKS	☐	☐	☐	☐	☐
29	Voting App	☐	☐	☐	☐	☐
30	Blog App (final)	☐	☐	☐	☐	☐

Les ressources officielles à garder en favoris

Sujet	Où
AWS	docs.aws.amazon.com · <i>AWS Skill Builder</i> (cours gratuits) · <i>AWS Pricing Calculator</i>
Azure	learn.microsoft.com (parcours gratuits, bacs à sable)
Kubernetes	kubernetes.io/docs · killercode.com (labs gratuits dans le navigateur)
Terraform	developer.hashicorp.com/terraform · registry.terraform.io
Docker	docs.docker.com
Jenkins	jenkins.io/doc
GitHub Actions / GitLab CI	docs.github.com/actions · docs.gitlab.com/ee/ci
Argo CD	argo-cd.readthedocs.io
Prometheus / Grafana	prometheus.io/docs · grafana.com/docs · grafana.com/grafana/dashboards
Sécurité	owasp.org · aquasecurity.github.io/trivy · docs.sonarsource.com
Le programme d'origine	github.com/ZaheerrAhmed/DevOps-Projects

Bonne route. Lis les messages d'erreur, détruis tes clusters, et note tout.